

BayesFilter

Bayesian Estimation for Structural State-Space Models
HMC-Safe Filtering, Analytic Likelihood Derivatives,
and Industrial-Scale Inference

Chak Wong

June 21, 2026

Contents

I	Scope and Interfaces	1
1	Introduction	2
1.1	Central Design Thesis	2
1.2	Provenance and Claim Discipline	3
1.3	Reader Map	3
2	State-Space Model Contracts	4
2.1	Degenerate Transitions and Lag Stacks	5
2.2	Structural Nonlinear Transition Contract	5
2.3	Masks and Mixed Frequency	6
2.4	Backend Classes	6
3	Posterior Targets and HMC Requirements	7
3.1	Finite Failure Policy	7
3.2	Compilation and Differentiability	8
4	BayesFilter API Design	9
4.1	Core Objects	9
4.2	Structural Model Protocol	10
4.3	Filter Run Metadata	10
4.4	Structural Filter Implementation Requirements	11
4.5	Return Contract	11
4.6	Backend Promotion	12
4.7	Client Adapter Boundary	12
II	Linear Gaussian Filtering	13
5	Prediction-Error Decomposition	14
5.1	Linear Gaussian Specialization	14
5.2	Kalman Recursion	14
5.3	Log Likelihood	15
5.4	Initialization	15
5.5	Reference Role	16
6	Stable Linear Filtering	17
6.1	Backend Ladder	17
6.2	Covariance Form as Reference Algebra	17
6.3	Solve Form	18

6.4	Square-Root and Spectral Backends	18
6.5	Backend Agreement Tests	18
6.6	Diagnostics	19
7	Missing Data and Mixed Frequency	20
7.1	Selection Form	20
7.2	Mixed-Frequency State Augmentation	20
7.3	Implementation Policy	21
7.4	Derivative Deferral	21
8	Large-Scale Linear Gaussian State-Space Models	22
8.1	Scale Targets	22
8.2	Backend Registry	22
8.3	Validation Ladder	23
8.4	Stress Scenarios	23
8.5	Readiness Criteria	24
8.6	Derivative Consolidation Hypotheses	24
III	Analytic Derivatives	26
9	Kalman Score Recursions	27
9.1	Prediction Derivatives	27
9.2	Innovation Derivatives	27
9.3	Solve-Form Score	28
9.4	Gain and Update Derivatives	28
9.5	Masks and Time-Varying Observation Blocks	29
9.6	Evidence Status	29
10	Kalman Hessian and Observed Information	30
10.1	Second-Order Prediction	30
10.2	Second-Order Innovation Objects	31
10.3	Solve-Form Hessian	31
10.4	Second-Order Gain and Update	31
10.5	Information Objects and Sign Convention	32
10.6	Hessian-Ready Backend Contract	32
10.7	Evidence Status	32
11	Structural Derivatives	34
11.1	Provider Map	34
11.2	Parameter Transforms	35
11.3	Initial Conditions	35
11.4	Sparsity, Masks, and Zero Blocks	36
11.5	Metadata and Shape Contract	36
11.6	Provider Validation	37

12 QR and Cholesky Factor Derivatives	38
12.1 Reconstruction Contract	38
12.2 Cholesky Factor Derivatives	39
12.3 QR Factor Derivatives	39
12.4 Square-Root Kalman Stacks	40
12.5 Spectral Factors	41
12.6 Backend Parity Gates	41
13 Custom-Gradient Wrappers for HMC	42
13.1 Same-Scalar Contract	42
13.2 Gradient Callback Contract	43
13.3 Hessian and Observed-Information Path	43
13.4 Static-Shape and Compilation Contract	44
13.5 HMC Safety Labels	44
14 Derivative Validation	45
14.1 Validation Ladder	45
14.2 What Is Being Compared	45
14.3 Recommended Rungs	46
14.4 Finite-Difference Tolerances	47
14.5 Backend-Specific Gates	47
14.6 Validation Artifact	47
14.7 Audit Record	48
IV Nonlinear Filtering	49
15 Extended Kalman Filtering	50
15.1 Local Linearization	50
15.2 Likelihood Status	50
15.3 HMC Contract	51
16 Sigma-Point Filters	52
16.1 Moment-Matching Interface	52
16.2 Conjugate Unscented Transform Rules	52
16.3 Approximation Boundary	54
16.4 Derivative and Compilation Requirements	54
16.5 Evidence Ladder	54
17 Square-Root Sigma-Point Filters	56
17.1 Backend Contract	56
17.2 HMC-Relevant Risks	56
17.3 When to Prefer Square-Root Forms	57
17.4 CUT With Square-Root Backends	57
18 SVD Sigma-Point Filters	58
18.1 What the Source Audit Established	58
18.2 Spectral Differentiation Risk	58
18.3 Analytic Score and Hessian Recursion	59

18.3.1 Map and Moment Derivatives	60
18.3.2 UKF Update Objects	60
18.3.3 Likelihood Score and Hessian	62
18.4 Structural Fixed-Support Score	62
18.5 Spectral Factor Derivative Contract	64
18.6 Principal-Square-Root Placement Backend	65
18.7 Validation Targets for SVD Score and Hessian	66
18.8 SVD-CUT Score and Hessian	66
18.8.1 SVD-CUT Moment Derivatives	68
18.8.2 SVD-CUT Likelihood Gradient and Hessian	69
18.8.3 Point Count, GPU, and XLA	70
18.9 Status Labels for SVD-HMC	70
18.10 BayesFilter Policy	71
19 Structural DSGE Dynamics	72
19.1 The Structural Split	72
19.1.1 Literature Map for This Chapter	73
19.2 Why Linear Kalman Filtering Can Hide the Problem	74
19.3 Why Nonlinear Filters Must Respect the Structural Map	75
19.4 Reusable BayesFilter Structural Filtering Step	76
19.5 Degenerate Transitions	77
19.6 Standard UKF, Structural UKF, and the Predictive Law	77
19.6.1 The Original Unscented-Transform Pattern	77
19.6.2 Standard Additive-Noise UKF Algorithm	79
19.6.3 Structural UKF Algorithm	80
19.6.4 UKF in Sequential-Monte-Carlo Kernel Notation	81
19.6.5 Why These Algorithms Differ	83
19.6.6 Why the Sigma-Point Variable Uses Pre-Transition Uncertainty	84
19.7 Worked Example A: Nonlinear Structural Transition	86
19.7.1 Reviewer Edge Case: Does $\phi = 0$ Collapse the Example?	89
19.7.2 Numerical Illustration	89
19.8 Degenerate Linear Transition Example	92
19.8.1 Worked structural derivation for the degenerate linear-transition case	93
19.8.2 Worked Degenerate Linear-Transition Example	95
19.8.3 What Is Similar to the Structural-Deterministic Case?	98
19.8.4 What Is Different from the Earlier Nonlinear-State Example?	98
19.9 Structural Correctness Boundary	98
19.10 Degeneracy and SVD	99
19.11 Pruned DSGE and Adapter Implications	106
19.12 Source-Project Lesson	107
19.13 Validation Gates and Final Policy Rule	107
19.13.1 Common Misunderstandings	108
20 Particle-Filter Foundations	109
20.1 Objects, assumptions, and claim-status vocabulary	109
20.2 DPF-block notation and claim-status index	110
20.3 Nonlinear state-space model and filtering recursion	111
20.4 Empirical filtering measures	112

20.5	Sequential importance sampling	113
20.6	Sequential importance resampling and the bootstrap filter	113
20.7	The bootstrap particle-filter likelihood estimator	114
20.8	Differentiability boundary	116
20.9	Degeneracy and effective sample size	117
20.10	Baseline audit table	117
20.11	What this chapter does and does not claim	118
21	Particle-Flow Foundations	119
21.1	Objects, notation, and source roles	119
21.2	From discrete reweighting to continuous transport	120
21.3	Homotopy path between predictive and filtering laws	121
21.4	Continuity equation and transport PDE	122
21.5	EDH under Gaussian closure	123
21.6	Linear-Gaussian recovery as the exact special case	125
21.7	LEDH and local linearization	125
21.8	Stiffness and discretization	127
21.9	Exactness and approximation ledger	128
21.10	What this chapter does and does not claim	129
22	Particle-Flow Particle Filters and Proposal Correction	130
22.1	Objects, assumptions, and source roles	130
22.2	Why proposal correction is needed	131
22.3	Change of variables under the flow map	131
22.4	Proposal-corrected PF-PF weights	132
22.5	EDH/PF and LEDH/PF	132
22.5.1	EDH/PF	132
22.5.2	LEDH/PF	133
22.6	Li–Coates Algorithm 1 as an implementation contract	133
22.6.1	What the later OT chapters inherit from PF-PF	134
22.7	Jacobian determinant and log-determinant evolution	135
22.8	What the correction restores, and what it does not	136
22.9	Implementation audit obligations	137
22.10	Why this chapter matters for the later HMC discussion	137
22.11	What this chapter does and does not claim	137
23	Soft Differentiable Resampling	139
23.1	What problem this chapter solves	139
23.2	Running two-particle discontinuity cell	139
23.3	Classical resampling bottleneck	140
23.4	Soft resampling rule	140
23.5	Worked two-particle calculation	140
23.6	Algorithm: soft differentiable resampling	141
23.7	Filtering-loop insertion point	141
23.8	Verification contract	141
23.9	What this chapter does and does not claim	142

24 Deterministic OT Equal-Weighting Baseline	143
24.1 What problem this chapter solves	143
24.2 Running three-particle cell: deterministic OT version	144
24.3 Objects and notation	144
24.4 Weighted cloud, equal-weight cloud, and the coupling set	145
24.4.1 Why the filter needs a transport object, not just a scalar	145
24.5 The unregularized deterministic OT baseline	146
24.6 Barycentric projection: from coupling to equal-weight cloud	146
24.7 Algorithm: deterministic OT equal-weighting	147
24.8 Filtering-loop insertion point	147
24.9 Verification and failure interpretation	147
24.10 What this chapter does and does not claim	148
25 Entropic OT, Sinkhorn, and Barycentric Projection	149
25.1 What changes when entropy is added	149
25.2 Running three-particle cell with entropic smoothing	149
25.3 Entropic OT teacher object	150
25.3.1 Exact teacher versus exact engineering route	150
25.4 Why the optimizer factorizes	151
25.5 Algorithm: finite Sinkhorn equal-weighting	151
25.5.1 Exact online and streaming Sinkhorn as the reference lane	151
25.6 Worked balancing pass on the running cell	152
25.7 Barycentric projection as the output map	152
25.8 Differentiation contracts	153
25.9 Filtering-loop insertion point	153
25.10 Verification contract	153
25.11 What this chapter does and does not claim	154
26 Custom Gradients for LEDH-PFPF-OT	155
26.1 The scalar whose derivative is wanted	155
26.2 Forward decomposition	156
26.3 Transport orientation	156
26.4 VJP of the barycentric product	157
26.5 A normalized transport matrix	157
26.6 VJP of the quadratic cost	158
26.7 Softmin and Sinkhorn adjoints	159
26.8 Why a custom VJP is memory efficient	161
26.9 Embedding the OT VJP in the filtering loop	161
26.10 Implementation contract	162
26.11 Relation to forward-mode JVP	162
26.12 What this chapter does and does not claim	163
27 Retained-Teacher Neural OT Warm Starts	164
27.1 Objects and conventions	164
27.1.1 Retained-teacher approximation family, not one architecture	164
27.2 What the teacher computes	165
27.3 Algorithm: retained-teacher warm-started Sinkhorn	165
27.4 Training objective and deployment contract	165
27.5 Permutation symmetry	166

27.6	Residual hierarchy	166
27.6.1	How the broader retained-teacher approximations fit here	166
27.7	Filtering-loop insertion point	167
27.8	Verification notes	167
27.9	What this chapter does and does not claim	168
28	ICNN, Brenier Maps, and Monge-Gap Direct Map Learning	169
28.1	Reader orientation: what changes in this chapter?	169
28.2	Object inventory for direct map learning	170
28.3	Running direct-map cell	170
28.4	Brenier map viewpoint	171
28.5	ICNN parameterization and training contract	171
28.6	Monge-gap regularization	172
28.7	Structured direct-map variants	174
28.8	Why this chapter is not the default immediate route	174
28.8.1	Direct-map families as a different scalable-OT answer	174
28.9	Verification and non-claims	175
29	Dynamic Geodesic and Operator Learners, and the Filtering Target Contract	176
29.1	Reader orientation: what changes here?	176
29.2	Object inventory for dynamic and operator learners	176
29.3	Static map versus dynamic path: a toy contrast	177
29.4	Dynamic geodesic learners	177
29.5	Operator-learning families	178
29.6	Adjacent boundary-marker families	178
29.6.1	Sliced, subspace, and localization families as target-changing lanes	179
29.7	Same-scalar rule revisited	179
29.8	Consistency notes for richer learned objects	180
29.9	Relation to the full HMC target chapter	181
29.10	What this chapter does and does not claim	181
30	DPF HMC Target Correctness and Structural-Model Suitability	182
30.1	Objects in the HMC target contract	182
30.2	Target-status taxonomy	184
30.3	Rung-by-rung target analysis	185
30.3.1	EDH and LEDH flow proposals	185
30.3.2	PF-PF proposal correction	185
30.3.3	Soft differentiable resampling	186
30.3.4	OT and entropic OT resampling	186
30.3.5	Learned or amortized OT	186
30.4	Structured target maps	187
30.5	Pseudo-marginal and surrogate distinctions	188
30.6	Why nonlinear DSGE models sharpen the issue	188
30.7	Why MacroFinance models sharpen the issue	189
30.8	Relation to surrogate-HMC acceleration	189
30.9	Evidence gates for banking language	190
30.10	BayesFilter recommendation	191
30.11	What this chapter does and does not claim	192

31 DPF Literature-to-Debugging Verification Contract	193
31.1 Diagnostic order	193
31.2 Proposal log-probability autodiff topology	194
31.3 BayesFilter/FilterFlow deterministic tie-out contract	196
31.4 Equation-to-test matrix	197
31.5 Minimal controlled examples	198
31.6 Diagnostic specification	199
31.7 Source and implementation map	201
31.8 Scalable OT reuse-risk and validation doctrine	201
31.8.1 Code availability and reuse risk are prefilters, not verdicts	202
31.8.2 Validation ladder before promotion	202
31.8.3 Lane-specific narrow interpretations	203
31.9 Promotion thresholds	203
31.10 Boundary of this contract	204
32 Filter Choice	205
32.1 Decision Register	205
32.2 Recommended Order	206
32.3 Industrial Default	206
V High-Dimensional Nonlinear Filtering	207
33 High-Dimensional Filtering Foundations	208
33.1 The Filtering Problem In Its Least Forgiving Form	208
33.2 Exact Discrete-Time Recursion And Likelihood	209
33.3 Analytical Likelihood-Gradient And Filtering Sensitivities	211
33.4 Zakai, Kushner–Stratonovich, And DMZ Equations	212
33.5 Pathwise Robust DMZ And Memoryless Computation	213
33.6 Approximate Targets And The HMC Boundary	213
33.7 Why Dimension Breaks Filters	214
33.8 Defects Passed To Synthesis	214
33.9 Methodological Boundary	215
33.10 Bibliographic Limits	215
34 Deterministic Gaussian Projection and Point-Rule Foundations	216
34.1 Introduction And Scope	216
34.1.1 Problem Setting And Deterministic Gaussian Lanes	217
34.1.2 The Deterministic Gaussian Lane In One Page	217
34.1.3 Approximation Hierarchy And Reading Guide	217
34.2 Exact Filtering And Gaussian Projection	218
34.2.1 Object Map And Deterministic Gaussian Conventions	218
34.3 Deterministic Point-Rule Families	220
34.4 Exports To Sparse-Grid Specialization And Later Chapters	220
35 Gaussian And High-Order Filters	222
35.1 Purpose And Scope	222
35.2 Low-Dimensional Construction Of FixedSGQF	222
35.2.1 One nonlinear Gaussian moment as the basic quadrature problem	223

35.2.2	Tensor-product growth in two dimensions	223
35.2.3	One-Dimensional Rules And Their Tensor Products	223
35.2.4	Three-dimensional preview: why sparse grids help	224
35.3	Sparse-Grid Rule Reconstructed In Source Order	225
35.3.1	From Formula To Point Cloud	225
35.3.2	Exactness, Point Count, And The UKF Relation	227
35.4	What The Fixed Cloud Exports Forward	227
35.5	Methodological Boundary And Non-Claims	227
36	Fixed-Cloud Filtering, Same-Branch SGQF, and Lane-Specific Validation	228
36.1	Purpose And Scope	228
36.2	Filtering Value Path On A Fixed Cloud	228
36.2.1	Prediction Stage	228
36.2.2	Observation Stage	229
36.3	Worked One-Step Example With A Numeric Oracle	230
36.4	General Analytical Derivative Framework For FixedSGQF	233
36.4.1	Derivative Ledger	233
36.4.2	Square-Root Branch	233
36.4.3	Prediction Sensitivities	234
36.4.4	Observation Sensitivities And Innovation Score	234
36.5	Same-Branch Fixed-Cloud Scalar Contract	235
36.6	Lane-Specific Validation Burden	236
36.7	Implementation Contract For FixedSGQF	238
36.7.1	Input, Output, Invariant, And Failure Contract	238
36.7.2	One-Step Value-Path Contract	239
36.7.3	Default Numerical Constants	239
36.8	End-To-End Mathematical Algorithm	241
36.9	Exports To Later Chapters	241
37	Low-Rank Density Filters, KR Maps, and Retained Objects	243
37.1	Orientation And Scope	243
37.2	Running Example And Notation	243
37.3	Exact Filtering Target And Recursive Bottleneck	244
37.4	Tensor-Train Approximation Toolkit	244
37.4.1	Rank-Count Plausibility	245
37.5	Coordinate Systems, Conditional Maps, And Retained Objects	245
37.5.1	One Observation Step Through All Coordinate Systems	246
37.6	TT Sequential Learning And Conditional KR Transports	247
37.7	What This Chapter Exports Forward	247
37.8	Tensor-Network Kalman And Square-Root Caution	247
37.9	Complexity, Diagnostics, And Exports	247
37.10	Methodological Boundary And Sources	248
38	Squared-TT Recursion and Fixed-Branch Likelihood Construction	249
38.1	Purpose And Scope	249
38.2	Squared-TT Filtering Object And Recursive Closure	249
38.3	What Must Be Saved For The Next Time Step	250
38.4	Fixed-Branch Likelihood Construction	250

38.4.1	What A Core Object Looks Like	251
38.5	Consolidated Fixed Least-Squares Formulation	252
38.5.1	One Concrete Branch, Fully Instantiated	253
38.5.2	Square-Root Fitting Object Versus Density Audit Object	255
38.6	Initialization And Sweep Order	256
38.6.1	Forward Core Update	256
38.6.2	Derivative Core Update	256
38.6.3	Alternating-Sweep Recomputations	257
38.6.4	Stopping Rule And Failure Exits	257
38.7	One Observation Step Through The Stored Objects	258
38.8	Exports To The Same-Scalar Derivative Chapter	259
39	Fixed-Branch Approximate Likelihoods and Same-Scalar Gradients	260
39.1	Why Fixed-Branch Discipline Is Shared Across The Two Lanes	260
39.2	A Canonical Branch Record	260
39.3	Fixed-Branch Approximate Likelihoods Across The Three Operational Layers	261
39.4	Finite-Difference Contract And Branch-Validity Logic	261
39.5	Value Path, Derivative Path, And What Is Actually Differentiated	262
39.6	Approximate Targets, Exact-Target Corrections, And HMC Admissibility	265
39.7	Exports To Validation And Promotion Chapters	266
39.8	Implementation Boundary	266
40	Validation, Defect Calculus, and Method Promotion	267
40.1	Validation Architecture And Benchmark Roles	267
40.2	Finite-Difference Verification As The First Shared Gate	268
40.3	Why The Synthesis Must Start With Defects	268
40.4	Lane-Specific Validation Ladders Before Promotion	269
40.4.1	FixedSGQF ladder	270
40.4.2	Retained-object TT/KR ladder	270
40.5	Mathematical Veto Ledger Before Interpretation	271
40.6	Particle Collapse	272
40.7	Validation Benchmarks And Promotion Logic	273
40.8	Defect Calculus And Promotion Rules	274
VI	HMC, Geometry, and Diagnostics	275
41	HMC for State-Space Likelihoods	276
41.1	Target Contract	276
41.2	Implementation Position	276
41.3	Diagnostic Use of NUTS	277
42	Mass Matrices and Local Geometry	278
42.1	Sources of Curvature	278
42.2	Regularization Policy	278

43 Boundary Handling and Safe Gradients	279
43.1 Finite Invalid Returns	279
43.2 Boundary Checklist	279
44 XLA and JIT Compilation	280
44.1 Full-Pipeline Requirement	280
44.2 Device and Shape Constraints	280
45 Diagnostics and Failure Taxonomy	281
45.1 Filter and Derivative Diagnostics	281
45.2 Sampler Diagnostics	281
45.3 Failure Taxonomy	281
46 Transport Maps and Surrogate Geometry	283
46.1 Transported Targets	283
46.2 NeuTra Position	283
46.3 Surrogate Targets	284
46.4 Diagnostics	284
46.5 BayesFilter Policy	285
VII Industrial Applications and Case Studies	286
47 LGSSM Validation Ladder	287
47.1 Evidence Already Available	287
47.2 Validation Rungs	287
47.3 Interpretation	288
48 Nonlinear SSM Validation Ladder	289
48.1 Controlled Models	289
48.1.1 Model A: Affine Gaussian Structural Oracle	289
48.1.2 Model B: Nonlinear Accumulation	289
48.1.3 Model C: Nonlinear Growth Benchmark	290
48.1.4 Deferred Models	290
48.2 Current V1 Filter Evidence	290
48.3 Required Outputs	292
48.4 Stop Rules	292
49 NK SVD Case Study	293
49.1 What Passed	293
49.2 What Remains Blocked	293
49.3 Lesson for BayesFilter	294
50 CIP/AFNS Case Study	295
50.1 Filtering Lessons	295
50.2 BayesFilter Boundary	295
50.3 Readiness Use	295

51 NAWM-Scale Design Target	297
51.1 Why NAWM Changes the Standard	297
51.2 Design Requirements	297
51.3 Hypothesis	298
52 Production Checklist	299
52.1 Target and Source	299
52.2 Derivative and Filter	299
52.3 Sampler and JIT	300
52.4 Claim Discipline	300
Appendices	302
A Notation and Shape Conventions	302
A.1 Derivative Shapes	302
A.2 Label Namespace	303
B Matrix Calculus Identities	304
C Factor Derivative Proofs	305
D MathDevMCP Workflows	306
E ResearchAssistant Workflows	307
E.1 Workspace Policy	307
E.2 Ingestion Workflow	307
E.3 Claim Gate	308
E.4 Current Blockers	308
F Source Map	309
G Experiment Templates	310
G.1 Filter Reference Template	310
G.2 HMC Smoke Template	310
G.3 Medium Recovery Template	311
G.4 Strict Convergence Template	311

Part I

Scope and Interfaces

Chapter 1

Introduction

BayesFilter is a monograph and software-design project for Bayesian estimation of structural state-space models. The word structural is important. The latent state is not merely an anonymous vector passed to a numerical routine: its coordinates may be stochastic shocks, deterministic lags, accounting identities, endogenous completions, auxiliary variables, or measurement-only summaries. A filtering backend that ignores those roles can evaluate the wrong likelihood, produce the wrong gradient, or make a sampler explore states the model never generates.

The project was extracted from DSGE, macro-finance, and analytic Kalman derivative work, but its purpose is broader and cleaner than any one source project. BayesFilter owns the reusable contracts for state-space likelihoods, derivatives, diagnostics, and compiled Bayesian sampling. The economics of NK, EZ, SGU, CIP, AFNS, and future NAWM-scale applications remain downstream case studies that stress those contracts.

The organizing premise is that filtering is not a small implementation detail inside Bayesian state-space inference. For Hamiltonian Monte Carlo, the filter is part of the posterior target: it supplies the log likelihood, the gradient path, the invalid-parameter behavior, and much of the geometry seen by the sampler. A filtering backend that is adequate for point likelihood evaluation may still be unsuitable for HMC if it has hidden non-finite branches, unstable spectral derivatives, dynamic shapes, structural state violations, or opaque compilation behavior.

This monograph separates four layers:

- (i) the structural state-space model contract;
- (ii) the likelihood and derivative backend;
- (iii) the HMC target and diagnostic policy;
- (iv) application case studies that stress the generic contracts.

The separation lets the linear Gaussian and analytic derivative spine be audited before nonlinear sigma-point filters, particle filters, transport maps, or DSGE-specific experiments are used as evidence.

1.1 Central Design Thesis

BayesFilter follows one design thesis:

model projects own structural maps, and BayesFilter owns filters.

For an autoregression, the structural map may be a companion-form lag shift. For a macro-finance model, it may be an affine no-arbitrage state recursion with masked yields. For a DSGE model, it may be a perturbation solution that separates exogenous shocks from endogenous predetermined variables. These maps belong to the model project. The common filtering machinery, derivative validation, diagnostics, and HMC target discipline belong to BayesFilter.

This separation prevents each application project from rediscovering the same numerical failure modes. It also prevents a successful linear Gaussian implementation from being misused as a nonlinear structural filter simply because both expose a state vector and covariance matrix.

1.2 Provenance and Claim Discipline

The source documents for this project are read-only inputs. BayesFilter consolidates them into one notation system and one production-readiness policy, but it does not treat old prose as automatically authoritative. Every substantive claim should be traceable to one of four sources: a local source chapter or reset memo listed in `docs/source_map.yml`, a reviewed literature artifact, a code/test reference, or an explicitly recorded project decision such as the TensorFlow Probability NUTS benchmark.

The most important negative rule is equally simple: no backend is described as HMC-safe merely because it returns a finite likelihood on a smoke test. HMC safety requires a target contract, a derivative policy, finite failure handling, static-shape or compilation discipline where compilation is claimed, and diagnostics that can reveal when the sampler is exploring a numerically fragile region.

1.3 Reader Map

The book is written from the stable core outward. Part I fixes notation and interfaces, including structural state partitions. Part II develops the exact linear Gaussian likelihood because it is the regression oracle for more complicated filters. Part III records analytic derivative and custom-gradient policy before nonlinear filters are promoted into HMC experiments. Part IV discusses nonlinear filters, deterministic completions, and spectral risks with those derivative requirements already in place. Part V states the HMC, JIT, and diagnostic contracts. The case studies appear last so they can be judged against the generic BayesFilter checklist rather than driving the definitions.

Chapter 2

State-Space Model Contracts

BayesFilter treats a state-space model as a contract between a structural model and a filtering backend. The structural model may be a DSGE perturbation solver, an affine term-structure model, a macro-finance system, or a synthetic validation problem. The filter should not need to know that origin; it should receive typed maps with stable shapes and declared derivative policies.

Definition 2.1 (BayesFilter state-space contract). *For parameter vector $\theta \in \Theta \subseteq \mathbb{R}^{d_\theta}$, a BayesFilter state-space contract supplies latent states $x_t \in \mathbb{R}^{n_x}$, observations $y_t \in \mathbb{R}^{n_y}$, transition law*

$$p_\theta(x_{t+1} \mid x_t),$$

observation law

$$p_\theta(y_t \mid x_t),$$

initial law $p_\theta(x_0)$, and an observation mask or selection rule when the available components of y_t vary over time. A linear Gaussian specialization supplies matrices and vectors

$$c_\theta, \quad F_\theta, \quad Q_\theta, \quad a_\theta, \quad H_\theta, \quad R_\theta,$$

with dimensions fixed by the model instance.

Definition 2.2 (Structural state partition). *A structural state partition is model meta-data that decomposes the latent state coordinates into declared roles. The minimal BayesFilter partition records*

$$x_t = \text{pack}(s_t, d_t, a_t),$$

where s_t is the stochastic block receiving current innovations directly or through a declared stochastic transition, d_t is a deterministic-completion block, and a_t is an optional auxiliary block used for lags, bookkeeping, or backend-specific summaries. The partition also records the innovation dimension and the pack/unpack order.

The partition is part of the model contract. It should not be inferred only from a covariance matrix. A zero row of a shock-impact matrix is a useful diagnostic, but it is not a complete timing declaration. Conversely, a small positive nugget inserted for numerical regularization does not turn a deterministic identity into a structural source of randomness.

Assumption 2.1 (Shape and value contract). *For a fixed model instance, tensor ranks and leading dimensions are static throughout a compiled likelihood evaluation. Parameter-dependent covariance objects are symmetric positive semidefinite or positive definite as*

required by the backend, and invalid parameter values are reported through an explicit finite-failure policy rather than accidental linear-algebra exceptions.

The contract separates model maps from filter maps. A model map builds state-space objects from θ . A filter map consumes those objects and the data. A derivative provider supplies first or second derivatives of the state-space objects, or declares that a backend relies on autodiff through primitive operations. This distinction is essential for large models: the derivative of a structural solver and the derivative of a Kalman or sigma-point recursion are different mathematical layers.

Remark 2.1 (Structural deterministic dynamics are part of the contract). *For DSGE models, the state-space contract must not hide the distinction between shock-driven exogenous states and endogenous or accounting states that are deterministic conditional on lagged states and current shocks. A collapsed linear transition may be sufficient for an exact Kalman filter, but nonlinear sigma-point and particle filters need the structural transition map. This is a release-gating issue; see Chapter 19.*

2.1 Degenerate Transitions and Lag Stacks

Structural deterministic coordinates are not special to DSGE models. An autoregression of order two already has the same issue. One companion-form representation is

$$m_t = \phi_1 m_{t-1} + \phi_2 \ell_{t-1} + \sigma \varepsilon_t, \quad (2.1)$$

$$\ell_t = m_{t-1}, \quad (2.2)$$

$$x_t = (m_t, \ell_t). \quad (2.3)$$

The lag coordinate ℓ_t is deterministic conditional on the previous state. In an exact linear Kalman backend, the corresponding transition covariance is singular:

$$Q = \begin{bmatrix} \sigma^2 & 0 \\ 0 & 0 \end{bmatrix}.$$

That singularity is structural information, not an implementation failure. For nonlinear sigma-point or particle filters, quadrature or simulation should be applied to the innovation or stochastic block and then lifted through the lag-shift map.

The same contract covers accumulated sums, stock-flow identities, mixed-frequency aggregation states, and deterministic no-arbitrage recursions. BayesFilter should therefore test degenerate transitions directly rather than only through full-rank synthetic examples.

2.2 Structural Nonlinear Transition Contract

For a nonlinear structural model, a backend should know whether it integrates over innovations, stochastic states, or the full latent state. A convenient contract is

$$s_t = T_s(s_{t-1}, \varepsilon_t; \theta), \quad (2.4)$$

$$d_t = T_d(d_{t-1}, s_{t-1}, s_t; \theta), \quad (2.5)$$

$$x_t = \text{pack}(s_t, d_t, a_t), \quad (2.6)$$

$$y_t \sim p_\theta(y_t \mid x_t). \quad (2.7)$$

When a mixed structural model is evaluated by a full-state Gaussian approximation, the result must be labeled as an approximation. The default nonlinear filter for a mixed structural model should preserve deterministic identities pointwise for propagated sigma points or particles.

Assumption 2.2 (Approximation labeling). *If a backend integrates over a larger stochastic space than the declared model transition, or injects artificial noise into deterministic coordinates, the run metadata must record an approximation label. The label is part of the likelihood provenance and must be visible to HMC diagnostics and case-study reports.*

2.3 Masks and Mixed Frequency

A sparse panel should be represented by a deterministic mask or selection operator, not by changing the model definition at each time. At time t , the filter may work with the observed subvector $y_{t,\mathcal{O}_t}$ and the corresponding selected observation map. The public contract must state whether an all-missing time step performs only prediction, whether ragged panels are batched by pattern, and whether the implementation keeps static shapes for JIT compilation.

2.4 Backend Classes

BayesFilter distinguishes the following backend roles:

- exact linear Gaussian likelihood backends;
- square-root or solve-form variants of exact linear Gaussian backends;
- approximate nonlinear Gaussian filters such as EKF, UKF, CKF, and SVD-factored sigma-point filters;
- particle-filter or simulation-based likelihood estimators;
- diagnostic or surrogate backends that do not define the final HMC target without an explicit correction policy.

The same model can be evaluated by more than one backend, but the monograph must state which backend defines the posterior target and which backends are only diagnostic approximations.

Chapter 3

Posterior Targets and HMC Requirements

HMC samples an unconstrained or transformed parameter vector by integrating Hamiltonian dynamics for a log target. In BayesFilter, that log target usually contains a filtering likelihood. The target contract must therefore be stated before discussing sampler tuning.

Definition 3.1 (Filtering posterior target). *Let $u \in \mathbb{R}^{d_u}$ denote the coordinates used by the sampler and let $\theta = T(u)$ be the model parameter transform. A filtering posterior target has the form*

$$\log \pi(u \mid y_{1:T}) = \log p(T(u)) + \ell(T(u); y_{1:T}) + \log |\det DT(u)|,$$

where ℓ is the log likelihood returned by the chosen filtering backend.

For HMC, the value alone is not enough. The backend and transform jointly define the gradient seen by the sampler. If a custom gradient is used, it must be the gradient of the same scalar target value, not the gradient of a nearby surrogate unless the sampler is explicitly corrected for that surrogate. In particular, if the filtering backend exposes likelihood derivatives in model parameter coordinates θ , the HMC wrapper must still construct the full sampler-coordinate target in u -space by adding the prior and transform Jacobian terms from Definition 3.1.

3.1 Finite Failure Policy

Invalid parameters are ordinary events during HMC warmup and exploration. A BayesFilter target must decide how to handle them. The preferred policy is to return a deterministic low log density together with the exact derivative of that same returned fallback scalar for known invalid regions, while separately recording diagnostics for the reason. If the fallback scalar is constant on the invalid region, the corresponding gradient must be zero. A finite surrogate direction that is not the derivative of the returned scalar is not HMC-safe unless the sampler is explicitly corrected for that surrogate. A linear-algebra failure that aborts the process is not an HMC-compatible failure mode.

The policy must be explicit for:

- parameter transforms and prior support;
- stationarity or determinacy restrictions;

- covariance positive-definiteness checks;
- failed factorizations or solve residuals;
- non-finite likelihood or gradient values.

3.2 Compilation and Differentiability

When a backend is advertised as JIT or XLA compatible, the claim refers to the complete value-and-gradient path needed by the sampler. Partial tracing of an outer function is not enough if the filter, derivative provider, or failure branch executes outside the compiled region.

The derivative policy should be one of:

- analytic score or Hessian recursion;
- custom gradient wrapping a certified analytic derivative;
- autodiff through smooth primitive operations;
- stopped-gradient approximation used only with an explicit correction or diagnostic label;
- not HMC-safe.

Chapter 4

BayesFilter API Design

The monograph is not an API reference, but its mathematical contracts should be precise enough that an implementation can be audited against them. The API shape below is therefore a documentation contract rather than a final Python interface.

4.1 Core Objects

A BayesFilter implementation should expose five separable objects:

- a parameter transform from sampler coordinates u to model parameters θ ;
- a structural state partition describing stochastic, deterministic, and auxiliary coordinates;
- a model provider that builds state-space objects from θ ;
- a filter backend that returns log likelihood and diagnostics;
- a derivative provider or autodiff policy that returns the score and, when needed, curvature information.

For a linear Gaussian model, the model provider returns

$$(c, F, Q, a, H, R, m_0, P_0)$$

and optionally their first and second derivatives with respect to θ . For a nonlinear model, it returns transition and observation maps together with the sigma-point, linearization, or simulation policy required by the selected backend.

4.2 Structural Model Protocol

The exact software interface may change, but implementations should be auditable against the following documentation contract:

```

    partition : state names, role indices, innovation dimension,
    initial_law :  $(m_0, P_0)$  or a sampling rule,
    innovation_law :  $\Sigma_\varepsilon$  or an innovation sampler,
    unpack_state, pack_state :  $x \leftrightarrow (s, d, a)$ ,
    propagate_stochastic :  $(s_{t-1}, \varepsilon_t, \theta) \mapsto s_t$ ,
    complete_deterministic :  $(d_{t-1}, s_{t-1}, s_t, \theta) \mapsto d_t$ ,
    observe :  $x_t \mapsto p_\theta(y_t \mid x_t)$ .

```

Here s_t denotes the declared stochastic block, d_t the deterministic completion block, and a_t any auxiliary or backend-specific block. A model with no deterministic block can set d_t empty; a linear Gaussian companion form can still expose the partition even if the exact Kalman backend consumes only the collapsed matrices.

DSGE, MacroFinance, and other application projects should implement this protocol through adapters. BayesFilter should not require those projects to copy filter internals, and those projects should not define private near-duplicates of BayesFilter filters.

4.3 Filter Run Metadata

Every backend should return metadata that records what target was actually evaluated. At minimum, diagnostics should include:

- backend name and version;
- structural partition summary;
- integration space: innovation, stochastic-state, or full-state;
- deterministic-completion policy;
- approximation label, if any;
- derivative policy;
- compiled/eager execution status when relevant;
- finite-failure or warning codes.

This metadata prevents a collapsed approximation, a diagnostic surrogate, or a partially compiled run from being misreported as a structural HMC target.

For mixed structural models, the metadata must also be strong enough to audit deterministic completion. A sigma-point, cubature, or particle backend should therefore expose:

- whether deterministic identities were checked pointwise;
- the maximum deterministic identity residual when such a check is available;

- whether a full-state integration path was used for a mixed model;
- the approximation label attached to any enlarged stochastic space;
- whether artificial jitter was only a numerical linear-algebra device or part of the model transition being evaluated.

These fields are not cosmetic. They are the evidence that the backend followed the `structural_filter_step` contract from Chapter 19. If they are missing, the safest interpretation is that the result has not yet earned a structural filtering claim.

4.4 Structural Filter Implementation Requirements

A BayesFilter structural filter implementation should satisfy the following requirements before a client adapter uses it as an HMC likelihood:

- (i) **Fail-closed partition validation:** mixed models must declare stochastic, deterministic-completion, and auxiliary coordinates before a non-full-state backend runs.
- (ii) **Visible integration space:** the result must state whether the backend integrated over innovations, stochastic states, or the full state.
- (iii) **Deterministic identity diagnostics:** test fixtures with known identities, such as AR(p) lag shifts or $k_t - \phi k_{t-1} - \gamma m_t^2 = 0$, must be checked pointwise for propagated points or particles.
- (iv) **Approximation-label propagation:** if a mixed model is evaluated by a full-state Gaussian approximation, the label must survive into the likelihood result, HMC target wrapper, case-study tables, and reset memo.
- (v) **Reference-oracle tests:** affine Gaussian cases must recover the exact Kalman value, and low-dimensional nonlinear cases should be compared with dense quadrature, high-particle simulation, or another explicit reference.
- (vi) **Derivative and compilation gates:** value-side structural correctness is not a gradient guarantee. SVD/eigen derivatives, static shapes, eager/compiled parity, and HMC diagnostics need separate gates.

DSGE and MacroFinance adapters should supply model timing and economics. BayesFilter should supply this generic validation and filtering machinery. That boundary keeps the common structural-filtering rule in one place while still allowing client projects to own their own solution methods and priors.

4.5 Return Contract

The minimum likelihood return is

$$(\ell, d),$$

where ℓ is a scalar log likelihood and d is a diagnostics record. A gradient-capable backend returns

$$(\ell, \nabla_{\theta} \ell, d),$$

and a curvature backend returns

$$(\ell, \nabla_{\theta}\ell, \nabla_{\theta\theta}^2\ell, d).$$

Diagnostics should include enough information to decide whether the result is a valid target evaluation, a finite invalid-region return, or a numerical warning that should influence sampler adaptation or backend selection.

4.6 Backend Promotion

BayesFilter should promote a backend through stages:

- (i) value-only likelihood evaluation;
- (ii) gradient validation on small problems;
- (iii) compiled value-and-gradient execution;
- (iv) stress tests under masks, boundaries, and near-singular covariances;
- (v) HMC diagnostics on controlled targets;
- (vi) application case studies.

No backend should skip directly from value-only likelihood tests to industrial HMC use.

4.7 Client Adapter Boundary

The adapter boundary is intentionally strict. A client project owns:

- structural equations and timing;
- parameter transformations and priors;
- model-specific observation equations;
- perturbation, no-arbitrage, or equilibrium solution objects;
- application-level HMC targets and case-study data.

BayesFilter owns:

- generic filter recursions;
- structural partition validation;
- square-root and SVD numerical backends;
- derivative validation ladders;
- finite-failure and diagnostic conventions;
- backend promotion gates.

The boundary is a maintenance device. It lets MacroFinance reuse analytic Kalman derivatives, DSGE reuse structural nonlinear filtering, and future large models reuse the same validation ladder without forking the numerical core.

Part II

Linear Gaussian Filtering

Chapter 5

Prediction-Error Decomposition

The linear Gaussian model is the first BayesFilter reference target because its likelihood is exact and its derivative recursions can be audited against code and automatic-differentiation tests. This chapter records the value-only likelihood spine; score and Hessian recursions are deferred to Chapters 9–10.

5.1 Linear Gaussian Specialization

Under the contract in Definition 2.1, the linear Gaussian specialization is

$$x_{t+1} = c(\theta) + F(\theta)x_t + u_{t+1}, \quad u_{t+1} \sim \mathcal{N}(0, Q(\theta)), \quad (5.1)$$

$$y_t = a(\theta) + H(\theta)x_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, R(\theta)). \quad (5.2)$$

For the exact likelihood backend, $Q(\theta) \succeq 0$ and $R(\theta) \succ 0$ are required at every accepted target evaluation. The initial moments are denoted $m_{0|0}$ and $P_{0|0}$.

The semidefinite allowance for Q is intentional. Companion-form lag stacks, mixed-frequency accumulators, and structural deterministic coordinates often produce process covariances with zero rows or deficient rank. This is not a reason to abandon the exact linear Gaussian likelihood. It is a requirement that the backend distinguish structural degeneracy from accidental numerical failure.

Assumption 5.1 (Innovation regularity). *For every time point contributing an observation, the selected innovation covariance S_t defined below is positive definite on the observed coordinates. If the model or data mask makes S_t singular, the likelihood backend must return a named finite-failure diagnostic or use a separately documented diffuse/singular-observation treatment.*

5.2 Kalman Recursion

Let

$$m_{t|s} = \mathbb{E}[x_t \mid y_{1:s}], \quad P_{t|s} = \text{Var}(x_t \mid y_{1:s}).$$

The prediction step is

$$m_{t|t-1} = c + Fm_{t-1|t-1}, \quad (5.3)$$

$$P_{t|t-1} = FP_{t-1|t-1}F^\top + Q. \quad (5.4)$$

The innovation and innovation covariance are

$$v_t = y_t - a - Hm_{t|t-1}, \quad (5.5)$$

$$S_t = HP_{t|t-1}H^\top + R. \quad (5.6)$$

When $S_t \succ 0$, the Kalman gain and filtered moments are

$$K_t = P_{t|t-1}H^\top S_t^{-1}, \quad (5.7)$$

$$m_{t|t} = m_{t|t-1} + K_tv_t, \quad (5.8)$$

$$P_{t|t} = (I - K_tH)P_{t|t-1}. \quad (5.9)$$

The covariance update in (5.9) is the compact textbook form. Production implementations may use the Joseph form

$$P_{t|t} = (I - K_tH)P_{t|t-1}(I - K_tH)^\top + K_tRK_t^\top$$

or may update an algebraically equivalent factor rather than materializing a full covariance matrix. These are numerical representations of the same exact linear Gaussian likelihood, not different statistical models.

5.3 Log Likelihood

The prediction-error decomposition gives

$$\ell(\theta) = -\frac{1}{2} \sum_{t=1}^T [\log \det S_t + v_t^\top S_t^{-1} v_t + n_{y,t} \log(2\pi)], \quad (5.10)$$

where $n_{y,t}$ is the number of observed components at time t . In a dense panel, $n_{y,t} = n_y$. In a masked panel, the same formula applies after selecting the observed components and the corresponding rows and columns of the observation objects.

If no components are observed at a time point, the measurement update and likelihood contribution are skipped and the filter carries the prediction forward. This convention treats missingness as part of the observation contract rather than as a change in the latent transition law.

Equation (5.10) is the exact likelihood for (5.1)–(5.2). It is also the reference value against which approximate nonlinear filters and alternative linear algebra backends should be tested whenever a controlled linear Gaussian case is available.

5.4 Initialization

Initial moments are part of the model contract. If $m_{0|0}$ and $P_{0|0}$ are fixed by design, derivative initial conditions in later chapters are zero. If $P_{0|0}$ is the stationary covariance, then its derivatives are governed by the derivative Lyapunov equations for the transition system. BayesFilter must record which initialization policy defines the target, because changing it changes the likelihood.

5.5 Reference Role

The prediction-error likelihood is the value oracle for the rest of the monograph. Later chapters may evaluate it through covariance, solve-form, Cholesky, QR, or SVD linear-algebra primitives, but they must agree on (5.10) within documented tolerances. Non-linear filters should also recover this value when their model is specialized to a linear Gaussian case.

Derivative chapters differentiate this same scalar likelihood. A custom gradient, analytic score recursion, or Hessian recursion is valid for HMC only if it corresponds to the value defined here, plus the prior and parameter transform terms in Chapter 3.

Chapter 6

Stable Linear Filtering

The covariance-form equations in Chapter 5 are algebraically correct, but they are not always the best production primitive. Large panels, nearly collinear observations, low measurement noise, and repeated covariance updates can turn small roundoff differences into large differences in precision matrices and Hessian contractions.

6.1 Backend Ladder

BayesFilter uses a backend ladder rather than a single Kalman implementation:

- (i) covariance-form recursion for clarity and small reference problems;
- (ii) solve-form likelihood evaluation using factored innovation covariances;
- (iii) Cholesky square-root propagation where positive pivots are reliable;
- (iv) QR square-root propagation when reconstruction and solve diagnostics are needed;
- (v) SVD or spectral fallbacks only with explicit diagnostic labels.

The ladder is a production policy, not a mathematical change to the likelihood. All exact linear Gaussian backends must agree with (5.10) within documented tolerances.

6.2 Covariance Form as Reference Algebra

The covariance-form recursion is the clearest way to state the likelihood, but it is not automatically the safest way to compute it. In finite precision, the compact covariance update can lose symmetry or positive semidefiniteness after many time steps. Implementations should therefore symmetrize only as a documented numerical projection, not as a way to hide model invalidity.

For small reference tests, covariance form is useful because it exposes the same moments that appear in the mathematical recursion. For production likelihoods, BayesFilter should prefer solve and factor forms that avoid explicit inverses and provide residual diagnostics.

6.3 Solve Form

Let $S_t = L_t L_t^\top$ and define $z_t = L_t^{-1} v_t$. Then

$$\log \det S_t = 2 \sum_k \log L_{t,kk}, \quad v_t^\top S_t^{-1} v_t = z_t^\top z_t.$$

The likelihood can therefore be evaluated with triangular solves and log diagonal pivots rather than by explicitly forming S_t^{-1} . This is the preferred reference form for implementation because the same solves appear in the analytic score and Hessian recursions.

The solve-form contract is:

- factor the selected innovation covariance;
- solve for the whitened innovation;
- accumulate log determinant and quadratic form from the factor and solve result;
- record factor pivots and solve residuals.

The backend should not form an explicit inverse except in small diagnostic tests where the numerical error is itself being measured.

6.4 Square-Root and Spectral Backends

Cholesky square-root filters are efficient when positive pivots are reliable. QR square-root filters are useful when the implementation needs stable stack updates, reconstruction checks, and a clear path to derivative-factor validation. SVD or eigenspectral factorizations are useful for rank-deficient or nearly rank-deficient linear Gaussian problems, but they should be reported as spectral backend choices with their own diagnostics.

This distinction matters because the word SVD appears in two different parts of the monograph. An SVD-factored exact linear Gaussian backend is still evaluating (5.10). A nonlinear SVD sigma-point filter is an approximate nonlinear Gaussian filter whose value and gradient policy require separate tests. Success of the former does not certify the latter.

6.5 Backend Agreement Tests

Every exact linear Gaussian backend should be tested against at least one reference backend on controlled models. A minimal agreement test records:

- log-likelihood difference;
- maximum filtered-mean difference;
- maximum filtered-covariance or reconstructed-factor difference;
- minimum pivot or singular value;
- solve residual norm;
- whether the process covariance is full rank or structurally singular.

For singular Q examples, the test should assert that the likelihood is finite and that the structural degeneracy is reported as model structure or backend telemetry, not as an unexplained numerical exception.

6.6 Diagnostics

Stable filtering backends should return diagnostics, not only a scalar likelihood. The MacroFinance large-scale backend ladder tests track:

- minimum factor pivots;
- reconstruction errors for prediction, innovation, and filtered factors;
- solve residual norms;
- finite log likelihood, gradient, and Hessian values;
- backend deltas against the solve-form reference.

Those tests classify SVD fallback triggers as no trigger, candidate trigger, or confirmed trigger. BayesFilter should preserve that caution: a spectral fallback is a diagnostic event, not evidence that the HMC gradient path is automatically safe.

The diagnostic record should travel with the likelihood into HMC reports. A finite log likelihood with repeated tiny pivots or frequent fallback triggers is not the same engineering result as a finite log likelihood with comfortable factor margins. Both may be mathematically valid evaluations, but they imply different sampler risk.

Chapter 7

Missing Data and Mixed Frequency

Real macro-finance panels are rarely dense. Missing observations, mixed-frequency releases, publication lags, and ragged starts must be handled as part of the filtering contract rather than as ad hoc data cleaning outside the likelihood.

7.1 Selection Form

Let M_t be a deterministic selection matrix that extracts the observed components of y_t . The dense observation equation

$$y_t = a + Hx_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, R)$$

becomes

$$M_t y_t = M_t a + M_t H x_t + M_t \varepsilon_t, \quad M_t \varepsilon_t \sim \mathcal{N}(0, M_t R M_t^\top).$$

The prediction-error likelihood in (5.10) then uses the selected innovation and selected innovation covariance. If no components are observed at time t , the filter performs prediction without a measurement update.

Equivalently, write $\mathcal{O}_t$ for the observed row set. The per-period objects are

$$v_{t,\mathcal{O}_t} = y_{t,\mathcal{O}_t} - a_{\mathcal{O}_t} - H_{\mathcal{O}_t} m_{t|t-1}, \quad S_{t,\mathcal{O}_t} = H_{\mathcal{O}_t} P_{t|t-1} H_{\mathcal{O}_t}^\top + R_{\mathcal{O}_t,\mathcal{O}_t}.$$

The likelihood contribution uses the dimension $|\mathcal{O}_t|$. If $\mathcal{O}_t$ is empty, the contribution is zero and the filtered moments equal the predicted moments. This convention is important for ragged panels because it makes missingness explicit in the likelihood rather than hidden in preprocessed data.

7.2 Mixed-Frequency State Augmentation

Mixed-frequency models often require deterministic auxiliary states. Examples include cumulators for quarterly growth, publication-lag states, and running averages used to align daily, monthly, and quarterly releases. These coordinates are part of the structural state contract from Definition 2.2. Their process covariance may be singular because the auxiliary rows are deterministic functions of lagged states and current high-frequency states.

BayesFilter should therefore distinguish two operations:

- (i) augmenting the state so the observation equation is linear or conditionally linear at the target sampling frequency;
- (ii) selecting the observed rows at each time point.

Both operations preserve the prediction-error likelihood when the augmented transition and observation maps are the declared model. A small ridge added only for numerical conditioning must be recorded as numerical regularization, not as a new source of structural shock variation.

7.3 Implementation Policy

The public backend must state whether it supports sparse panels. A dense-only backend should fail before likelihood evaluation if the observations contain NaNs, infinities, or masks that imply unsupported missing-data behavior. The MacroFinance missing-data policy tests explicitly check that unsupported sparse observation masks raise a named pending-support error rather than leaking NaNs into the Kalman recursion.

For compiled likelihoods, there are two viable policies:

- group time steps by mask pattern and keep each compiled path static;
- keep a fixed dense shape and use selection or masking operations whose shape is known at trace time.

Either policy is acceptable if the likelihood value and derivative contract are unchanged and the diagnostics identify which observations contributed to each time step.

The diagnostic record should include at least:

- number of observed rows at each time or by mask pattern;
- count of all-missing prediction-only steps;
- mask-pattern identifiers used for compiled grouping;
- whether missingness was handled by row selection, dense masking, or an unsupported fallback;
- any regularization applied to augmented deterministic rows.

7.4 Derivative Deferral

The value contract above is independent of the derivative implementation. Later derivative chapters must state whether mask patterns are differentiated as fixed data, whether all-missing steps contribute zero derivative to the measurement likelihood, and whether mixed-frequency augmentation parameters enter the transition, the observation map, or both. This chapter fixes the likelihood object; it does not yet certify any derivative backend.

Chapter 8

Large-Scale Linear Gaussian State-Space Models

Large linear Gaussian models are the bridge between textbook Kalman filtering and industrial Bayesian state-space inference. They are still exact, but their dimension exposes the numerical and compilation issues that small examples can hide.

8.1 Scale Targets

A BayesFilter scale target should record:

- state dimension n_x ;
- observation dimension n_y ;
- time length T ;
- parameter dimension d_θ ;
- dense or sparse observation policy;
- backend ladder and fallback rules;
- memory layout and compilation policy.

The MacroFinance large-scale LGSSM tests include scenarios such as `baseline_10x3x5`, low-measurement-noise collinearity, ill-conditioned process noise, and near-rank-loss candidates. BayesFilter uses these as infrastructure evidence, not as a claim that every NAWM-sized model is already solved.

8.2 Backend Registry

Large-scale runs should be described by a backend registry rather than by a single method name. A registry entry should identify:

- the value backend, such as covariance form, solve form, Cholesky square root, QR square root, or spectral fallback;

- the derivative backend, such as tape autodiff, analytic score, analytic Hessian, custom gradient, or finite-difference diagnostic;
- the initialization policy;
- the missing-data policy;
- static-shape and compilation assumptions;
- finite-failure diagnostics and fallback labels.

This registry is part of the scientific result. A reported HMC run without backend metadata is not reproducible enough to diagnose whether a divergence, small effective sample size, or slow compilation came from the posterior geometry, the filtering algebra, the derivative path, or the execution mode.

8.3 Validation Ladder

The validation ladder should move from exact values to sampler behavior:

- (i) value parity against the prediction-error decomposition in Chapter 5;
- (ii) filtered-moment and backend-delta parity across covariance, solve, and square-root implementations on controlled small cases;
- (iii) stress cases with low measurement noise, nearly collinear observations, singular or ill-conditioned process covariance, and long panels;
- (iv) derivative parity on small cases before the derivative backend is used by HMC;
- (v) compiled-path checks for static shape, finite diagnostics, and stable recompilation behavior;
- (vi) short HMC smoke runs that exercise the target without being used as convergence evidence;
- (vii) medium and strict HMC recovery only after the value and derivative gates have passed.

The same ladder should be reused for MacroFinance adapters, DSGE adapters, and future BayesFilter-native examples. Passing it for one model family is useful evidence about the backend, but it does not automatically transfer to another state partition, observation mask policy, or parameterization.

8.4 Stress Scenarios

The stress suite should include both numerical and modeling edge cases:

- structurally singular Q from AR(p) companion states, mixed-frequency accumulators, or deterministic structural coordinates;

- nearly singular innovation covariance from collinear measurements or very small measurement noise;
- long panels where repeated updates test covariance symmetry and factor reconstruction;
- ragged panels and all-missing time steps;
- parameter draws near admissibility boundaries;
- backend fallback triggers and spectral-gap telemetry.

Each scenario should define the expected diagnostic outcome before it is used as an HMC target. Some cases should pass with comfortable numerical margins; others should return named finite failures. Both outcomes are useful if they are intentional and reproducible.

8.5 Readiness Criteria

A large-scale LGSSM backend is ready for HMC experiments only after:

- (i) the value backend matches a solve-form or covariance-form oracle;
- (ii) gradient and Hessian backends match finite differences or autodiff on small controlled cases;
- (iii) square-root or QR backends match the solve-form backend within stated tolerances on dimension-scaled cases;
- (iv) diagnostics remain finite under stress scenarios;
- (v) shape and compilation behavior are documented.

The current evidence supports this ladder as a validation design. It does not remove the need to rerun the ladder for each new model family, parameterization, or missing-data policy.

8.6 Derivative Consolidation Hypotheses

The next analytic-derivative pass should test the following hypotheses against the MacroFinance derivation and implementation:

- (i) the Chapter 5 value contract is identical to the likelihood differentiated in the MacroFinance analytic Kalman note;
- (ii) analytic score recursions match autodiff or finite differences on small dense and singular- Q cases;
- (iii) Hessian recursions match finite-difference-on-gradient diagnostics on small cases before they are used for mass-matrix or curvature proposals;
- (iv) solve-form derivatives avoid explicit innovation-covariance inverses in production code;

- (v) derivative diagnostics identify the first time step and matrix block responsible for nonfinite values or large parity errors.

These are hypotheses, not completed certification. They should be promoted to claims only after source derivations, code, and numerical tests agree.

Part III

Analytic Derivatives

Chapter 9

Kalman Score Recursions

The score recursion differentiates the prediction-error decomposition from Chapter 5. In BayesFilter this is the first derivative contract for exact linear Gaussian HMC targets.

For parameter component θ_i , write

$$\dot{z}^{(i)} = \frac{\partial z}{\partial \theta_i}.$$

All objects in the filtering recursion may depend on θ :

$$c_t, \quad F_t, \quad Q_t, \quad a_t, \quad H_t, \quad R_t.$$

Time subscripts are kept in this chapter because missing-data masks and mixed-frequency observation systems can make the effective observation equation time varying even when the underlying structural model is time homogeneous.

9.1 Prediction Derivatives

The differentiated prediction step is

$$\dot{m}_{t|t-1}^{(i)} = \dot{c}_t^{(i)} + \dot{F}_t^{(i)} m_{t-1|t-1} + F_t \dot{m}_{t-1|t-1}^{(i)}, \quad (9.1)$$

$$\dot{P}_{t|t-1}^{(i)} = \dot{F}_t^{(i)} P_{t-1|t-1} F_t^\top + F_t \dot{P}_{t-1|t-1}^{(i)} F_t^\top + F_t P_{t-1|t-1} \dot{F}_t^{(i)\top} + \dot{Q}_t^{(i)}. \quad (9.2)$$

These are source labels `eq:dm_pred_note` and `eq:dP_pred_note` in the MacroFinance analytic derivative note. They also define the initial-condition contract: if the implementation fixes $(m_{0|0}, P_{0|0})$ as data, the initial derivatives are zero; if it uses a stationary initial covariance, the derivatives must come from the corresponding derivative Lyapunov equations and be reported as part of the structural provider.

9.2 Innovation Derivatives

The source note derives the innovation derivatives

$$\dot{v}_t^{(i)} = -\dot{a}_t^{(i)} - \dot{H}_t^{(i)} m_{t|t-1} - H_t \dot{m}_{t|t-1}^{(i)}, \quad (9.3)$$

$$\dot{S}_t^{(i)} = \dot{H}_t^{(i)} P_{t|t-1} H_t^\top + H_t \dot{P}_{t|t-1}^{(i)} H_t^\top + H_t P_{t|t-1} \dot{H}_t^{(i)\top} + \dot{R}_t^{(i)}. \quad (9.4)$$

These are source labels `eq:dv_note` and `eq:dS_note`. Using $\partial_i S^{-1} = -S^{-1} \dot{S}^{(i)} S^{-1}$, the per-time score contribution is

$$\frac{\partial \ell_t}{\partial \theta_i} = -\frac{1}{2} \left[\text{tr}(S_t^{-1} \dot{S}_t^{(i)}) + 2\dot{v}_t^{(i)\top} S_t^{-1} v_t - v_t^\top S_t^{-1} \dot{S}_t^{(i)} S_t^{-1} v_t \right]. \quad (9.5)$$

This is consolidated from source label `eq:score_note` in the MacroFinance analytic Kalman derivative note.

9.3 Solve-Form Score

For production code, BayesFilter prefers the solve form. Let $S_t w_t = v_t$. Then the same score can be written as

$$\frac{\partial \ell_t}{\partial \theta_i} = -\frac{1}{2} \left[\text{tr}(S_t^{-1} \dot{S}_t^{(i)}) + 2\dot{v}_t^{(i)\top} w_t - w_t^\top \dot{S}_t^{(i)} w_t \right]. \quad (9.6)$$

This is source label `eq:solve_score_proved`. The implementation reference is the MacroFinance solve differentiated Kalman backend. The trace term may be computed by explicit materialization in small tests, but production code should prefer factor solves or trace estimators appropriate to the dimension and backend.

9.4 Gain and Update Derivatives

The score contribution at time t can be computed as soon as (9.3)–(9.4) are available. The recursion must still propagate derivatives to the next time step. For the covariance-form Kalman update,

$$K_t = P_{t|t-1} H_t^\top S_t^{-1},$$

the gain derivative is

$$\dot{K}_t^{(i)} = \dot{P}_{t|t-1} H_t^\top S_t^{-1} + P_{t|t-1} \dot{H}_t^{(i)\top} S_t^{-1} - P_{t|t-1} H_t^\top S_t^{-1} \dot{S}_t^{(i)} S_t^{-1}. \quad (9.7)$$

Equivalently, if the implementation treats the gain as the solved matrix $S_t K_t^\top = H_t P_{t|t-1}$, then

$$\dot{K}_t^{(i)\top} = \mathcal{S}_{S_t} \left(\dot{H}_t^{(i)} P_{t|t-1} + H_t \dot{P}_{t|t-1}^{(i)} - \dot{S}_t^{(i)} K_t^\top \right). \quad (9.8)$$

The filtered mean and covariance derivatives are

$$\dot{m}_{t|t}^{(i)} = \dot{m}_{t|t-1}^{(i)} + \dot{K}_t^{(i)} v_t + K_t \dot{v}_t^{(i)}, \quad (9.9)$$

$$\dot{P}_{t|t}^{(i)} = -\dot{K}_t^{(i)} H_t P_{t|t-1} - K_t \dot{H}_t^{(i)} P_{t|t-1} + (I - K_t H_t) \dot{P}_{t|t-1}^{(i)}. \quad (9.10)$$

Equation (9.10) differentiates the simplified covariance update $P_{t|t} = (I - K_t H_t) P_{t|t-1}$. A Joseph-form or square-root backend may use a different algebraic update for numerical reasons, but it must produce derivatives of the same filtered covariance it uses in the next prediction step.

9.5 Masks and Time-Varying Observation Blocks

For missing data, let M_t select the observed rows at time t . The effective observation objects are

$$y_t^o = M_t y_t, \quad a_t^o = M_t a_t, \quad H_t^o = M_t H_t, \quad R_t^o = M_t R_t M_t^\top.$$

The score recursion above applies after replacing (y_t, a_t, H_t, R_t) by $(y_t^o, a_t^o, H_t^o, R_t^o)$. The mask is data, not a differentiated parameter, so $\dot{M}_t = 0$. If no rows are observed, the likelihood and score contribution for that time are zero and the update is the identity:

$$m_{t|t} = m_{t|t-1}, \quad P_{t|t} = P_{t|t-1},$$

with the same equalities for first derivatives. This convention ensures that the derivative path corresponds to the same masked likelihood value used by the filter.

9.6 Evidence Status

The current MathDevMCP AST audit identifies Cholesky/solve, prediction, update, quadratic-form, and derivative-recursion structure in that file, but it does not by itself certify the algebra. The stronger evidence is the MacroFinance test suite comparing solve, QR square-root, autodiff, and finite difference results on controlled LGSSMs.

Chapter 10

Kalman Hessian and Observed Information

The Hessian layer is necessary for observed-information diagnostics, local identification, and mass-matrix construction, but it is also the easiest place to overstate the state of the mathematics. BayesFilter therefore records the Hessian as a source-backed contract and validation target, not as a finished proof for every backend.

For second derivatives, write

$$\ddot{z}^{(ij)} = \frac{\partial^2 z}{\partial \theta_i \partial \theta_j}.$$

The MacroFinance note expands the second-order prediction, innovation, and gain recursions by product rule. In solve form, the source introduces

$$\dot{w}_t^{(j)} = \mathcal{S}_{S_t} \left(\dot{v}_t^{(j)} - \dot{S}_t^{(j)} w_t \right), \quad (10.1)$$

where $\mathcal{S}_{S_t}(b)$ denotes solving $S_t x = b$. It then decomposes the per-time Hessian contribution into log-determinant, innovation-linear, and quadratic terms:

$$\frac{\partial^2 \ell_t}{\partial \theta_i \partial \theta_j} = -\frac{1}{2} \left[T_{ij}^{\log \det} + T_{ij}^v - T_{ij}^S \right]. \quad (10.2)$$

This statement is consolidated from source label `eq:solve.hessian.proved`.

10.1 Second-Order Prediction

The second-order prediction step is the product-rule derivative of (9.1)–(9.2). For the predicted mean,

$$\ddot{m}_{t|t-1}^{(ij)} = \ddot{c}_t^{(ij)} + \ddot{F}_t^{(ij)} m_{t-1|t-1} + \dot{F}_t^{(i)} \dot{m}_{t-1|t-1}^{(j)} + \dot{F}_t^{(j)} \dot{m}_{t-1|t-1}^{(i)} + F_t \ddot{m}_{t-1|t-1}^{(ij)}. \quad (10.3)$$

For the predicted covariance,

$$\begin{aligned} \ddot{P}_{t|t-1}^{(ij)} = & \ddot{F}_t^{(ij)} P_{t-1|t-1} F_t^\top + \dot{F}_t^{(i)} \dot{P}_{t-1|t-1}^{(j)} F_t^\top + \dot{F}_t^{(i)} P_{t-1|t-1} \dot{F}_t^{(j)\top} \\ & + \dot{F}_t^{(j)} \dot{P}_{t-1|t-1}^{(i)} F_t^\top + F_t \ddot{P}_{t-1|t-1}^{(ij)} F_t^\top + F_t \dot{P}_{t-1|t-1}^{(i)} \dot{F}_t^{(j)\top} \\ & + \dot{F}_t^{(j)} P_{t-1|t-1} \dot{F}_t^{(i)\top} + F_t \dot{P}_{t-1|t-1}^{(j)} \dot{F}_t^{(i)\top} + F_t P_{t-1|t-1} \ddot{F}_t^{(ij)\top} + \ddot{Q}_t^{(ij)}. \end{aligned} \quad (10.4)$$

These equations are the second-order source recursions in the MacroFinance analytic derivative note.

10.2 Second-Order Innovation Objects

The innovation derivatives are

$$\ddot{v}_t^{(ij)} = -\ddot{a}_t^{(ij)} - \ddot{H}_t^{(ij)} m_{t|t-1} - \dot{H}_t^{(i)} \dot{m}_{t|t-1}^{(j)} - \dot{H}_t^{(j)} \dot{m}_{t|t-1}^{(i)} - H_t \ddot{m}_{t|t-1}^{(ij)}, \quad (10.5)$$

$$\begin{aligned} \ddot{S}_t^{(ij)} = & \ddot{H}_t^{(ij)} P_{t|t-1} H_t^\top + \dot{H}_t^{(i)} \dot{P}_{t|t-1}^{(j)} H_t^\top + \dot{H}_t^{(i)} P_{t|t-1} \dot{H}_t^{(j)\top} \\ & + \dot{H}_t^{(j)} \dot{P}_{t|t-1}^{(i)} H_t^\top + \dot{H}_t^{(j)} P_{t|t-1} \dot{H}_t^{(i)\top} + H_t \ddot{P}_{t|t-1}^{(ij)} H_t^\top \\ & + H_t \dot{P}_{t|t-1}^{(i)} \dot{H}_t^{(j)\top} + H_t \dot{P}_{t|t-1}^{(j)} \dot{H}_t^{(i)\top} + H_t P_{t|t-1} \ddot{H}_t^{(ij)\top} + \ddot{R}_t^{(ij)}. \end{aligned} \quad (10.6)$$

Together with the first-order innovation objects from Chapter 9, these are the only model-specific inputs needed by the solve-form likelihood Hessian.

10.3 Solve-Form Hessian

The compact statement above is useful only if the three terms are kept explicit. Let $S_t w_t = v_t$ and define $\dot{w}_t^{(j)}$ by (10.1). The second derivative of the solved vector is

$$\ddot{w}_t^{(ij)} = \mathcal{S}_{S_t} \left(\ddot{v}_t^{(ij)} - \ddot{S}_t^{(ij)} w_t - \dot{S}_t^{(i)} \dot{w}_t^{(j)} - \dot{S}_t^{(j)} \dot{w}_t^{(i)} \right). \quad (10.7)$$

The log-determinant term is

$$T_{ij}^{\log \det} = \text{tr} \left(S_t^{-1} \ddot{S}_t^{(ij)} - S_t^{-1} \dot{S}_t^{(j)} S_t^{-1} \dot{S}_t^{(i)} \right). \quad (10.8)$$

The innovation-linear term is

$$T_{ij}^v = 2\ddot{v}_t^{(ij)\top} w_t + 2\dot{v}_t^{(i)\top} \dot{w}_t^{(j)}. \quad (10.9)$$

The quadratic term is

$$T_{ij}^S = 2\dot{w}_t^{(j)\top} \dot{S}_t^{(i)} w_t + w_t^\top \ddot{S}_t^{(ij)} w_t. \quad (10.10)$$

Substituting (10.8)–(10.10) into (10.2) gives the observed Hessian contribution. The formula requires only the innovation objects

$$v_t, \quad S_t, \quad \dot{v}_t^{(i)}, \quad \dot{S}_t^{(i)}, \quad \ddot{v}_t^{(ij)}, \quad \ddot{S}_t^{(ij)}.$$

It is therefore backend-neutral. A linear Kalman filter, a square-root Kalman filter, or a sigma-point filter may all use the same likelihood Hessian layer once their own recursions supply these six objects.

10.4 Second-Order Gain and Update

The covariance-form backend also propagates the Hessian of the filtered state objects. Define

$$\dot{S}_t^{-1(i)} = -S_t^{-1} \dot{S}_t^{(i)} S_t^{-1}, \quad (10.11)$$

$$\ddot{S}_t^{-1(ij)} = S_t^{-1} \dot{S}_t^{(j)} S_t^{-1} \dot{S}_t^{(i)} S_t^{-1} + S_t^{-1} \dot{S}_t^{(i)} S_t^{-1} \dot{S}_t^{(j)} S_t^{-1} - S_t^{-1} \ddot{S}_t^{(ij)} S_t^{-1}. \quad (10.12)$$

Then

$$\dot{K}_t^{(i)} = \dot{P}_{t|t-1}^{(i)} H_t^\top S_t^{-1} + P_{t|t-1} \dot{H}_t^{(i)\top} S_t^{-1} + P_{t|t-1} H_t^\top \dot{S}_t^{-1(i)}, \quad (10.13)$$

$$\begin{aligned} \ddot{K}_t^{(ij)} = & \ddot{P}_{t|t-1}^{(ij)} H_t^\top S_t^{-1} + \dot{P}_{t|t-1}^{(i)} \dot{H}_t^{(j)\top} S_t^{-1} + \dot{P}_{t|t-1}^{(j)} H_t^\top \dot{S}_t^{-1(j)} \\ & + \dot{P}_{t|t-1}^{(j)} \dot{H}_t^{(i)\top} S_t^{-1} + P_{t|t-1} \ddot{H}_t^{(ij)\top} S_t^{-1} + P_{t|t-1} \dot{H}_t^{(i)\top} \dot{S}_t^{-1(j)} \\ & + \dot{P}_{t|t-1}^{(j)} H_t^\top \dot{S}_t^{-1(i)} + P_{t|t-1} \dot{H}_t^{(j)\top} \dot{S}_t^{-1(i)} + P_{t|t-1} H_t^\top \ddot{S}_t^{-1(ij)}. \end{aligned} \quad (10.14)$$

The filtered mean Hessian is

$$\ddot{m}_{t|t}^{(ij)} = \ddot{m}_{t|t-1}^{(ij)} + \ddot{K}_t^{(ij)} v_t + \dot{K}_t^{(i)} \dot{v}_t^{(j)} + \dot{K}_t^{(j)} \dot{v}_t^{(i)} + K_t \ddot{v}_t^{(ij)}. \quad (10.15)$$

For the simplified covariance update,

$$\begin{aligned} \ddot{P}_{t|t}^{(ij)} = & -\ddot{K}_t^{(ij)} H_t P_{t|t-1} - \dot{K}_t^{(i)} \dot{H}_t^{(j)} P_{t|t-1} - \dot{K}_t^{(i)} H_t \dot{P}_{t|t-1}^{(j)} \\ & - \dot{K}_t^{(j)} \dot{H}_t^{(i)} P_{t|t-1} - K_t \ddot{H}_t^{(ij)} P_{t|t-1} - K_t \dot{H}_t^{(i)} \dot{P}_{t|t-1}^{(j)} \\ & - \dot{K}_t^{(j)} H_t \dot{P}_{t|t-1}^{(i)} - K_t \dot{H}_t^{(j)} \dot{P}_{t|t-1}^{(i)} + (I - K_t H_t) \ddot{P}_{t|t-1}^{(ij)}. \end{aligned} \quad (10.16)$$

The gain and covariance formulas are useful as a covariance-form audit oracle. Solve-form or square-root implementations may avoid explicit (10.11)–(10.12) internally, but they must agree with these recursions on controlled well-conditioned systems.

10.5 Information Objects and Sign Convention

This chapter computes Hessians of the log likelihood contribution $\ell_t(\theta)$. The observed information contribution is

$$\mathcal{J}_t(\theta) = -\nabla_\theta^2 \ell_t(\theta),$$

and posterior precision at a mode additionally includes the negative Hessian of the log prior. BayesFilter should not call a matrix a mass matrix, local covariance, or identification object unless the sign convention and prior contribution have been stated explicitly.

10.6 Hessian-Ready Backend Contract

A backend is Hessian-ready when it can return, for every time step and active parameter pair,

$$v_t, \quad S_t, \quad \dot{v}_t^{(i)}, \quad \dot{S}_t^{(i)}, \quad \ddot{v}_t^{(ij)}, \quad \ddot{S}_t^{(ij)}$$

for the same likelihood value it evaluates. To propagate the scan, it must also return either the covariance-form update derivatives (10.15)–(10.16) or an equivalent factor/backend-specific derivative update with reconstruction diagnostics. This is the bridge used later by the SVD sigma-point chapter: the nonlinear backend supplies approximate innovation derivatives, and the solve-form Hessian layer remains the same.

10.7 Evidence Status

The source derivation is detailed, but BayesFilter should keep three evidence levels distinct:

- formula provenance from the MacroFinance analytic derivative note;
- structural code evidence from MathDevMCP AST audits;
- numerical parity evidence from tests against autodiff and finite differences.

The current MathDevMCP audit of the MacroFinance solve differentiated Kalman backend found the Kalman operation graph and solve structure but reported a mismatch for a strict `logdet/trace` operation query and missing explicit shape and covariance guards. This should be treated as a production-readiness prompt, not as a proof failure. The generic LGSSM autodiff-validation tests are the stronger mathematical check for the current implementation because they compare likelihood, gradient, and Hessian against solve backends, TensorFlow autodiff, and finite differences on controlled small-dimensional models.

Chapter 11

Structural Derivatives

Kalman derivative recursions assume that the derivatives of the state-space objects are already available. Structural derivatives are the upstream layer: they map model parameters into

$$c_\theta, F_\theta, Q_\theta, a_\theta, H_\theta, R_\theta$$

and their first and second derivatives.

11.1 Provider Map

Let $\psi \in \mathbb{R}^p$ denote the parameter vector used by the numerical provider. In HMC this is usually an unconstrained or transformed parameter vector, not necessarily the raw economic vector. The structural provider declares the map

$$g(\psi) = \text{vec}(c(\psi), F(\psi), Q(\psi), a(\psi), H(\psi), R(\psi), m_0(\psi), P_0(\psi)). \quad (11.1)$$

The MacroFinance source map uses the same contract for (b, F, Q, a, H, R) under source label `eq:structural_map_g`; BayesFilter adds (m_0, P_0) because the filtering scan cannot be differentiated without an initial-condition policy.

For every active coordinate ψ_i , the first-order provider contract is

$$\dot{G}^{(i)} = \left(\dot{c}^{(i)}, \dot{F}^{(i)}, \dot{Q}^{(i)}, \dot{a}^{(i)}, \dot{H}^{(i)}, \dot{R}^{(i)}, \dot{m}_0^{(i)}, \dot{P}_0^{(i)} \right). \quad (11.2)$$

For every active pair (ψ_i, ψ_j) , the second-order contract is

$$\ddot{G}^{(ij)} = \left(\ddot{c}^{(ij)}, \ddot{F}^{(ij)}, \ddot{Q}^{(ij)}, \ddot{a}^{(ij)}, \ddot{H}^{(ij)}, \ddot{R}^{(ij)}, \ddot{m}_0^{(ij)}, \ddot{P}_0^{(ij)} \right). \quad (11.3)$$

Equations (11.2)–(11.3) are the objects consumed by Chapters 9 and 10. A backend may store them as dense tensors, sparse blocks, named arrays, or lazy linear operators, but the contract is mathematical: each block must mean a derivative of the same state-space law used to evaluate the likelihood.

For BayesFilter, a derivative provider must declare:

- parameter order and units;
- shape of every first and second derivative tensor;
- whether derivatives are analytic, autodiff, finite-difference, or unavailable;
- which parameters affect which reduced-form objects;
- whether cross-derivatives are implemented or intentionally zero.

11.2 Parameter Transforms

Let $\phi = T(\psi)$ be the raw economic parameter vector and let $M(\phi)$ be any reduced-form block among $c, F, Q, a, H, R, m_0, P_0$. Write

$$M_\alpha = \frac{\partial M}{\partial \phi_\alpha}, \quad M_{\alpha\beta} = \frac{\partial^2 M}{\partial \phi_\alpha \partial \phi_\beta},$$

and

$$T_{\alpha,i} = \frac{\partial \phi_\alpha}{\partial \psi_i}, \quad T_{\alpha,ij} = \frac{\partial^2 \phi_\alpha}{\partial \psi_i \partial \psi_j}.$$

Then

$$\dot{M}^{(i)} = \sum_{\alpha} M_{\alpha} T_{\alpha,i}, \quad (11.4)$$

$$\ddot{M}^{(ij)} = \sum_{\alpha, \beta} M_{\alpha\beta} T_{\alpha,i} T_{\beta,j} + \sum_{\alpha} M_{\alpha} T_{\alpha,ij}. \quad (11.5)$$

These are the BayesFilter versions of the MacroFinance transformed-parameter chain-rule equations. They matter because HMC often runs on $\psi = \log \sigma$, $\psi = \log \kappa$, Cholesky coordinates, Fisher transforms, or stationarity transforms. A correct derivative with respect to a raw standard deviation is wrong for HMC if it is passed off as the derivative with respect to the log standard deviation.

11.3 Initial Conditions

The initial moments in (11.1) are part of the model law. There are three common cases:

- (i) fixed diffuse or fixed proper initialization: $\dot{m}_0 = \dot{P}_0 = \ddot{m}_0 = \ddot{P}_0 = 0$ because the initial moments are data of the filtering problem;
- (ii) stationary initialization: m_0 and P_0 solve model equations and must be differentiated together with those equations;
- (iii) estimated or hierarchical initialization: m_0 and P_0 have their own parameter blocks and cross derivatives.

For a stationary linear system with $m_0 = c + Fm_0$, the first derivative satisfies

$$(I - F)\dot{m}_0^{(i)} = \dot{c}^{(i)} + \dot{F}^{(i)}m_0. \quad (11.6)$$

For $P_0 = FP_0F^\top + Q$, the first derivative satisfies the Lyapunov derivative equation

$$\dot{P}_0^{(i)} = F\dot{P}_0^{(i)}F^\top + \dot{F}^{(i)}P_0F^\top + FP_0\dot{F}^{(i)\top} + \dot{Q}^{(i)}. \quad (11.7)$$

Second derivatives follow by differentiating (11.6) and (11.7). BayesFilter should record whether these equations were solved analytically, by a differentiable Lyapunov solver, by autodiff through the stationary solver, or intentionally not used.

11.4 Sparsity, Masks, and Zero Blocks

A derivative tensor entry equal to zero is different from a derivative entry that is unavailable. The provider must distinguish:

$$\ddot{M}^{(ij)} = 0 \quad \text{declared structural zero,} \quad \ddot{M}^{(ij)} \text{ not implemented.}$$

Structural zeros are useful for large models because they allow the score and Hessian scan to skip blocks without changing the mathematical target. Missing entries are not a mathematical shortcut; they mean the backend cannot certify the requested derivative object.

Observation masks are data-side selections. If J_t selects observed coordinates at time t , the provider may expose either the full (a_t, H_t, R_t) together with J_t or the already-masked blocks

$$a_t^o = J_t a_t, \quad H_t^o = J_t H_t, \quad R_t^o = J_t R_t J_t^\top.$$

The same selection applies to derivatives. Since J_t is data, not a parameter, no derivative of the mask is taken. This convention is the structural upstream counterpart of the masking rule in Section 9.5.

11.5 Metadata and Shape Contract

The provider must make the following metadata inspectable before likelihood evaluation:

- names and order of ψ_i ;
- transform type and raw unit for each parameter;
- state and observation coordinate names;
- dimensions of each reduced-form block;
- tensor layout conventions, including whether Hessian tensors store all pairs or only the symmetric upper triangle;
- symmetry and positive-definiteness policies for Q , R , and P_0 ;
- deterministic or algebraic state-coordinate roles when the provider is used by a structural nonlinear filter.

These declarations are not decorative. Without them, a finite likelihood can be computed from a different parameter ordering, unit system, or deterministic state convention than the derivative tensors assume.

The MacroFinance large-scale derivative-provider tests check derivative shapes, finite-difference parity by object type, reduced-form rank diagnostics, and parameter-unit rescaling. BayesFilter should preserve these as public contract tests before a structural model is allowed to feed an HMC target.

11.6 Provider Validation

The minimum validation ladder for a structural provider is:

- (i) finite value check: all blocks in (11.1) are finite, conformable, symmetric where required, and admissible before filtering;
- (ii) shape check: (11.2)–(11.3) have the declared dimensions for every active parameter and pair;
- (iii) transform check: (11.4)–(11.5) agree with finite differences in transformed coordinates;
- (iv) block finite-difference check: each reduced-form object is checked before it is inserted into a long filtering scan;
- (v) end-to-end check: the resulting likelihood score and Hessian agree with finite differences or autodiff on small systems where that comparison is reliable;
- (vi) diagnostic check: structural zeros, unavailable derivatives, regularization, rejected parameter points, and mask policies are reported.

Only after these checks pass should the structural provider feed an HMC target or an observed-information diagnostic. This is the upstream analogue of the factor parity gates in Chapter 12.

Chapter 12

QR and Cholesky Factor Derivatives

Square-root filters propagate factors rather than covariance matrices. Their derivative contract is therefore not only a Kalman contract; it is also a factor calculus contract. The likelihood score and Hessian in Chapters 9–10 are expressed in terms of v_t , S_t , and their derivatives. A factor backend is acceptable only if the factors it propagates reproduce those covariance objects and the same derivative objects.

BayesFilter uses three levels of factor evidence:

- (i) Cholesky or QR identities stated in the source derivation;
- (ii) reconstruction diagnostics proving that differentiated factors reconstruct the differentiated covariance objects;
- (iii) backend parity against solve-form likelihood, score, and Hessian.

The QR square-root tests in MacroFinance require prediction, innovation, and filtered factor derivative reconstruction errors below tight tolerances on small-dimensional parameter grids, and they compare QR likelihood, gradient, and Hessian against the solve-form backend. Those tests are currently the right standard for BayesFilter factor-derivative chapters: a factor formula is not production-ready until both reconstruction and end-to-end likelihood parity pass.

12.1 Reconstruction Contract

Let $A(\theta)$ be any covariance-like symmetric positive semidefinite object used by the filter, and let an implemented factor satisfy

$$A_{\star}(\theta) = C(\theta)C(\theta)^{\top}.$$

The subscript $\star$ is deliberate. For a Cholesky backend, $A_{\star} = A$ on a positive-definite branch. For a spectral backend, $A_{\star}$ may be a rank-truncated, floored, or symmetrized version of A . The first derivative reconstructed by the factor is

$$\dot{A}_{\star}^{(i)} = \dot{C}^{(i)}C^{\top} + C\dot{C}^{(i)\top}. \quad (12.1)$$

The second derivative reconstructed by the factor is

$$\ddot{A}_{\star}^{(ij)} = \ddot{C}^{(ij)}C^{\top} + C\ddot{C}^{(ij)\top} + \dot{C}^{(i)}\dot{C}^{(j)\top} + \dot{C}^{(j)}\dot{C}^{(i)\top}. \quad (12.2)$$

Equations (12.1)–(12.2) are the universal audit equations for every factor backend. The derivative may be obtained by analytic formulas, custom gradients, or autodiff, but the resulting C , $\dot{C}$, and $\ddot{C}$ must reconstruct the implemented covariance branch that produced the likelihood value.

12.2 Cholesky Factor Derivatives

Suppose $A = LL^\top$, where A is positive definite and L is lower triangular with positive diagonal. Define the lower-triangular half-diagonal operator

$$\Phi(X)_{ab} = \begin{cases} X_{ab}, & a > b, \\ X_{aa}/2, & a = b, \\ 0, & a < b. \end{cases}$$

For a first derivative, form

$$B^{(i)} = L^{-1} \dot{A}^{(i)} L^{-\top}, \quad G^{(i)} = \Phi(B^{(i)}).$$

Then the Cholesky factor derivative is

$$\dot{L}^{(i)} = LG^{(i)}. \quad (12.3)$$

This is the BayesFilter restatement of source label `eq:first_cholesky_derivative`.

For a second derivative, remove the known first-order products before applying the same triangular split:

$$C^{(ij)} = L^{-1} \left(\ddot{A}^{(ij)} - \dot{L}^{(i)} \dot{L}^{(j)\top} - \dot{L}^{(j)} \dot{L}^{(i)\top} \right) L^{-\top}, \quad H^{(ij)} = \Phi(C^{(ij)}).$$

Then

$$\ddot{L}^{(ij)} = LH^{(ij)}. \quad (12.4)$$

This restates source label `eq:second_cholesky_derivative`. The formulas are local smooth-branch formulas: they require a positive diagonal and become numerically fragile near a Cholesky pivot boundary. A backend that adds jitter before Cholesky must report whether its derivatives target the original A or the implemented $A + \delta I$.

12.3 QR Factor Derivatives

QR square-root filters often factor a stack rather than a covariance matrix. Let $M(\theta) \in \mathbb{R}^{m \times n}$ have full column rank, $m \geq n$, and use the thin convention

$$M = QR, \quad Q^\top Q = I_n, \quad R \text{ upper triangular}, \quad R_{aa} > 0.$$

The positive diagonal fixes the local sign convention. Define

$$A^{(i)} = Q^\top \dot{M}^{(i)} R^{-1}.$$

Let $\Omega^{(i)}$ be the skew-symmetric matrix identified from

$$\Omega_{ab}^{(i)} = \begin{cases} A_{ab}^{(i)}, & a > b, \\ 0, & a = b, \\ -A_{ba}^{(i)}, & a < b, \end{cases} \quad T^{(i)} = A^{(i)} - \Omega^{(i)}.$$

Then

$$\dot{R}^{(i)} = T^{(i)} R, \quad (12.5)$$

$$\dot{Q}^{(i)} = Q\Omega^{(i)} + (I - QQ^\top)\dot{M}^{(i)}R^{-1}. \quad (12.6)$$

Equation (12.5) restates source label `eq:first_qr_r_derivative`.

For second derivatives, define the effective second-order stack derivative

$$E^{(ij)} = \ddot{M}^{(ij)} - \dot{Q}^{(i)}\dot{R}^{(j)} - \dot{Q}^{(j)}\dot{R}^{(i)}$$

and

$$B^{(ij)} = Q^\top E^{(ij)} R^{-1}, \quad C_Q^{(ij)} = -\dot{Q}^{(i)\top}\dot{Q}^{(j)} - \dot{Q}^{(j)\top}\dot{Q}^{(i)}.$$

The matrix $\Gamma^{(ij)} = Q^\top \ddot{Q}^{(ij)}$ is determined by

$$\Gamma_{ab}^{(ij)} = \begin{cases} B_{ab}^{(ij)}, & a > b, \\ C_{Q,aa}^{(ij)}/2, & a = b, \\ C_{Q,ab}^{(ij)} - B_{ba}^{(ij)}, & a < b, \end{cases} \quad U^{(ij)} = B^{(ij)} - \Gamma^{(ij)}.$$

Then

$$\ddot{R}^{(ij)} = U^{(ij)} R, \quad (12.7)$$

$$\ddot{Q}^{(ij)} = Q\Gamma^{(ij)} + (I - QQ^\top)E^{(ij)}R^{-1}. \quad (12.8)$$

Equation (12.7) restates source label `eq:second_qr_r_derivative`. The QR formulas are local to a fixed rank and sign branch. Pivoting, column-rank loss, or diagonal sign corrections are branch events and must be reported as derivative-policy events, not hidden inside a smooth Hessian claim.

12.4 Square-Root Kalman Stacks

The generic formulas above enter the filter through stack factorizations. If $C_{t-1|t-1}C_{t-1|t-1}^\top = P_{t-1|t-1}$ and $G_t G_t^\top = Q_t$, the prediction covariance can be declared by the stack

$$M_t^P = [F_t C_{t-1|t-1} \quad G_t], \quad P_{t|t-1} = M_t^P M_t^{P\top}.$$

Equivalently, a thin QR factorization of $M_t^{P\top} = Q_t^P R_t^P$ gives the lower factor $C_{t|t-1} = R_t^{P\top}$. The first and second derivatives of M_t^P are obtained by product rule from the derivatives of F_t , $C_{t-1|t-1}$, and G_t , and (12.5)–(12.7) then give the derivatives of $C_{t|t-1}$.

Likewise, if $E_t E_t^\top = R_t$,

$$M_t^S = [H_t C_{t|t-1} \quad E_t], \quad S_t = M_t^S M_t^{S\top},$$

and QR of $M_t^{S\top}$ gives the innovation factor used by the solve-form likelihood. The filtered covariance can be audited by the Joseph stack

$$M_t^U = [(I - K_t H_t) C_{t|t-1} \quad K_t E_t], \quad P_{t|t} = M_t^U M_t^{U\top}.$$

Derivatives of K_t are those in (9.7) and (10.14). Therefore a square-root implementation has two equivalent audit paths: reconstruct (12.1)–(12.2) from differentiated factors, and compare the resulting score/Hessian objects to the covariance-form recursions in Chapters 9–10.

12.5 Spectral Factors

SVD sigma-point filters use a spectral factor, not a triangular factor, to generate point displacements. If

$$P = U\Lambda U^\top, \quad C = U\Lambda_+^{1/2},$$

then the factor actually reconstructs

$$P_+ = CC^\top = U\Lambda_+U^\top,$$

where Λ_+ includes the rank, floor, or smoothing policy used by the implementation. Therefore spectral factor validation must distinguish two questions:

- (i) Do C and its derivatives reconstruct P_+ and its derivatives?
- (ii) If $P_+ \neq P$, is the difference a declared model approximation, a finite-arithmetic regularization, or an invalid fallback?

This distinction is essential for HMC. A hard eigenvalue floor can produce a finite likelihood while changing the derivative target at the floor boundary. If the floor is hard, derivatives are branch derivatives of P_+ , not derivatives of the pre-regularized covariance P . If the floor is smooth, the floor derivative is part of the model actually differentiated. Either way, the factor reconstruction equations in (12.1)–(12.2) must be checked against P_+ .

Chapter 18 therefore treats spectral derivatives as a separate contract with gap telemetry, active-floor reporting, and branch diagnostics. Its simple-spectrum SVD/eigen formulas define one smooth-branch policy; they do not remove the need for finite-difference checks across spectral gap, rank, and floor regimes.

12.6 Backend Parity Gates

A factor backend is ready to support analytic gradients and Hessians only after the following checks pass on controlled systems:

- (i) value reconstruction: $\|A_\star - CC^\top\|$ is within tolerance for prediction, innovation, and filtered covariance objects;
- (ii) first- and second-derivative reconstruction: (12.1)–(12.2) match the declared $\dot{A}_\star$ and $\ddot{A}_\star$;
- (iii) solve parity: the factor solve gives the same w_t , score, and Hessian contribution as the covariance solve on well-conditioned systems;
- (iv) branch reporting: jitter, rank truncation, pivoting, sign flips, active floors, and fallback paths are emitted as diagnostics;
- (v) backend parity: Cholesky-refactorized, QR square-root, and SVD/spectral backends agree where their mathematical branches represent the same covariance law.

These gates prevent a common mistake: a numerically stable factor can make the likelihood finite while silently changing the derivative target. The factor chapter therefore supplies the algebraic contracts that the validation chapter turns into executable tests.

Chapter 13

Custom-Gradient Wrappers for HMC

A custom gradient is safe for HMC only if it returns the derivative of the same scalar log target used in the Metropolis correction. A fast gradient of a nearby surrogate changes the dynamics and can bias the chain unless the sampler is explicitly corrected.

13.1 Same-Scalar Contract

Let the production value path return the full sampler-coordinate posterior target

$$\mathcal{L}(u) = \ell(T(u); y_{1:T}) + \log p(T(u)) + \log |\det DT(u)|, \quad (13.1)$$

where the likelihood term uses the same structural provider, masks, factor branch, regularization policy, and parameter transforms declared in Chapters 11–12. When a chapter later writes a likelihood-only derivative such as $\nabla_{\theta} \ell$, that object should be read as one component of the full sampler-coordinate target gradient, not as the entire HMC target by itself. The custom-gradient path is valid only if it returns

$$g_i(u) = \frac{\partial \mathcal{L}(u)}{\partial u_i}. \quad (13.2)$$

If the value path evaluates a floored spectral covariance P_+ , the gradient must be the derivative of that implemented value path. If the value path uses a jittered innovation covariance $S_t + \delta I$, the gradient must include the same jitter policy or clearly report that the parameter point is outside the certified smooth branch. The wrapper must not evaluate one scalar and return the gradient of another.

BayesFilter’s custom-gradient wrapper policy is:

- (i) compute the primal log likelihood with the production backend;
- (ii) return an analytic score that has been validated against the same primal value on controlled cases;
- (iii) expose diagnostics for finite failures and backend fallbacks;
- (iv) keep tensor shapes static inside compiled sampler paths;

- (v) use Hessians for mass matrices or diagnostics, not inside every HMC leapfrog step unless explicitly justified.

This policy reflects the MacroFinance motivation: nested tape Hessians through full Kalman scans are useful as small-model validation oracles, but they are not the intended large-model production path.

13.2 Gradient Callback Contract

A custom-gradient wrapper should expose a contract equivalent to the following mathematical tuple:

$$\text{value_and_score}(u) = (\mathcal{L}(u), \nabla_u \mathcal{L}(u), d(u)),$$

where $d(u)$ is a diagnostic record. The diagnostic record must include:

- the structural provider and parameter transform identifiers;
- the active observation-mask policy;
- the covariance/factor backend used by the value path;
- any jitter, nugget, PSD projection, rank truncation, active spectral floor, pivoting, sign correction, or fallback branch;
- the minimum relevant Cholesky pivot, QR diagonal, or spectral gap;
- finite-value and finite-gradient status.

The gradient callback should return a failure status rather than silently replacing the derivative with zeros, prior gradients, or a different backend. Emergency fallbacks can be useful for optimization diagnostics, but HMC must know whether it is using the certified derivative of the returned scalar target. If the wrapper deliberately returns a finite invalid-region fallback scalar, then the only HMC-safe fallback gradient is the exact derivative of that fallback scalar. In particular, a constant low-density fallback must have zero gradient; a nonzero surrogate direction defines a different target unless accompanied by an explicit correction scheme.

13.3 Hessian and Observed-Information Path

The HMC hot path needs values and gradients. Hessians are usually used outside the leapfrog loop for mass-matrix initialization, local identification, and curvature diagnostics. If a wrapper exposes a Hessian, it should target

$$H_{ij}(\psi) = \frac{\partial^2 \mathcal{L}(\psi)}{\partial \psi_i \partial \psi_j}$$

for the same scalar in (13.1). The observed information is $-H(\psi)$, possibly plus a sign-adjusted convention for priors if the likelihood-only and posterior targets are both reported. A Hessian path that omits prior curvature, transformed-parameter curvature, or jitter/floor branch terms must label itself as a partial diagnostic, not as posterior curvature.

The recommended production pattern is:

- (i) value-plus-score path for MAP and HMC;
- (ii) analytic or custom Hessian path for mode diagnostics and mass-matrix construction;
- (iii) small-model autodiff Hessian path as an oracle, not as the large-model default.

13.4 Static-Shape and Compilation Contract

Compiled HMC paths require stable shapes. Parameter dimension, state dimension, observation dimension, maximum active observation size, and the layout of diagnostic records should be static across leapfrog calls. Missing data should be represented by fixed-shape masks or predeclared selection patterns rather than by dynamically changing tensor ranks. Fixed sigma-point rules must keep the number and ordering of points stable. Factor backends must report branch events without changing the differentiable return signature.

This static-shape rule is an implementation constraint, but it has mathematical content: changing the set of coordinates, points, or factors during a compiled scan can turn a smooth derivative contract into a piecewise branch derivative without making the branch visible to the sampler.

13.5 HMC Safety Labels

BayesFilter should distinguish the following labels:

- **value-correct**: the scalar likelihood or posterior value matches a reference value path;
- **gradient-correct**: the returned score matches finite differences or autodiff of that same scalar on certified cases;
- **curvature-correct**: the Hessian or observed information matches the same scalar and sign convention;
- **sampler-usable**: short HMC runs have finite log probabilities, finite gradients, and acceptable diagnostic telemetry;
- **production-gated**: compiled/eager parity, branch diagnostics, mask tests, stress tests, and representative workload tests all pass.

A finite HMC run is not by itself a gradient proof. Conversely, a correct gradient on small smooth systems does not certify spectral floors, rank changes, or DSGE deterministic-completion branches unless those regimes have been validated explicitly.

Chapter 14

Derivative Validation

Derivative validation is a ladder. Passing one rung is useful evidence, but it does not certify the whole industrial target.

14.1 Validation Ladder

BayesFilter derivative providers should pass:

- (i) shape and finite-value checks;
- (ii) finite-difference checks on scalar or tiny LGSSMs;
- (iii) autodiff parity on small models where tape Hessians are feasible;
- (iv) backend parity among covariance, solve, Cholesky, and QR variants;
- (v) factor reconstruction and solve-residual diagnostics;
- (vi) compiled/eager parity for TensorFlow backends;
- (vii) stress tests for low noise, collinearity, near rank loss, and masks.

MacroFinance currently supplies examples of these tests, including QR square-root parity against autodiff and finite differences, solve backend agreement with the existing differentiated backend, TensorFlow eager/compiled parity, and large-scale backend ladder diagnostics. BayesFilter should import the testing discipline before importing any claim of production readiness.

14.2 What Is Being Compared

Every derivative test must name the scalar or tensor being compared. For a sampler-coordinate posterior target,

$$\mathcal{L}(u) = \ell(T(u); y_{1:T}) + \log p(T(u)) + \log |\det DT(u)|,$$

the score test compares

$$\hat{g}_i(u) \quad \text{to} \quad \frac{\mathcal{L}(u + he_i) - \mathcal{L}(u - he_i)}{2h}$$

or to an autodiff score of that same scalar. The Hessian test compares

$$\hat{H}_{ij}(u) \quad \text{to} \quad \frac{\hat{g}_i(u + he_j) - \hat{g}_i(u - he_j)}{2h}$$

or to a tape Hessian of the same scalar. If the reported analytic derivative is likelihood-only but the finite difference includes the prior or transform Jacobian, the test is invalid. Conversely, if a chapter validates $\nabla_{\theta}\ell$ in model-parameter coordinates, that result should be read as a component-level backend check rather than as the complete HMC target gradient unless the prior and Jacobian terms are added consistently. If the value path contains jitter, floors, masks, transformed parameters, or invalid-region fallback values, the comparison value must contain the same choices.

14.3 Recommended Rungs

The derivative validation ladder should be run in this order:

- (i) **Finite values and shapes.** Check that value, score, Hessian, state moments, factors, provider tensors, and diagnostics are finite and have declared dimensions.
- (ii) **Symmetry and admissibility.** Check that covariance-like objects are symmetric, positive semidefinite/definite under their declared policy, and that Hessians are symmetric within tolerance.
- (iii) **Block finite differences.** Validate structural derivatives of $c, F, Q, a, H, R, m_0, P_0$ before running a long filter scan.
- (iv) **Likelihood finite differences.** Compare score and Hessian of the complete scalar target on tiny systems.
- (v) **Autodiff parity.** Use taped gradients and Hessians as small-system oracles where nested tapes are feasible.
- (vi) **Backend parity.** Compare covariance, solve, Cholesky, QR, and SVD/spectral backends on systems where they represent the same law.
- (vii) **Factor reconstruction.** Check (12.1)–(12.2) for prediction, innovation, and filtered covariance factors.
- (viii) **Mask tests.** Test dense all-observed masks, sparse deterministic masks, and all-missing periods using the convention in Section 9.5.
- (ix) **Spectral and branch stress tests.** Vary spectral gaps, Cholesky pivots, QR diagonals, rank thresholds, floors, jitter, and fallback branches while checking diagnostics.
- (x) **Eager/compiled parity.** Compare values and derivatives under eager execution, compiled execution, and XLA/JIT paths when those paths are intended for production.
- (xi) **Sampler smoke tests.** Run short HMC/NUTS checks only after deterministic value-and-gradient tests pass, and interpret them as sampler usability evidence, not proof of the derivative formulas.

14.4 Finite-Difference Tolerances

Finite differences are not exact tests; they are numerical experiments. The step size should be selected relative to the transformed parameter scale, for example

$$h_i = \eta(1 + |\psi_i|),$$

with several η values checked when possible. A central difference should show a stable error plateau before the result is accepted. Parameters near constraints, branch boundaries, or spectral floors should be tested separately from smooth interior points, because the expected derivative may be a branch derivative rather than a smooth derivative of the unregularized model.

14.5 Backend-Specific Gates

Each backend has a distinct failure mode:

- covariance form can lose symmetry or positive semidefiniteness under finite arithmetic;
- solve form can hide ill conditioning unless solve residuals and log-determinant policies are reported;
- Cholesky form depends on positive pivots and any jitter policy;
- QR form depends on full column rank, diagonal sign convention, and pivot policy;
- SVD/spectral form depends on gap, rank, floor, sign/order, and fallback policy;
- sigma-point form additionally depends on fixed point rules, weights, nonlinear map derivatives, and moment reconstruction.

Backend parity is meaningful only when the compared backends target the same law. A covariance backend using P and an SVD backend using a floored P_+ are not required to have identical derivatives unless $P_+ = P$ on that test case.

14.6 Validation Artifact

Every promoted derivative backend should leave an artifact containing:

- model name, parameter vector, transform policy, and reference point;
- backend name and value branch;
- tests run, tolerances, and maximum absolute/relative errors;
- finite-value, finite-gradient, and finite-Hessian status;
- branch diagnostics: jitter, PSD projection, spectral floors, pivots, rank truncation, sign/order corrections, and fallbacks;
- interpretation: which claim is supported and which regimes remain untested.

This artifact is the bridge between a mathematical derivation and an HMC claim. Without it, phrases such as “analytic Hessian”, “SVD sigma-point gradient”, or “HMC-ready backend” are too ambiguous to audit.

14.7 Audit Record

Each derivative chapter should record the source labels, code files, and tests that support it. If a tool audit is inconclusive, the monograph should say so. An inconclusive symbolic audit can still be valuable: it identifies which guards, shapes, or noncommutative matrix steps need manual proof before a formula is promoted to peer-review-ready status.

For the analytic-derivative chapter set, the current audit record is:

- MacroFinance analytic derivative labels provide the source formulas for score, Hessian, structural provider, Cholesky, and QR contracts;
- MathDevMCP is useful for label lookup and local-context provenance, but it does not replace manual proof of noncommutative matrix calculus steps;
- ResearchAssistant is local/offline and has no current summary hits for the exact SVD/eigen derivative-validation cluster used here;
- production readiness remains a test claim, not a prose claim.

Part IV

Nonlinear Filtering

Chapter 15

Extended Kalman Filtering

The extended Kalman filter is the most direct nonlinear extension of the linear Gaussian spine in Chapter 5. It replaces nonlinear transition and observation maps by local linear approximations and then applies the same prediction-error likelihood form to the resulting Gaussian approximation. In BayesFilter this is an approximate likelihood backend, not an exact replacement for the Kalman filter except in the affine-Gaussian special case.

15.1 Local Linearization

Consider the nonlinear contract

$$x_{t+1} = f_\theta(x_t) + u_{t+1}, \quad u_{t+1} \sim \mathcal{N}(0, Q_\theta), \quad (15.1)$$

$$y_t = h_\theta(x_t) + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, R_\theta). \quad (15.2)$$

Let

$$F_t = \left. \frac{\partial f_\theta(x)}{\partial x^\top} \right|_{x=m_{t-1|t-1}}, \quad H_t = \left. \frac{\partial h_\theta(x)}{\partial x^\top} \right|_{x=m_{t|t-1}}.$$

The EKF prediction and update are

$$m_{t|t-1} = f_\theta(m_{t-1|t-1}), \quad (15.3)$$

$$P_{t|t-1} = F_t P_{t-1|t-1} F_t^\top + Q_\theta, \quad (15.4)$$

$$v_t = y_t - h_\theta(m_{t|t-1}), \quad (15.5)$$

$$S_t = H_t P_{t|t-1} H_t^\top + R_\theta, \quad (15.6)$$

$$K_t = P_{t|t-1} H_t^\top S_t^{-1}, \quad (15.7)$$

$$m_{t|t} = m_{t|t-1} + K_t v_t. \quad (15.8)$$

The covariance may be updated in simplified form or in a Joseph-style linearized form. The backend must state which form defines the numerical recursion, because this choice affects finite-arithmetic behavior even when the first-order algebra is equivalent.

15.2 Likelihood Status

The EKF log likelihood uses the same innovation expression as (5.10), but the innovations and innovation covariances come from local linearization. Therefore the EKF target is

an approximate target unless f_θ and h_θ are affine on the region reached by the filter. An HMC chain using the EKF samples from the posterior induced by that approximate likelihood and prior, not from the exact nonlinear state-space posterior.

For BayesFilter documentation and tests, the EKF should be used in three roles:

- (i) a baseline nonlinear Gaussian approximation;
- (ii) a regression oracle in cases where the nonlinear maps collapse to affine maps and the exact Kalman likelihood is available;
- (iii) a diagnostic comparison for sigma-point and particle backends.

15.3 HMC Contract

An EKF backend is HMC-eligible only after it declares:

- how the Jacobians F_t and H_t are computed;
- whether gradients with respect to θ are tape-based, analytic, or custom;
- what happens when S_t is singular, non-finite, or numerically indefinite;
- whether all loop lengths, masks, and state dimensions remain static under compilation;
- which parity tests compare the EKF to exact linear Gaussian cases.

The older nonlinear-filtering chapters in the source projects motivate this contract, but BayesFilter tightens it for HMC: differentiability and finite failure behavior are part of the target definition, not implementation afterthoughts.

Chapter 16

Sigma-Point Filters

Sigma-point filters replace local linearization with deterministic quadrature. They propagate a finite set of points through nonlinear maps and reconstruct approximate Gaussian moments from weighted averages. This makes them more accurate than the EKF for some nonlinear transformations, while preserving the Gaussian filtering closure used by the prediction-error likelihood. The price is that the likelihood remains approximate and the derivative path becomes more complicated.

16.1 Moment-Matching Interface

At each time step, a sigma-point backend starts from a Gaussian approximation

$$x_t \mid y_{1:t-1} \approx \mathcal{N}(m_{t|t-1}, P_{t|t-1}).$$

A deterministic rule constructs points $\chi_{t|t-1}^{(j)}$ and weights $w_j^{(m)}, w_j^{(c)}$. Passing the points through the observation map gives

$$z_t^{(j)} = h_\theta(\chi_{t|t-1}^{(j)}).$$

The backend then forms approximate predicted observations, innovation covariances, and cross covariances:

$$\bar{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \quad (16.1)$$

$$P_{zz,t} = \sum_j w_j^{(c)} (z_t^{(j)} - \bar{z}_t)(z_t^{(j)} - \bar{z}_t)^\top + R_\theta, \quad (16.2)$$

$$P_{xz,t} = \sum_j w_j^{(c)} (\chi_{t|t-1}^{(j)} - m_{t|t-1})(z_t^{(j)} - \bar{z}_t)^\top. \quad (16.3)$$

The gain is $K_t = P_{xz,t} P_{zz,t}^{-1}$ and the innovation is $v_t = y_t - \bar{z}_t$. The same Gaussian innovation likelihood formula is then used, with $P_{zz,t}$ replacing S_t .

16.2 Conjugate Unscented Transform Rules

The ordinary unscented transform is not the only deterministic quadrature rule that can be used in the interface above. The conjugate unscented transform (CUT) of [Adurthi](#)

et al. [2012a,b] constructs higher-order Gaussian point sets by combining symmetric conjugate point families. The experimental file

`/home/chakwong/python/src/dsge_hmc/filters/CUTSRUKF.py`

implements a deprecated TensorFlow square-root CUT4 demonstration for a coordinated-turn tracking model. It is not wired into the current BayesFilter filter hierarchy, but it is a useful design reference because it separates the sigma-point rule from the square-root covariance backend.

For a standard normal variable $Z \in \mathbb{R}^n$, the CUT4-G rule used there has

$$N_{\text{CUT4}} = 2n + 2^n$$

points: $2n$ axis points and 2^n hypercube-corner points. In the convention used by the demo, for $n \geq 3$,

$$r_a^2 = \frac{n+2}{2}, \quad r_c^2 = \frac{n+2}{n-2},$$

with weights

$$w_a = \frac{4}{(n+2)^2}, \quad w_c = \frac{(n-2)^2}{2^n(n+2)^2}.$$

More explicitly, define

$$\mathcal{A} = \{\pm r_a e_i : i = 1, \dots, n\}, \quad \mathcal{C} = \{r_c s : s \in \{-1, +1\}^n\}.$$

The CUT4-G quadrature operator is

$$\mathcal{Q}_{\text{CUT4}}[\varphi] = w_a \sum_{\xi \in \mathcal{A}} \varphi(\xi) + w_c \sum_{\xi \in \mathcal{C}} \varphi(\xi). \quad (16.4)$$

Proposition 16.1 (CUT4-G Gaussian moment exactness). *For $n \geq 3$, the point set (16.4) integrates every standard-normal monomial in $Z \in \mathbb{R}^n$ of total degree at most five exactly. Equivalently, it is a degree-five Gaussian rule.*

Proof. Both $\mathcal{A}$ and $\mathcal{C}$ are centrally symmetric and sign-symmetric in every coordinate. Hence every monomial with at least one odd coordinate exponent has zero quadrature value, matching the standard normal. This covers all odd total degrees through five and also the mixed even-odd monomials.

It remains to check total degrees 0, 2, and 4. The total mass is

$$2nw_a + 2^n w_c = 1.$$

For a coordinate i ,

$$\mathcal{Q}_{\text{CUT4}}[z_i^2] = 2w_a r_a^2 + 2^n w_c r_c^2 = 1 = \mathbb{E}[Z_i^2],$$

and, for $i \neq j$, $\mathcal{Q}_{\text{CUT4}}[z_i z_j] = 0 = \mathbb{E}[Z_i Z_j]$ by sign symmetry. The fourth marginal moment is

$$\mathcal{Q}_{\text{CUT4}}[z_i^4] = 2w_a r_a^4 + 2^n w_c r_c^4 = 3 = \mathbb{E}[Z_i^4],$$

while the mixed fourth moment is

$$\mathcal{Q}_{\text{CUT4}}[z_i^2 z_j^2] = 2^n w_c r_c^4 = 1 = \mathbb{E}[Z_i^2 Z_j^2], \quad i \neq j.$$

All remaining fourth-degree monomials contain an odd coordinate exponent and therefore vanish on both sides. These identities exhaust the Gaussian monomials of total degree at most five, so the rule has degree-five exactness. $\square$

Thus the rule is symmetric, has positive weights, and integrates Gaussian polynomials through degree five. By contrast, the usual $2n + 1$ unscented rule is commonly described as third-degree accurate for Gaussian moment propagation. The extra corner points are what allow CUT4-G to match cross moments such as $\mathbb{E}[Z_i^2 Z_j^2]$ that axis-only rules do not represent in the same way.

The price is exponential growth in the corner set. CUT4-G uses 42 points at $n = 5$, 1044 points at $n = 10$, and becomes unattractive in large structural states unless the quadrature is applied to a low-dimensional stochastic block. This reinforces the structural lesson of Chapter 19: higher-order quadrature should be spent on the declared stochastic integration coordinates, not on deterministic-completion directions that add no stochastic law.

16.3 Approximation Boundary

BayesFilter should label sigma-point filters as approximate likelihood backends unless a special model structure supplies an exactness argument. The source SR-UKF material contains model-specific claims about quadratic observation equations and pruned DSGE structure. Those claims are useful case-study material, but they should not be promoted to a general BayesFilter theorem without a separate derivation in BayesFilter notation and a regression test against an exact reference.

For HMC, this distinction matters. If the sigma-point likelihood is used in the target, the chain targets the sigma-point posterior. If the sigma-point filter is only a surrogate for an exact nonlinear likelihood, then a correction or delayed-acceptance policy is needed before exact-posterior claims are made.

16.4 Derivative and Compilation Requirements

Sigma-point HMC requires differentiating through:

- the covariance factor used to generate points;
- the nonlinear transition and observation maps;
- the weighted covariance assembly;
- linear solves or factorizations for $P_{zz,t}$;
- any PSD projection, jitter, clipping, or fallback branch.

The derivative provider must return the derivative of the same approximate log likelihood that supplies the Metropolis value. Static shapes are also non-negotiable: the number of sigma points, state dimension, observed-data mask shape, and scan length must not vary inside the compiled transition.

16.5 Evidence Ladder

The minimum evidence ladder for a sigma-point backend is:

- (i) exact recovery on affine Gaussian systems against the Kalman backend;

- (ii) finite value and finite gradient on controlled nonlinear systems;
- (iii) eager/compiled parity where a compiled sampler is claimed;
- (iv) comparison to a denser quadrature, long simulation, or trusted reference on low-dimensional nonlinear examples;
- (v) HMC diagnostics that remain explicitly separated from convergence proof.

This ladder is the consolidation of the source-project SVD/HMC validation plans, not a completed BayesFilter certification result.

CUT backends should pass the same ladder plus one rule-specific check: for standard-normal polynomial test functions up to the declared degree, the implemented points and weights should reproduce the analytic Gaussian moments before the points are used in a filter. This catches mistakes in point enumeration, weights, dimension restrictions, and transformed-coordinate scaling before nonlinear filtering errors obscure the quadrature issue.

Chapter 17

Square-Root Sigma-Point Filters

Square-root sigma-point filters propagate a factor of the covariance rather than the covariance matrix itself. Their purpose is numerical: avoid avoidable loss of symmetry and positive semi-definiteness, reduce solve instability, and make the filtering recursion easier to monitor. In BayesFilter, square-root bookkeeping is a backend discipline. It does not by itself make an approximate likelihood exact or an autodiff gradient safe for HMC.

17.1 Backend Contract

A square-root backend stores a factor C_t such that $P_t = C_t C_t^\top$ or an equivalent factorization. The factor may be Cholesky, QR-derived, SVD/eigen-derived, or another auditable representation. The backend contract must state:

- the factor orientation and reconstruction identity;
- the predict update and measurement update used for the factor;
- whether covariance matrices are ever reconstructed;
- how rank loss, jitter, and PSD projection are handled;
- which operations define the differentiable target path.

The source SR-UKF material is useful here precisely because it exposed a maintenance risk: a method can be called square-root while still reconstructing full covariances and applying simplified covariance updates internally. The BayesFilter monograph should therefore describe the actual implemented recursion, not only the label attached to the filter.

17.2 HMC-Relevant Risks

For HMC, the relevant question is not only whether the factor recursion keeps P_t positive semi-definite in floating point. The sampler also needs a stable derivative of the scalar log target. Cholesky, QR, and SVD factors have different derivative behavior near rank loss or repeated singular values. A backend that is robust for value evaluation can still be fragile as a raw tape-gradient path.

BayesFilter should therefore classify square-root sigma-point evidence in two columns:

- (i) value-recursion evidence, such as finite likelihoods, factor reconstruction residuals, and affine-Gaussian parity;
- (ii) derivative evidence, such as finite gradients, AD/custom-gradient parity, spectral-gap telemetry, and HMC leapfrog stability.

Only the second column is directly relevant to HMC production readiness.

17.3 When to Prefer Square-Root Forms

Square-root forms are natural when observation noise is small, latent covariances are nearly singular, or long scans accumulate asymmetry. They are also useful for large LGSSMs where factor diagnostics expose numerical problems earlier than a final log likelihood does. They should not be treated as a universal answer to nonlinear filtering: if the moment closure is poor, the filter is still approximating the wrong target; if the derivative of the factorization is unstable, HMC can still fail.

17.4 CUT With Square-Root Backends

The conjugate unscented transform in Section 16.2 is a point-and-weight rule. It can be combined with any square-root factor backend that maps standardized points $\xi^{(j)}$ into state points

$$\chi^{(j)} = m + C\xi^{(j)}, \quad CC^\top = P_\star.$$

The experimental `/home/chakwong/python/src/dsge_hmc/filters/CUTSRUKF.py` module uses positive CUT4-G weights together with QR square-root covariance updates. It also keeps rank-deficient process noise as a rectangular factor G with $Q = GG^\top$, avoiding an invalid Cholesky factorization of a singular Q .

An SVD/eigen factor can be used in the same location:

$$P = U\Lambda U^\top, \quad C = U\Lambda_+^{1/2}.$$

This may make value evaluation more robust when P is singular or nearly singular. It does not change CUT's quadrature degree; it changes the numerical factor used to place the CUT points. Therefore a stable CUT-SVD filter must report the same factor diagnostics as an SVD sigma-point filter: the covariance actually reconstructed, active floors or rank truncations, spectral gaps, sign or order choices, and fallback branches. In short, CUT can be combined with SVD, but the result is a higher-order quadrature rule on top of an SVD factor policy, not a new theorem that removes spectral derivative risk.

Chapter 18

SVD Sigma-Point Filters

The SVD sigma-point filter is the most important nonlinear-filtering lesson from the recent source-project debugging work. It improved value-side robustness by using spectral covariance factors and explicit finite-arithmetic guards, but it also exposed why raw autodiff through spectral decompositions is not a comfortable industrial HMC default. BayesFilter should preserve both lessons.

18.1 What the Source Audit Established

The math/code audit recorded in the source-project SVD filter audit result from 2026-04-25 established the following bounded facts:

- the linear SVD Kalman path uses a Joseph-style covariance update;
- the nonlinear sigma-point path uses the simplified update

$$P^+ = P^- - KP_{zz}K^\top;$$

- PSD projection occurs after the candidate covariance is formed, so it cannot prevent overflow or non-finite values during the formation of P^+ ;
- the conditioning guard with threshold `_SVD_CONDITIONING_MAX_ABS = 1e100` is an engineering finite-arithmetic guard, not a mathematically derived posterior threshold;
- prior short HMC diagnostics were numerical-path regressions, not convergence evidence.

These are implementation facts, not general theorems about SVD filtering. BayesFilter should cite them as project evidence and should avoid converting them into unsupported production-readiness claims.

18.2 Spectral Differentiation Risk

The Phase 2B literature gate accepted the Matrix Backprop source for the bounded claim that gradients through SVD or eigendecomposition contain terms with denominators involving singular-value or eigenvalue gaps. When two singular values or eigenvalues

are equal, the decomposition is not uniquely differentiable; when they are merely close, derivative terms can become large and numerically unstable.

This is directly relevant to HMC. A large DSGE or NAWM-sized model can visit parameter regions where covariance factors are ill conditioned, nearly rank deficient, or spectrally clustered. A value-side SVD factor may still return a finite covariance approximation, while the tape gradient through the spectral basis becomes noisy, discontinuous, or non-finite. Larger dimensions make close spectral gaps more likely in practice, so BayesFilter should require gap telemetry and derivative validation before using SVD sigma-point gradients inside production HMC.

18.3 Analytic Score and Hessian Recursion

The SVD sigma-point derivative problem has two layers. The likelihood layer is not new: after the sigma-point update has produced

$$v_t, \quad S_t, \quad \dot{v}_t^{(i)}, \quad \dot{S}_t^{(i)}, \quad \ddot{v}_t^{(ij)}, \quad \ddot{S}_t^{(ij)},$$

the score and Hessian are exactly the solve-form expressions in Chapter 10. The new work is the sigma-point layer: differentiate the point cloud, the nonlinear maps, and the weighted moment assemblies that generate these innovation objects.

Thus this chapter plugs into the analytic-derivative stack as follows. Chapter 11 supplies derivatives of structural maps and parameter transforms; Chapter 12 defines what differentiated SVD factors must reconstruct; Chapter 13 requires the SVD score to differentiate the same implemented scalar value; and Chapter 14 supplies the finite-difference, backend-parity, branch, and HMC-gating tests.

Let a Gaussian input at some predict or update substep be

$$X \sim \mathcal{N}(m(\theta), P(\theta)).$$

Let an SVD/eigendecomposition factor be written

$$P = U\Lambda U^\top, \quad C = U\Lambda_+^{1/2},$$

where Λ_+ denotes the eigenvalue policy actually used for point generation: active positive eigenvalues, a smooth floor, or a hard floor. A fixed sigma-point rule can be written as

$$\chi^{(r)} = m + C\xi^{(r)}, \quad r = 0, \dots, N-1, \quad (18.1)$$

where the standardized offsets $\xi^{(r)}$ and weights $w_r^{(m)}, w_r^{(c)}$ are fixed by the UKF, CKF, or another deterministic quadrature rule. For the UKF, for example, the nonzero offsets are $\pm\gamma e_a$. If tuning parameters such as α or κ are estimated, the same formulas hold with additional $\dot{\xi}^{(r)}, \ddot{\xi}^{(r)}, \dot{w}_r$, and $\ddot{w}_r$ terms. BayesFilter's default derivative contract treats the rule as fixed.

With fixed quadrature offsets, the point derivatives are

$$\dot{\chi}^{(r,i)} = \dot{m}^{(i)} + \dot{C}^{(i)}\xi^{(r)}, \quad (18.2)$$

$$\ddot{\chi}^{(r,ij)} = \ddot{m}^{(ij)} + \ddot{C}^{(ij)}\xi^{(r)}. \quad (18.3)$$

Thus all spectral difficulty enters through $\dot{C}$ and $\ddot{C}$; the remaining recursion is ordinary chain rule and product rule.

18.3.1 Map and Moment Derivatives

Let g_θ denote either the transition map in the predict step or the observation map in the update step, and define

$$\eta^{(r)} = g_\theta(\chi^{(r)}).$$

Using $D_x g_\theta$ for the Jacobian in the state argument, and $D_{xx}^2 g_\theta[a, b]$ for the second directional derivative in directions a, b , the propagated point derivatives are

$$\dot{\eta}^{(r,i)} = D_x g_\theta(\chi^{(r)}) \dot{\chi}^{(r,i)} + \partial_i g_\theta(\chi^{(r)}), \quad (18.4)$$

$$\begin{aligned} \ddot{\eta}^{(r,ij)} &= D_x g_\theta(\chi^{(r)}) \ddot{\chi}^{(r,ij)} + D_{xx}^2 g_\theta(\chi^{(r)})[\dot{\chi}^{(r,i)}, \dot{\chi}^{(r,j)}] \\ &\quad + D_{x\theta_i}^2 g_\theta(\chi^{(r)}) \dot{\chi}^{(r,j)} + D_{x\theta_j}^2 g_\theta(\chi^{(r)}) \dot{\chi}^{(r,i)} + \partial_{ij}^2 g_\theta(\chi^{(r)}). \end{aligned} \quad (18.5)$$

These equations are the exact place where the structural model derivatives enter the SVD sigma-point score and Hessian.

The sigma-point mean and covariance of the transformed cloud are

$$\bar{\eta} = \sum_{r=0}^{N-1} w_r^{(m)} \eta^{(r)}, \quad (18.6)$$

$$M_\eta = \sum_{r=0}^{N-1} w_r^{(c)} \Delta_\eta^{(r)} \Delta_\eta^{(r)\top} + \Omega_\theta, \quad \Delta_\eta^{(r)} = \eta^{(r)} - \bar{\eta}, \quad (18.7)$$

where Ω_θ is the process-noise covariance in a predict step or the measurement-noise covariance in an update step. Their derivatives are

$$\dot{\bar{\eta}}^{(i)} = \sum_r w_r^{(m)} \dot{\eta}^{(r,i)}, \quad (18.8)$$

$$\ddot{\bar{\eta}}^{(ij)} = \sum_r w_r^{(m)} \ddot{\eta}^{(r,ij)}, \quad (18.9)$$

and, with $\dot{\Delta}_\eta^{(r,i)} = \dot{\eta}^{(r,i)} - \dot{\bar{\eta}}^{(i)}$,

$$\dot{M}_\eta^{(i)} = \sum_r w_r^{(c)} \left[\dot{\Delta}_\eta^{(r,i)} \Delta_\eta^{(r)\top} + \Delta_\eta^{(r)} \dot{\Delta}_\eta^{(r,i)\top} \right] + \dot{\Omega}_\theta^{(i)}, \quad (18.10)$$

$$\begin{aligned} \ddot{M}_\eta^{(ij)} &= \sum_r w_r^{(c)} \left[\ddot{\Delta}_\eta^{(r,ij)} \Delta_\eta^{(r)\top} + \Delta_\eta^{(r)} \ddot{\Delta}_\eta^{(r,ij)\top} + \dot{\Delta}_\eta^{(r,i)} \dot{\Delta}_\eta^{(r,j)\top} + \dot{\Delta}_\eta^{(r,j)} \dot{\Delta}_\eta^{(r,i)\top} \right] + \ddot{\Omega}_\theta^{(ij)}. \end{aligned} \quad (18.11)$$

Equations (18.2)–(18.11) are the analytic derivative recursion for any sigma-point predict moment.

18.3.2 UKF Update Objects

At the observation update, take the input mean and covariance to be $m_t^- = m_{t|t-1}$ and $P_t^- = P_{t|t-1}$, construct points $\chi_t^{(r)}$ by (18.1), and propagate

$$z_t^{(r)} = h_\theta(\chi_t^{(r)}).$$

Equations (18.8)–(18.11) give

$$\bar{z}_t, \quad S_t = \sum_r w_r^{(c)} (z_t^{(r)} - \bar{z}_t)(z_t^{(r)} - \bar{z}_t)^\top + R_\theta,$$

and their first and second derivatives. The innovation is

$$v_t = y_t - \bar{z}_t, \quad \dot{v}_t^{(i)} = -\dot{\bar{z}}_t^{(i)}, \quad \ddot{v}_t^{(ij)} = -\ddot{\bar{z}}_t^{(ij)}, \quad (18.12)$$

unless the data transformation itself depends on parameters.

The cross-covariance used in the Kalman gain is

$$P_{xz,t} = \sum_r w_r^{(c)} \Delta_x^{(r)} \Delta_z^{(r)\top}, \quad \Delta_x^{(r)} = \chi_t^{(r)} - m_t^-, \quad \Delta_z^{(r)} = z_t^{(r)} - \bar{z}_t. \quad (18.13)$$

Its first and second derivatives are

$$\dot{P}_{xz,t}^{(i)} = \sum_r w_r^{(c)} \left[\dot{\Delta}_x^{(r,i)} \Delta_z^{(r)\top} + \Delta_x^{(r)} \dot{\Delta}_z^{(r,i)\top} \right], \quad (18.14)$$

$$\ddot{P}_{xz,t}^{(ij)} = \sum_r w_r^{(c)} \left[\ddot{\Delta}_x^{(r,ij)} \Delta_z^{(r)\top} + \Delta_x^{(r)} \ddot{\Delta}_z^{(r,ij)\top} + \dot{\Delta}_x^{(r,i)} \dot{\Delta}_z^{(r,j)\top} + \dot{\Delta}_x^{(r,j)} \dot{\Delta}_z^{(r,i)\top} \right]. \quad (18.15)$$

The gain is best differentiated as a solved matrix equation. Since $K_t = P_{xz,t} S_t^{-1}$, equivalently

$$S_t K_t^\top = P_{xz,t}^\top.$$

Therefore

$$\dot{K}_t^{(i)\top} = \mathcal{S}_{S_t} \left(\dot{P}_{xz,t}^{(i)\top} - \dot{S}_t^{(i)} K_t^\top \right), \quad (18.16)$$

$$\ddot{K}_t^{(ij)\top} = \mathcal{S}_{S_t} \left(\ddot{P}_{xz,t}^{(ij)\top} - \ddot{S}_t^{(ij)} K_t^\top - \dot{S}_t^{(i)} \dot{K}_t^{(j)\top} - \dot{S}_t^{(j)} \dot{K}_t^{(i)\top} \right). \quad (18.17)$$

Finally, the usual sigma-point update

$$m_t^+ = m_t^- + K_t v_t, \quad (18.18)$$

$$P_t^+ = P_t^- - K_t S_t K_t^\top \quad (18.19)$$

has derivatives

$$\dot{m}_t^{+(i)} = \dot{m}_t^{-(i)} + \dot{K}_t^{(i)} v_t + K_t \dot{v}_t^{(i)}, \quad (18.20)$$

$$\ddot{m}_t^{+(ij)} = \ddot{m}_t^{-(ij)} + \ddot{K}_t^{(ij)} v_t + \dot{K}_t^{(i)} \dot{v}_t^{(j)} + \dot{K}_t^{(j)} \dot{v}_t^{(i)} + K_t \ddot{v}_t^{(ij)}. \quad (18.21)$$

For the covariance, define

$$\dot{P}_t^{+(i)} = \dot{P}_t^{-(i)} - \dot{K}_t^{(i)} S_t K_t^\top - K_t \dot{S}_t^{(i)} K_t^\top - K_t S_t \dot{K}_t^{(i)\top}, \quad (18.22)$$

and

$$\begin{aligned} \ddot{P}_t^{+(ij)} = & \ddot{P}_t^{-(ij)} - \ddot{K}_t^{(ij)} S_t K_t^\top - \dot{K}_t^{(i)} \dot{S}_t^{(j)} K_t^\top - \dot{K}_t^{(j)} S_t \dot{K}_t^{(i)\top} \\ & - \dot{K}_t^{(j)} \dot{S}_t^{(i)} K_t^\top - K_t \ddot{S}_t^{(ij)} K_t^\top - K_t \dot{S}_t^{(i)} \dot{K}_t^{(j)\top} \\ & - \dot{K}_t^{(j)} S_t \dot{K}_t^{(i)\top} - K_t \dot{S}_t^{(j)} \dot{K}_t^{(i)\top} - K_t S_t \ddot{K}_t^{(ij)\top}. \end{aligned} \quad (18.23)$$

The next time step then refactors P_t^+ and differentiates the new SVD factor. This is where the spectral contract below enters the recursion again.

18.3.3 Likelihood Score and Hessian

The SVD sigma-point likelihood contribution is the Gaussian innovation likelihood

$$\ell_t = -\frac{1}{2} [\log \det S_t + v_t^\top S_t^{-1} v_t + n_y \log(2\pi)],$$

where S_t and v_t are the sigma-point objects above. Let $S_t w_t = v_t$. The score is

$$\partial_i \ell_t = -\frac{1}{2} [\text{tr}(S_t^{-1} \dot{S}_t^{(i)}) + 2\dot{v}_t^{(i)\top} w_t - w_t^\top \dot{S}_t^{(i)} w_t], \quad (18.24)$$

and the Hessian is obtained by substituting the sigma-point $\dot{v}_t, \dot{S}_t, \ddot{v}_t, \ddot{S}_t$ into (10.1)–(10.10). Thus the UKF/SVD Hessian is not a separate likelihood theorem. It is the Kalman solve-form Hessian applied to approximate moments generated by (18.1)–(18.23).

18.4 Structural Fixed-Support Score

Chapter 19 explains why a structural sigma-point filter places points on the pre-transition uncertainty variable

$$A_t = (x_{t-1}, \varepsilon_t), \quad A_t \mid y_{1:t-1} \approx \mathcal{N}(\mu_{a,t}, P_{a,t}), \quad (18.25)$$

and then completes the state pointwise:

$$A_t^{(r)} = \mu_{a,t} + C_{a,t} \xi^{(r)}, \quad X_t^{(r)} = F_\theta(A_t^{(r)}), \quad Z_t^{(r)} = h_\theta(X_t^{(r)}). \quad (18.26)$$

Equations (18.25)–(18.26) are the derivative version of Proposition 19.1. The filtered state is still x_t ; the sigma-point variable is the pre-transition uncertainty whose push-forward generates the predictive law. This is the object that must be differentiated for deterministic-completion examples such as Model C in Chapter 48.

The subtle case is a fixed structural zero direction. Write

$$P_{a,t} = U_t \Lambda_t U_t^\top, \quad \Lambda_t = \text{diag}(\lambda_{1t}, \dots, \lambda_{dt}),$$

and split the spectral indices into

$$\mathcal{I}_t^+ = \{a : \lambda_{at} > \tau\}, \quad \mathcal{I}_t^0 = \{a : \lambda_{at} \leq \tau\}.$$

The structural fixed-support branch assumes that the zero-support block is part of the model law, not a numerical floor:

$$u_b^\top \dot{P}_{a,t}^{(i)} u_c = 0 \quad \text{whenever } b \in \mathcal{I}_t^0 \text{ or } c \in \mathcal{I}_t^0, \quad (18.27)$$

$$|\lambda_{at} - \lambda_{bt}| > g_{\min} \quad \text{for every differentiated active pair.} \quad (18.28)$$

Here “differentiated active pair” means any pair that enters the derivative of an active factor column: active-active pairs and active-null pairs. Gaps between two null directions do not matter because their factor columns are identically zero on this branch.

The implemented factor is

$$C_{a,t} = [\sqrt{\lambda_{1t}} u_{1t} \quad \cdots \quad \sqrt{\lambda_{dt}} u_{dt}], \quad \sqrt{\lambda_{at}} u_{at} = 0 \quad \text{for } a \in \mathcal{I}_t^0. \quad (18.29)$$

For an active column $a \in \mathcal{I}_t^+$,

$$\dot{u}_{at}^{(i)} = \sum_{b \neq a} u_{bt} \frac{u_{bt}^\top \dot{P}_{a,t}^{(i)} u_{at}}{\lambda_{at} - \lambda_{bt}}, \quad (18.30)$$

$$\dot{c}_{at}^{(i)} = \sqrt{\lambda_{at}} \dot{u}_{at}^{(i)} + \frac{u_{at}^\top \dot{P}_{a,t}^{(i)} u_{at}}{2\sqrt{\lambda_{at}}} u_{at}. \quad (18.31)$$

For a null column $a \in \mathcal{I}_t^0$,

$$c_{at} = 0, \quad \dot{c}_{at}^{(i)} = 0. \quad (18.32)$$

Under (18.27), these column rules reconstruct the derivative of the same pre-transition covariance:

$$\dot{P}_{a,t}^{(i)} = \dot{C}_{a,t}^{(i)} C_{a,t}^\top + C_{a,t} \dot{C}_{a,t}^{(i)\top}. \quad (18.33)$$

The reconstruction target is not a nuggetted covariance. A positive floor that turns a structural zero direction into positive variance changes the law and must be reported as a different regularized branch.

With this fixed-support factor, the score recursion is the ordinary sigma-point chain rule. For a scalar parameter component θ_i ,

$$\dot{A}_t^{(r,i)} = \dot{\mu}_{a,t}^{(i)} + \dot{C}_{a,t}^{(i)} \xi^{(r)}, \quad (18.34)$$

$$\dot{X}_t^{(r,i)} = D_a F_\theta(A_t^{(r)}) \dot{A}_t^{(r,i)} + \partial_i F_\theta(A_t^{(r)}), \quad (18.35)$$

$$\dot{Z}_t^{(r,i)} = D_x h_\theta(X_t^{(r)}) \dot{X}_t^{(r,i)} + \partial_i h_\theta(X_t^{(r)}). \quad (18.36)$$

The moment derivatives are

$$\dot{\hat{x}}_t^{(i)} = \sum_r w_r^{(m)} \dot{X}_t^{(r,i)}, \quad \dot{\hat{z}}_t^{(i)} = \sum_r w_r^{(m)} \dot{Z}_t^{(r,i)}, \quad (18.37)$$

$$\dot{S}_t^{(i)} = \sum_r w_r^{(c)} \left[\dot{\Delta}_z^{(r,i)} \Delta_z^{(r)\top} + \Delta_z^{(r)} \dot{\Delta}_z^{(r,i)\top} \right] + \dot{R}_t^{(i)}, \quad (18.38)$$

$$\dot{P}_{xz,t}^{(i)} = \sum_r w_r^{(c)} \left[\dot{\Delta}_x^{(r,i)} \Delta_z^{(r)\top} + \Delta_x^{(r)} \dot{\Delta}_z^{(r,i)\top} \right], \quad (18.39)$$

where

$$\Delta_x^{(r)} = X_t^{(r)} - \hat{x}_t, \quad \Delta_z^{(r)} = Z_t^{(r)} - \hat{z}_t,$$

and dotted centered quantities subtract the corresponding dotted means. The innovation derivative remains

$$v_t = y_t - \hat{z}_t, \quad \dot{v}_t^{(i)} = -\dot{\hat{z}}_t^{(i)}.$$

Therefore the per-period score is

$$\partial_i \ell_t = -\frac{1}{2} \left[\text{tr}(S_t^{-1} \dot{S}_t^{(i)}) + 2\dot{v}_t^{(i)\top} S_t^{-1} v_t - v_t^\top S_t^{-1} \dot{S}_t^{(i)} S_t^{-1} v_t \right]. \quad (18.40)$$

This sign convention is the direct derivative of the likelihood in (19.91); the term $\dot{v}_t = -\dot{\hat{z}}_t$ is where the observation mean derivative enters.

The diagnostics for this branch must report at least the fixed null count, the maximum fixed-null derivative residual in (18.27), the active spectral gap in (18.28), the factor reconstruction residual in (18.33), and the deterministic-completion residual of the propagated points. Passing these checks means that the score differentiates the implemented structural sigma-point likelihood. It does not certify a nonlinear Hessian, HMC target, or GPU/XLA scaling claim.

18.5 Spectral Factor Derivative Contract

The preceding section assumes $\dot{C}$ and $\ddot{C}$ are available. This subsection states what those derivatives mean for an SVD/eigen factor. Suppose first that $P(\theta)$ has a smooth simple spectrum and no active hard floor:

$$P = \sum_{a=1}^n \lambda_a u_a u_a^\top, \quad \lambda_a > 0, \quad \lambda_a \neq \lambda_b \ (a \neq b).$$

For a parameter component θ_i ,

$$\dot{\lambda}_a^{(i)} = u_a^\top \dot{P}^{(i)} u_a, \quad (18.41)$$

$$\dot{u}_a^{(i)} = \sum_{b \neq a} u_b \frac{u_b^\top \dot{P}^{(i)} u_a}{\lambda_a - \lambda_b}. \quad (18.42)$$

For the column factor $c_a = \sqrt{\lambda_a} u_a$,

$$\dot{c}_a^{(i)} = \sqrt{\lambda_a} \dot{u}_a^{(i)} + \frac{\dot{\lambda}_a^{(i)}}{2\sqrt{\lambda_a}} u_a. \quad (18.43)$$

Second derivatives can be written in the same eigenbasis. Define $A_a^{(i)} = \dot{P}^{(i)} - \dot{\lambda}_a^{(i)} I$ and $A_a^{(ij)} = \ddot{P}^{(ij)} - \ddot{\lambda}_a^{(ij)} I$. Then

$$\ddot{\lambda}_a^{(ij)} = u_a^\top \ddot{P}^{(ij)} u_a + 2 \sum_{b \neq a} \frac{(u_b^\top \dot{P}^{(i)} u_a)(u_b^\top \dot{P}^{(j)} u_a)}{\lambda_a - \lambda_b}. \quad (18.44)$$

The second derivative of the eigenvector is determined by differentiating the eigen-equation and the normalization condition:

$$\ddot{u}_a^{(ij)} = -u_a \dot{u}_a^{(i)\top} \dot{u}_a^{(j)} + \sum_{b \neq a} u_b \frac{u_b^\top \left(A_a^{(ij)} u_a + A_a^{(i)} \dot{u}_a^{(j)} + A_a^{(j)} \dot{u}_a^{(i)} \right)}{\lambda_a - \lambda_b}. \quad (18.45)$$

Consequently

$$\begin{aligned} \ddot{c}_a^{(ij)} &= \sqrt{\lambda_a} \ddot{u}_a^{(ij)} + \frac{\dot{\lambda}_a^{(i)}}{2\sqrt{\lambda_a}} \dot{u}_a^{(j)} + \frac{\dot{\lambda}_a^{(j)}}{2\sqrt{\lambda_a}} \dot{u}_a^{(i)} \\ &\quad + \left(\frac{\ddot{\lambda}_a^{(ij)}}{2\sqrt{\lambda_a}} - \frac{\dot{\lambda}_a^{(i)} \dot{\lambda}_a^{(j)}}{4\lambda_a^{3/2}} \right) u_a. \end{aligned} \quad (18.46)$$

Equations (18.41)–(18.46) are the branch-local eigenderivative formulas for an eigenfactor covariance square root on a smooth simple-spectrum branch, before any active hard-floor or ordering/sign fallback is crossed.

The formulas also show why the derivative path is fragile. Eigenvector derivatives contain denominators $\lambda_a - \lambda_b$; the Hessian contains the same gaps again through $\dot{u}_a$ and through differentiation of the gap. This is the spectral-gap risk identified by matrix-backpropagation calculus [Ionescu et al., 2015]. If eigenvalues are repeated, individual eigenvectors are not uniquely differentiable. If a hard floor $\lambda_a^+ = \max(\lambda_a, \epsilon)$ is used, the

derivative is branch dependent and undefined at $\lambda_a = \epsilon$. A smooth floor $\lambda_a^+ = \phi_\epsilon(\lambda_a)$ replaces $\dot{\lambda}_a$ and $\ddot{\lambda}_a$ by

$$\dot{\lambda}_a^+ = \phi'_\epsilon(\lambda_a)\dot{\lambda}_a, \quad \ddot{\lambda}_a^+ = \phi'_\epsilon(\lambda_a)\ddot{\lambda}_a + \phi''_\epsilon(\lambda_a)\dot{\lambda}_a^{(i)}\dot{\lambda}_a^{(j)},$$

but it does not remove the eigenvector gap denominators.

For this reason BayesFilter should validate SVD factor derivatives through reconstruction identities, not merely by checking that autodiff returns finite numbers. The reconstruction target is the covariance represented by the implemented factor,

$$P_+ = CC^\top = U\Lambda_+U^\top,$$

not necessarily the pre-floor covariance P :

$$\dot{P}_+^{(i)} \stackrel{?}{=} \dot{C}^{(i)}C^\top + C\dot{C}^{(i)\top}, \quad (18.47)$$

$$\ddot{P}_+^{(ij)} \stackrel{?}{=} \ddot{C}^{(ij)}C^\top + C\ddot{C}^{(ij)\top} + \dot{C}^{(i)}\dot{C}^{(j)\top} + \dot{C}^{(j)}\dot{C}^{(i)\top}. \quad (18.48)$$

The residuals in (18.47)–(18.48), the minimum spectral gap, the active-floor set, and any fallback branch are part of the derivative output contract.

18.6 Principal-Square-Root Placement Backend

The preceding eigenderivative formulas define the historical `tf_svd_ukf` branch. BayesFilter also has a distinct experimental backend, `tf_principal_sqrt_ukf`, whose factor is the principal matrix square root rather than a signed or ordered eigenvector column factor. The two names are backend contracts, not aliases.

On a strict-SPD branch, let

$$P \in \mathbb{S}_{++}^n, \quad S = P^{1/2} = S^\top \succ 0, \quad S^2 = P.$$

For a covariance perturbation dP , the Frechet derivative of the principal square root is the unique solution dS of the Sylvester equation

$$S dS + dS S = dP. \quad (18.49)$$

Equivalently, for a parameter component θ_i ,

$$S \dot{S}^{(i)} + \dot{S}^{(i)} S = \dot{P}^{(i)}. \quad (18.50)$$

Because the eigenvalues of strict-SPD S are positive, the linear Sylvester operator $X \mapsto SX + XS$ is invertible on all matrices. This is the reason the principal-square-root derivative does not contain the active repeated-positive-eigenvalue denominators that appear in (18.42). This statement is only a strict-SPD square-root derivative statement. It is not a rank-loss theorem, not a structural-null completion rule, and not an HMC convergence result.

For second derivatives on the same strict-SPD branch, differentiate (18.50) once more:

$$S \ddot{S}^{(ij)} + \ddot{S}^{(ij)} S = \ddot{P}^{(ij)} - \dot{S}^{(i)} \dot{S}^{(j)} - \dot{S}^{(j)} \dot{S}^{(i)}. \quad (18.51)$$

Thus the same Sylvester primitive supplies the Hessian-level square-root factor derivative once $\dot{S}^{(i)}$, $\dot{S}^{(j)}$, and $\ddot{P}^{(ij)}$ are available.

The sigma-point rule has square-root gauge freedom. If $CC^\top = P$, then the standardized point cloud $\chi^{(r)} = m + C\xi^{(r)}$ matches the same first and second Gaussian moments whenever the chosen rule's offsets and weights have the required standardized moment identities. Replacing an eigenvector factor $C = U\Lambda^{1/2}$ by the principal factor $S = P^{1/2}$ is therefore a change of square-root gauge for moment placement. It does not imply that nonlinear propagated values may differ only by roundoff: after a nonlinear map g , the clouds $\{g(m + C\xi^{(r)})\}$ and $\{g(m + S\xi^{(r)})\}$ may differ. BayesFilter must therefore validate the implemented likelihood and score, not claim nonlinear value identity across gauges.

The current principal-square-root backend is strict-SPD only. It must keep fixed-null support disabled unless a separate reviewed branch specifies how structural nulls, active floors, and rank-deficient covariance laws are represented and differentiated. Required diagnostics for this backend include the Sylvester residual in (18.50), the reconstruction residual $\dot{P}^{(i)} - (\dot{S}^{(i)}S + S\dot{S}^{(i)})$, minimum eigenvalue of P , any jitter or floor policy, and eager/compiled parity for the complete value-and-score path.

The Phase 2026-06 strict-SPD implementation evidence is bounded: repeated positive-spectrum fixtures, reconstruction checks, compiled value/score smoke, and the two-candidate NK replay are engineering artifacts. They do not establish posterior correctness, HMC convergence, runtime superiority, GPU readiness, default readiness, or full baseline launch readiness.

18.7 Validation Targets for SVD Score and Hessian

The analytical derivation above leads to concrete tests:

- (i) On affine Gaussian systems, the SVD sigma-point score and Hessian must match the exact Kalman score and Hessian from Chapters 9–10.
- (ii) On small nonlinear systems, the analytical score and Hessian must match finite differences of the same SVD sigma-point likelihood.
- (iii) The factor residuals (18.47)–(18.48) must remain small on well-separated spectra.
- (iv) Tests must deliberately include clustered eigenvalues, zero eigenvalues, floor crossings, and sign/order changes, and report when the derivative is a branch derivative rather than the derivative of a smooth mathematical SVD.
- (v) HMC diagnostics must record whether gradients came from the structural analytic recursion, a custom spectral derivative, raw autodiff, or a regularized fallback.

These tests turn the phrase “SVD sigma-point Hessian” into an auditable object: a sigma-point moment derivative recursion, a solve-form likelihood Hessian, and a declared spectral-factor derivative policy.

18.8 SVD-CUT Score and Hessian

The conjugate unscented transform can be inserted into the same spectral sigma-point filter. The precise object is an SVD-placed CUT rule: use an SVD/eigen factor to place the CUT offsets, then use the ordinary sigma-point moment and likelihood recursions.

This is sometimes called CUT-SVD and sometimes SVD-CUT. The name is less important than the contract

$$P_\star = CC^\top,$$

where $P_\star$ is the covariance actually represented by the implemented factor. If eigenvalues have been floored, truncated, or otherwise regularized, $P_\star$ is the floored or truncated covariance, not necessarily the pre-regularized covariance submitted to the factor routine.

Let the SVD branch return a factor $C(\theta) \in \mathbb{R}^{n_x \times q}$ and let $Z \sim \mathcal{N}(0, I_q)$. A CUT rule in the factor coordinates supplies standardized offsets and weights

$$\{(\xi^{(r)}, w_r) : r = 0, \dots, N_{\text{CUT}} - 1\}, \quad \xi^{(r)} \in \mathbb{R}^q.$$

For CUT4-G, when $q \geq 3$,

$$N_{\text{CUT4}} = 2q + 2^q.$$

The SVD-CUT points are

$$\chi^{(r)} = m + C\xi^{(r)}, \quad r = 0, \dots, N_{\text{CUT}} - 1. \quad (18.52)$$

The quadrature approximation to an expectation under the implemented Gaussian law $X_\star = m + CZ \sim \mathcal{N}(m, P_\star)$ is

$$\mathcal{Q}_{\text{SVD-CUT}}[\varphi] = \sum_{r=0}^{N_{\text{CUT}}-1} w_r \varphi(m + C\xi^{(r)}). \quad (18.53)$$

Proposition 18.1 (Polynomial exactness under affine SVD placement). *Suppose the standardized CUT rule is exact for standard-normal polynomials on $\mathbb{R}^q$ through degree d . If $CC^\top = P_\star$, then (18.53) integrates every polynomial $p : \mathbb{R}^{n_x} \rightarrow \mathbb{R}$ of total degree at most d exactly under $X_\star \sim \mathcal{N}(m, P_\star)$. The statement remains true when $P_\star$ is singular.*

Proof. For any polynomial p of degree at most d , define

$$\psi(z) = p(m + Cz).$$

The affine composition cannot increase total degree, so ψ is a polynomial on $\mathbb{R}^q$ of degree at most d . By standardized CUT exactness,

$$\sum_r w_r \psi(\xi^{(r)}) = \mathbb{E}[\psi(Z)].$$

Substituting the definition of ψ gives

$$\sum_r w_r p(m + C\xi^{(r)}) = \mathbb{E}[p(m + CZ)] = \mathbb{E}[p(X_\star)].$$

No inverse of C is used. Hence rank deficiency of $P_\star = CC^\top$ does not affect the formal quadrature exactness statement; it only changes the affine support on which the degenerate Gaussian law lives. $\square$

This proposition is the clean mathematical answer to the accuracy question: abstracting from finite arithmetic, SVD placement does not by itself destroy CUT accuracy. It is not a recommendation to hide a structural state split inside a large singular covariance. A full zero-padded factor may reproduce the same law, but it also creates duplicate or nearly duplicate points in null directions and pushes the derivative path through the larger spectral branch.

18.8.1 SVD-CUT Moment Derivatives

The analytic gradient and Hessian follow by differentiating (18.52)–(18.53). Assume first that the CUT offsets and weights are fixed. If CUT tuning parameters are estimated, the same derivation applies after adding the explicit $\dot{\xi}^{(r)}$, $\ddot{\xi}^{(r)}$, $\dot{w}_r$, and $\ddot{w}_r$ terms.

For a parameter component θ_i , the point derivatives are

$$\dot{\chi}^{(r,i)} = \dot{m}^{(i)} + \dot{C}^{(i)}\xi^{(r)}, \quad (18.54)$$

$$\ddot{\chi}^{(r,ij)} = \ddot{m}^{(ij)} + \ddot{C}^{(ij)}\xi^{(r)}. \quad (18.55)$$

Let g_θ be the nonlinear map being integrated. In a predict step this is the transition map; in an update step it is the observation map. Write

$$y^{(r)} = g_\theta(\chi^{(r)}), \quad \bar{y} = \sum_r w_r y^{(r)}, \quad d^{(r)} = y^{(r)} - \bar{y}.$$

The propagated point derivatives are

$$\dot{y}^{(r,i)} = D_x g_\theta(\chi^{(r)}) \dot{\chi}^{(r,i)} + \partial_i g_\theta(\chi^{(r)}), \quad (18.56)$$

$$\begin{aligned} \ddot{y}^{(r,ij)} = & D_x g_\theta(\chi^{(r)}) \ddot{\chi}^{(r,ij)} + D_{xx}^2 g_\theta(\chi^{(r)}) [\dot{\chi}^{(r,i)}, \dot{\chi}^{(r,j)}] \\ & + D_{x\theta_i}^2 g_\theta(\chi^{(r)}) \dot{\chi}^{(r,j)} + D_{x\theta_j}^2 g_\theta(\chi^{(r)}) \dot{\chi}^{(r,i)} + \partial_{ij}^2 g_\theta(\chi^{(r)}). \end{aligned} \quad (18.57)$$

Therefore

$$\dot{\bar{y}}^{(i)} = \sum_r w_r \dot{y}^{(r,i)}, \quad (18.58)$$

$$\ddot{\bar{y}}^{(ij)} = \sum_r w_r \ddot{y}^{(r,ij)}. \quad (18.59)$$

Let Ω_θ denote the additive covariance in the moment being formed: Q_θ in a predict step or R_θ in an observation update. The SVD-CUT covariance moment is

$$M = \sum_r w_r d^{(r)} d^{(r)\top} + \Omega_\theta. \quad (18.60)$$

With

$$\dot{d}^{(r,i)} = \dot{y}^{(r,i)} - \dot{\bar{y}}^{(i)}, \quad \ddot{d}^{(r,ij)} = \ddot{y}^{(r,ij)} - \ddot{\bar{y}}^{(ij)},$$

its derivatives are

$$\dot{M}^{(i)} = \sum_r w_r \left[\dot{d}^{(r,i)} d^{(r)\top} + d^{(r)} \dot{d}^{(r,i)\top} \right] + \dot{\Omega}_\theta^{(i)}, \quad (18.61)$$

$$\ddot{M}^{(ij)} = \sum_r w_r \left[\ddot{d}^{(r,ij)} d^{(r)\top} + d^{(r)} \ddot{d}^{(r,ij)\top} + \dot{d}^{(r,i)} \dot{d}^{(r,j)\top} + \dot{d}^{(r,j)} \dot{d}^{(r,i)\top} \right] + \ddot{\Omega}_\theta^{(ij)}. \quad (18.62)$$

Derivation. Equations (18.54) and (18.55) are the first and second derivatives of the affine point map $\chi^{(r)} = m + C\xi^{(r)}$, using fixed $\xi^{(r)}$. Equations (18.56) and (18.57) are the first and second multivariate chain rules for $g_\theta(\chi^{(r)}(\theta))$. Equations (18.58) and (18.59) follow from linearity of the weighted sum. Finally, differentiating $d^{(r)} d^{(r)\top}$ once and twice gives

$$\partial_i (dd^\top) = \dot{d}_i d^\top + d \dot{d}_i^\top$$

and

$$\partial_{ij}^2 (dd^\top) = \ddot{d}_{ij} d^\top + d \ddot{d}_{ij}^\top + \dot{d}_i \dot{d}_j^\top + \dot{d}_j \dot{d}_i^\top,$$

which proves (18.61) and (18.62) after summing over the fixed weights and adding the derivatives of Ω_θ . $\square$

18.8.2 SVD-CUT Likelihood Gradient and Hessian

For an observation update, set $g_\theta = h_\theta$ and identify

$$\bar{z}_t = \bar{y}, \quad S_t = M, \quad v_t = y_t^{\text{obs}} - \bar{z}_t.$$

If the recorded observation does not depend on θ , then

$$\dot{v}_t^{(i)} = -\dot{\bar{z}}_t^{(i)}, \quad \ddot{v}_t^{(ij)} = -\ddot{\bar{z}}_t^{(ij)}. \quad (18.63)$$

Let $S_t w_t = v_t$. The per-time SVD-CUT log likelihood is

$$\ell_t = -\frac{1}{2} [\log \det S_t + v_t^\top S_t^{-1} v_t + n_y \log(2\pi)]. \quad (18.64)$$

Its score is

$$\partial_i \ell_t = -\frac{1}{2} [\text{tr}(S_t^{-1} \dot{S}_t^{(i)}) + 2\dot{v}_t^{(i)\top} w_t - w_t^\top \dot{S}_t^{(i)} w_t]. \quad (18.65)$$

For the Hessian, define

$$\dot{w}_t^{(j)} = \mathcal{S}_{S_t} (\dot{v}_t^{(j)} - \dot{S}_t^{(j)} w_t). \quad (18.66)$$

Then

$$T_{ij}^{\log \det} = \text{tr} \left(S_t^{-1} \ddot{S}_t^{(ij)} - S_t^{-1} \dot{S}_t^{(j)} S_t^{-1} \dot{S}_t^{(i)} \right), \quad (18.67)$$

$$T_{ij}^v = 2\ddot{v}_t^{(ij)\top} w_t + 2\dot{v}_t^{(i)\top} \dot{w}_t^{(j)}, \quad (18.68)$$

$$T_{ij}^S = 2\dot{w}_t^{(j)\top} \dot{S}_t^{(i)} w_t + w_t^\top \ddot{S}_t^{(ij)} w_t, \quad (18.69)$$

and

$$\partial_{ij}^2 \ell_t = -\frac{1}{2} [T_{ij}^{\log \det} + T_{ij}^v - T_{ij}^S]. \quad (18.70)$$

Derivation. Equations (18.61) and (18.62), with $\Omega_\theta = R_\theta$, supply $\dot{S}_t$ and $\ddot{S}_t$. Equation (18.63) supplies $\dot{v}_t$ and $\ddot{v}_t$. Differentiating the solve $S_t w_t = v_t$ gives

$$S_t \dot{w}_t^{(j)} + \dot{S}_t^{(j)} w_t = \dot{v}_t^{(j)},$$

hence (18.66). The differential identities $\partial_i \log \det S = \text{tr}(S^{-1} \dot{S}_i)$ and $\partial_i S^{-1} = -S^{-1} \dot{S}_i S^{-1}$ give (18.65). Differentiating the three pieces of (18.64) a second time gives (18.67) and (18.70). This is the same solve-form likelihood Hessian as Chapter 10; the SVD-CUT-specific work is the construction of $v_t, S_t, \dot{v}_t, \dot{S}_t, \ddot{v}_t, \ddot{S}_t$ from the CUT point cloud. $\square$

All dependence on the SVD backend enters through $C, \dot{C}, \ddot{C}$. Consequently the derivative must reconstruct the same implemented covariance branch:

$$\dot{P}_\star^{(i)} = \dot{C}^{(i)} C^\top + C \dot{C}^{(i)\top},$$

and

$$\ddot{P}_\star^{(ij)} = \ddot{C}^{(ij)} C^\top + C \ddot{C}^{(ij)\top} + \dot{C}^{(i)} \dot{C}^{(j)\top} + \dot{C}^{(j)} \dot{C}^{(i)\top}.$$

These are exactly the reconstruction identities (18.47)–(18.48). If a hard eigenvalue floor is active, the equations describe the branch derivative of the floored covariance $P_\star$, not the derivative of the pre-floor covariance. If eigenvalues are clustered, repeated, reordered, or sign-flipped, the smooth-branch formulas in Section 18.5 may fail or become ill-conditioned.

18.8.3 Point Count, GPU, and XLA

Modern GPUs and XLA make large sigma-point clouds much less painful than scalar Python loops, but they do not make the point count disappear. The CUT map evaluations are embarrassingly parallel over points and chains, and the point dimension is static when q is fixed. This fits the XLA discipline in Chapter 44: construct a fixed tensor of offsets, evaluate the transition or observation map in batch, and reduce over the point axis inside the compiled value-and-gradient path.

The remaining costs still scale with

$$N_{\text{CUT4}} = 2q + 2^q.$$

For one observation update with output dimension n_y , the value-side moment assembly contains

$$O(N_{\text{CUT}} c_h) \quad \text{map work,} \quad O(N_{\text{CUT}} n_y^2) \quad \text{innovation-covariance reductions,}$$

plus $O(N_{\text{CUT}} n_x n_y)$ if the cross covariance and Kalman gain are formed. With p parameters, a direct analytic score stores or recomputes $O(N_{\text{CUT}} p n_y)$ propagated first derivatives, and a direct Hessian stores or recomputes $O(N_{\text{CUT}} p^2 n_y)$ propagated second derivatives, in addition to the SVD-factor derivative work. XLA can fuse many element-wise operations and run the point axis in parallel, but memory traffic, reductions, spectral factorization, compile time, and the sequential time scan remain real constraints.

Thus the correct engineering conclusion is conditional. A large number of CUT points may be acceptable on modern GPU/XLA hardware when the stochastic factor dimension q is modest, the maps are sufficiently expensive to amortize compile and launch overhead, tensor shapes are static, and chains or particles provide enough batch parallelism. It is not generally acceptable to collapse a large DSGE state into a full-state CUT rule and rely on the GPU to absorb the 2^q corner set. The preferred design remains structural SVD-CUT: apply higher-order CUT quadrature only to the declared stochastic integration block, then complete deterministic coordinates through the structural transition map and use SVD regularization only where the covariance factor actually needs it.

18.9 Status Labels for SVD-HMC

The SVD/HMC gap-closure plan introduced four labels that BayesFilter should reuse:

filter-correct the likelihood and gradients agree with a reference behavior on controlled cases;

sampler-usable short or medium HMC runs produce finite chains and useful diagnostics;

converged strict multi-chain posterior diagnostics pass on the intended target;

production-XLA-gated fixed-solution, full-solve, boundary rejection, gradient, and tiny HMC paths compile and run under the production compilation policy.

These labels prevent a recurring mistake: a clean smoke run is not a convergence result, and a finite likelihood is not a validated HMC gradient.

18.10 BayesFilter Policy

BayesFilter may use SVD sigma-point filters as robust approximate likelihood infrastructure and as a diagnostic backend. To use them as HMC production targets, the project must additionally provide:

- (i) affine-Gaussian recovery against the exact Kalman backend;
- (ii) nonlinear controlled recovery against dense quadrature or a trusted reference;
- (iii) finite value and finite gradient stress tests across invalid and near-boundary parameter proposals;
- (iv) eigenvalue or singular-value gap telemetry during HMC;
- (v) eager/compiled parity for the complete value-and-gradient path;
- (vi) a custom-gradient or analytic-gradient policy if raw spectral autodiff fails the preceding checks.

For industrial-size models, the conservative default is prevention rather than debugging: certify a derivative path before relying on SVD-tape gradients in a long HMC run.

Chapter 19

Structural Deterministic Dynamics in DSGE Filtering

Production warning. Do not implement DSGE nonlinear filtering by silently treating every declared state coordinate as an exchangeable noisy Gaussian coordinate. Many DSGE models contain a structural split between shock-driven exogenous states and endogenous or accounting states that are deterministic conditional on lagged states, controls, and current shocks. This chapter proves the law-level reason: for mixed structural models, the nonlinear prediction target is the pushforward of the lagged filtering law and the current innovation law through the structural transition. A full-state additive-noise approximation is therefore valid only when it preserves that law or is explicitly labeled and tested as an approximation.

19.1 The Structural Split

The state vector in a DSGE likelihood is not merely a numerical vector. It usually carries economic timing. In the language of the structural state partition from Chapter 2, a common DSGE representation separates a declared stochastic block from a deterministic-completion block.

Definition 19.1 (Structural transition notation). *Throughout this chapter, write the full structural state as $x_t = (m_t, k_t)$, where m_t is the declared stochastic or exogenous block and k_t is the deterministic-completion block. A mixed structural transition is*

$$m_t = T_m(m_{t-1}, \varepsilon_t; \theta), \quad (19.1)$$

$$k_t = T_k(k_{t-1}, m_{t-1}, m_t; \theta), \quad (19.2)$$

$$x_t = (m_t, k_t). \quad (19.3)$$

Equivalently, define the structural map

$$F_\theta(x_{t-1}, \varepsilon_t) = (T_m(m_{t-1}, \varepsilon_t; \theta), T_k(k_{t-1}, m_{t-1}, T_m(m_{t-1}, \varepsilon_t; \theta); \theta)). \quad (19.4)$$

Let $\pi_{t-1|t-1}^x$ denote the filtering law of x_{t-1} and let ν_θ^ε denote the law of ε_t . The pre-transition law is

$$\pi_t^a = \pi_{t-1|t-1}^x \otimes \nu_\theta^\varepsilon$$

when the current innovation is conditionally independent of the lagged filtering law. The predictive law of x_t is

$$\pi_{t|t-1}^x(B) = \Pr_\theta(x_t \in B \mid y_{1:t-1})$$

for measurable state sets B . The pushforward of π_t^a by F_θ is the measure

$$(F_\theta)_\# \pi_t^a(B) = \pi_t^a(F_\theta^{-1}(B)).$$

To place sigma points on a random variable means to choose weighted points $\{a_t^{(j)}, w_j^{(m)}, w_j^{(c)}\}$ whose weighted mean and covariance match the Gaussian approximation to that variable, then propagate $x_t^{(j)} = F_\theta(a_t^{(j)})$ point by point.

Thus, for mixed structural models,

$$\pi_{t|t-1}^x = (F_\theta)_\# \pi_t^a,$$

or, in density notation when the densities exist,

$$p_\theta(x_t \mid y_{1:t-1}) = \iint \delta(x_t - F_\theta(x_{t-1}, \varepsilon_t)) p(\varepsilon_t) p_\theta(x_{t-1} \mid y_{1:t-1}) d\varepsilon_t dx_{t-1}.$$

In a perturbation solution, the same economics may also be represented by a compact law of motion

$$x_t = h_x x_{t-1} + \eta \varepsilon_t$$

at first order, with zero or rank-deficient innovation rows for the deterministic-completion block. The compact representation is useful, but it must not erase the timing contract or the structural metadata described in Chapter 4. In nonlinear filtering, the correct random input is therefore the innovation or declared stochastic block. The endogenous block is then generated by the model-implied deterministic-completion map. Here “deterministic” means no independent innovation in that coordinate once (x_{t-1}, ε_t) is fixed; it does not mean zero predictive variance conditional only on the lagged filtering state.

This is the distinction emphasized in the DSGE filtering literature. It is also an instance of the generic BayesFilter structural-transition contract from Chapter 2: the backend integrates over the declared stochastic block and completes the deterministic block pointwise. Standard Bayesian DSGE expositions write the likelihood in state-space form after the model solution has supplied a transition law for predetermined variables and an observation equation for measured macroeconomic series [Herbst and Schorfheide, 2015, An and Schorfheide, 2007]. Pruned perturbation methods make the same point in a different language: the first-order component is propagated stochastically, while the higher-order correction is propagated by a separate deterministic recursion driven by the lower-order state [Kim et al., 2008, Andreasen et al., 2018]. The filtering backend must honor the state-space law supplied by the structural solution, not merely the dimension of the state vector.

19.1.1 Literature Map for This Chapter

The chapter uses four strands of literature, and each supports a different claim.

- (i) **State-space likelihoods and DSGE solutions.** Standard state-space and Bayesian DSGE references provide the filtering likelihood setup and the distinction between a structural model solution and the numerical recursion used to evaluate it [Durbin and Koopman, 2012, Herbst and Schorfheide, 2015, An and Schorfheide, 2007].

- (ii) **Pruned nonlinear DSGE state spaces.** Pruning papers show that nonlinear DSGE approximations naturally separate lower-order stochastic propagation from higher-order deterministic correction recursions [Kim et al., 2008, Andreassen et al., 2018]. That literature motivates the structural partition used here, but it does not by itself choose a sigma-point implementation.
- (iii) **Unscented and sigma-point filtering.** Julier and Uhlmann justify deterministic sigma points as a moment-matching approximation for nonlinear transformations, while van der Merwe’s sigma-point Kalman-filter treatment supplies the dynamic state-space notation and augmented-noise filtering pattern used below [Julier and Uhlmann, 1996, 1997, van der Merwe, 2004].
- (iv) **Particle and factor backends.** Particle-filter references support the same law-level distinction in a simulation backend: sample the declared transition and weight by the observation density [Gordon et al., 1993, Doucet et al., 2001, Andrieu et al., 2010]. Square-root and SVD chapters in this monograph address numerical factorization and derivative risk; they do not replace the structural-law requirement.

The contribution of this chapter is to connect these strands: in a mixed DSGE transition, sigma-point, SVD sigma-point, cubature, and particle methods should approximate the pushforward law generated by the structural map $F_\theta(x_{t-1}, \varepsilon_t)$.

19.2 Why Linear Kalman Filtering Can Hide the Problem

For a linear Gaussian state-space model,

$$x_t = c + Fx_{t-1} + u_t, \quad u_t \sim \mathcal{N}(0, Q), \quad (19.5)$$

$$y_t = a + Hx_t + e_t, \quad e_t \sim \mathcal{N}(0, R), \quad (19.6)$$

the Kalman filter only needs the conditional mean and covariance of $x_t \mid x_{t-1}$. If a DSGE solution implies a singular or nearly singular Q , the exact linear Gaussian likelihood remains well defined when the collapsed representation preserves the same conditional law induced by the structural transition. Numerically stable implementations may then handle that degenerate conditional Gaussian law through diffuse, square-root, solve-form, or carefully regularized recursions as appropriate [Durbin and Koopman, 2012].

This is why the following collapsed first-order representation can be acceptable in an exact Kalman path:

$$Q = \eta\eta^\top, \quad P_{t|t-1} = h_x P_{t-1|t-1} h_x^\top + Q.$$

The key point is not that structural determinism disappears. The key point is that the collapsed linear representation still encodes the same one-step conditional Gaussian law. Deterministic-completion coordinates are not wrong merely because they are included in x_t . They obtain randomness through their dependence on lagged stochastic coordinates, and their direct innovation variance may be zero. The exact linear Kalman recursion uses only the correct first two conditional moments.

The danger is that this success can train the implementation culture into a bad habit: it makes the state vector look like a homogeneous Gaussian vector. That habit becomes

wrong when the backend changes from exact linear filtering to nonlinear sigma-point or particle filtering. The relevant design decision is then the integration space recorded in the filter metadata, not merely the numerical dimension of x_t .

A related misunderstanding should be removed explicitly. A deterministic-completion coordinate need not have zero one-step predictive variance given the lagged state. It can inherit randomness through the current stochastic block. In the Chapter 2 AR(2) lag-stack example, the lag coordinate has no new innovation of its own but still remains random under the lagged filtering distribution. Likewise, in DSGE models with (19.1)–(19.2), k_t may satisfy

$$\text{Var}(k_t \mid x_{t-1}) > 0$$

even though k_t has no independent innovation once (x_{t-1}, ε_t) is fixed.

19.3 Why Nonlinear Filters Must Respect the Structural Map

Sigma-point and particle filters approximate a transition law, not just a matrix covariance update. In a generic nonlinear state-space model,

$$x_t = f_\theta(x_{t-1}, \varepsilon_t), \quad y_t = g_\theta(x_t, e_t),$$

the filter has to decide which variables are integrated over and which are deterministic transforms. In BayesFilter language, this is the choice of integration space recorded in the filter metadata. For DSGE models with (19.1)–(19.2), the natural nonlinear prediction step is:

- (i) represent the filtering distribution of the lagged structural state;
- (ii) draw, integrate, or choose quadrature points for the exogenous innovation ε_t or current exogenous state m_t ;
- (iii) compute each corresponding endogenous state k_t using $T_k(k_{t-1}, m_{t-1}, m_t; \theta)$;
- (iv) evaluate the observation map on the resulting structurally valid points.

The endogenous coordinates should not receive arbitrary independent “sigma-point noise” merely because they are entries of x_t . If a sigma point moves an accounting identity or predetermined endogenous state outside the model-implied support of the structural transition, the filter is no longer approximating the intended DSGE state-space model. It is approximating a different artificial model relative to the declared structural law. This is exactly the approximation boundary from Chapter 16: quadrature may approximate the target law, but injecting additional stochastic coordinates changes the target unless that change is declared and justified.

Particle-filter literature makes the same distinction operationally. A particle filter propagates particles through the state transition and weights them by the observation density [Gordon et al., 1993, Doucet et al., 2001, Andrieu et al., 2010]. Written in the structural notation, a basic propagation and weighting step is

$$x_t^{(i)} = F_\theta\left(x_{t-1}^{(a_i)}, \varepsilon_t^{(i)}\right), \quad w_t^{(i)} \propto p_\theta\left(y_t \mid x_t^{(i)}\right).$$

When part of the transition is deterministic, particles should be propagated by sampling the declared stochastic block and then deterministically completing the remaining coordinates. Adding artificial noise to deterministic coordinates can be used only as a declared proposal or regularization device, with the proposal/target correction and approximation status stated explicitly. It is not the default filtering law.

19.4 Reusable BayesFilter Structural Filtering Step

The doctrine above can be stated as a reusable backend contract. At every time step, the filter should treat the structural transition as the object being approximated:

$$x_t = F_\theta(x_{t-1}, \varepsilon_t), \quad y_t \sim p_\theta(y_t | x_t).$$

The filtered state is still the full state x_t . The integration variable is the pre-transition uncertainty that generates the predictive law.

Algorithm: structural_filter_step.

- (1) Start from the filtered approximation to $p_\theta(x_{t-1} | y_{1:t-1})$ and the declared innovation law $p_\theta(\varepsilon_t)$.
- (2) Form sigma points, quadrature points, particles, or linearization points in the declared integration space: (x_{t-1}, ε_t) , $(s_{t-1}, d_{t-1}, \varepsilon_t)$, or an equivalent stochastic-state parameterization.
- (3) For each point, call the structural transition map:

$$s_t^{(j)} = T_s(s_{t-1}^{(j)}, \varepsilon_t^{(j)}; \theta), \quad d_t^{(j)} = T_d(d_{t-1}^{(j)}, s_{t-1}^{(j)}, s_t^{(j)}; \theta).$$

Deterministic-completion coordinates are outputs of this map, not independent new stochastic coordinates.

- (4) Pack $x_t^{(j)} = \text{pack}(s_t^{(j)}, d_t^{(j)}, a_t^{(j)})$ and evaluate the observation map or density on these structurally valid points.
- (5) Apply the selected update: exact Kalman, Gaussian sigma-point closure, particle weighting/resampling, or another declared backend.
- (6) Return the likelihood contribution, filtered approximation, and metadata recording integration space, deterministic-completion policy, approximation label, derivative policy, and finite-failure diagnostics.

This algorithm is intentionally backend-neutral. For a linear Gaussian Kalman backend, the structural transition may collapse to matrices (F, Q) as long as the same conditional Gaussian law is preserved. For UKF, CKF, SVD sigma-point, and particle backends, the pointwise structural transition is the default. A mixed-model full-state Gaussian approximation may still be offered as a diagnostic or legacy path, but it must carry an approximation label and must be tested against structural references.

19.5 Degenerate Transitions Are Structural

Degenerate transition covariance is not an exceptional corner case in DSGE models. It is often the mathematical expression of timing, identities, and endogenous accumulation. Capital, debt, lagged output growth, risk-premium terms, and other predetermined variables may be deterministic conditional on the previous state and current shocks. A full covariance matrix with small positive diagonal padding can keep linear algebra routines alive, but it does not by itself define the intended economic transition. BayesFilter therefore separates three ideas that are often confused in practice:

- (i) an exact degenerate transition law that already matches the structural model;
- (ii) numerical regularization used to evaluate that law stably; and
- (iii) an altered transition law that injects new stochastic degrees of freedom into deterministic-completion coordinates.

Only the first two preserve the original structural model automatically.

The implementation policy is therefore:

- distinguish structural determinism from numerical regularization;
- never let a nugget or PSD projection change the declared stochastic dimension without a written approximation label;
- expose deterministic blocks in diagnostics, rather than burying them under a full-rank covariance;
- test degenerate and rank-deficient transition cases directly.

This point is separate from the SVD spectral-gradient issue in Chapter 18. SVD factors may improve value-side robustness for nearly singular covariances, and an SVD sigma-point filter can be the right numerical backend for a structurally declared sigma-point rule. But SVD is a factorization choice, not a state-law declaration. It repairs the reviewer’s degeneracy concern only if the sigma-point backend factors the covariance of the correct integration variable, such as (x_{t-1}, ε_t) , and then completes deterministic coordinates pointwise. If the backend first invents a full-rank post-transition covariance for deterministic-completion coordinates, an SVD factor merely represents that altered law more stably.

19.6 Standard UKF, Structural UKF, and the Predictive Law

19.6.1 The Original Unscented-Transform Pattern

Julier and Uhlmann’s original unscented-transform construction starts from a random variable $X \in \mathbb{R}^L$ with mean $\bar{x}$ and covariance P_x and approximates the moments of $Y = g(X)$ by propagating deterministic sigma points through g [Julier and Uhlmann, 1996, 1997]. The (α, β, κ) scaled rule used for the formulas below belongs to the later

sigma-point Kalman filter convention [van der Merwe, 2004]. In the notation used here, one convenient scaled UKF rule forms

$$\chi_0 = \bar{x}, \quad (19.7)$$

$$\chi_i = \bar{x} + \left[\sqrt{(L + \lambda)P_x} \right]_i, \quad i = 1, \dots, L, \quad (19.8)$$

$$\chi_{i+L} = \bar{x} - \left[\sqrt{(L + \lambda)P_x} \right]_i, \quad i = 1, \dots, L, \quad (19.9)$$

where $\left[\sqrt{(L + \lambda)P_x} \right]_i$ denotes the i th column or row of the chosen matrix square root. With

$$\lambda = \alpha^2(L + \kappa) - L,$$

the usual scaled weights are

$$w_0^{(m)} = \frac{\lambda}{L + \lambda}, \quad w_0^{(c)} = \frac{\lambda}{L + \lambda} + 1 - \alpha^2 + \beta, \quad (19.10)$$

$$w_i^{(m)} = w_i^{(c)} = \frac{1}{2(L + \lambda)}, \quad i = 1, \dots, 2L. \quad (19.11)$$

The propagated points are $v_i = g(\chi_i)$, and the transformed moments are approximated by

$$\bar{y} \approx \sum_{i=0}^{2L} w_i^{(m)} v_i, \quad (19.12)$$

$$P_y \approx \sum_{i=0}^{2L} w_i^{(c)} (v_i - \bar{y})(v_i - \bar{y})^\top. \quad (19.13)$$

The early unscented-transform report analyzes this construction by Taylor expansion: the method matches the input mean and covariance and pushes the resulting deterministic ensemble through the nonlinear map, yielding lower-order moment accuracy that improves on first-order linearization under the smoothness conditions stated there [Julier and Uhlmann, 1996]. Van der Merwe's sigma-point Kalman filter treatment uses the same family logic: sigma points are selected to capture the important first and second moments of the prior Gaussian random variable and then weighted after nonlinear propagation [van der Merwe, 2004].

This chapter uses that scaled UKF convention for exposition. Later UKF, CDKF, and square-root variants differ in point placement, weights, and factorization details, but the structural question is invariant: which random variable is the sigma-point rule approximating? If the wrong variable is chosen, the transform may approximate the wrong law accurately.

In the additive-noise filtering setting, the unscented-transform idea is usually applied to an augmented Gaussian variable containing the prior state and current additive-noise variables. The sigma points therefore approximate the variables that generate the next predicted state and predicted observation. For mixed structural DSGE transitions, that same principle is exactly why the sigma-point variable is (x_{t-1}, ε_t) or an equivalent pre-transition parameterization, not an artificially perturbed post-transition full state.

19.6.2 Standard Additive-Noise UKF Algorithm

To avoid confusion, write the additive-noise UKF pattern explicitly. Suppose the model is declared as

$$x_t = f_\theta(x_{t-1}, u_t), \quad y_t = h_\theta(x_t, e_t),$$

with independent Gaussian state innovation $u_t \sim \mathcal{N}(0, Q_t)$ and measurement innovation $e_t \sim \mathcal{N}(0, R_t)$. Given a previous Gaussian filtering approximation, one additive-noise UKF step is:

U1. Inputs. Start from

$$x_{t-1} \mid y_{1:t-1} \approx \mathcal{N}(m_{t-1|t-1}, P_{t-1|t-1}).$$

The implementation receives f_θ , h_θ , Q_t , R_t , the observed value y_t , sigma-point parameters (α, β, κ) , and a declared covariance-factorization policy.

U2. Augmented variable. Form

$$b_t = (x_{t-1}, u_t, e_t), \quad b_t \mid y_{1:t-1} \approx \mathcal{N}(\mu_b, P_b),$$

with

$$\mu_b = (m_{t-1|t-1}, 0, 0), \quad P_b = \text{blockdiag}(P_{t-1|t-1}, Q_t, R_t).$$

If a variant handles measurement noise by adding R_t after observation propagation, omit e_t from b_t and add R_t in step U6.

U3. Sigma points. Generate $\{b_t^{(j)}, w_j^{(m)}, w_j^{(c)}\}_{j=0}^{2L_b}$ from (μ_b, P_b) using (19.9)–(19.11). Write $b_t^{(j)} = (x_{t-1}^{(j)}, u_t^{(j)}, e_t^{(j)})$.

U4. State propagation. Propagate

$$x_t^{(j)} = f_\theta(x_{t-1}^{(j)}, u_t^{(j)}).$$

Compute

$$\hat{x}_{t|t-1} = \sum_j w_j^{(m)} x_t^{(j)}, \quad P_{xx,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_{t|t-1})(x_t^{(j)} - \hat{x}_{t|t-1})^\top.$$

U5. Observation propagation. Form

$$z_t^{(j)} = h_\theta(x_t^{(j)}, e_t^{(j)})$$

or $z_t^{(j)} = h_\theta(x_t^{(j)}, 0)$ with R_t added below, depending on the chosen variant.

U6. Innovation moments. Compute

$$\hat{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \tag{19.14}$$

$$S_t = \sum_j w_j^{(c)} (z_t^{(j)} - \hat{z}_t)(z_t^{(j)} - \hat{z}_t)^\top + R_t^{\text{add}}, \tag{19.15}$$

$$P_{xz,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_{t|t-1})(z_t^{(j)} - \hat{z}_t)^\top, \tag{19.16}$$

where $R_t^{\text{add}} = 0$ if measurement noise was included in b_t and $R_t^{\text{add}} = R_t$ otherwise.

U7. Gaussian update and likelihood. With $v_t = y_t - \hat{z}_t$,

$$K_t = P_{xz,t} S_t^{-1}, \quad (19.17)$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t v_t, \quad (19.18)$$

$$P_{t|t} = P_{xx,t} - K_t S_t K_t^\top, \quad (19.19)$$

$$\ell_t = -\frac{1}{2} [d_y \log(2\pi) + \log |S_t| + v_t^\top S_t^{-1} v_t]. \quad (19.20)$$

U8. Return metadata. Return $(\ell_t, \hat{x}_{t|t}, P_{t|t})$ with metadata recording the augmented variable b_t , the factorization method, sigma-point parameters, whether measurement noise was augmented or added, and any PSD/jitter/finiteness interventions.

This is the textbook pattern many readers have in mind. It is perfectly natural when the declared state transition itself is an additive-noise model on the full state.

19.6.3 Structural UKF Algorithm

For a mixed structural model, the update algebra remains the same, but the sigma-point variable is chosen differently because the predictive law is generated differently. For the structural transition (19.1)–(19.2), a structural UKF step is:

S1. Inputs. Start from the filtering approximation

$$x_{t-1} \mid y_{1:t-1} \approx \mathcal{N}(\mu_{t-1}, P_{t-1}).$$

The implementation receives T_m , T_k , the observation map h_θ , the innovation law $\varepsilon_t \sim \mathcal{N}(0, \Sigma_{\varepsilon,t})$, observation covariance R_t , the observed value y_t , sigma-point parameters, and a factorization policy.

S2. Pre-transition variable. Form

$$a_t = (x_{t-1}, \varepsilon_t), \quad a_t \mid y_{1:t-1} \approx \mathcal{N}(\mu_a, P_a),$$

with

$$\mu_a = (\mu_{t-1}, 0), \quad P_a = \text{blockdiag}(P_{t-1}, \Sigma_{\varepsilon,t}).$$

This is the sigma-point variable because $\pi_{t|t-1}^x = (F_\theta)_\# \pi_t^a$.

S3. Sigma points. Generate $\{a_t^{(j)}, w_j^{(m)}, w_j^{(c)}\}_{j=0}^{2L_a}$ from (μ_a, P_a) . Write $a_t^{(j)} = (m_{t-1}^{(j)}, k_{t-1}^{(j)}, \varepsilon_t^{(j)})$.

S4. Stochastic-block propagation. For each point compute

$$m_t^{(j)} = T_m(m_{t-1}^{(j)}, \varepsilon_t^{(j)}; \theta).$$

S5. Deterministic completion. For the same point compute

$$k_t^{(j)} = T_k(k_{t-1}^{(j)}, m_{t-1}^{(j)}, m_t^{(j)}; \theta), \quad x_t^{(j)} = (m_t^{(j)}, k_t^{(j)}).$$

There is no independent sigma-point coordinate for k_t unless the model declares one.

S6. State moments. Compute

$$\hat{x}_{t|t-1} = \sum_j w_j^{(m)} x_t^{(j)}, \quad P_{xx,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_{t|t-1})(x_t^{(j)} - \hat{x}_{t|t-1})^\top.$$

S7. Observation points and moments. Evaluate $z_t^{(j)} = h_\theta(x_t^{(j)}, 0)$ or the appropriate noise-augmented observation variant, then compute

$$\hat{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \quad (19.21)$$

$$S_t = \sum_j w_j^{(c)} (z_t^{(j)} - \hat{z}_t)(z_t^{(j)} - \hat{z}_t)^\top + R_t, \quad (19.22)$$

$$P_{xz,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_{t|t-1})(z_t^{(j)} - \hat{z}_t)^\top. \quad (19.23)$$

S8. Gaussian update and likelihood. Apply exactly the update from U7:

$$v_t = y_t - \hat{z}_t, \quad K_t = P_{xz,t} S_t^{-1}, \quad \hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t v_t,$$

with $P_{t|t} = P_{xx,t} - K_t S_t K_t^\top$ and the same Gaussian innovation log-likelihood formula.

S9. Return metadata. Return the likelihood contribution and filtered Gaussian approximation with metadata recording `integration_space` equal to (x_{t-1}, ε_t) , `deterministic_completion` equal to `pointwise`, structural identity diagnostics, sigma-point parameters, factorization method, and any approximation label.

Structural UKF is not a different Kalman-gain formula. It is the same Gaussian update algebra applied to a predictive sigma-point cloud that respects the structural transition law.

19.6.4 UKF in Sequential-Monte-Carlo Kernel Notation

The same point can be expressed in the probability-kernel notation used in [Chopin and Papaspiliopoulos \[2020\]](#). A state-space model is specified by an initial law $P_{\theta,0}(dx_0)$, transition kernels $P_{\theta,t}(x_{t-1}, dx_t)$, and observation densities $f_{\theta,t}(y_t | x_t)$. In the bootstrap Feynman–Kac formalism, the transition kernel is

$$M_{\theta,t}(x_{t-1}, dx_t) = P_{\theta,t}(x_{t-1}, dx_t),$$

and the potential is

$$G_{\theta,t}(x_t) = f_{\theta,t}(y_t | x_t).$$

If η_{t-1} denotes the filtering law of x_{t-1} given $y_{1:t-1}$, the exact one-step recursion is

$$\xi_t(dx_t) = \int \eta_{t-1}(dx_{t-1}) P_{\theta,t}(x_{t-1}, dx_t), \quad (19.24)$$

$$\lambda_t = \xi_t(G_{\theta,t}) = \int G_{\theta,t}(x_t) \xi_t(dx_t), \quad (19.25)$$

$$\eta_t(dx_t) = \frac{G_{\theta,t}(x_t) \xi_t(dx_t)}{\xi_t(G_{\theta,t})}. \quad (19.26)$$

Equations (19.24)–(19.26) are the filtering target. A particle filter approximates the measures in these equations by random weighted particles. A UKF instead approximates the same prediction/update recursion by deterministic quadrature and Gaussian projection.

For the mixed structural transition in (19.4), the transition kernel is the pushforward kernel

$$P_{\theta,t}^{\text{str}}(x, B) = \int \mathbf{1}\{F_{\theta}(x, \varepsilon) \in B\} \nu_{\theta,t}^{\varepsilon}(d\varepsilon) = \int \delta_{F_{\theta}(x, \varepsilon)}(B) \nu_{\theta,t}^{\varepsilon}(d\varepsilon), \quad (19.27)$$

for measurable state sets B . This is a perfectly valid probability kernel even when it is singular with respect to Lebesgue measure. Kernel notation is therefore useful here: it does not force a degenerate structural transition to pretend that it has a full-dimensional transition density. The predictive integral in (19.24) becomes, for any test function ψ ,

$$\xi_t^{\text{str}}(\psi) = \iint \psi(F_{\theta}(x_{t-1}, \varepsilon_t)) \eta_{t-1}(dx_{t-1}) \nu_{\theta,t}^{\varepsilon}(d\varepsilon_t). \quad (19.28)$$

Thus the kernel version of the structural statement is:

$$P_{\theta,t} = P_{\theta,t}^{\text{str}}, \quad M_{\theta,t} = P_{\theta,t}^{\text{str}}, \quad G_{\theta,t}(x) = f_{\theta,t}(y_t \mid x).$$

Now suppose the incoming filter is approximated by a Gaussian $\hat{\eta}_{t-1} = \mathcal{N}(\hat{m}_{t-1}, \hat{P}_{t-1})$. Let

$$\hat{\alpha}_t(da) = \hat{\eta}_{t-1}(dx_{t-1}) \nu_{\theta,t}^{\varepsilon}(d\varepsilon_t), \quad a = (x_{t-1}, \varepsilon_t).$$

The structural UKF replaces integrals against $\hat{\alpha}_t$ by the deterministic sigma-point rule

$$\mathcal{U}_t^{(r)}[\psi] = \sum_j w_j^{(r)} \psi(a_t^{(j)}), \quad r \in \{m, c\}.$$

With

$$x_t^{(j)} = F_{\theta}(a_t^{(j)}), \quad z_t^{(j)} = h_{\theta}(x_t^{(j)}),$$

the UKF predictive and observation moments are

$$\hat{m}_t^- = \sum_j w_j^{(m)} x_t^{(j)}, \quad (19.29)$$

$$\hat{P}_t^- = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{m}_t^-)(x_t^{(j)} - \hat{m}_t^-)^{\top}, \quad (19.30)$$

$$\hat{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \quad (19.31)$$

$$\hat{S}_t = \sum_j w_j^{(c)} (z_t^{(j)} - \hat{z}_t)(z_t^{(j)} - \hat{z}_t)^{\top} + R_t, \quad (19.32)$$

$$\hat{P}_{xz,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{m}_t^-)(z_t^{(j)} - \hat{z}_t)^{\top}. \quad (19.33)$$

Equivalently, UKF builds the Gaussian joint approximation

$$\hat{J}_t(dx_t, dy_t) = \mathcal{N}\left(\begin{bmatrix} x_t \\ y_t \end{bmatrix}; \begin{bmatrix} \hat{m}_t^- \\ \hat{z}_t \end{bmatrix}, \begin{bmatrix} \hat{P}_t^- & \hat{P}_{xz,t} \\ \hat{P}_{xz,t}^{\top} & \hat{S}_t \end{bmatrix}\right) dx_t dy_t. \quad (19.34)$$

The UKF likelihood factor and update are then the marginal and conditional Gaussian objects induced by (19.34):

$$\widehat{\lambda}_t^{\text{UKF}} = \varphi_{d_y}(y_t; \widehat{z}_t, \widehat{S}_t), \quad (19.35)$$

$$\begin{aligned} \widehat{\eta}_t^{\text{UKF}}(dx_t) &= \widehat{J}_t(dx_t \mid y_t) \\ &= \mathcal{N}\left(dx_t; \widehat{m}_t^- + \widehat{P}_{xz,t} \widehat{S}_t^{-1}(y_t - \widehat{z}_t), \widehat{P}_t^- - \widehat{P}_{xz,t} \widehat{S}_t^{-1} \widehat{P}_{xz,t}^\top\right). \end{aligned} \quad (19.36)$$

The corresponding contribution to the log likelihood is $\log \widehat{\lambda}_t^{\text{UKF}}$. If measurement noise is augmented rather than added, replace $a = (x_{t-1}, \varepsilon_t)$ by $a = (x_{t-1}, \varepsilon_t, e_t)$, set $z_t^{(j)} = h_\theta(F_\theta(x_{t-1}^{(j)}, \varepsilon_t^{(j)}), e_t^{(j)})$, and omit the extra R_t term in (19.33).

This notation also states the equivalence condition for collapsed SVD constructions. If a full-state degenerate SVD implementation uses another kernel

$$\widetilde{P}_{\theta,t}(x, B) = \int \delta_{\widetilde{F}_\theta(x, \eta)}(B) \widetilde{\nu}_{\theta,t}(d\eta),$$

then it is a law-equivalent implementation of the structural model only when

$$\widetilde{P}_{\theta,t}(x, B) = P_{\theta,t}^{\text{str}}(x, B) \quad \text{for the relevant } x \text{ and measurable } B, \quad (19.37)$$

or at least when the two kernels induce the same predictive measure from the current filtering approximation. Equality of a covariance matrix, or existence of an SVD factor for a singular covariance, is not by itself equality of probability kernels. This is the kernel-notation version of the structural warning in this chapter.

19.6.5 Why These Algorithms Differ

Object	Standard UKF	additive-noise	Structural UKF
Filtered object	Full state x_t		Full state x_t
Sigma-point variable	Declared augmented additive-noise variable (x_{t-1}, u_t, e_t)		Pre-transition structural variable (x_{t-1}, ε_t)
Direct process noise	Whatever the additive model declares		Only the declared stochastic block
Deterministic coordinates	None unless constraints are declared		Completed by T_k pointwise
Target law	Additive-noise predictive law		Structural pushforward law
Failure if misused	Moment approximation error		Model-law error plus moment approximation error
SVD/square-root role	Covariance factorization		Covariance factorization, not structural repair

The update equations are shared. The exact difference is upstream: which random variable receives sigma points and which map generates the predictive state cloud.

19.6.6 Why the Sigma-Point Variable Uses Pre-Transition Uncertainty

Proposition 19.1 (Predictive pushforward and sigma-point space). *Under the structural transition*

$$m_t = T_m(m_{t-1}, \varepsilon_t), \quad (19.38)$$

$$k_t = T_k(k_{t-1}, m_{t-1}, m_t), \quad (19.39)$$

$$x_t = (m_t, k_t), \quad (19.40)$$

with innovation law $p(\varepsilon_t)$ independent of $x_{t-1} \mid y_{1:t-1}$, the following hold:

- (i) the one-step predictive law $p(x_t \mid y_{1:t-1})$ is the pushforward of the joint law of (x_{t-1}, ε_t) under the structural map;
- (ii) a UKF that approximates this predictive law should place sigma points on (x_{t-1}, ε_t) , or on an equivalent parameterization of the same pre-transition uncertainty;
- (iii) a naive full-state additive-noise UKF targets the same predictive law only if its induced predictive law coincides with that pushforward law; if it adds a direct perturbation to k_t , it targets a different law.

Proof. Define the structural map

$$F(x_{t-1}, \varepsilon_t) = (T_m(m_{t-1}, \varepsilon_t), T_k(k_{t-1}, m_{t-1}, T_m(m_{t-1}, \varepsilon_t))). \quad (19.41)$$

Because the current innovation is independent of the lagged filtering law, the joint pre-transition uncertainty factorizes as

$$p(x_{t-1}, \varepsilon_t \mid y_{1:t-1}) = p(x_{t-1} \mid y_{1:t-1})p(\varepsilon_t). \quad (19.42)$$

Let φ be any bounded test function on the state space of x_t . Then

$$\mathbb{E}[\varphi(x_t) \mid y_{1:t-1}] = \iint \varphi(F(x_{t-1}, \varepsilon_t)) p(x_{t-1} \mid y_{1:t-1}) p(\varepsilon_t) d\varepsilon_t dx_{t-1}. \quad (19.43)$$

Equation (19.43) is exactly the expectation of φ under the pushforward of the joint law of (x_{t-1}, ε_t) through F , which proves part (i).

Moreover, under the structural transition,

$$x_t = F(x_{t-1}, \varepsilon_t) \quad \text{almost surely.} \quad (19.44)$$

So once (x_{t-1}, ε_t) is fixed, both m_t and k_t are fixed. There is no independent new perturbation attached to k_t under the intended model. Therefore any sigma-point rule intended to approximate $p(x_t \mid y_{1:t-1})$ should be applied to the pre-transition uncertainty variables that generate that law, namely (x_{t-1}, ε_t) or an exactly equivalent parameterization. This proves part (ii).

Finally, if a full-state additive-noise UKF omits ε_t , then it misses genuine current-shock uncertainty. If it instead adds a direct perturbation to k_t , then it enlarges the stochastic space and defines a different predictive law unless that perturbation is degenerate in exactly the way required to reproduce the same pushforward distribution in (19.43). This proves part (iii). $\square$

The proposition does not say that the filter stops targeting x_t . The filtered state remains x_t . The point is that the sigma-point variable is the pre-transition uncertainty whose pushforward generates the predictive law of x_t . This is analogous to process-noise augmentation in a standard additive-noise UKF, but in the mixed structural setting the distinction is sharper: ε_t is a genuine new source of uncertainty, while k_t is a structural completion of that uncertainty rather than an independently shocked coordinate.

Proposition 19.2 (Local structural UKF Taylor accuracy). *Let $A_t = (x_{t-1}, \varepsilon_t)$ have a Gaussian approximation with mean μ_a and covariance P_a , and let $X_t = F_\theta(A_t)$ where F_θ is the structural map in (19.4). Write $P_a = \eta^2 \Sigma_a$, with Σ_a fixed and η measuring the local scale of the input uncertainty. Suppose the sigma-point deviations $\xi_j = a_t^{(j)} - \mu_a$ satisfy*

$$\sum_j w_j^{(m)} \xi_j = 0, \quad \sum_j w_j^{(m)} \xi_j \xi_j^\top = P_a, \quad \sum_j w_j^{(c)} \xi_j \xi_j^\top = P_a,$$

and suppose F_θ is three-times continuously differentiable on a neighborhood containing the relevant input mass and sigma points. Let J be the Jacobian of F_θ at μ_a , and let H_r be the Hessian of the r th component of F_θ at μ_a . Then the structural UKF prediction matches the transformed mean through the quadratic Taylor term:

$$\mathbb{E}[X_{t,r} \mid y_{1:t-1}] = F_{\theta,r}(\mu_a) + \frac{1}{2} \text{tr}(H_r P_a) + O(\eta^3), \quad (19.45)$$

$$\hat{x}_{t|t-1,r}^{\text{UKF}} = F_{\theta,r}(\mu_a) + \frac{1}{2} \text{tr}(H_r P_a) + O(\eta^3). \quad (19.46)$$

It also matches the leading second-order covariance term:

$$\text{Cov}(X_t \mid y_{1:t-1}) = J P_a J^\top + O(\eta^3), \quad (19.47)$$

$$P_{xx,t}^{\text{UKF}} = J P_a J^\top + O(\eta^3). \quad (19.48)$$

Thus, under the local small-uncertainty Taylor expansion used here, the structural UKF matches the transformed mean through the quadratic term and matches the leading covariance term for the Gaussian approximation on the correct input law (x_{t-1}, ε_t) . This is a local expansion statement for the selected structural input law, not a blanket exactness theorem for nonlinear structural transitions. It does not apply to a naive post-transition full-state sigma-point cloud unless that cloud induces the same law as $(F_\theta)_\# \pi_t^a$.

Proof. By Proposition 19.1, the structural predictive law of x_t is the law of $F_\theta(A_t)$. The structural UKF therefore applies the unscented-transform rule to the transformation $g = F_\theta$ and the input variable A_t .

For a component r , Taylor expansion around μ_a gives

$$F_{\theta,r}(\mu_a + \delta) = F_{\theta,r}(\mu_a) + J_r \delta + \frac{1}{2} \delta^\top H_r \delta + O(\|\delta\|^3).$$

Taking expectation under the Gaussian approximation for $A_t - \mu_a$ yields the first line of (19.46), because $\mathbb{E}[A_t - \mu_a] = 0$ and $\mathbb{E}[(A_t - \mu_a)(A_t - \mu_a)^\top] = P_a$. Taking the weighted sum over sigma points gives the second line of (19.46) by the same first- and second-moment identities for ξ_j .

For covariance, subtract the corresponding mean expansion. The leading centered term is $J(A_t - \mu_a)$ for the true law and $J\xi_j$ for the sigma points. Their second moments

are both $JP_a J^\top$ by moment matching. The quadratic Taylor terms have size $O(\eta^2)$, so their covariance contribution with the leading linear term is $O(\eta^3)$ and their self-covariance contribution is $O(\eta^4)$. This proves (19.48). These are the local unscented-transform moment calculations for the selected sigma-point rule [Julier and Uhlmann, 1996, van der Merwe, 2004], applied to the structural input variable. If a naive full-state cloud is constructed from a different input law, the same UT calculation applies only to that different law; it gives no accuracy guarantee for the structural pushforward target. $\square$

19.7 Worked Example A: Nonlinear Structural Transition

Let

$$m_t = \rho m_{t-1} + \sigma \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 1), \quad (19.49)$$

$$k_t = \phi k_{t-1} + \gamma m_t^2, \quad (19.50)$$

$$y_t = m_t + k_t + e_t, \quad e_t \sim \mathcal{N}(0, R). \quad (19.51)$$

The state is $x_t = (m_t, k_t)$, but only the innovation ε_t is new stochastic input at time t . Conditional on $(m_{t-1}, k_{t-1}, \varepsilon_t)$, the coordinate k_t is deterministic.

Suppose the previous filtering distribution is Gaussian:

$$x_{t-1} \mid y_{1:t-1} \approx \mathcal{N}(\mu_{t-1}, P_{t-1}), \quad \mu_{t-1} = \begin{bmatrix} \bar{m} \\ \bar{k} \end{bmatrix}.$$

The structural UKF should augment the previous state with the innovation. This step needs to be motivated carefully, because it is exactly where the structural UKF departs from the naive full-state construction. The point is not to pad the state arbitrarily. The point is to place sigma points on the variables that actually generate the one-step uncertainty in the structural transition. In this example, the predictive law is generated by the joint uncertainty in the lagged state (m_{t-1}, k_{t-1}) and the current shock ε_t . Once those three quantities are fixed, both m_t and k_t are fixed by the structural map.

That is why the augmented variable is

$$a_t = (x_{t-1}, \varepsilon_t), \quad (19.52)$$

rather than an artificially padded post-transition full state. Standard additive-noise UKF presentations often augment with process noise for the same reason: sigma points should live on the pre-transition uncertainty variables. For a mixed structural model, however, that principle has a sharper consequence. The shock ε_t is a genuine new source of uncertainty, while k_t is not. If the augmentation omitted ε_t , the propagated sigma points would miss current shock uncertainty. If the augmentation instead added a direct perturbation for k_t , the sigma points would describe a different transition law from the one declared by the model.

So the structural UKF should augment the previous state with the innovation. This places sigma points in the same pre-transition uncertainty variables that define the predictive law, rather than in an artificially padded post-transition full state:

$$a_t = \begin{bmatrix} m_{t-1} \\ k_{t-1} \\ \varepsilon_t \end{bmatrix}, \quad a_t \mid y_{1:t-1} \approx \mathcal{N} \left(\begin{bmatrix} \bar{m} \\ \bar{k} \\ 0 \end{bmatrix}, \begin{bmatrix} P_{t-1} & 0 \\ 0 & 1 \end{bmatrix} \right). \quad (19.53)$$

Let $\{a_t^{(j)}, w_j^{(m)}, w_j^{(c)}\}_{j=0}^{2n_a}$ be the usual unscented points and weights for this $n_a = 3$ dimensional augmented Gaussian [Julier and Uhlmann, 1997]. For every point $a_t^{(j)} = (m_{t-1}^{(j)}, k_{t-1}^{(j)}, \varepsilon_t^{(j)})$, the structural transition is evaluated as

$$m_t^{(j)} = \rho m_{t-1}^{(j)} + \sigma \varepsilon_t^{(j)}, \quad (19.54)$$

$$k_t^{(j)} = \phi k_{t-1}^{(j)} + \gamma \left(m_t^{(j)}\right)^2, \quad (19.55)$$

$$x_t^{(j)} = \begin{bmatrix} m_t^{(j)} \\ k_t^{(j)} \end{bmatrix}. \quad (19.56)$$

This is the crucial step: the filter never creates an independent sigma-point perturbation for k_t . It creates sigma points for the previous state and the current innovation, then computes k_t by the structural map (19.50).

The structural and naive full-state UKF constructions now differ at the level that matters for the update. The structural UKF forms sigma points for (x_{t-1}, ε_t) , propagates them through the structural transition, and therefore computes moments of the target law declared by the model. A naive full-state UKF instead behaves as if there were an additive-noise model of the form

$$x_t = \tilde{f}(x_{t-1}) + \tilde{u}_t, \quad \tilde{u}_t \sim \mathcal{N}(0, \tilde{Q}),$$

with a full-state covariance $\tilde{Q}$ that assigns direct new variance to all or most coordinates of x_t . The three failure claims can now be derived in the notation of this toy model.

Prediction-law failure. Under the structural model,

$$m_t = \rho m_{t-1} + \sigma \varepsilon_t, \quad k_t = \phi k_{t-1} + \gamma m_t^2.$$

So conditional on (m_{t-1}, k_{t-1}) , the one-step predictive law is induced by a single scalar shock ε_t . Eliminating the shock-driven update for m_t gives the structural identity

$$k_t - \phi k_{t-1} - \gamma m_t^2 = 0. \quad (19.57)$$

Every propagated sigma point in the structural UKF satisfies this identity, so its sigma-point cloud lies on the support of the structural predictive law. A naive full-state UKF instead propagates points

$$\tilde{x}_t^{(j)} = (\tilde{m}_t^{(j)}, \tilde{k}_t^{(j)})$$

from an additive full-state covariance model. In general those points satisfy

$$\tilde{k}_t^{(j)} - \phi \tilde{k}_{t-1}^{(j)} - \gamma (\tilde{m}_t^{(j)})^2 \neq 0,$$

so the naive sigma-point cloud approximates moments of a different one-step law. That is the prediction-law failure.

Cross-covariance failure. For the observation equation $z_t = m_t + k_t$, the structural UKF computes

$$P_{xz,t}^{\text{struct}} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_{t|t-1}) (z_t^{(j)} - \hat{z}_t),$$

with

$$z_t^{(j)} = m_t^{(j)} + k_t^{(j)}, \quad k_t^{(j)} = \phi k_{t-1}^{(j)} + \gamma (m_t^{(j)})^2.$$

Hence all variation in the second state coordinate enters through lagged-state variation and the common shock channel already represented in $m_t^{(j)}$. Under the naive full-state UKF,

$$P_{xz,t}^{\text{naive}} = \sum_j w_j^{(c)} (\tilde{x}_t^{(j)} - \tilde{x}_{t|t-1}) (\tilde{z}_t^{(j)} - \tilde{z}_t),$$

with

$$\tilde{z}_t^{(j)} = \tilde{m}_t^{(j)} + \tilde{k}_t^{(j)}.$$

Because $\tilde{k}_t^{(j)}$ now contains a direct artificial perturbation channel, the second component of $P_{xz,t}^{\text{naive}}$ contains covariance terms between the observation and that artificial variation. Those terms do not exist in the structural model, where k_t is completed deterministically from the same shock path that drives m_t . That is the cross-covariance failure.

Update-law failure. The familiar UKF update equations,

$$K_t = P_{xz,t} S_t^{-1}, \quad \hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t v_t, \quad P_{t|t} = P_{t|t-1} - K_t S_t K_t^\top,$$

do not fail algebraically. They fail semantically when $(\hat{x}_{t|t-1}, S_t, P_{xz,t})$ are computed from the wrong predictive law. Once the naive full-state approximation changes either $P_{xz,t}$ or S_t , it changes the gain itself:

$$K_t^{\text{naive}} = P_{xz,t}^{\text{naive}} (S_t^{\text{naive}})^{-1} \neq P_{xz,t}^{\text{struct}} (S_t^{\text{struct}})^{-1} = K_t^{\text{struct}}$$

in general. The existing numerical comparison later in this example shows this explicitly: adding artificial variance to k_t changes the innovation variance from $S_t = 0.61217$ to $S_t = 0.65217$, which then changes the approximate log-likelihood contribution. The same change propagates into the Kalman gain and posterior update. So the update law is not wrong as linear algebra; it is correctly updating the wrong approximate model.

The predicted state moments are

$$\hat{x}_{t|t-1} = \sum_j w_j^{(m)} x_t^{(j)}, \tag{19.58}$$

$$P_{xx,t} = \sum_j w_j^{(c)} \left(x_t^{(j)} - \hat{x}_{t|t-1} \right) \left(x_t^{(j)} - \hat{x}_{t|t-1} \right)^\top. \tag{19.59}$$

For the observation equation (19.51), define

$$z_t^{(j)} = m_t^{(j)} + k_t^{(j)}.$$

Then the predicted observation, innovation variance, and cross covariance are

$$\hat{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \tag{19.60}$$

$$S_t = \sum_j w_j^{(c)} (z_t^{(j)} - \hat{z}_t)^2 + R, \tag{19.61}$$

$$P_{xz,t} = \sum_j w_j^{(c)} \left(x_t^{(j)} - \hat{x}_{t|t-1} \right) (z_t^{(j)} - \hat{z}_t). \tag{19.62}$$

The Gaussian measurement update is

$$v_t = y_t - \hat{z}_t, \quad (19.63)$$

$$K_t = P_{xz,t} S_t^{-1}, \quad (19.64)$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t v_t, \quad (19.65)$$

$$P_{t|t} = P_{xx,t} - K_t S_t K_t^\top, \quad (19.66)$$

and the one-step approximate log likelihood contribution is

$$\ell_t = -\frac{1}{2} \left[\log(2\pi S_t) + \frac{v_t^2}{S_t} \right].$$

19.7.1 Reviewer Edge Case: Does $\phi = 0$ Collapse the Example?

The model does not collapse just because $\phi = 0$. In that case

$$k_t = \gamma m_t^2, \quad m_t = \rho m_{t-1} + \sigma \varepsilon_t,$$

so k_t is still random whenever $\gamma \neq 0$ and the conditional law of m_t is nondegenerate. What disappears is the inherited lagged- k channel, not the current-shock channel. The support becomes the lower-dimensional manifold

$$\{(m, k) : k = \gamma m^2\},$$

conditional on the parameters and the lagged state distribution. It collapses to a point only in degenerate cases, for example if $\gamma = 0$ or if m_t is itself degenerate. With the numerical calibration below but setting $\phi = 0$, the recomputed structural UKF moments are

$$\hat{x}_{t|t-1} = \begin{bmatrix} 0 \\ 0.11024 \end{bmatrix}, \quad P_{xx,t} = \begin{bmatrix} 0.27560 & 0 \\ 0 & 0.04247 \end{bmatrix},$$

and the observation moments are

$$\hat{z}_t = 0.11024, \quad S_t = 0.56807, \quad P_{xz,t} = \begin{bmatrix} 0.27560 \\ 0.04247 \end{bmatrix}.$$

The k variance is smaller because lagged k_{t-1} no longer feeds into k_t , but it is not zero: it is induced by the quadratic transformation of the shock-driven m_t .

19.7.2 Numerical Illustration

Use the parameters

$$\rho = 0.8, \quad \sigma = 0.5, \quad \phi = 0.7, \quad \gamma = 0.4, \quad R = 0.25,$$

and previous filtering moments

$$\mu_{t-1} = (0, 0)^\top, \quad P_{t-1} = \text{diag}(0.04, 0.09).$$

Use the scaled unscented convention $\alpha = 1$, $\beta = 2$, and $\kappa = 0$. Since $n_a = 3$, this gives $\lambda = \alpha^2(n_a + \kappa) - n_a = 0$, central mean weight $w_0^{(m)} = 0$, central covariance weight $w_0^{(c)} = 2$, and six off-center mean/covariance weights equal to $1/6$. The augmented covariance is

$$\text{diag}(0.04, 0.09, 1),$$

so $\sqrt{(n_a + \lambda)P_a}$ has diagonal entries 0.34641, 0.51962, and 1.73205. The seven sigma points and their deterministic structural completion are:

j	$m_{t-1}^{(j)}$	$k_{t-1}^{(j)}$	$\varepsilon_t^{(j)}$	$m_t^{(j)}$	$k_t^{(j)}$	$z_t^{(j)}$
0	0	0	0	0	0	0
1	0.34641	0	0	0.27713	0.03072	0.30785
2	0	0.51962	0	0	0.36373	0.36373
3	0	0	1.73205	0.86603	0.30000	1.16603
4	-0.34641	0	0	-0.27713	0.03072	-0.24641
5	0	-0.51962	0	0	-0.36373	-0.36373
6	0	0	-1.73205	-0.86603	0.30000	-0.56603

For example, the third positive innovation point gives $m_t = 0.5(1.73205) = 0.86603$ and $k_t = 0.4(0.86603)^2 = 0.30000$. No row contains an independently sampled “new” shock for k_t .

Using the stated covariance weights, the structural propagation gives

$$\hat{x}_{t|t-1} = \begin{bmatrix} 0 \\ 0.11024 \end{bmatrix}, \quad P_{xx,t} = \begin{bmatrix} 0.27560 & 0 \\ 0 & 0.08657 \end{bmatrix}.$$

The predicted observation moments are

$$\hat{z}_t = 0.11024, \quad S_t = 0.61217, \quad P_{xz,t} = \begin{bmatrix} 0.27560 \\ 0.08657 \end{bmatrix}.$$

For an observation $y_t = 0.30$,

$$v_t = 0.18976, \quad K_t = \begin{bmatrix} 0.45020 \\ 0.14141 \end{bmatrix},$$

and therefore

$$\hat{x}_{t|t} = \begin{bmatrix} 0.08543 \\ 0.13707 \end{bmatrix}, \quad P_{t|t} = \begin{bmatrix} 0.15152 & -0.03897 \\ -0.03897 & 0.07433 \end{bmatrix}.$$

The one-step log likelihood contribution is approximately

$$\ell_t = -0.70297.$$

These numbers are illustrative calculations under the stated sigma-point convention and should be read as a worked example rather than general performance evidence.

Side-by-side prediction/update contrast. For this toy model, the structural UKF treats

$$a_t = (x_{t-1}, \varepsilon_t)$$

as the sigma-point object and computes propagated points by

$$m_t^{(j)} = \rho m_{t-1}^{(j)} + \sigma \varepsilon_t^{(j)}, \quad k_t^{(j)} = \phi k_{t-1}^{(j)} + \gamma (m_t^{(j)})^2.$$

It therefore forms

$$P_{xz,t}^{\text{struct}} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_{t|t-1}) (z_t^{(j)} - \hat{z}_t),$$

where every $x_t^{(j)}$ lies on the support generated by the structural map. A naive full-state UKF instead produces an artificial cloud $\tilde{x}_t^{(j)} = (\tilde{m}_t^{(j)}, \tilde{k}_t^{(j)})$ with

$$\tilde{k}_t^{(j)} \neq \phi k_{t-1}^{(j)} + \gamma (\tilde{m}_t^{(j)})^2 \text{ in general,}$$

and therefore forms

$$P_{xz,t}^{\text{naive}} = \sum_j w_j^{(c)} (\tilde{x}_t^{(j)} - \tilde{x}_{t|t-1}) (\tilde{z}_t^{(j)} - \tilde{z}_t),$$

which mixes genuine shock-driven variation with artificial perturbations of the deterministic-completion coordinate.

Structural identity test. A simple way to see the difference is to check whether propagated points satisfy

$$k_t - \phi k_{t-1} - \gamma m_t^2 = 0.$$

The structural UKF satisfies this identity point by point for every propagated sigma point. A naive full-state UKF with direct perturbations to k_t does not. That is the diagnostic place where the structural target and this naive full-state approximation part company.

Now compare this with an off-manifold full-state approximation that adds an independent artificial innovation variance 0.04 directly to k_t after the structural map. Nothing in (19.49)–(19.51) justifies that extra shock. It changes the innovation variance from $S_t = 0.61217$ to $S_t = 0.65217$ and changes the same observation's log likelihood contribution from -0.70297 to -0.73282 . The numerical difference is small in this toy calibration, but the conceptual difference is large: the latter filter is approximating a different approximate state-space model relative to the declared structural law.

To connect these numbers back to the three failures above, write the naive full-state approximation as using

$$\tilde{P}_{xx,t} = \begin{bmatrix} 0.27560 & 0 \\ 0 & 0.12657 \end{bmatrix},$$

which differs from the structural covariance only by the artificial extra variance in the k_t coordinate. Under the same observation equation $z_t = m_t + k_t$, this gives

$$\tilde{S}_t = 0.65217, \quad \tilde{P}_{xz,t} = \begin{bmatrix} 0.27560 \\ 0.12657 \end{bmatrix},$$

and therefore

$$\tilde{K}_t = \begin{bmatrix} 0.42259 \\ 0.19408 \end{bmatrix}, \quad \tilde{x}_{t|t} = \begin{bmatrix} 0.08019 \\ 0.14707 \end{bmatrix},$$

for the same observation $y_t = 0.30$.

These values make the three failures concrete:

- (i) **Prediction-law failure:** the structural update uses $P_{xx,t} = \text{diag}(0.27560, 0.08657)$, while the naive full-state approximation uses $\tilde{P}_{xx,t} = \text{diag}(0.27560, 0.12657)$ because it has inserted artificial uncertainty directly into k_t .

(ii) **Cross-covariance failure:** the structural update uses

$$P_{xz,t} = \begin{bmatrix} 0.27560 \\ 0.08657 \end{bmatrix},$$

whereas the naive full-state approximation uses

$$\tilde{P}_{xz,t} = \begin{bmatrix} 0.27560 \\ 0.12657 \end{bmatrix}.$$

The second component is larger only because the naive construction lets the observation load directly on artificial variation in k_t .

(iii) **Update-law failure:** the structural gain and posterior mean are

$$K_t = \begin{bmatrix} 0.45020 \\ 0.14141 \end{bmatrix}, \quad \hat{x}_{t|t} = \begin{bmatrix} 0.08543 \\ 0.13707 \end{bmatrix},$$

whereas the naive full-state approximation gives

$$\tilde{K}_t = \begin{bmatrix} 0.42259 \\ 0.19407 \end{bmatrix}, \quad \tilde{x}_{t|t} = \begin{bmatrix} 0.08019 \\ 0.14707 \end{bmatrix}.$$

The update formula has not changed, but the gain and posterior correction do because they are now computed from the wrong predictive moments.

So the numerical illustration does not merely show a slightly different likelihood value. It shows that once the naive approximation changes the predictive covariance and cross covariance, the entire measurement update shifts with it.

The lesson is not that UKF is wrong. The lesson is that the UKF must be applied to the correct structural transition. Sigma points belong in the declared stochastic integration space. Endogenous deterministic coordinates are completed by the model point by point. In BayesFilter terms, this is a design rule for mixed structural models and an approximation-label boundary for any backend that instead enlarges the stochastic space.

19.8 Worked Example B: Degenerate Linear Transition with Nonlinear Measurement

The most practically relevant counterpart to the previous example is not a fully stochastic linear-Gaussian state transition. It is a degenerate linear transition together with a nonlinear measurement equation. That case is closer to many DSGE and affine term-structure applications than a fully nonlinear latent transition. The key point is that the latent transition may still have a structural split even when it is linear. A nonlinear measurement equation does not remove that split.

Consider the linear structural transition

$$m_t = \rho m_{t-1} + \sigma \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 1), \quad (19.67)$$

$$k_t = \phi k_{t-1} + \psi m_{t-1} + \chi m_t, \quad (19.68)$$

$$x_t = (m_t, k_t), \quad (19.69)$$

and the nonlinear exponential-affine measurement equation

$$y_t = \exp(A^\top x_t + \mu) + e_t, \quad e_t \sim \mathcal{N}(0, R). \quad (19.70)$$

The latent transition is linear, but it is still structurally degenerate: conditional on (x_{t-1}, ε_t) , the endogenous coordinate k_t is completed deterministically from the same shock path that drives m_t . So the structural issue from the previous section does not disappear. What changes is where the nonlinearity appears. The latent transition is linear, while the measurement map is nonlinear.

This means there are now two layers to keep separate:

- (i) the latent predictive law is still generated by the pre-transition uncertainty variables (x_{t-1}, ε_t) , not by direct full-state perturbations of x_t ;
- (ii) even after the latent cloud is propagated correctly, the UKF must still approximate nonlinear observation moments and cross covariances for the map $x_t \mapsto \exp(A^\top x_t + \mu)$.

The first issue is structural-modeling correctness. The second is observation-side quadrature quality.

19.8.1 Worked structural derivation for the degenerate linear-transition case

Accepted assumptions. For this section, treat the following as explicit modeling assumptions:

- (A1) the innovation is conditionally independent of the lagged filtering law, so that

$$p(x_{t-1}, \varepsilon_t \mid y_{1:t-1}) = p(x_{t-1} \mid y_{1:t-1})p(\varepsilon_t); \quad (19.71)$$

- (A2) the latent map, observation map, and composed map defined below are measurable and have the finite moments needed for the stated expectations;
- (A3) whenever we refer to the observation-side-only error regime, we assume that the latent predictive law from (19.74) is preserved exactly;
- (A4) no invertibility of T_k is required: it is enough that k_t is a well-defined measurable deterministic completion of (k_{t-1}, m_{t-1}, m_t) ;
- (A5) if one wants to replace the innovation variable ε_t by the current exogenous state m_t as the sigma-point variable, then for fixed m_{t-1} the map

$$\varepsilon_t \mapsto T_m(m_{t-1}, \varepsilon_t) \quad (19.72)$$

should be injective, or invertible onto its image with measurable inverse. This extra assumption is only needed for that reparameterization step, not for the pushforward identities proved below.

Latent predictive pushforward. Define the degenerate linear structural map

$$F_{\text{lin}}(x_{t-1}, \varepsilon_t) = \left(\rho m_{t-1} + \sigma \varepsilon_t, \phi k_{t-1} + \psi m_{t-1} + \chi(\rho m_{t-1} + \sigma \varepsilon_t) \right). \quad (19.73)$$

Then for any bounded test function φ on the latent-state space,

$$\mathbb{E}[\varphi(x_t) \mid y_{1:t-1}] = \iint \varphi(F_{\text{lin}}(x_{t-1}, \varepsilon_t)) p(x_{t-1} \mid y_{1:t-1}) p(\varepsilon_t) d\varepsilon_t dx_{t-1}. \quad (19.74)$$

Hence the latent predictive law is the pushforward of the joint law of (x_{t-1}, ε_t) through F_{lin} .

Derivation. Under the state transition (19.67)–(19.69) and the factorization assumption (19.71), the conditional expectation of any bounded test function $\varphi(x_t)$ is obtained by substituting $x_t = F_{\text{lin}}(x_{t-1}, \varepsilon_t)$ and integrating over the joint law of (x_{t-1}, ε_t) . This gives (19.74), which is exactly the pushforward identity.

Deterministic-completion identity. Under (19.67)–(19.68), all admissible latent states satisfy

$$k_t - \phi k_{t-1} - \psi m_{t-1} - \chi m_t = 0. \quad (19.75)$$

Therefore, once (x_{t-1}, ε_t) is fixed, both m_t and k_t are fixed; there is no independent new perturbation attached to k_t in the latent transition.

Derivation. Substitute (19.67) into (19.68) and rearrange terms. The resulting identity is exactly (19.75).

Observation predictive pushforward. Define the nonlinear noise-free observation map

$$h(x_t) = \exp(A^\top x_t + \mu). \quad (19.76)$$

and the composed map

$$G(x_{t-1}, \varepsilon_t) = h(F_{\text{lin}}(x_{t-1}, \varepsilon_t)). \quad (19.77)$$

Then for any bounded test function ψ_0 on the noise-free observation space,

$$\mathbb{E}[\psi_0(h(x_t)) \mid y_{1:t-1}] = \iint \psi_0(G(x_{t-1}, \varepsilon_t)) p(x_{t-1} \mid y_{1:t-1}) p(\varepsilon_t) d\varepsilon_t dx_{t-1}. \quad (19.78)$$

Hence the noise-free observation predictive law is the pushforward of the same pre-transition uncertainty through the composed map.

Derivation. Compose the latent pushforward identity (19.74) with the measurable observation map (19.76). This yields (19.78).

Naive full-state perturbation changes the latent law. If a naive full-state UKF introduces a direct perturbation to the endogenous coordinate k_t , then its propagated latent sigma-point cloud does not, in general, satisfy (19.75) pointwise. Therefore it does not target the latent predictive law characterized by (19.74).

Diagnostic check. By the deterministic-completion identity above, every admissible latent state under the structural model satisfies (19.75). A naive full-state UKF that adds an independent perturbation to k_t creates propagated points whose k_t values are not determined solely by (x_{t-1}, ε_t) , so the identity fails in general. Hence the resulting sigma-point cloud cannot represent the same latent predictive law.

Failure propagation and observation-side-only error. Under the preceding worked identities:

- (i) if a naive full-state UKF directly perturbs k_t , then prediction-law failure, cross-covariance failure, and update-law failure all still apply;
- (ii) if the latent predictive law from (19.74) is preserved exactly and only the transformed observation moments from (19.78) are approximated by quadrature, then the error is not structural-model failure; it is only observation-side quadrature error.

Reason. Part (i) follows because the naive construction changes the latent predictive law, and the nonlinear observation law is the pushforward of that same latent law. Once the latent law changes, the observation moments, state-observation cross covariance, and therefore the UKF update objects also change. Part (ii) follows from Assumption (A3): if the latent law is preserved and only the nonlinear observation moments are approximated, then the remaining error arises only in the observation-side quadrature.

The formal statements above separate three layers that are easy to blur in practice: accepted modeling assumptions, exact pushforward identities, and the interpretation of failure modes. The filtered state is still x_t , but the latent predictive law is generated by the pre-transition uncertainty variables (x_{t-1}, ε_t) . The nonlinear measurement equation adds another approximation layer, but it does not remove the need to respect the degenerate latent transition. Assumption (A5) is stronger than the pushforward arguments need: it is included only to justify replacing ε_t by m_t as an equivalent sigma-point variable when that reparameterization is desired.

19.8.2 Worked Degenerate Linear-Transition Example

Use

$$\rho = 0.8, \quad \sigma = 0.5, \quad \phi = 0.7, \quad \psi = 0.3, \quad \chi = 0.4,$$

with previous filtering moments

$$\mu_{t-1} = (0, 0)^\top, \quad P_{t-1} = \text{diag}(0.04, 0.09),$$

and measurement parameters

$$A = \begin{bmatrix} 1.1 \\ 0.6 \end{bmatrix}, \quad \mu = 0.10, \quad R = 0.04.$$

Because the latent transition remains structurally degenerate, the correct sigma-point object is still

$$a_t = (x_{t-1}, \varepsilon_t),$$

not a naively perturbed full post-transition state. Using the same scaled unscented construction as in the previous example, the propagated latent points are obtained from

$$m_t^{(j)} = \rho m_{t-1}^{(j)} + \sigma \varepsilon_t^{(j)}, \tag{19.79}$$

$$k_t^{(j)} = \phi k_{t-1}^{(j)} + \psi m_{t-1}^{(j)} + \chi m_t^{(j)}. \tag{19.80}$$

For each propagated point, define the nonlinear noise-free observation

$$z_t^{(j)} = \exp\left(1.1m_t^{(j)} + 0.6k_t^{(j)} + 0.10\right).$$

The same seven sigma points used earlier now produce the following latent and measurement values:

j	$m_{t-1}^{(j)}$	$k_{t-1}^{(j)}$	$\varepsilon_t^{(j)}$	$m_t^{(j)}$	$k_t^{(j)}$	$z_t^{(j)}$
0	0	0	0	0	0	1.10517
1	0.34641	0	0	0.27713	0.21477	1.70524
2	0	0.51962	0	0	0.36373	1.37470
3	0	0	1.73205	0.86603	0.34641	3.52709
4	-0.34641	0	0	-0.27713	-0.21477	0.71626
5	0	-0.51962	0	0	-0.36373	0.88848
6	0	0	-1.73205	-0.86603	-0.34641	0.34629

The predicted latent moments are

$$\hat{x}_{t|t-1} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad P_{xx,t} = \begin{bmatrix} 0.27560 & 0.11984 \\ 0.11984 & 0.09948 \end{bmatrix}.$$

The nonlinear predicted observation moments are

$$\hat{z}_t = 1.42635, \quad S_t = 1.32191, \quad P_{xz,t} = \begin{bmatrix} 0.50479 \\ 0.24852 \end{bmatrix}.$$

So the structural UKF gain is

$$K_t = \begin{bmatrix} 0.38186 \\ 0.18800 \end{bmatrix}.$$

If the realized observation is $y_t = 1.50$, then

$$v_t = 0.07365, \quad \hat{x}_{t|t} = \begin{bmatrix} 0.02813 \\ 0.01385 \end{bmatrix}.$$

As in the previous example, these numbers are illustrative calculations under the stated sigma-point convention. They demonstrate the law and moment objects being compared; they are not standalone performance evidence for a production filter.

Now compare this with a naive full-state approximation that adds an artificial latent perturbation variance 0.04 directly to the deterministic-completion coordinate k_t . To keep the comparison reproducible, take the naive rule to form a four-dimensional sigma-point variable

$$\tilde{a}_t = (x_{t-1}, \varepsilon_t, \delta_t), \quad \delta_t \sim \mathcal{N}(0, 0.04),$$

and to propagate

$$\tilde{k}_t = \phi k_{t-1} + \psi m_{t-1} + \chi m_t + \delta_t.$$

That artificial perturbation changes the latent predictive covariance to

$$\tilde{P}_{xx,t} = \begin{bmatrix} 0.27560 & 0.11984 \\ 0.11984 & 0.13948 \end{bmatrix},$$

while leaving the mean unchanged. Under the same nonlinear measurement map, the corresponding naive moment approximation is

$$\tilde{z}_t = 1.44483, \quad \tilde{S}_t = 1.57838, \quad \tilde{P}_{xz,t} = \begin{bmatrix} 0.53757 \\ 0.28867 \end{bmatrix}.$$

Hence the naive full-state gain becomes

$$\tilde{K}_t = \begin{bmatrix} 0.34058 \\ 0.18289 \end{bmatrix},$$

and for the same observation $y_t = 1.50$ the naive posterior mean update is

$$\tilde{x}_{t|t} = \begin{bmatrix} 0.01879 \\ 0.01009 \end{bmatrix}.$$

These numbers make the three failures concrete in this linear-transition / nonlinear-measurement setting:

- (i) **Prediction-law failure:** the structural latent law uses

$$P_{xx,t} = \begin{bmatrix} 0.27560 & 0.11984 \\ 0.11984 & 0.09948 \end{bmatrix},$$

whereas the naive full-state approximation uses

$$\tilde{P}_{xx,t} = \begin{bmatrix} 0.27560 & 0.11984 \\ 0.11984 & 0.13948 \end{bmatrix},$$

because it has inserted artificial uncertainty directly into k_t .

- (ii) **Cross-covariance failure:** the structural update uses

$$P_{xz,t} = \begin{bmatrix} 0.50479 \\ 0.24852 \end{bmatrix},$$

whereas the naive approximation uses

$$\tilde{P}_{xz,t} = \begin{bmatrix} 0.53757 \\ 0.28867 \end{bmatrix}.$$

The extra direct perturbation of k_t changes how the nonlinear observation loads on the second state coordinate.

- (iii) **Update-law failure:** the structural gain and posterior mean are

$$K_t = \begin{bmatrix} 0.38186 \\ 0.18800 \end{bmatrix}, \quad \hat{x}_{t|t} = \begin{bmatrix} 0.02813 \\ 0.01385 \end{bmatrix},$$

whereas the naive full-state approximation gives

$$\tilde{K}_t = \begin{bmatrix} 0.34058 \\ 0.18289 \end{bmatrix}, \quad \tilde{x}_{t|t} = \begin{bmatrix} 0.01879 \\ 0.01009 \end{bmatrix}.$$

The update formula itself is unchanged, but the predicted moments and cross covariance are not, so the gain and posterior correction are computed for a different approximate model.

19.8.3 What Is Similar to the Structural-Deterministic Case?

The similarity is that the structural latent transition is still generated by (x_{t-1}, ε_t) , not by independent perturbations of every coordinate of x_t . If a naive full-state UKF directly perturbs k_t , it again changes the latent predictive law before the observation map is even evaluated. So the structural warning remains active even though the latent transition is linear.

The nonlinear observation adds a second approximation layer. Once the latent cloud is propagated correctly, the UKF still has to approximate

$$\hat{y}_t, \quad S_t, \quad P_{xz,t}$$

for the map $x_t \mapsto \exp(A^\top x_t + \mu)$. Poor quadrature here still changes the gain and posterior update, just as in other nonlinear Gaussian filters.

19.8.4 What Is Different from the Earlier Nonlinear-State Example?

In the earlier toy example, the latent state equation itself was nonlinear, so the UKF had to approximate both the latent pushforward through the nonlinear state map and the resulting observation moments. Here the latent transition is linear, but structurally degenerate. That removes one source of approximation error but not the structural state-partition issue. The UKF still needs the correct pre-transition uncertainty variable because k_t remains a deterministic-completion coordinate.

So this example isolates a different practical regime:

- (i) the latent transition is linear but not full-state stochastic;
- (ii) the nonlinear burden sits in the measurement equation;
- (iii) the UKF can still fail in two distinct ways:
 - structurally, if it perturbs k_t directly and therefore changes the latent predictive law;
 - numerically, if it uses the correct latent construction but poorly approximates the nonlinear observation moments.

This is exactly why the structural deterministic-dynamics chapter should not be read as applying only to nonlinear state transitions. The same latent state-partition discipline remains necessary when the state equation is linear but degenerate and the observation map is nonlinear.

19.9 What Structural Correctness Does and Does Not Guarantee

Structural correctness guarantees that the filter is approximating the intended latent predictive law rather than an altered full-state approximation. It does not, by itself, guarantee that the resulting Gaussian closure is exact, that nonlinear observation moments are approximated with negligible error, or that the derivative path is stable enough

for HMC. In BayesFilter terms, structural correctness is a law-specification requirement; it is not a blanket certificate of moment accuracy, derivative accuracy, or production readiness.

In particular, structural correctness does not solve four other problems that must be evaluated separately:

- (i) it does not make the Gaussian sigma-point closure exact for a genuinely nonlinear transformation;
- (ii) it does not remove observation-side quadrature error in models such as (19.70);
- (iii) it does not resolve numerical covariance-factor issues when square roots, SVD factors, or PSD projections are used;
- (iv) it does not guarantee a stable value-gradient path for HMC, especially when spectral decompositions or fallback branches are differentiated.

Thus the right mental model is layered: first make sure the latent law is correct, then measure the quality of the nonlinear approximation, then audit the numerical and derivative path.

19.10 Structural Degeneracy, Numerical Degeneracy, and SVD

The literature and the source-project audit motivate a distinction that should be made explicit:

Issue	What it means	What fixes it
Structural degeneracy	Some latent coordinates are deterministic-completion variables under the model law	Respect the declared stochastic integration space and complete the deterministic block pointwise
Numerical degeneracy	A covariance or innovation matrix is singular or ill-conditioned in computation	Use a stable solve, square-root, or spectral factorization appropriate to the declared law
SVD/square-root representation	A numerical backend chooses a robust factorization of a covariance object	This may improve value-side robustness, but it does not by itself correct a structurally wrong latent law
Derivative/spectral risk	Raw autodiff through spectral or fallback branches may be unstable	Requires derivative audits, stress tests, and possibly custom gradients

This is why an SVD sigma-point filter does not automatically solve the problem discussed in this chapter. It may solve a factorization problem while leaving the latent-law specification problem untouched. Conversely, a structural SVD sigma-point filter is a sensible backend when it factors the covariance of the declared pre-transition variable,

propagates those points through F_θ , and records the usual value and derivative diagnostics from Chapters 17 and 18. The question is therefore not “SVD or structural UKF?” The correct question is:

Which law is being factored and propagated?

Proposition 19.3 (Degenerate covariance and formal sigma-point accuracy). *Let $A_\delta = \mu + \delta Z$ be a possibly degenerate Gaussian random variable in $\mathbb{R}^L$, with $\mathbb{E}[Z] = 0$ and $\text{Cov}(Z) = \Sigma \succeq 0$. Thus $\text{Cov}(A_\delta) = P_\delta = \delta^2 \Sigma$ may be singular. Let $\{a_\delta^{(j)}, w_j^{(m)}, w_j^{(c)}\}$ be any sigma-point rule whose deviations $\xi_j = a_\delta^{(j)} - \mu = \delta \zeta_j$ have fixed standardized points ζ_j and satisfy*

$$\sum_j w_j^{(m)} \xi_j = 0, \quad \sum_j w_j^{(m)} \xi_j \xi_j^\top = P_\delta, \quad \sum_j w_j^{(c)} \xi_j \xi_j^\top = P_\delta.$$

If $G : \mathbb{R}^L \rightarrow \mathbb{R}^d$ is three-times continuously differentiable and its third-order Taylor remainders around μ have expected size $O(\delta^3)$ under the Gaussian law and weighted size $O(\delta^3)$ under the sigma-point cloud, then the sigma-point transformed mean and covariance have the same local order as in the nonsingular case:

$$\mathbb{E}[G_r(A_\delta)] = G_r(\mu) + \frac{1}{2} \text{tr}(H_r P_\delta) + O(\delta^3), \quad (19.81)$$

$$\hat{g}_r^{\text{SP}} = G_r(\mu) + \frac{1}{2} \text{tr}(H_r P_\delta) + O(\delta^3), \quad (19.82)$$

$$\text{Cov}(G(A_\delta)) = J P_\delta J^\top + O(\delta^3), \quad (19.83)$$

$$P_G^{\text{SP}} = J P_\delta J^\top + O(\delta^3), \quad (19.84)$$

where J is the Jacobian of G at μ and H_r is the Hessian of the r th component. Positive definiteness of P_δ is not used. Consequently, abstracting from finite arithmetic and factor-construction errors, rank deficiency of the state-noise covariance does not by itself lower the formal local sigma-point moment accuracy for the law being propagated.

Proof. The usual Taylor proof only uses the first two moments of the input variable and the matching first two weighted moments of the sigma-point cloud. For a component r ,

$$G_r(\mu + \Delta) = G_r(\mu) + J_r \Delta + \frac{1}{2} \Delta^\top H_r \Delta + O(\|\Delta\|^3).$$

Taking expectations with $\Delta = A_\delta - \mu$ gives the first expansion because $\mathbb{E}[\Delta] = 0$ and $\mathbb{E}[\Delta \Delta^\top] = P_\delta$. Taking the weighted sum with $\Delta = \xi_j$ gives the sigma-point mean expansion by the same identities. For covariance, the leading centered term is $J \Delta$, whose second moment is $J P_\delta J^\top$ both for the true Gaussian law and for the sigma-point cloud. The quadratic remainder terms enter at order $O(\delta^3)$ or smaller. No inverse, Cholesky pivot, or full-rank support argument appears in this proof, so the conclusion is unchanged when P_δ is only positive semidefinite. $\square$

Proposition 19.3 verifies the reviewer’s claim in its precise value-side form. If a model is written as a full-state transition with a degenerate Gaussian innovation, and if that transition induces exactly the same conditional law as the structural pushforward in Proposition 19.1, then an exact SVD sigma-point rule for that degenerate law has the same formal local moment order as the corresponding nondegenerate rule. Degeneracy

is not, by itself, a mathematical objection to sigma-point filtering. This is a formal value-side statement abstracting from finite arithmetic, factor-construction choices, and derivative-branch effects. Those implementation issues still require the diagnostics below.

That statement is an equivalence result, not a recommendation to hide the structural split inside a large singular covariance matrix. To make the comparison precise, let the structural route use

$$A_t = (x_{t-1}, \varepsilon_t), \quad x_t = F_\theta(A_t), \quad \pi_t^x = (F_\theta)_\# \pi_t^a,$$

where $\pi_t^a = \pi_{t-1|t-1}^x \otimes \nu_\theta^\varepsilon$. Let a collapsed full-state SVD route instead use an augmented variable

$$\tilde{A}_t = (x_{t-1}, \tilde{u}_t), \quad \tilde{u}_t \sim \mathcal{N}(0, \tilde{Q}_t), \quad \tilde{x}_t = \tilde{F}_\theta(\tilde{A}_t),$$

with $\tilde{Q}_t = \tilde{C}_t \tilde{C}_t^\top$ obtained by an SVD or spectral rule. The collapsed route is acceptable only when the following mathematical diagnostics are satisfied.

- (i) **Law declaration is pushforward equality.** The two declarations are equivalent only if

$$(\tilde{F}_\theta)_\# \left(\pi_{t-1|t-1}^x \otimes \mathcal{N}(0, \tilde{Q}_t) \right) = (F_\theta)_\# \left(\pi_{t-1|t-1}^x \otimes \nu_\theta^\varepsilon \right),$$

or, equivalently, if $\mathbb{E}[\varphi(\tilde{x}_t) \mid y_{1:t-1}] = \mathbb{E}[\varphi(x_t) \mid y_{1:t-1}]$ for every bounded test function φ . Matching a covariance matrix is only a necessary moment check, not this equality of laws.

Example. If $m_t = \varepsilon_t$ and $k_t = \varepsilon_t^2$, then the structural conditional law is supported on the parabola $\{(m, k) : k = m^2\}$. A collapsed additive Gaussian transition $\tilde{x}_t = \mu + \tilde{u}_t$ with any covariance $\tilde{Q}_t$ has affine Gaussian support. It cannot equal the parabolic pushforward unless ε_t is degenerate. A singular covariance matrix by itself therefore does not encode the nonlinear timing contract.

- (ii) **Sigma-point support is a constraint residual.** Let $c_\theta(a, x) = 0$ collect the deterministic identities implied by the structural transition, for example $c_\theta(A_t, x_t) = x_t - F_\theta(A_t)$. A structural sigma-point cloud satisfies

$$\Delta_{\text{id}}^{\text{str}} = \max_j \|c_\theta(a_t^{(j)}, F_\theta(a_t^{(j)}))\| = 0.$$

The collapsed cloud must satisfy the corresponding check

$$\Delta_{\text{id}}^{\text{col}} = \max_j \|c_\theta(\tilde{a}_t^{(j)}, \tilde{F}_\theta(\tilde{a}_t^{(j)}))\| = 0,$$

after translating its variables into the structural coordinates. If this residual is not defined because the collapsed adapter no longer exposes the relevant shock or timing variables, the adapter cannot demonstrate structural equivalence.

Example. For the linear deterministic identity $x_t = (m_t, k_t) = (\varepsilon_t, \varepsilon_t)$, the support condition is $c(m, k) = k - m = 0$. The exact rank-one covariance $\tilde{Q} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ has an SVD factor whose points lie on $k = m$. After a nugget, $\tilde{Q}_\eta = \tilde{Q} + \eta^2 I$, sigma points have normal coordinate displacement $\sqrt{L + \lambda} \eta$ in the direction $(1, -1)/\sqrt{2}$, so $\Delta_{\text{id}}^{\text{col}} > 0$.

- (iii) **Dimension and cost are point-count identities.** For a symmetric $2L + 1$ rule, the number of model evaluations is determined by the dimension L of the factored variable. If the previous state has dimension n and the declared shock has dimension q , the structural augmented rule uses

$$N_{\text{str}} = 2(n + q) + 1.$$

A full-state additive-noise route that augments by an n -dimensional degenerate noise vector uses

$$N_{\text{col}} = 2(n + n) + 1, \quad N_{\text{col}} - N_{\text{str}} = 2(n - q).$$

This overhead is pure bookkeeping if the extra $n - q$ directions are zero-noise directions.

Example. With $n = 100$ state coordinates and $q = 10$ genuine shocks, the structural augmented rule uses 221 points, while the full-state degenerate-noise augmentation uses 401 points. In the two-coordinate toy covariance $\text{diag}(1, 0)$ treated as a two-dimensional factor, the zero column produces two sigma points equal to the mean; they add evaluations and alter the weight bookkeeping without adding stochastic information.

- (iv) **Finite arithmetic is normal-support leakage.** Let M_t be a matrix whose columns span the exact linear support of a degenerate Gaussian factor, and let N_t span the normal space $(\text{col } M_t)^\perp$. Exact support preservation requires

$$N_t^\top \tilde{C}_t = 0 \quad \Longleftrightarrow \quad N_t^\top \tilde{Q}_t N_t = 0.$$

A nugget, PSD projection, rank threshold, or asymmetric reconstruction should therefore be reported through

$$\ell_{\perp, t} = \|N_t^\top \tilde{Q}_t N_t\|_{\text{op}} \quad \text{or} \quad \max_j \|N_t^\top (\tilde{x}_t^{(j)} - \bar{x}_t)\|.$$

Example. In the identity example $k = m$, the normal vector is $N = (1, -1)^\top / \sqrt{2}$. For $\tilde{Q}_\eta = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} + \eta^2 I$,

$$N^\top \tilde{Q}_\eta N = \eta^2.$$

Hence the nominally deterministic relation has positive artificial variance η^2 in finite arithmetic. This is not a harmless representation of the same law; it is a regularized law unless explicitly removed before propagation.

- (v) **Gradient risk is controlled by spectral gaps.** If $\tilde{C}_t(\theta)$ is produced from an eigendecomposition or SVD, derivatives of the spectral vectors contain gap denominators. For a symmetric eigendecomposition, schematically,

$$du_i = \sum_{j \neq i} u_j \frac{u_j^\top (d\tilde{Q}_t) u_i}{\lambda_i - \lambda_j} + \text{terms along } u_i.$$

Therefore the gradient audit must track

$$g_t^{\min} = \min_{i \neq j} |\lambda_i(\tilde{Q}_t) - \lambda_j(\tilde{Q}_t)|, \quad \|\partial_\theta \tilde{C}_t\| \lesssim \frac{\|\partial_\theta \tilde{Q}_t\|}{g_t^{\min}}$$

away from rank and ordering branches. Structural completion does not remove factor-gradient issues, but it avoids adding spectral choices for artificial zero-noise coordinates.

Example. Let $\tilde{Q}(\theta) = \begin{pmatrix} 1 & \theta \\ \theta & 1 \end{pmatrix}$. The eigenvalues are $1 + \theta$ and $1 - \theta$, with gap $2|\theta|$. At $\theta = 0$ the covariance is perfectly smooth but the ordered eigenspace is not unique; raw autodiff through the spectral factor can be discontinuous or unstable even though the value-side covariance is benign.

- (vi) **Diagnostics must separate model law from regularization.** A collapsed route should report both the identity residual ((ii)) and the normal leakage ((iv)). Otherwise a finite likelihood can be produced by a different state-space model. For a scalar observation that loads on a deterministic coordinate, this difference is explicit: if the structural law is

$$k_t = 0, \quad y_t = k_t + e_t, \quad e_t \sim \mathcal{N}(0, r),$$

then $y_t \sim \mathcal{N}(0, r)$. If a collapsed covariance injects artificial variance η^2 into k_t , then the implemented likelihood uses $y_t \sim \mathcal{N}(0, r + \eta^2)$, and the one-period log-likelihood change is

$$\Delta \ell_t = -\frac{1}{2} \left[\log \frac{r + \eta^2}{r} + y_t^2 \left(\frac{1}{r + \eta^2} - \frac{1}{r} \right) \right].$$

A finite value of this likelihood is therefore not evidence that the structural transition was respected; it may only show that the nugget made the altered model numerically evaluable.

These diagnostics matter for the UKF because the UKF update is only as correct as the sigma-point cloud used to build its predictive moments. For a noise-free observation map h_θ and observation noise covariance R_t , the structural UKF forms points

$$x_t^{(j)} = F_\theta(a_t^{(j)}), \quad z_t^{(j)} = h_\theta(x_t^{(j)}),$$

and then computes

$$\hat{x}_t = \sum_j w_j^{(m)} x_t^{(j)}, \quad \hat{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \quad (19.85)$$

$$P_{xx,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_t)(x_t^{(j)} - \hat{x}_t)^\top, \quad (19.86)$$

$$S_t = \sum_j w_j^{(c)} (z_t^{(j)} - \hat{z}_t)(z_t^{(j)} - \hat{z}_t)^\top + R_t, \quad (19.87)$$

$$P_{xz,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_t)(z_t^{(j)} - \hat{z}_t)^\top. \quad (19.88)$$

The Gaussian UKF update then uses

$$v_t = y_t - \hat{z}_t, \quad K_t = P_{xz,t} S_t^{-1}, \quad (19.89)$$

$$\hat{x}_{t|t} = \hat{x}_t + K_t v_t, \quad P_{t|t} = P_{xx,t} - K_t S_t K_t^\top, \quad (19.90)$$

with log-likelihood contribution

$$\ell_t = -\frac{1}{2} \left[d_y \log(2\pi) + \log \det S_t + v_t^\top S_t^{-1} v_t \right]. \quad (19.91)$$

Thus a collapsed SVD route is not merely changing an implementation detail. It replaces the point cloud $\{F_\theta(a_t^{(j)})\}$ in (19.88) by $\{\tilde{F}_\theta(\tilde{a}_t^{(j)})\}$. The error enters through

$$\Delta \hat{z}_t = \tilde{z}_t - \hat{z}_t, \quad (19.92)$$

$$\Delta S_t = \tilde{S}_t - S_t, \quad (19.93)$$

$$\Delta P_{xz,t} = \tilde{P}_{xz,t} - P_{xz,t}, \quad (19.94)$$

$$\Delta K_t = \tilde{P}_{xz,t} \tilde{S}_t^{-1} - P_{xz,t} S_t^{-1}. \quad (19.95)$$

Even when the formal sigma-point rule is accurate for its own input law, (19.95) is controlled by whether the collapsed cloud matches the structural cloud on the particular functions h_θ , $x h_\theta(x)^\top$, and $h_\theta(x) h_\theta(x)^\top$ used in the UKF update. Equality of predictive laws is the robust sufficient condition; accidental equality of a few low-order moments is not a model-contract proof. This is the precise UKF sense in which the law-level diagnostics above are not optional metadata.

The six diagnostics above correspond to the UKF recursion as follows.

- (i) **Pushforward equality** is the condition that the two point clouds approximate the same expectations in (19.88). If ((i)) fails, then $\tilde{x}_t$, $\tilde{z}_t$, $\tilde{S}_t$, and $\tilde{P}_{xz,t}$ are moments of a different predictive law.
- (ii) **Identity residuals** test whether every propagated point used in (19.88) is an admissible structural state. If $\Delta_{\text{id}}^{\text{col}} > 0$, then the UKF averages include states that the DSGE transition assigns probability zero.
- (iii) **Point-count overhead** affects the number of evaluations of F_θ and h_θ required to build the same objects $\hat{x}_t$, $\hat{z}_t$, S_t , $P_{xz,t}$. Extra zero-noise directions do not add information; they add factorization and map-evaluation work before the update in (19.90).
- (iv) **Normal-support leakage** enters the UKF through S_t and $P_{xz,t}$. Artificial variance in a deterministic-completion direction changes the innovation covariance and the state-observation cross covariance, so it changes both the likelihood (19.91) and the gain $K_t = P_{xz,t} S_t^{-1}$.
- (v) **Spectral-gap gradient risk** is the derivative of the same UKF likelihood and update with respect to parameters. If $\partial_\theta \tilde{C}_t$ is unstable, then the gradients of $\tilde{S}_t$, $\tilde{P}_{xz,t}$, $\tilde{K}_t$, and $\tilde{\ell}_t$ inherit that instability.
- (vi) **Diagnostic separation** tells the reader whether (19.90) and (19.91) were evaluated under the structural law or under a regularized law. Without the identity and leakage diagnostics, a finite UKF likelihood cannot distinguish those two cases.

A one-dimensional observation example shows the mechanism without requiring filtering expertise. Let

$$m_t = \varepsilon_t, \quad k_t = m_t, \quad y_t = k_t + e_t, \quad \varepsilon_t \sim \mathcal{N}(0, q), \quad e_t \sim \mathcal{N}(0, r).$$

The structural predictive state has

$$\text{Var}(m_t) = q, \quad \text{Var}(k_t) = q, \quad \text{Cov}(m_t, k_t) = q, \quad S_t^{\text{str}} = q + r.$$

The Kalman gain components implied by the UKF moment objects are therefore

$$K_t^{\text{str}} = \frac{\text{Cov}((m_t, k_t), y_t)}{\text{Var}(y_t)} = \frac{1}{q+r} \begin{pmatrix} q \\ q \end{pmatrix}.$$

If the collapsed SVD route adds a small independent numerical variance η^2 to the deterministic-completion coordinate k_t , then

$$\tilde{k}_t = m_t + \eta \xi_t, \quad \xi_t \sim \mathcal{N}(0, 1),$$

and the UKF objects become

$$\tilde{S}_t = q + \eta^2 + r, \quad \tilde{K}_t = \frac{1}{q + \eta^2 + r} \begin{pmatrix} q \\ q + \eta^2 \end{pmatrix}.$$

The likelihood and update have changed:

$$\tilde{\ell}_t - \ell_t = -\frac{1}{2} \left[\log \frac{q + \eta^2 + r}{q + r} + y_t^2 \left(\frac{1}{q + \eta^2 + r} - \frac{1}{q + r} \right) \right], \quad (19.96)$$

$$\tilde{K}_{m,t} - K_{m,t} = -\frac{q\eta^2}{(q+r)(q+\eta^2+r)}, \quad (19.97)$$

$$\tilde{K}_{k,t} - K_{k,t} = \frac{\eta^2 r}{(q+r)(q+\eta^2+r)}. \quad (19.98)$$

In this example, the six diagnostics above appear in concrete form:

- (i) **Pushforward equality** is the difference between the structural law $k_t = m_t$ and the collapsed law $\tilde{k}_t = m_t + \eta \xi_t$. If $\eta > 0$, the predictive law is no longer supported on $k = m$, and the UKF moments in (19.10) are not the moments in (19.10).
- (ii) **Identity residuals** are measured by $c(m, k) = k - m$. The structural points have $c = 0$ pointwise, while the collapsed points have $c(\tilde{m}_t, \tilde{k}_t) = \eta \xi_t$ and therefore positive residual whenever the artificial direction is sampled.
- (iii) **Point-count overhead** is hidden in the construction of $\tilde{k}_t$. The structural rule needs points for the one genuine shock ε_t ; the collapsed route adds a second numerical direction ξ_t before computing the same scalar observation y_t .
- (iv) **Normal-support leakage** is the extra term η^2 in $\tilde{S}_t = q + \eta^2 + r$ in (19.10). That term is exactly the artificial variance normal to the structural line $k = m$, and it changes the likelihood and gain in (19.98).
- (v) **Spectral-gradient risk** enters if η^2 is produced by an SVD threshold, nugget branch, or spectral factor of a nearly singular covariance. Then derivatives of $\tilde{S}_t$, $\tilde{K}_t$, and $\tilde{\ell}_t$ inherit the derivative behavior of that spectral branch.
- (vi) **Diagnostic separation** is the need to report whether the likelihood is based on $S_t^{\text{str}} = q + r$ or on $\tilde{S}_t = q + \eta^2 + r$. Both are finite UKF likelihoods, but (19.98) shows that they are not the same likelihood unless $\eta = 0$.

So the nugget changes both the likelihood weight assigned to the observation and the way the observation is fed back into m_t and k_t . This is the UKF problem in concrete terms: the filter can remain numerically stable and still perform the prediction, likelihood evaluation, and update for a different state-space model.

The practical conclusion is therefore conservative. SVD is a good numerical backend for singular or nearly singular covariance factors, and it can be used inside a structurally correct sigma-point filter. But using SVD on a collapsed full-state covariance is usually a harder implementation contract: it has more linear algebra to audit, more fragile derivatives for HMC, and more ways for a regularization choice to become an unrecorded model change. The structural adapter is preferred because it states the economic law directly and leaves SVD to do the narrower job of factoring the covariance that actually belongs to that law.

For HMC promotion, one more distinction matters. Matrix backpropagation formulas for spectral decompositions contain denominators involving singular-value or eigenvalue gaps, so repeated or nearly repeated spectral values are derivative-risk regions [Ionescu et al., 2015]. That risk is orthogonal to the structural-law issue. A structurally correct SVD sigma-point value path still needs spectral-gap telemetry, finite-gradient stress tests, and compiled parity before it can be treated as a production HMC target.

19.11 Pruned DSGE and Adapter Implications

For pruned second-order DSGE approximations, the source DSGE project already uses the standard first-order/second-order decomposition:

$$x_t^f = h_x x_{t-1}^f + \eta \varepsilon_t, \quad (19.99)$$

$$x_t^s = h_x x_{t-1}^s + \frac{1}{2} H_{xx}(x_{t-1}^f \otimes x_{t-1}^f) + \frac{1}{2} h_{ss}, \quad (19.100)$$

$$y_t = g_x(x_t^f + x_t^s) + \frac{1}{2} G_{xx}(x_t^f \otimes x_t^f) + \frac{1}{2} g_{ss} + e_t. \quad (19.101)$$

That decomposition is necessary, but it is not sufficient for large DSGE filtering. It separates perturbation order; it does not automatically separate exogenous shock states from endogenous deterministic states inside x_t^f . A robust BayesFilter DSGE adapter should therefore carry both bookkeeping axes,

$$\text{perturbation order: } (x^f, x^s), \quad \text{structural timing: } (m, k).$$

The adapter-facing implication is simple: the model must expose the exogenous transition, the deterministic endogenous transition, the observation map, and a declaration of which variables define the stochastic integration space. For a sigma-point implementation, that adapter contract should compile to the following concrete objects:

- (i) dimensions and names for the filtered state, the declared innovation, and deterministic-completion coordinates;
- (ii) a function that maps each point $(x_{t-1}^{(j)}, \varepsilon_t^{(j)})$ to $x_t^{(j)} = F_\theta(x_{t-1}^{(j)}, \varepsilon_t^{(j)})$;
- (iii) a deterministic-identity residual, evaluated on every propagated point;
- (iv) an observation map evaluated only after structural completion;

- (v) an approximation label if any legacy full-state or regularized covariance path is used instead.

The pruned perturbation decomposition tells the adapter what equations exist; the structural metadata tells the filter which variables receive quadrature.

19.12 Source-Project Lesson

The recent source-project audit found a concrete instance of the structural failure discussed in this chapter. The default DSGE SVD sigma-point adapter uses a collapsed transition of the form

$$Q_w = \eta\eta^\top + \epsilon I, \quad x_{t|t-1}^f = h_x x_{t-1|t-1}^f,$$

and propagates the covariance as

$$P_{t|t-1} = h_x P_{t-1|t-1} h_x^\top + Q_w.$$

It tracks the pruned x^s correction separately, but it does not expose a partition between exogenous and endogenous coordinates inside x^f . For the smallest NK example, where the states are only the exogenous demand and monetary-policy shocks, this is not a serious modeling error. For Rotemberg NK, SGU, EZ, and NAWM-scale DSGE models, however, the state vector contains endogenous/predetermined components, so treating every coordinate as an exchangeable noisy Gaussian coordinate becomes a structural filtering bug rather than a mere numerical tuning issue.

This does not mean the generic SVD sigma-point machinery is unusable. The source audit found a more precise split: a generic sigma-point path that accepts user-supplied transition points or a structural transition map can be reused as an approximate backend, while the default mixed-DSGE adapter must be gated unless it exposes the exogenous/endogenous partition and deterministic completion. In other words, reuse the factorization and diagnostics; do not reuse an adapter that silently changes the stochastic integration space.

19.13 Validation Gates and Final Policy Rule

No future BayesFilter report should claim that a nonlinear DSGE filter is value-valid, gradient-valid, sampler-usable, converged, or production-ready in the sense of Chapter 52 unless the following structural tests are passed or explicitly waived with written justification:

1. **Metadata test:** the model exposes exogenous, endogenous, and deterministic state blocks, and their dimensions match the shock-impact matrix and transition maps.
2. **Constraint-support test:** propagated sigma points or particles satisfy deterministic identities to numerical tolerance.
3. **Linear recovery test:** when the structural transition is linear, the structural nonlinear filter recovers the exact Kalman likelihood up to the declared approximation tolerance.

4. **Degenerate-transition test:** zero-innovation endogenous rows remain deterministic except for explicitly declared numerical regularization.
5. **SVD/factorization test:** if a square-root or SVD sigma-point backend is used, diagnostics state whether the factor belongs to the structural integration variable or to a legacy full-state approximation; the latter must carry an approximation label.
6. **Toy DSGE test:** a controlled model with

$$m_t = \rho m_{t-1} + \sigma \varepsilon_t, \quad k_t = f(k_{t-1}, m_{t-1}, m_t),$$

compares structural propagation against a dense quadrature or high-particle reference.

7. **Case-study test:** Rotemberg NK, SGU, and any EZ/NAWM target must document which state coordinates are shock-driven, which are deterministic conditional on the shock path, and how the filter honors that timing.
8. **Derivative-promotion test:** any HMC claim through SVD or eigenspectral factors must pass spectral-gap, finite-gradient, and eager/compiled parity checks.

19.13.1 Common Misunderstandings

The most common reader errors in this area are:

- (i) a deterministic endogenous state need not have zero one-step predictive variance; it can inherit randomness through the declared stochastic block;
- (ii) a singular or rank-deficient transition covariance is not, by itself, a modeling error in the exact linear-Gaussian case;
- (iii) numerical regularization is not the same thing as declaring a new stochastic state coordinate;
- (iv) a full-state nonlinear approximation is not automatically forbidden, but it must be labeled as an approximation if it enlarges the stochastic space beyond the structural transition contract.

For nonlinear DSGE filtering, the stochastic integration variables are not automatically the entries of the state vector. They are the variables declared stochastic by the structural transition. Deterministic endogenous states must be computed, not noised.

If an implementation violates this rule, it must be labeled as an artificial approximation and justified against a reference. Otherwise, the filter is solving a different approximate state-space model relative to the declared structural law. Failing one of the validation gates above does not automatically make a backend useless, but it does mean the strongest claim labels from Chapter 52 are not yet justified.

Chapter 20

Particle-Filter Foundations

This chapter develops the particle-filter baseline for the differentiable particle-filter program. The goal is not yet to make the filter differentiable. The goal is to establish, in BayesFilter notation, the nonlinear filtering recursion, the empirical particle approximation, the bootstrap particle-filter likelihood estimator, and the exact point at which the later differentiable program departs from the classical construction.

Particle methods matter because the nonlinear filtering problem is generally not closed under finite-dimensional moments. When the predictive law or filtering law is materially non-Gaussian, a Gaussian closure such as the EKF or UKF may be a useful approximation but no longer defines the exact filtering object. The particle filter instead approximates the filtering law directly as a weighted random measure. This makes it the natural baseline for any later attempt to construct particle-flow, resampling-relaxed, or differentiable particle methods.

20.1 Objects, assumptions, and claim-status vocabulary

The later DPF chapters rely on this chapter as the reference object. The following notation is therefore fixed before introducing any differentiable relaxation.

Object	Meaning in this chapter
$x_t \in \mathbb{R}^{n_x}$	latent state at time t
$y_t \in \mathbb{R}^{n_y}$	observation at time t
$\theta \in \Theta$	fixed model parameter while the filtering recursion is defined
$p_\theta(x_t \mid x_{t-1})$	transition density or Markov kernel density
$p_\theta(y_t \mid x_t)$	observation density, evaluated at the realized observation y_t
$p_\theta(x_t \mid y_{1:t-1})$	predictive law before assimilating y_t
$p_\theta(x_t \mid y_{1:t})$	filtering law after assimilating y_t
$p_\theta(y_t \mid y_{1:t-1})$	one-step likelihood factor
$q_\theta(x_t \mid x_{0:t-1}, y_{1:t})$	sequential proposal density
$\tilde{w}_t^{(i)}, w_t^{(i)}$	unnormalized and normalized particle weights
$a_{t-1}^{(i)}$	ancestor index selected during resampling
$\hat{\pi}_t^N$	weighted empirical approximation to the filtering law
Z_t^N	average incremental bootstrap weight at time t
$\hat{p}_\theta^N(y_{1:T})$	bootstrap particle-filter estimator of the marginal likelihood

The standing assumptions for the formulas below are the standard ones needed to make the displayed densities and expectations well defined: the transition and observation kernels are dominated by reference measures; the proposal has support wherever the trajectory target has support; the relevant integrals are finite; and resampling schemes are conditionally unbiased in the sense that the expected empirical measure after resampling equals the normalized empirical measure before resampling. These assumptions are not cosmetic. They mark the difference between an exact model identity, a finite-particle Monte Carlo estimator statement, and an implementation diagnostic.

The chapter uses the following claim-status vocabulary. The filtering recursion and likelihood factorization are exact model identities. Weighted particles are finite- N Monte Carlo approximations. The bootstrap likelihood estimator is an unbiased estimator of the likelihood, not of the log likelihood and not of the score. ESS and ancestor-count summaries are diagnostics, not proofs of posterior correctness or failure.

20.2 DPF-block notation and claim-status index

This chapter begins the DPF block, so the following index fixes the notation used across the later particle-flow, PF-PF, resampling/OT, learned-OT, HMC, and debugging chapters. A symbol may acquire a method-specific subscript in a later chapter, but its mathematical role should not change silently.

Table 20.1: DPF-block notation index.

Category	Symbols	Fixed role across the DPF block
State and observations	$x_t, x_{0:t}, y_t, y_{1:t}$	Latent state, latent trajectory, current observation, and observation history for the state-space model.
Parameters	θ, u	Structural/model parameter and, in HMC chapters, unconstrained sampler coordinate after any parameter transform.
Filtering laws	$p_\theta(x_t y_{1:t-1}), p_\theta(x_t y_{1:t})$	Predictive and filtering laws; these are the reference statistical objects.
Particles and ancestors	$x_t^{(i)}, \tilde{x}_t^{(j)}, a_{t-1}^{(i)}, a_j$	Pre- or post-update particles, equal-weight output particles, and discrete ancestor indices.
Weights and empirical measures	$\tilde{w}_t^{(i)}, w_t^{(i)}, \hat{\pi}_t^N, \hat{\pi}_{t,\text{eq}}^N$	Unnormalized/normalized weights and weighted/equal-weight empirical measures.
Likelihood estimates	$Z_t^N, \hat{p}_\theta^N(y_{1:T}), \widehat{L}(y \theta, \omega)$	Average incremental particle weight, bootstrap likelihood estimator, and generic randomized likelihood estimator used in pseudo-marginal discussion.
Proposals	$q_\theta, q_{t,0}, q_{t,1}, \gamma_t$	Sequential proposal, pre-flow proposal, post-flow proposal, and unnormalized one-step target density.
Flow maps and Jacobians	$\pi_{t,\lambda}, v_{t,\lambda}, \Phi_t, J_t, D\Phi_t$	Homotopy density, pseudo-time velocity field, flow map, and forward Jacobian of the flow.
Transport couplings	$\Pi, \Pi_0^*, \Pi_\varepsilon^*, \Pi_{\varepsilon,K}$	Unregularized, entropy-regularized, and finite-Sinkhorn transport couplings.
Sinkhorn scalings	$C, K_\varepsilon, a, b, \varepsilon, K$	Cost matrix, Gibbs kernel, row/column scalings, regularization level, and finite iteration count.
Learned maps	$T_\psi, Y_\tau, R_{\psi,\tau}, \mathcal{D}$	Learned student map, teacher output, student-teacher residual, and training distribution.
HMC scalar and gradient	$\ell_\star(u), U_\star(u), g(u), H_\star(u, p)$	Named log target, potential, gradient used by the integrator, and Hamiltonian. The value and gradient must refer to the same scalar.

Table 20.2: DPF-block claim-status vocabulary.

Status phrase	Use only when	Disallowed shortcut
Exact model identity	The statement follows from the specified state-space model, Bayes' rule, change of variables, or a named exact special case.	Calling a closure, relaxation, solver, or learned map exact because it is written in equations.
Unbiased particle estimator	The expectation is taken over the stated particle randomness and the target object is the likelihood or normalizing constant.	Inferring unbiased log likelihood, score, pathwise gradient, or HMC validity.
Consistent approximation	A convergence regime and target object are stated.	Treating increasing N or decreasing tolerance as a finite-sample proof.
Approximate closure	A Gaussian, local-linear, finite-particle, or numerical closure replaces the reference filtering object.	Delaying the approximation label until a later chapter.
Relaxed target	A smooth resampling or transport object replaces the classical categorical operation and the scalar target is the relaxed object.	Implying the original posterior is preserved without a correction.
Learned surrogate	A trained map approximates a named teacher under a declared training distribution.	Using runtime, smoothness, or training loss as posterior validity evidence.
Engineering hypothesis	The claim is a bounded recommendation or diagnostic rule, with required tests stated.	Presenting implementation preference as mathematical validation.
Unsupported claim	The statement lacks a local derivation, reviewed source support, or documented test evidence.	Leaving the statement in the main argument without weakening or removing it.

20.3 Nonlinear state-space model and filtering recursion

Let $x_t \in \mathbb{R}^{n_x}$ denote the latent state and $y_t \in \mathbb{R}^{n_y}$ the observation at time t . Fix a parameter vector $\theta \in \Theta \subset \mathbb{R}^{d_\theta}$. The nonlinear state-space model is specified by a transition density and an observation density,

$$x_t \mid x_{t-1} \sim p_\theta(x_t \mid x_{t-1}), \quad (20.1)$$

$$y_t \mid x_t \sim p_\theta(y_t \mid x_t). \quad (20.2)$$

Write $y_{1:t} = (y_1, \dots, y_t)$ for the observation history. The predictive law and the filtering law satisfy the standard Bayesian recursion

$$p_\theta(x_t \mid y_{1:t-1}) = \int p_\theta(x_t \mid x_{t-1}) p_\theta(x_{t-1} \mid y_{1:t-1}) dx_{t-1}, \quad (20.3)$$

$$p_\theta(x_t \mid y_{1:t}) = \frac{p_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})}{p_\theta(y_t \mid y_{1:t-1})}, \quad (20.4)$$

where the one-step predictive density of the observation is

$$p_\theta(y_t \mid y_{1:t-1}) = \int p_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1}) dx_t. \quad (20.5)$$

The prediction equation is the Chapman–Kolmogorov identity applied to the conditional law of x_{t-1} given the observations already seen. For any measurable set B ,

$$\mathbb{P}_\theta(x_t \in B \mid y_{1:t-1}) = \int \mathbb{P}_\theta(x_t \in B \mid x_{t-1}) p_\theta(x_{t-1} \mid y_{1:t-1}) dx_{t-1}.$$

When the transition kernel has density $p_\theta(x_t \mid x_{t-1})$, this set identity gives (20.3). The update equation is Bayes' rule with the predictive law as prior and the observation density as likelihood:

$$p_\theta(x_t \mid y_{1:t}) = \frac{p_\theta(y_t \mid x_t, y_{1:t-1})p_\theta(x_t \mid y_{1:t-1})}{p_\theta(y_t \mid y_{1:t-1})}.$$

The conditional-independence convention of the state-space model reduces $p_\theta(y_t \mid x_t, y_{1:t-1})$ to $p_\theta(y_t \mid x_t)$, giving (20.4). Finally, repeated use of the chain rule gives the marginal likelihood factorization

$$p_\theta(y_{1:T}) = p_\theta(y_1) \prod_{t=2}^T p_\theta(y_t \mid y_{1:t-1}),$$

with the same notation covering $t = 1$ by interpreting $y_{1:0}$ as the empty history. The marginal likelihood then factorizes as

$$p_\theta(y_{1:T}) = \prod_{t=1}^T p_\theta(y_t \mid y_{1:t-1}). \quad (20.6)$$

Equations (20.3)–(20.6) are exact. The difficulty is computational rather than conceptual: outside special classes such as the linear-Gaussian case, the integrals are rarely available in closed form. Particle filters replace these exact distributions by random empirical approximations whose quality improves with particle count under appropriate assumptions.

20.4 Empirical filtering measures

A particle filter represents the filtering law by a weighted empirical measure. At time t , let $\{(x_t^{(i)}, w_t^{(i)})\}_{i=1}^N$ be particles and normalized weights, with $w_t^{(i)} \geq 0$ and $\sum_{i=1}^N w_t^{(i)} = 1$. The empirical filtering measure is

$$\hat{\pi}_t^N(dx) = \sum_{i=1}^N w_t^{(i)} \delta_{x_t^{(i)}}(dx), \quad (20.7)$$

where δ_z is the Dirac point mass at z . The approximation of a test function φ under the filtering law is then

$$\int \varphi(x) \hat{\pi}_t^N(dx) = \sum_{i=1}^N w_t^{(i)} \varphi(x_t^{(i)}). \quad (20.8)$$

Thus the filter approximates the entire posterior law by a finite atomic measure, not only its first two moments.

This is the main structural distinction between particle methods and the Gaussian-approximation filters of the previous chapters. The particle filter is not built by closing the recursion in a finite-dimensional moment family. Instead, it propagates a random discrete approximation to the law itself.

20.5 Sequential importance sampling

To derive the basic particle recursion, introduce a proposal density $q_\theta(x_t \mid x_{0:t-1}, y_{1:t})$ from which particles can be sampled. Assume that the proposal is positive whenever the trajectory target below is positive. Without this support condition the importance ratio is not a valid correction; particles may miss regions that carry target probability. Suppose a full trajectory proposal factors as

$$q_\theta(x_{0:t} \mid y_{1:t}) = q_\theta(x_0 \mid y_1) \prod_{s=1}^t q_\theta(x_s \mid x_{0:s-1}, y_{1:s}). \quad (20.9)$$

The target filtering distribution over trajectories is proportional to

$$p_\theta(x_{0:t}, y_{1:t}) = p_\theta(x_0) \prod_{s=1}^t p_\theta(x_s \mid x_{s-1}) p_\theta(y_s \mid x_s). \quad (20.10)$$

The unnormalized importance weight is therefore

$$\tilde{w}_t(x_{0:t}) = \frac{p_\theta(x_{0:t}, y_{1:t})}{q_\theta(x_{0:t} \mid y_{1:t})}. \quad (20.11)$$

This ratio is a trajectory-level object. It is not yet a resampling rule and not yet a differentiable likelihood map. It is the Radon–Nikodym correction that converts expectations under the proposal path law into expectations under the unnormalized trajectory target, provided the support and integrability conditions above hold. Using the factorization in (20.9)–(20.10), one obtains the recursive update

$$\tilde{w}_t^{(i)} = \tilde{w}_{t-1}^{(i)} \frac{p_\theta(y_t \mid x_t^{(i)}) p_\theta(x_t^{(i)} \mid x_{t-1}^{(i)})}{q_\theta(x_t^{(i)} \mid x_{0:t-1}^{(i)}, y_{1:t})}. \quad (20.12)$$

Normalizing gives

$$w_t^{(i)} = \frac{\tilde{w}_t^{(i)}}{\sum_{j=1}^N \tilde{w}_t^{(j)}}. \quad (20.13)$$

Equation (20.12) is the generic sequential importance sampling (SIS) update. It already shows the two main mathematical stresses of particle filtering:

- (i) the proposal must be chosen carefully to avoid explosive weight variability;
- (ii) even when the estimator is well defined, the normalized weights tend to concentrate over time, which leads to degeneracy.

20.6 Sequential importance resampling and the bootstrap filter

Sequential importance sampling alone is typically unstable over long horizons, because the weights become increasingly uneven. Sequential importance resampling (SIR) introduces a resampling step to refresh the cloud when the weight distribution becomes too concentrated.

The most classical special case is the bootstrap particle filter, which uses the transition prior itself as the proposal:

$$q_\theta(x_t \mid x_{0:t-1}, y_{1:t}) = p_\theta(x_t \mid x_{t-1}). \quad (20.14)$$

Substituting (20.14) into (20.12) gives the simplified incremental weight

$$\tilde{w}_t^{(i)} \propto w_{t-1}^{(i)} p_\theta(y_t \mid x_t^{(i)}). \quad (20.15)$$

The bootstrap recursion at time t can therefore be written as:

- Step 1: Select ancestors:** if resampling is performed before propagation, draw $a_{t-1}^{(i)}$ from the current normalized weights; otherwise keep $a_{t-1}^{(i)} = i$.
- Step 2: Propagate:** sample $x_t^{(i)} \sim p_\theta(x_t \mid x_{t-1}^{(a_{t-1}^{(i)})})$.
- Step 3: Weight:** compute $\tilde{w}_t^{(i)} = p_\theta(y_t \mid x_t^{(i)})$ and normalize the weights.
- Step 4: Resample:** after recording the likelihood factor, either resample using the normalized weights and reset weights to $1/N$, or carry the weighted cloud forward until the next resampling decision.

This yields the standard bootstrap/SIR particle filter of [Gordon et al. \[1993\]](#), elaborated in the SMC text of [Doucet et al. \[2001\]](#). The particle-MCMC literature [[Andrieu and Roberts, 2009](#), [Andrieu et al., 2010](#)] later uses the likelihood estimator generated by this construction, but that pseudo-marginal role is a statement about an unbiased value-side estimator inside a Metropolis–Hastings construction. It is not a statement that the realized resampling path is smooth in θ .

20.7 The bootstrap particle-filter likelihood estimator

The bootstrap particle filter supplies a natural estimator of the factors in the marginal-likelihood decomposition (20.6). Define the average incremental weight at time t by

$$Z_t^N = \frac{1}{N} \sum_{i=1}^N \tilde{w}_t^{(i)}. \quad (20.16)$$

The corresponding likelihood estimator is

$$\hat{p}_\theta^N(y_{1:T}) = \prod_{t=1}^T Z_t^N. \quad (20.17)$$

The estimator-status result BayesFilter needs at this stage is the following.

Proposition 20.1 (Bootstrap particle-filter likelihood-estimator status). *Fix the observations $y_{1:T}$ and the parameter θ . Assume the model densities are dominated and integrable, the bootstrap propagation step samples from $p_\theta(x_t \mid x_{t-1})$, and the Feynman–Kac quantities used below are finite: for every bounded measurable φ used in the induction, the*

operator $Q_t\varphi$ is well defined, the incremental weights have finite conditional expectation, and the displayed conditional expectations are finite. Assume also that every resampling step used by the algorithm is conditionally unbiased for the current normalized empirical measure. Here conditional unbiasedness means that, for every bounded measurable test function ψ in the same finite-expectation class, the expected post-resampling empirical average equals the pre-resampling weighted empirical average, conditional on the current weighted cloud. Biased or deterministic approximation schemes that do not satisfy this identity are not covered by the proposition. Then the estimator (20.17) is an unbiased estimator of the marginal likelihood:

$$\mathbb{E}[\hat{p}_\theta^N(y_{1:T})] = p_\theta(y_{1:T}).$$

This claim is about the likelihood itself. It does not claim that $\log \hat{p}_\theta^N(y_{1:T})$ is an unbiased estimator of $\log p_\theta(y_{1:T})$, and it does not claim that differentiating a realized particle system gives an unbiased score.

Proof. For compactness, define the bootstrap Feynman–Kac operator

$$(Q_t\varphi)(x_{t-1}) = \int \varphi(x_t) p_\theta(y_t | x_t) p_\theta(x_t | x_{t-1}) dx_t$$

for bounded measurable test functions φ . Also define the unnormalized filtering functional

$$\gamma_t(\varphi) = p_\theta(y_{1:t}) \int \varphi(x_t) p_\theta(x_t | y_{1:t}) dx_t.$$

The prediction/update recursion implies $\gamma_t(\varphi) = \gamma_{t-1}(Q_t\varphi)$ and $\gamma_t(1) = p_\theta(y_{1:t})$.

The filtration convention is as follows. At the end of time $t - 1$ the algorithm has a weighted empirical measure $\hat{\pi}_{t-1}^N$ and a normalizing-constant estimate $\prod_{s=1}^{t-1} Z_s^N$. Any resampling randomness used to choose the particles propagated at time t is then drawn, and the transition propagation at time t is conditionally independent across particles given that post-resampling cloud. All test functions in the induction are bounded measurable, or otherwise integrable enough that the conditional expectations below are finite.

Let $\mathcal{F}_{t-1}^N$ denote the sigma-field generated by the particle system just before propagation at time t , after any resampling decision at time $t - 1$. Let $\bar{\pi}_{t-1}^N$ be the empirical measure of the particles from which the transition propagation is drawn. If resampling is used, the assumption of conditional unbiasedness means

$$\mathbb{E}[\bar{\pi}_{t-1}^N(\psi) | \hat{\pi}_{t-1}^N] = \hat{\pi}_{t-1}^N(\psi)$$

for bounded ψ ; if resampling is not used, then $\bar{\pi}_{t-1}^N = \hat{\pi}_{t-1}^N$.

After bootstrap propagation and weighting,

$$Z_t^N \hat{\pi}_t^N(\varphi) = \frac{1}{N} \sum_{i=1}^N p_\theta(y_t | x_t^{(i)}) \varphi(x_t^{(i)}).$$

Conditional on $\mathcal{F}_{t-1}^N$, the propagated particles are drawn from the transition kernel, so

$$\mathbb{E}[Z_t^N \hat{\pi}_t^N(\varphi) | \mathcal{F}_{t-1}^N] = \bar{\pi}_{t-1}^N(Q_t\varphi).$$

Taking the conditional expectation over any resampling randomness replaces $\bar{\pi}_{t-1}^N$ by $\hat{\pi}_{t-1}^N$ in expectation. Multiplying by the previous normalizing-constant estimate, which is measurable with respect to the previous particle history, gives

$$\mathbb{E} \left[\left(\prod_{s=1}^t Z_s^N \right) \hat{\pi}_t^N(\varphi) \right] = \mathbb{E} \left[\left(\prod_{s=1}^{t-1} Z_s^N \right) \hat{\pi}_{t-1}^N(Q_t \varphi) \right].$$

Iterating this identity and using $\gamma_t(\varphi) = \gamma_{t-1}(Q_t \varphi)$ gives

$$\mathbb{E} \left[\left(\prod_{s=1}^t Z_s^N \right) \hat{\pi}_t^N(\varphi) \right] = \gamma_t(\varphi).$$

Taking $\varphi \equiv 1$ yields

$$\mathbb{E} \left[\prod_{s=1}^t Z_s^N \right] = \gamma_t(1) = p_\theta(y_{1:t}).$$

Setting $t = T$ proves the displayed likelihood-unbiasedness identity. This is the local BayesFilter version of the standard SMC normalizing-constant unbiasedness argument; see [Doucet et al. \[2001\]](#) and [Andrieu et al. \[2010\]](#) for the full particle-system treatment. $\square$

This proposition is mathematically central for the later DPF program. It tells us that the classical bootstrap particle filter supplies an unbiased estimator of the marginal likelihood, even though it does not supply a pathwise differentiable likelihood map. Later differentiable constructions must therefore be compared against this baseline carefully: a differentiable surrogate may be smoother to optimize or differentiate, but it need not preserve this exact estimator status.

20.8 Differentiability boundary

The unbiasedness result above is a value-side Monte Carlo statement. It should not be read as a pathwise differentiability statement. For a fixed random seed, the propagation step may be reparameterizable in some models, but the resampling step is a categorical map from weights to ancestor indices. Small changes in θ can leave the selected ancestor unchanged over an interval and then switch it discontinuously when a cumulative weight boundary is crossed. The realized resampled cloud is therefore not a smooth function of θ .

This distinction creates three separate objects that later chapters must not blur:

- (i) the true likelihood $p_\theta(y_{1:T})$;
- (ii) an unbiased randomized estimator $\hat{p}_\theta^N(y_{1:T})$ of that likelihood;
- (iii) a pathwise differentiable scalar produced by a relaxed, transported, or learned particle computation.

The first object is the statistical target, the second can support pseudo-marginal value-side constructions under their own assumptions, and the third may be useful for optimization or HMC only after its target status is stated. A gradient through the third object is not automatically a gradient of the first or of the expectation of the second.

20.9 Degeneracy and effective sample size

The main numerical weakness of the particle filter is weight degeneracy. As the observation sequence grows, the normalized weights frequently concentrate on a small subset of particles. A standard summary is the effective sample size (ESS),

$$\text{ESS}_t = \frac{1}{\sum_{i=1}^N (w_t^{(i)})^2}. \quad (20.18)$$

The extreme cases are informative:

- if all weights are equal, then $\text{ESS}_t = N$;
- if one particle carries almost all the mass, then $\text{ESS}_t \approx 1$.

When ESS collapses, the atomic approximation (20.7) still exists, but its practical variance may become extremely large. Resampling alleviates this by discarding low-weight particles and duplicating high-weight particles, but resampling does not remove the deeper dimensionality problem. The classical literature shows that in many high-dimensional settings the number of particles needed to stabilize the weight distribution can grow very rapidly with the effective dimension of the observation problem [Bengtsson et al., 2008, Snyder et al., 2008].

This is the exact point where the later chapters enter. Particle-flow methods, PF-PF corrections, differentiable resampling, OT relaxations, and learned OT operators are all responses to some combination of:

- (i) degeneracy under standard resampling;
- (ii) lack of differentiability of the resampling step;
- (iii) computational burden of high-quality resampling or transport maps;
- (iv) mismatch between a useful learning objective and a valid HMC target.

A mathematically clean DPF monograph must therefore begin here: by making the classical particle-filter object precise before modifying it.

20.10 Baseline audit table

Table 20.3: Classical particle-filter baseline audit.

Object or claim	Status	Assumptions	Implementation diagnostic
Filtering recursion and likelihood factorization	Exact model identity	Dominated transition and observation kernels; fixed θ ; finite normalizers.	Compare predictive and filtering factors against a Kalman reference in a linear-Gaussian model.
Trajectory importance ratio	Exact algebraic identity for the specified target/proposal pair	Proposal support covers target support; finite weights.	Check target and proposal log densities separately on a small trajectory.
Weighted empirical measure	Finite- N Monte Carlo approximation	Particle system generated by the declared proposal and weighting rule.	Compare bounded test-function expectations across increasing N .

Object or claim	Status	Assumptions	Implementation diagnostic
Bootstrap likelihood estimator	Unbiased estimator of likelihood, not log likelihood or score	Bootstrap proposal; conditionally unbiased resampling; integrable likelihood factors.	In a controlled model, average repeated estimates and compare to the analytic likelihood.
ESS and ancestor diagnostics	Engineering diagnostic, not posterior-validity proof	Weights normalized and finite; diagnostic interpreted with model dimension and observation informativeness.	Track ESS, log-weight variance, and unique ancestor count by time and parameter region.
Pathwise gradient through resampling	Not supplied by the classical bootstrap filter	Categorical ancestor map is used without relaxation.	Finite-difference a fixed-seed resampled path and identify discontinuities or zero-almost-everywhere regions.

20.11 What this chapter does and does not claim

This chapter establishes the exact nonlinear filtering recursion and the classical bootstrap particle-filter baseline. It does *not* yet claim any of the following:

- a pathwise differentiable likelihood map;
- a particle-flow approximation;
- a proposal-corrected flow filter;
- a differentiable resampling rule;
- an HMC-ready particle backend.

Those belong to the later chapters. The role of the present chapter is more basic and more important: it fixes the reference object against which every later DPF construction must be interpreted.

Chapter 21

Particle-Flow Foundations

This chapter develops the mathematical foundations of particle-flow methods. Where the previous chapter fixed the classical particle-filter baseline, the present chapter studies a different idea: instead of relying on repeated reweighting and resampling to move from the predictive law to the filtering law, one may attempt to transport particles continuously along a pseudo-time path.

This transport point of view is attractive because it addresses a real weakness of classical particle filters: in sharply informative or moderately high-dimensional problems, repeated weight concentration can make the empirical measure numerically poor long before the state-space model itself ceases to be well defined. Particle-flow methods attempt to reduce that instability by constructing a velocity field that pushes the predictive cloud toward the posterior cloud. The mathematical difficulty is that the transport law is not free: one must define the interpolating densities, the continuity equation, the velocity field, and the assumptions under which the resulting flow is exact or only approximate.

This chapter therefore treats particle flow first as a transport problem, not as a final likelihood engine. In particular, the present chapter does *not* yet claim that a raw particle flow defines the final HMC target. It first develops the homotopy and flow mathematics, identifies the Gaussian-closure approximation behind the classical EDH construction, and states the exact special case in which the transport recovers the Kalman update. The subsequent PF-PF chapter then asks how to restore a clearer proposal/target relationship by importance correction.

21.1 Objects, notation, and source roles

The particle-flow literature is easy to misread because the same word "flow" can refer to several distinct objects: a density path, a velocity field, a particle ODE, a numerical map produced by integrating that ODE, or a proposal mechanism later corrected by an importance weight. This chapter uses the following objects.

Symbol or phrase	Meaning in this chapter
$\pi_{t t-1}$	predictive law of x_t given $y_{1:t-1}$ before assimilating y_t
$\pi_{t t}$	filtering law of x_t given $y_{1:t}$ after assimilating y_t
$\pi_{t,\lambda}$	normalized homotopy density at pseudo-time $\lambda \in [0, 1]$
$Z_t(\lambda)$	normalizing constant of the homotopy path
$v_{t,\lambda}$	velocity field whose flow is intended to transport $\pi_{t,\lambda}$
$\Phi_{t,\lambda}$	flow map generated by the particle ODE from pseudo-time 0 to λ
$A_t(\lambda), b_t(\lambda)$	affine EDH velocity coefficients under a Gaussian closure
$P_{t,\lambda}, m_{t,\lambda}$	covariance and mean of the Gaussian homotopy closure
$G_t^{(i)}$	particle-local observation Jacobian in LEDH
$\Lambda_t, \Lambda_t^{(i)}$	global and particle-local likelihood precision contributions
$\eta_t^{(i)}$	particle-local Gaussian information vector in LEDH

The source role is also deliberately limited. Daum–Huang particle flow [Daum and Huang, 2008] motivates the EDH construction. Invertible particle-flow particle filters [Li and Coates, 2017] explain why later chapters must treat a flow as a proposal map with a density correction rather than as an automatically correct posterior sampler. Particle-flow applications such as Hu and van Leeuwen [2021] motivate numerical and high-dimensional caution. These sources locate the method family; they do not remove the need for the local derivations below, and they do not establish a final HMC target for BayesFilter.

21.2 From discrete reweighting to continuous transport

Let the predictive law at time t be

$$\pi_{t|t-1}(dx) = p_\theta(x_t \in dx \mid y_{1:t-1}),$$

and let the filtering law be

$$\pi_{t|t}(dx) = p_\theta(x_t \in dx \mid y_{1:t}).$$

The bootstrap particle filter approximates the transition from $\pi_{t|t-1}$ to $\pi_{t|t}$ by assigning observation-driven weights and then resampling the cloud. Particle-flow methods replace this discrete update by a continuous pseudo-time transport from a prior-like distribution to a posterior-like distribution. The intent is to move particles toward regions of high posterior density before resampling is needed, thereby reducing weight collapse.

The conceptual shift is therefore from

- (i) a *weighted empirical update* followed by discrete resampling,

to

- (ii) a *continuous transport path* whose endpoint approximates the filtering law.

The advantage of the second view is geometric: the cloud is moved by a velocity field rather than by repeated duplication and deletion of particles. The cost is mathematical and numerical: the transport field must be derived, integrated, and interpreted correctly.

21.3 Homotopy path between predictive and filtering laws

Particle-flow methods introduce a pseudo-time parameter $\lambda \in [0, 1]$ and construct an interpolating family of densities $\pi_{t,\lambda}$. Let

$$\bar{\pi}_{t,\lambda}(x) = p_\theta(x_t = x \mid y_{1:t-1})p_\theta(y_t \mid x)^\lambda$$

be the unnormalized path. The normalized path is

$$\pi_{t,\lambda}(x) = \frac{\bar{\pi}_{t,\lambda}(x)}{Z_t(\lambda)}.$$

Equivalently, in the classical Daum–Huang construction one defines the log-homotopy

$$\log \pi_{t,\lambda}(x) = \log p_\theta(x_t = x \mid y_{1:t-1}) + \lambda \log p_\theta(y_t \mid x) - \log Z_t(\lambda), \quad (21.1)$$

where

$$Z_t(\lambda) = \int p_\theta(x_t \mid y_{1:t-1})p_\theta(y_t \mid x_t)^\lambda dx_t \quad (21.2)$$

is the normalizing constant.

This definition deserves more emphasis than a short formula often receives. First, the endpoint interpretation is exact:

$$\pi_{t,0}(x) = p_\theta(x_t = x \mid y_{1:t-1}), \quad \pi_{t,1}(x) = p_\theta(x_t = x \mid y_{1:t}).$$

Thus the family joins the predictive law to the filtering law without changing what the endpoints mean. Second, the normalization is not merely a technical constant. It is the term that keeps the intermediate object a probability density rather than an unnormalized score. Third, the homotopy does not yet choose a transport field; it only chooses the density path the field is supposed to realize.

Differentiating (21.1) with respect to λ gives

$$\partial_\lambda \log \pi_{t,\lambda}(x) = \log p_\theta(y_t \mid x) - \partial_\lambda \log Z_t(\lambda). \quad (21.3)$$

The term involving $Z_t(\lambda)$ is crucial: it keeps the intermediate density normalized. Omitting it may produce a convenient algebraic simplification, but it changes the law being transported. This is one of the first places where a future implementation can quietly go wrong: if the code is written only in terms of an unnormalized path, the resulting transported object may no longer match the intended predictive-to-filtering homotopy.

One can make the normalizer term explicit. Differentiating (21.2) gives

$$\frac{dZ_t(\lambda)}{d\lambda} = \int p_\theta(x_t \mid y_{1:t-1})p_\theta(y_t \mid x_t)^\lambda \log p_\theta(y_t \mid x_t) dx_t.$$

This differentiation under the integral sign is not automatic. A sufficient local condition is that, on the pseudo-time interval being used, the likelihood is positive where the predictive density has support and there is an integrable envelope dominating

$$p_\theta(x_t \mid y_{1:t-1})p_\theta(y_t \mid x_t)^\lambda |\log p_\theta(y_t \mid x_t)|.$$

Equivalently, the predictive law must give finite expectation to the log-likelihood factor along the homotopy and permit dominated differentiation of $Z_t(\lambda)$. If this condition fails, the centered identity below should be treated as a formal diagnostic rather than as a justified derivative calculation. Therefore

$$\partial_\lambda \log Z_t(\lambda) = \int \log p_\theta(y_t | x) \pi_{t,\lambda}(x) dx.$$

The derivative in (21.3) is thus the centered log-likelihood under the intermediate law:

$$\partial_\lambda \log \pi_{t,\lambda}(x) = \log p_\theta(y_t | x) - \mathbb{E}_{\pi_{t,\lambda}}[\log p_\theta(y_t | X_t)].$$

This centered form is a useful audit identity. It shows that the density path preserves total probability, since integrating $\partial_\lambda \pi_{t,\lambda}$ over x gives zero.

For debugging purposes, the main lesson of this section is simple: if a claimed particle-flow implementation does not recover the correct endpoint target even in a simple controlled case, one of the first questions to ask is whether the normalized homotopy was implemented or whether an unnormalized shortcut was silently substituted.

21.4 Continuity equation and transport PDE

Suppose the particle cloud is transported by a velocity field $v_{t,\lambda}(x)$ through the pseudo-time ODE

$$\frac{dx(\lambda)}{d\lambda} = v_{t,\lambda}(x(\lambda)). \quad (21.4)$$

The following derivation is a classical strong-form calculation. It assumes that the density is differentiable in λ , continuously differentiable in x on the region of interest, that $\pi_{t,\lambda} v_{t,\lambda}$ is integrable, and that the boundary term in the integration-by-parts step vanishes. This is satisfied, for example, for smooth rapidly decaying densities on $\mathbb{R}^{n_x}$ with controlled velocity, for compactly supported smooth test functions in the weak form, or for bounded domains with an explicitly imposed no-flux or otherwise stated boundary condition. Without one of these admissible settings, the PDE should be read as a formal transport equation rather than a fully justified global statement. For reviewer-grade use, the weak-form identity with compactly supported test functions is the safer default; the displayed strong-form PDE is used only when the stated differentiability, integrability, and boundary-flux hypotheses hold. Mass conservation over any regular test region then implies that the evolving density must satisfy the continuity equation

$$\partial_\lambda \pi_{t,\lambda}(x) + \nabla \cdot (\pi_{t,\lambda}(x) v_{t,\lambda}(x)) = 0. \quad (21.5)$$

Equivalently, for any smooth compactly supported test function φ ,

$$\frac{d}{d\lambda} \int \varphi(x) \pi_{t,\lambda}(x) dx = \int \nabla \varphi(x)^\top v_{t,\lambda}(x) \pi_{t,\lambda}(x) dx,$$

and integration by parts gives (21.5). Dividing by $\pi_{t,\lambda}(x)$ where the density is positive yields

$$\partial_\lambda \log \pi_{t,\lambda}(x) = -\nabla \cdot v_{t,\lambda}(x) - \nabla \log \pi_{t,\lambda}(x)^\top v_{t,\lambda}(x). \quad (21.6)$$

Combining (21.3) and (21.6) gives the basic particle-flow PDE:

$$\log p_\theta(y_t | x) - \partial_\lambda \log Z_t(\lambda) = -\nabla \cdot v_{t,\lambda}(x) - \nabla \log \pi_{t,\lambda}(x)^\top v_{t,\lambda}(x). \quad (21.7)$$

This equation is exact as a conservation statement. What is not yet fixed is a unique velocity field. The PDE constrains the field but does not determine it without additional structure. In particular, if a vector field has zero weighted divergence with respect to $\pi_{t,\lambda}$, adding it to one solution can leave the same density evolution. EDH and LEDH should therefore be read as particular closure choices, not as the only possible solution of the transport PDE.

21.5 EDH under Gaussian closure

The classical exact Daum–Huang (EDH) construction becomes explicit after a Gaussian closure. The word "exact" in the name is historical and must be read with the regime stated here: the derivation is exact for the Gaussian family it postulates, and it is an exact filtering transport only when that Gaussian family is the true homotopy family.

Assume that the predictive law is approximated by

$$p_\theta(x_t = x \mid y_{1:t-1}) \approx \mathcal{N}(x; m_{t|t-1}, P_{t|t-1})$$

and that the observation density is Gaussian with linear observation map H_t and covariance R_t :

$$p_\theta(y_t \mid x) \propto \exp \left\{ -\frac{1}{2} (y_t - H_t x)^\top R_t^{-1} (y_t - H_t x) \right\}.$$

Here $P_{t|t-1}$ and R_t are assumed symmetric positive definite, and the matrix products have the conformable dimensions $H_t \in \mathbb{R}^{n_y \times n_x}$, $P_{t|t-1} \in \mathbb{R}^{n_x \times n_x}$, and $R_t \in \mathbb{R}^{n_y \times n_y}$. These assumptions are what make the inverse and solve notation below meaningful; an implementation should use linear solves and record conditioning diagnostics rather than treating the displayed inverses as a numerical instruction. At pseudo-time λ the interpolating density is then approximated by

$$\pi_{t,\lambda}(x) \approx \mathcal{N}(x; m_{t,\lambda}, P_{t,\lambda}).$$

The likelihood precision is

$$\Lambda_t = H_t^\top R_t^{-1} H_t. \quad (21.8)$$

Collecting the quadratic terms in the log-homotopy gives

$$P_{t,\lambda}^{-1} = P_{t|t-1}^{-1} + \lambda \Lambda_t,$$

so the homotopy covariance satisfies

$$P_{t,\lambda} = (P_{t|t-1}^{-1} + \lambda \Lambda_t)^{-1}, \quad (21.9)$$

and the matrix inverse derivative gives the covariance evolution. To see the step explicitly, set $M_{t,\lambda} = P_{t|t-1}^{-1} + \lambda \Lambda_t$, so that $M_{t,\lambda} P_{t,\lambda} = I$. Differentiating this identity yields $\Lambda_t P_{t,\lambda} + M_{t,\lambda} (dP_{t,\lambda}/d\lambda) = 0$ and hence

$$\frac{dP_{t,\lambda}}{d\lambda} = -P_{t,\lambda} \Lambda_t P_{t,\lambda}. \quad (21.10)$$

The corresponding information vector is

$$h_{t,\lambda} = P_{t|t-1}^{-1} m_{t|t-1} + \lambda H_t^\top R_t^{-1} y_t, \quad m_{t,\lambda} = P_{t,\lambda} h_{t,\lambda}.$$

Differentiating $m_{t,\lambda}$ gives

$$\frac{dm_{t,\lambda}}{d\lambda} = -P_{t,\lambda}\Lambda_t m_{t,\lambda} + P_{t,\lambda}H_t^\top R_t^{-1}y_t.$$

The EDH ansatz now chooses an affine velocity field

$$v_{t,\lambda}(x) = A_t(\lambda)x + b_t(\lambda), \quad (21.11)$$

whose mean and covariance evolution match the Gaussian homotopy. If $X_\lambda \sim \mathcal{N}(m_{t,\lambda}, P_{t,\lambda})$ evolves under this affine field, then

$$\frac{dm_{t,\lambda}}{d\lambda} = A_t(\lambda)m_{t,\lambda} + b_t(\lambda), \quad \frac{dP_{t,\lambda}}{d\lambda} = A_t(\lambda)P_{t,\lambda} + P_{t,\lambda}A_t(\lambda)^\top.$$

Choosing $A_t(\lambda) = -\frac{1}{2}P_{t,\lambda}\Lambda_t$ matches (21.10). The offset $b_t(\lambda)$ is then determined by the mean equation:

$$b_t(\lambda) = \frac{dm_{t,\lambda}}{d\lambda} - A_t(\lambda)m_{t,\lambda}.$$

Using

$$m_{t,\lambda} = m_{t|t-1} + \lambda P_{t,\lambda}H_t^\top R_t^{-1}(y_t - H_t m_{t|t-1})$$

rewrites this offset in the standard BayesFilter EDH convention. The coefficients are

$$A_t(\lambda) = -\frac{1}{2}P_{t,\lambda}\Lambda_t, \quad (21.12)$$

$$b_t(\lambda) = -A_t(\lambda)m_{t|t-1} + (I + \lambda A_t(\lambda))P_{t,\lambda}H_t^\top R_t^{-1}(y_t - H_t m_{t|t-1}). \quad (21.13)$$

Thus the EDH pseudo-time ODE is

$$\frac{dx}{d\lambda} = A_t(\lambda)x + b_t(\lambda). \quad (21.14)$$

The Gaussian closure should be read with care. The continuity equation and the homotopy construction are exact at their own level. The approximation enters when one assumes that the intermediate family remains Gaussian and then chooses an affine velocity field that is consistent with that Gaussian family. In other words, EDH is exact as a transport formula only when the closure assumption is exactly true; otherwise it is a mathematically structured approximation.

A future implementation should therefore separate four tasks explicitly:

- (i) evaluate the predictive Gaussian inputs $(m_{t|t-1}, P_{t|t-1})$;
- (ii) build the precision contribution Λ_t ;
- (iii) build the information contribution $H_t^\top R_t^{-1}y_t$ or its locally shifted analogue;
- (iv) integrate the affine pseudo-time dynamics (21.14) using a numerically stable scheme.

If the transported cloud behaves unexpectedly, the first debugging question is not merely whether the ODE solver was stable, but also whether the Gaussian closure itself was plausible for the state-space regime under study.

21.6 Linear-Gaussian recovery as the exact special case

The most important exactness statement for EDH is the linear-Gaussian recovery result. Suppose the predictive law is Gaussian and the observation model is truly linear-Gaussian. Then the homotopy family remains Gaussian for all λ . The covariance endpoint at $\lambda = 1$ is

$$P_{t,1} = (P_{t|t-1}^{-1} + H_t^\top R_t^{-1} H_t)^{-1}, \quad (21.15)$$

which is exactly the Kalman posterior covariance in information form. The mean endpoint follows from the endpoint information vector:

$$m_{t,1} = P_{t,1} \left(P_{t|t-1}^{-1} m_{t|t-1} + H_t^\top R_t^{-1} y_t \right). \quad (21.16)$$

Using the Kalman gain

$$K_t = P_{t|t-1} H_t^\top (H_t P_{t|t-1} H_t^\top + R_t)^{-1},$$

the same mean can be written as

$$m_{t,1} = m_{t|t-1} + K_t(y_t - H_t m_{t|t-1}).$$

Thus, in the linear-Gaussian case, EDH is not merely an approximation: it transports the predictive Gaussian law to the exact Kalman update, provided the affine ODE is integrated exactly or to controlled numerical tolerance.

This statement is crucial for BayesFilter because it supplies the exact special case against which any EDH implementation should be tested. It is also the boundary line for the rest of the chapter: outside this special case, the particle-flow path is no longer guaranteed exact merely by writing down the homotopy.

21.7 LEDH and local linearization

The local EDH (LEDH) construction replaces the single global observation linearization by a particle-specific local linearization. This is not merely an implementation refinement. It changes the closure from one Gaussian homotopy shared by the cloud to a family of particle-local Gaussian approximations.

For particle $x^{(i)}$, let

$$G_t^{(i)} = \left. \frac{\partial g_\theta(x)}{\partial x} \right|_{x=x^{(i)}} \quad (21.17)$$

and write the local linear approximation of the observation map as

$$g_\theta(x) \approx g_\theta(x^{(i)}) + G_t^{(i)}(x - x^{(i)}). \quad (21.18)$$

Equivalently, the local observation equation is approximated by

$$y_t \approx G_t^{(i)} x + (g_\theta(x^{(i)}) - G_t^{(i)} x^{(i)}) + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, R_t).$$

Collecting quadratic and linear terms in this local Gaussian likelihood gives the local precision

$$\Lambda_t^{(i)} = (G_t^{(i)})^\top R_t^{-1} G_t^{(i)} \quad (21.19)$$

and the local Gaussian information vector

$$\eta_t^{(i)} = (G_t^{(i)})^\top R_t^{-1} (y_t - g_\theta(x^{(i)}) + G_t^{(i)} x^{(i)}), \quad (21.20)$$

which is the particle-wise analogue of the global Gaussian information term $H_t^\top R_t^{-1} y_t$ in EDH. The corresponding local homotopy covariance is

$$P_{t,\lambda}^{(i)} = (P_{t|t-1}^{-1} + \lambda \Lambda_t^{(i)})^{-1}. \quad (21.21)$$

The associated local mean is

$$m_{t,\lambda}^{(i)} = P_{t,\lambda}^{(i)} \left(P_{t|t-1}^{-1} m_{t|t-1} + \lambda \eta_t^{(i)} \right).$$

The local linearization of the observation map near particle $x^{(i)}$ therefore produces particle-specific flow coefficients

$$A_t^{(i)}(\lambda) = -\frac{1}{2} P_{t,\lambda}^{(i)} \Lambda_t^{(i)}, \quad (21.22)$$

$$b_t^{(i)}(\lambda) = -A_t^{(i)}(\lambda) m_{t|t-1} + (I + \lambda A_t^{(i)}(\lambda)) P_{t,\lambda}^{(i)} \eta_t^{(i)}, \quad (21.23)$$

with pseudo-time evolution

$$\frac{dx^{(i)}}{d\lambda} = A_t^{(i)}(\lambda) x^{(i)} + b_t^{(i)}(\lambda). \quad (21.24)$$

For the Li-Coates PF-PF(LEDH) construction used in the next chapter, there is an additional implementation-facing distinction that must not be collapsed. The local linearization is not a single deterministic affine map shared by all particles, and it is not computed from a cloud-level average. For particle i , Algorithm 1 of [Li and Coates \[2017\]](#) first builds a particle-specific predicted covariance P^i and a zero-noise transition anchor

$$\bar{\eta}_0^i = g_k(x_{k-1}^i, 0).$$

It also samples an actual pre-flow proposal particle

$$\eta_0^i = g_k(x_{k-1}^i, v_k).$$

During pseudo-time integration, the Jacobian used in the LEDH coefficients is evaluated at the moving auxiliary state $\bar{\eta}_\lambda^i$, not at a fixed global mean and not at another particle. In the notation of [Li and Coates \[2017\]](#), the local observation linearization is

$$H^i(\lambda) = \left. \frac{\partial h(\eta, 0)}{\partial \eta} \right|_{\eta=\bar{\eta}_\lambda^i}, \quad e^i(\lambda) = h(\bar{\eta}_\lambda^i, 0) - H^i(\lambda) \bar{\eta}_\lambda^i. \quad (21.25)$$

With the particle-specific prediction covariance P^i held as the covariance input for that particle's flow step, the source-form LEDH coefficients are

$$A^i(\lambda) = -\frac{1}{2} P^i H^i(\lambda)^\top (\lambda H^i(\lambda) P^i H^i(\lambda)^\top + R)^{-1} H^i(\lambda), \quad (21.26)$$

$$b^i(\lambda) = (I + 2\lambda A^i(\lambda)) [(I + \lambda A^i(\lambda)) P^i H^i(\lambda)^\top R^{-1} \{z_k - e^i(\lambda)\} + A^i(\lambda) \bar{\eta}_0^i]. \quad (21.27)$$

The auxiliary anchor and the actual proposal particle are then migrated by the same particle-local affine update at each pseudo-time step:

$$\bar{\eta}_{\lambda_j}^i = \bar{\eta}_{\lambda_{j-1}}^i + \epsilon_j \left[A_j^i(\lambda_j) \bar{\eta}_{\lambda_{j-1}}^i + b_j^i(\lambda_j) \right], \quad (21.28)$$

$$\eta_{\lambda_j}^i = \eta_{\lambda_{j-1}}^i + \epsilon_j \left[A_j^i(\lambda_j) \eta_{\lambda_{j-1}}^i + b_j^i(\lambda_j) \right]. \quad (21.29)$$

This auxiliary/actual split is the key source-faithfulness point for LEDH in PF-PF. The auxiliary path determines the local linearization, while the actual path is the transported proposal particle whose density must later be corrected. Using a single global affine map, or linearizing every particle at a fixed cloud-level point, is therefore not Li-Coates Algorithm 1 LEDH.

The same source-faithfulness point applies to the per-particle covariance objects. The covariance P^i is part of the Li-Coates Algorithm 1 state used to construct the particle-local LEDH coefficients; BayesFilter does not treat this covariance state as a new filter invention. When a later differentiable OT or Sinkhorn resampling layer is attached to this Algorithm 1 route, the additional BayesFilter obligation is bookkeeping: specify how the covariance auxiliary state is carried through the chosen transport map, and test that this carry rule preserves the intended route. The transport layer supplies the resampling relaxation; it does not replace the source algorithm's covariance lifecycle or by itself prove HMC suitability.

This construction explains both the attraction and the danger of LEDH. The attraction is that each particle uses a local approximation intended to track nonlinear observation geometry more closely than a single global linearization. The danger is that the transported law now depends on a family of particle-specific linearizations, each of which may behave poorly if the Jacobian is unstable, ill-conditioned, or evaluated in a region where the local approximation is misleading. Hence LEDH is potentially more adaptive than EDH, but it is also more fragile numerically and interpretively. The chapter should not phrase LEDH as guaranteed improvement without model-specific evidence.

From an implementation and debugging standpoint, LEDH adds at least three new failure routes relative to EDH:

- (i) inaccurate or unstable local Jacobians;
- (ii) poor local-information-vector construction;
- (iii) path-dependent variation in stiffness across particles.

These are not secondary engineering details. They are direct consequences of how the local approximation enters the mathematics.

21.8 Stiffness and discretization

Even when the flow equations are well defined, their numerical integration may be difficult. The main difficulty is stiffness. When the observation noise is small or the effective likelihood precision is large, the eigenvalues of the drift matrix in the EDH or LEDH ODE can span several orders of magnitude. In that regime, explicit solvers require very small pseudo-time steps to avoid large truncation error or instability.

Formally, the stiffness is tied to the conditioning of the homotopy covariance or precision family. In the local case, for example, large eigenvalue spreads in $\Lambda_t^{(i)}$ may force

extremely fine pseudo-time resolution near the posterior end of the path. This matters for implementation because the flow construction is only useful if it can be integrated stably and repeatedly inside a filtering recursion.

The practical burden can be stated more concretely. If the pseudo-time ODE is integrated with an explicit method such as RK4, then the local truncation error is governed by powers of the step size, but the stability envelope is controlled by the operator norm and spectral spread of the drift matrix. In sharp-likelihood regimes, decreasing the observation noise or increasing the local precision can force the stable step size down dramatically. For LEDH this may happen non-uniformly across the cloud, because different particles see different local Jacobians and therefore different local precision matrices.

This is exactly why stiffness belongs in the monograph as more than a short warning. It creates a direct literature-to-debugging bridge:

- if a flow implementation behaves erratically only when observation noise becomes small, stiffness is a prime mathematical suspect;
- if certain particles show unstable trajectories while others remain well behaved, the local LEDH precision field is a prime suspect;
- if a nominally correct flow map fails under a coarse pseudo-time grid, the first question should be whether the flow is underresolved rather than whether the underlying theory is wrong.

For BayesFilter, the main consequence is therefore both conceptual and operational. Stiffness is not merely a performance issue. It is one of the main ways in which a formally attractive flow construction can become numerically unreliable enough to alter the practical behavior of the filter. That is why later implementation-facing chapters must record discretization policy, stiffness diagnostics, and stress tests explicitly, and why the literature pointers for stiffness should be treated as debugging tools rather than as optional background reading.

21.9 Exactness and approximation ledger

The particle-flow chapter has several layers that must not be collapsed.

Layer	Status	Main implementation diagnostic
Normalized homotopy endpoints	exact model identity when the predictive density and likelihood are the intended model objects	recover predictive law at $\lambda = 0$ and filtering law at $\lambda = 1$ in analytic test cases
Continuity equation	exact conservation statement under regularity and boundary assumptions	verify transported test-function moments against ODE integration in controlled problems
Velocity field choice	not unique; EDH/LEDH are closure choices	record the chosen field and compare alternatives on benchmark models
EDH Gaussian closure	exact for the Gaussian family; exact filtering only in linear-Gaussian recovery	test covariance and mean endpoints against the Kalman update
LEDH local linearization	approximate closure using particle-local Jacobians and information vectors	monitor Jacobian conditioning, local precision spectra, and particle-wise endpoint dispersion
Pseudo-time integration	numerical approximation to the chosen ODE	run step size, tolerance, stiffness, and log-det sensitivity checks

This ledger is deliberately conservative. It permits a strong statement about linear-Gaussian recovery, but it blocks any inference that a raw EDH or LEDH cloud is already a corrected nonlinear filter, a pseudo-marginal likelihood estimator, or an HMC-ready target.

21.10 What this chapter does and does not claim

This chapter develops the particle-flow transport mathematics itself. It does *not* yet claim:

- that the transported cloud alone defines the final corrected filtering law in the nonlinear case;
- that raw EDH or LEDH transport is already the final HMC target;
- that the Jacobian/proposal correction issue has been resolved;
- that differentiable resampling or OT relaxations have been introduced.

Those belong to the later PF-PF and resampling chapters. The role of the present chapter is narrower and mathematically prior: it explains how the transport path is constructed, where the Gaussian-closure or local-linearization approximation enters, and why the linear-Gaussian case supplies the exact special benchmark.

Chapter 22

Particle-Flow Particle Filters and Proposal Correction

The previous chapter developed particle flow as a transport construction. The main unresolved question left there was whether the transported cloud itself may already be treated as the corrected filtering object. In general, the answer is no. Outside exact special cases, the flow should be interpreted first as a proposal mechanism. The role of the present chapter is to make that proposal interpretation explicit and to derive the corresponding importance correction.

This chapter is therefore the mathematical bridge between raw flow transport and a more principled particle-based likelihood interpretation. It does not yet introduce differentiable resampling or optimal transport relaxations. Instead, it asks a prior question: once a particle has been moved by a deterministic flow map, what density does that transported particle actually have, and what importance weight is required to target the intended posterior law?

22.1 Objects, assumptions, and source roles

The PF-PF argument depends on naming the target and proposal objects before writing any weight. This chapter uses:

Object	Meaning
x_{t-1}	ancestor state after the previous filtering step
$x_{t,0}$	pre-flow particle sampled from a one-step proposal
$x_{t,1}$	post-flow particle after applying the flow map
$q_{t,0}$	pre-flow proposal density for $x_{t,0}$ conditional on (x_{t-1}, y_t)
Φ_t	differentiable flow map from pre-flow to post-flow coordinates
Φ_t^{-1}	inverse map used to express the pre-image of a post-flow particle
$J_t = D\Phi_t$	forward Jacobian matrix of the flow map
$q_{t,1}$	induced post-flow proposal density for $x_{t,1}$
γ_t	unnormalized one-step filtering target density
$\tilde{w}_t$	unnormalized importance weight correcting $q_{t,1}$ to γ_t

The source role is narrow. General SMC references [Doucet et al., 2001, Chopin and Papaspiliopoulos, 2020] supply the proposal and importance-weight framework. Invertible particle-flow particle filtering [Li and Coates, 2017] motivates treating particle flow as a

proposal transformation with a Jacobian correction. These sources do not remove the need to state the target/proposal ratio locally, and they do not imply that finite-particle, numerical, or HMC target issues have disappeared.

22.2 Why proposal correction is needed

In the linear-Gaussian special case studied in the previous chapter, EDH can recover the exact Kalman update. Outside that case, however, the transport map is typically built under Gaussian closure or local linearization. Even if the transport moves particles toward high posterior density, the moved cloud is not automatically distributed according to the exact filtering law.

The right interpretation is therefore proposal-theoretic. Let $x_{t,0}$ denote a pre-flow particle sampled from a proposal law, and let the flow map transport it to

$$x_{t,1} = \Phi_t(x_{t,0}). \quad (22.1)$$

The question is then not merely where the particle moved, but which density on $x_{t,1}$ is induced by the transformation and how that induced proposal density must be corrected to target the filtering posterior.

This is the basic PF-PF point emphasized in the invertible particle-flow literature: the flow is best viewed as a proposal-improving device until its proposal-density effect has been accounted for. The exact change-of-variables identity is not itself the approximation; the approximation enters through the choice of flow family, the numerical integration of the flow, and the finite particle system.

22.3 Change of variables under the flow map

Assume that for each fixed time step and observation, the flow map $\Phi_t : \mathbb{R}^{n_x} \rightarrow \mathbb{R}^{n_x}$ is a differentiable bijection on the region of interest and that its forward Jacobian $D\Phi_t(x_{t,0})$ is an $n_x \times n_x$ nonsingular matrix along the proposal paths being weighted. Equivalently, the calculation below is a local diffeomorphism calculation on the support actually used by the proposal. Let the pre-flow proposal density be

$$q_{t,0}(x_{t,0} \mid x_{t-1}, y_t). \quad (22.2)$$

Then the transported particle density is given by the standard change-of-variables formula,

$$q_{t,1}(x_{t,1} \mid x_{t-1}, y_t) = q_{t,0}(x_{t,0} \mid x_{t-1}, y_t) |\det D\Phi_t(x_{t,0})|^{-1}, \quad (22.3)$$

where $x_{t,0} = \Phi_t^{-1}(x_{t,1})$. The determinant in (22.3) is the determinant of the *forward* map from $x_{t,0}$ to $x_{t,1}$. Equivalently,

$$q_{t,1}(x_{t,1} \mid x_{t-1}, y_t) = q_{t,0}(\Phi_t^{-1}(x_{t,1}) \mid x_{t-1}, y_t) |\det D\Phi_t^{-1}(x_{t,1})|.$$

These two forms are identical because $|\det D\Phi_t^{-1}(x_{t,1})| = |\det D\Phi_t(x_{t,0})|^{-1}$. The chapter uses the forward-map convention because the flow ODE naturally evolves $x_{t,0}$ to $x_{t,1}$ and the log-determinant ODE below accumulates $\log |\det D\Phi_t(x_{t,0})|$.

Thus the Jacobian determinant is not an optional numerical decoration. It is part of the proposal density itself. Any flow-based importance scheme that forgets this factor changes the proposal law and therefore changes the targeting relationship of the resulting importance weights.

22.4 Proposal-corrected PF-PF weights

Let the intended one-step filtering target be proportional to

$$\gamma_t(x_t) \propto p_\theta(y_t | x_t) p_\theta(x_t | x_{t-1}). \quad (22.4)$$

This target is conditional on the ancestor state. In a full particle filter, the ancestor itself is random and selected from the previous weighted cloud. The present derivation is therefore a one-step conditional density-ratio calculation; the full likelihood estimator also depends on ancestor selection, resampling policy, particle count, and time recursion.

For a generic pre-flow proposal $q_{t,0}$, the corrected importance weight is

$$\tilde{w}_t \propto \frac{\gamma_t(x_{t,1})}{q_{t,1}(x_{t,1} | x_{t-1}, y_t)} = \frac{p_\theta(y_t | x_{t,1}) p_\theta(x_{t,1} | x_{t-1}) |\det D\Phi_t(x_{t,0})|}{q_{t,0}(x_{t,0} | x_{t-1}, y_t)}. \quad (22.5)$$

The bootstrap/prior-proposal formula below is the special case $q_{t,0} = p_\theta(\cdot | x_{t-1})$.

If the pre-flow particle is sampled from the transition prior,

$$q_{t,0}(x_{t,0} | x_{t-1}, y_t) = p_\theta(x_{t,0} | x_{t-1}), \quad (22.6)$$

and then transported to $x_{t,1} = \Phi_t(x_{t,0})$, the post-flow proposal density is

$$q_{t,1}(x_{t,1} | x_{t-1}, y_t) = p_\theta(x_{t,0} | x_{t-1}) |\det D\Phi_t(x_{t,0})|^{-1}. \quad (22.7)$$

The corresponding unnormalized importance weight is therefore

$$\tilde{w}_t \propto \frac{p_\theta(y_t | x_{t,1}) p_\theta(x_{t,1} | x_{t-1})}{q_{t,1}(x_{t,1} | x_{t-1}, y_t)} = \frac{p_\theta(y_t | x_{t,1}) p_\theta(x_{t,1} | x_{t-1}) |\det D\Phi_t(x_{t,0})|}{p_\theta(x_{t,0} | x_{t-1})}. \quad (22.8)$$

Equation (22.8) is the central correction formula for the PF-PF viewpoint. It shows exactly what proposal correction restores: once the transformation of proposal density has been accounted for, the flow-transported particle may be reweighted against the desired posterior factor in the usual importance-sampling way.

The formula also shows exactly what is *not* optional in code. A PF-PF implementation must evaluate, on the same path, the observation density at the post-flow particle, the transition density at the post-flow particle, the pre-flow proposal density at the pre-image, and the forward log determinant. Changing any one of these objects changes the proposal-to-target ratio being computed.

22.5 EDH/PF and LEDH/PF

The correction formula is structurally the same whether the flow map comes from EDH or LEDH. The difference lies in how the map itself is built.

22.5.1 EDH/PF

For EDH/PF, the transport map is generated by the global affine ODE from the previous chapter. The resulting proposal is easier to analyze because the flow field is common across the cloud. The main approximation point is the Gaussian closure used to derive the affine coefficients. If the affine ODE is integrated with a common map Φ_t , the same forward log determinant applies to every particle up to its path through the time-varying affine field.

22.5.2 LEDH/PF

For LEDH/PF, the map is particle-specific: each particle evolves under a local linearization and therefore under its own drift coefficients. This can improve local adaptation to nonlinear observation structure, but it also makes the proposal analysis more delicate. The local map remains proposal-correctable in principle through the same change-of-variables logic, but the numerical burden of evaluating particle-specific Jacobians or log-determinants becomes more significant. A code path that reuses an EDH log determinant for LEDH has changed the proposal density unless the local maps are actually identical.

The important mathematical point is that EDH/PF and LEDH/PF are not merely flow algorithms. They are proposal-corrected particle methods. That is why they occupy a different target-status category from raw flow transport.

22.6 Li-Coates Algorithm 1 as an implementation contract

Algorithm 1 of [Li and Coates \[2017\]](#) is more specific than the generic proposal-correction derivation above. It is a recursive particle filter in which each particle carries both a state and a covariance object. For BayesFilter, this algorithm must be read as an implementation contract, not as a vague instruction to apply a deterministic affine map.

At the start of time step k , particle i has an ancestor x_{k-1}^i , an importance weight w_{k-1}^i , and a posterior covariance P_{k-1}^i inherited from the previous time step. The covariance lifecycle in Algorithm 1 is

$$(x_{k-1}^i, P_{k-1}^i) \xrightarrow{\text{EKF/UKF prediction}} (m_{k|k-1}^i, P^i) \xrightarrow{\text{EKF/UKF update}} (m_{k|k}^i, P_k^i). \quad (22.9)$$

The source states that EKF or UKF covariance prediction equations may be used when the dynamic model is nonlinear. The source does not make optimal transport resampling part of Algorithm 1, and it should not be cited as evidence for any BayesFilter OT relaxation. In the BayesFilter rebuild, using UKF for the prediction and update arrows in (22.9) is a reviewed implementation choice requested for this program; it is not a claim that the paper's reported experiments used UKF in every scenario.

After the prediction arrow, Algorithm 1 defines two distinct state objects:

$$\bar{\eta}_0^i = g_k(x_{k-1}^i, 0), \quad (22.10)$$

$$\eta_0^i = g_k(x_{k-1}^i, v_k). \quad (22.11)$$

The first is the zero-noise transition anchor used to drive the local linearization. The second is the actual pre-flow proposal particle drawn through the dynamic model. The implementation must keep both objects. If the code transports only the actual proposal and never maintains the auxiliary zero-noise path, then the local Jacobian field is not the one specified by Algorithm 1.

For pseudo-time grid $0 = \lambda_0 < \lambda_1 < \dots < \lambda_{N_\lambda} = 1$ with $\epsilon_j = \lambda_j - \lambda_{j-1}$, Algorithm 1 repeats the following particle-specific update. Set $P = P^i$ and compute $A_j^i(\lambda_j)$ and $b_j^i(\lambda_j)$ from the LEDH formulas using the linearization at the current auxiliary state $\bar{\eta}^i$. Then move the auxiliary and actual states by the same local affine step:

$$\bar{\eta}^i \leftarrow \bar{\eta}^i + \epsilon_j \{A_j^i(\lambda_j) \bar{\eta}^i + b_j^i(\lambda_j)\}, \quad (22.12)$$

$$\eta_1^i \leftarrow \eta_1^i + \epsilon_j \{A_j^i(\lambda_j) \eta_1^i + b_j^i(\lambda_j)\}. \quad (22.13)$$

At the same pseudo-time step, the forward determinant multiplier is accumulated as

$$\theta^i \leftarrow \theta^i |\det\{I + \epsilon_j A_j^i(\lambda_j)\}|, \quad \theta_0^i = 1. \quad (22.14)$$

Thus, at the end of the pseudo-time integration,

$$\theta^i = \prod_{j=1}^{N_\lambda} |\det\{I + \epsilon_j A_j^i(\lambda_j)\}|. \quad (22.15)$$

This is the discrete Algorithm 1 version of the forward determinant in (22.8). It is particle-specific because the LEDH coefficients are particle-specific.

After the flow, Algorithm 1 sets $x_k^i = \eta_1^i$ and applies the PF-PF importance update

$$w_k^i \propto \frac{p(x_k^i | x_{k-1}^i) p(z_k | x_k^i) \theta^i}{p(\eta_0^i | x_{k-1}^i)} w_{k-1}^i. \quad (22.16)$$

The denominator is the density of the pre-flow transition sample, while the transition density in the numerator is evaluated at the post-flow particle x_k^i . Replacing both densities by the same value, dropping θ^i , or using a determinant from a different map changes the proposal-to-target ratio.

Finally, the weights are normalized, the EKF/UKF update produces P_k^i , and any resampling step resamples the triple

$$\{x_k^i, P_k^i, w_k^i\}_{i=1}^{N_p}. \quad (22.17)$$

The covariance component is part of the particle state for the next recursion. Resampling only the states while leaving covariances in their old order breaks the Algorithm 1 lifecycle. Likewise, introducing differentiable or optimal transport resampling may be a BayesFilter extension, but it must be labelled as an extension and audited separately from the Li-Coates source algorithm.

22.6.1 What the later OT chapters inherit from PF-PF

The OT survey companion note sharpens the next boundary in a useful way. After PF-PF correction, the BayesFilter object handed to a later resampling layer is not a vague “particle cloud.” It is a weighted empirical ensemble together with any carried auxiliary state. In the Algorithm 1 reading fixed above, the state handed forward is precisely the weighted triple (22.17). Thus a later optimal-transport layer does not merely need an OT scalar or distance certificate. It needs a usable transport object that can act on the weighted ensemble:

coupling matrix, barycentric transform, factorized transport operator, or directly transported equal-weighted (22.18)

A fast value of $\text{OT}(\mu, \nu)$ alone is not enough to continue the filtering recursion, because the filter must actually produce the next equal-weighted particle representation.

Write the PF-PF output cloud schematically as

$$\mu_t^N = \sum_{i=1}^{N_p} w_k^i \delta_{x_k^i}, \quad (22.19)$$

with carried covariance or particle-local state attached to each support point. A deterministic OT equal-weighting layer or entropic OT/Sinkhorn layer, in the spirit of ensemble-transform and differentiable OT resampling [Reich, 2013, Corenflos et al., 2021], then asks for a map from the weighted object μ_t^N to an equal-weight object. The next chapters will spell out that map through couplings, barycentric projections, and teacher-versus-numerical transport objects. The present chapter’s narrower role is only to make the handoff precise: PF-PF supplies the weighted source object, and any later OT resampling route must declare how that weighted object is actually transported rather than merely reporting a scalar transport score.

These details give a sharper audit checklist than the generic PF-PF equations:

- (i) each particle has its own P_{k-1}^i , predicted P^i , and updated P_k^i ;
- (ii) the local LEDH Jacobian is evaluated along the particle’s auxiliary zero-noise path;
- (iii) the actual proposal particle and the auxiliary path are both migrated;
- (iv) the determinant product in (22.15) uses the same local matrices that migrated the actual proposal particle;
- (v) the weight uses the post-flow transition density, the observation density, the determinant product, the pre-flow proposal density, and the previous particle weight;
- (vi) resampling, if used, resamples covariances together with states and weights.

22.7 Jacobian determinant and log-determinant evolution

Directly forming a full Jacobian matrix for every particle and time step may be numerically and computationally burdensome. A more useful identity is obtained by differentiating the Jacobian matrix along the flow.

Let

$$J_t(\lambda) = \frac{\partial x_t(\lambda)}{\partial x_t(0)}. \quad (22.20)$$

The terminal forward determinant in (22.3) is $\det J_t(1)$. The inverse-density form would use $-\log |\det J_t(1)|$ instead. This sign distinction is one of the most common PF-PF implementation errors.

If the flow satisfies

$$\frac{dx_t(\lambda)}{d\lambda} = v_{t,\lambda}(x_t(\lambda)), \quad (22.21)$$

then differentiation with respect to the initial point gives

$$\frac{dJ_t(\lambda)}{d\lambda} = \nabla_{v_{t,\lambda}(x_t(\lambda))} J_t(\lambda), \quad J_t(0) = I. \quad (22.22)$$

The derivation is direct: differentiate the flow equation with respect to the initial coordinate $x_t(0)$ and apply the chain rule. Applying Jacobi’s formula $d \log |\det J| / d\lambda = \text{tr}(J^{-1} dJ / d\lambda)$ then yields

$$\frac{d}{d\lambda} \log |\det J_t(\lambda)| = \text{tr}(\nabla_{v_{t,\lambda}(x_t(\lambda))}). \quad (22.23)$$

because $J_t^{-1}(\nabla v_{t,\lambda})J_t$ has the same trace as $\nabla v_{t,\lambda}$. Thus

$$\log |\det D\Phi_t(x_{t,0})| = \int_0^1 \text{tr}(\nabla v_{t,\lambda}(x_t(\lambda))) d\lambda.$$

For an affine flow $v_{t,\lambda}(x) = A_t(\lambda)x + b_t(\lambda)$, this simplifies to

$$\frac{d}{d\lambda} \log |\det J_t(\lambda)| = \text{tr}(A_t(\lambda)). \quad (22.24)$$

The practical consequence is clear: one may integrate a scalar log-determinant alongside the particle-flow ODE rather than carrying a full dense Jacobian for every particle in the affine case. In more general settings, the same identity still shows that the determinant is part of the dynamical system, not an external afterthought. This also gives a direct debugging criterion: if the corrected weights behave inconsistently with the transported cloud, the first mathematical suspects are the Jacobian/log-determinant path, the numerical flow integration, and the finite-particle approximation rather than the formal change-of-variables identity itself.

22.8 What the correction restores, and what it does not

The proposal correction in (22.8) restores a clearer importance-sampling interpretation. More precisely, it restores the correct proposal-to-target density ratio for the transported proposal generated by the flow map. This is mathematically important: it distinguishes PF-PF from a pure transport approximation that simply declares the endpoint cloud to be posterior- like without density correction.

What the correction does *not* automatically restore is an exact analytic filter in the nonlinear case. Several approximation layers may still remain:

- the flow may itself be derived under Gaussian closure or local linearization;
- the ODE is integrated numerically rather than in closed form;
- the particle system still uses a finite number of particles;
- Jacobian or log-determinant quantities may be computed with numerical approximation.

Therefore PF-PF is mathematically stronger than raw flow transport, but it is not automatically equivalent to an exact nonlinear filtering likelihood. The exact identity in this chapter is the change-of-variables correction for the proposal density; what remains approximate are the flow construction itself when closure or local linearization is used, the numerical integration of that flow, and the finite-particle system used to estimate the target. Its status is better described as a proposal-corrected particle approximation whose value-side interpretation is clearer than that of uncorrected flow transport.

At trajectory level, this one-step correction is multiplied through the filtering recursion in the same spirit as ordinary sequential importance sampling. For a particle path and ancestor history, the product of incremental corrections defines the path proposal weight. That statement is still conditional on using the stated proposal, the stated flow map, and the stated determinant convention at each time step. It is not a separate theorem that the finite-particle likelihood estimate is exact in the nonlinear case.

22.9 Implementation audit obligations

The formulas above imply concrete checks before PF-PF can be used as a value-side candidate in later HMC experiments.

- (i) **Affine density check:** choose a low-dimensional affine map with known determinant and compare (22.3) against a direct transformed-density calculation.
- (ii) **Forward/inverse sign check:** verify that the proposal density uses $-\log |\det D\Phi_t|$ while the numerator form in (22.8) contributes $+\log |\det D\Phi_t|$.
- (iii) **Jacobian ODE check:** compare the integrated trace in (22.23) with an automatic-differentiation or finite-difference determinant in small dimension.
- (iv) **Corrected versus uncorrected weights:** run the same transported cloud with and without the determinant/proposal-density term to confirm that the diagnostic detects the expected discrepancy.
- (v) **Path consistency:** on fixed random seeds, verify that the scalar value and any autodiff gradient use the same proposal density, flow map, and log-determinant path.

These are not tuning diagnostics. They are algebraic and numerical checks of the target/proposal object itself.

22.10 Why this chapter matters for the later HMC discussion

This chapter is the first point in the rebuilt DPF block where a sharper target question becomes possible. Raw flow transport gives a geometrically appealing approximation, but PF-PF introduces an explicit proposal density and an explicit correction factor. That is why later HMC-suitability analysis will treat proposal-corrected EDH/PF or LEDH/PF as the first rungs in this ladder with an explicit proposal-corrected value-side interpretation. That is a narrower statement than saying they are HMC-ready.

The key lesson is not that PF-PF solves all target issues. The key lesson is that it restores the proposal-correction logic that a valid particle method requires. This is exactly the right place in the monograph to separate flow-as-transport from flow-as-proposal.

22.11 What this chapter does and does not claim

This chapter derives the proposal-correction structure for flow-based particle methods. It does *not* yet claim:

- that differentiable resampling has been resolved;
- that the resulting method is already the final HMC target for nonlinear DSGE or MacroFinance models;
- that OT or learned transport relaxations have been introduced;

- that finite-particle, discretization, and Jacobian numerical issues are negligible.

Those belong to the later resampling and HMC-assessment chapters. The role of this chapter is more specific: it explains mathematically how a particle flow is turned into a proposal-corrected particle method and what that correction does and does not restore.

Chapter 23

Soft Differentiable Resampling

The previous chapters established the classical particle-filter setting in which a weighted cloud of particles is turned back into an equally weighted cloud by a resampling step. The mathematical difficulty is that the standard resampling map is discrete: it duplicates some particles, deletes others, and changes abruptly when weights cross threshold values. That makes naive pathwise differentiation break down.

This chapter studies the simplest smooth replacement of that discrete step. It does not introduce optimal transport yet. Its narrower task is to show, in the smallest possible setting, how differentiability can be bought by changing the resampling map itself.

23.1 What problem this chapter solves

Suppose a weighted particle cloud is

$$\hat{\pi}_t^N(dx) = \sum_{i=1}^N w_t^{(i)} \delta_{x_t^{(i)}}(dx), \quad w_t^{(i)} \geq 0, \quad \sum_{i=1}^N w_t^{(i)} = 1. \quad (23.1)$$

Classical multinomial resampling tries to convert this into an equally weighted cloud by drawing ancestor indices from the categorical law defined by the weights. This is fine as a simulation rule. It is not fine if one wants the realized output cloud to depend smoothly on the weights.

The chapter therefore asks one very specific question:

How can we make the resampled cloud depend more smoothly on the weights, and what bias is introduced when we do so?

23.2 Running two-particle discontinuity cell

The smallest useful example has two particles. Let

$$x_1 = -1, \quad x_2 = 2, \quad w = (p, 1 - p), \quad p \in (0, 1),$$

and let one fixed common random number $U \in (0, 1)$ be used to define the first resampled ancestor. Standard categorical resampling chooses

$$a(U, p) = \begin{cases} 1, & U \leq p, \\ 2, & U > p. \end{cases} \quad (23.2)$$

For a fixed U , this is a step function of p : small changes in the weight usually do nothing, and then a threshold crossing changes the realized ancestor abruptly.

Plain-language takeaway. If the algorithm literally copies an ancestor chosen by a discrete threshold, the output particle usually stays exactly the same when the weights move a little. A pathwise derivative of that realized output is therefore typically zero until a jump occurs.

23.3 Classical resampling bottleneck

For standard multinomial resampling, the equal-weight cloud is obtained by sampling ancestor indices

$$a_j \sim \text{Categorical}(w_t^{(1:N)})$$

and setting

$$\tilde{x}_t^{(j)} = x_t^{(a_j)}. \quad (23.3)$$

This map is exact for the classical resampling law, but it is not pathwise differentiable with respect to the weights because the realized ancestor path is piecewise constant in those weights.

23.4 Soft resampling rule

A simple smooth surrogate interpolates between the sampled ancestor and the weighted mean of the cloud. Define

$$\bar{x}_t = \sum_{i=1}^N w_t^{(i)} x_t^{(i)}, \quad (23.4)$$

and then set

$$\tilde{x}_t^{(j)} = \alpha x_t^{(a_j)} + (1 - \alpha) \bar{x}_t, \quad \alpha \in [0, 1]. \quad (23.5)$$

When $\alpha = 1$, the rule reduces to hard categorical resampling. When $\alpha < 1$, the output depends smoothly on the weighted mean, so the realized output cloud becomes less discontinuous as a function of the weights.

23.5 Worked two-particle calculation

In the two-particle cell,

$$\bar{x} = p(-1) + (1 - p)2 = 2 - 3p.$$

The soft-resampling output for one realized ancestor is therefore

$$\tilde{x}(U, p) = \alpha x_{a(U,p)} + (1 - \alpha)(2 - 3p). \quad (23.6)$$

Two immediate facts are now visible.

First, when the ancestor is held fixed, the dependence on p is smooth through the weighted mean term. Second, the object being computed has changed: the surrogate particle is no longer equal to the chosen ancestor itself unless $\alpha = 1$.

The weighted mean is preserved in conditional expectation because

$$\mathbb{E}[\tilde{x}_t^{(j)} \mid X, w] = \alpha \sum_i w_i x_i + (1 - \alpha) \bar{x}_t = \bar{x}_t. \quad (23.7)$$

So the surrogate keeps the first-moment center of mass, but not necessarily other nonlinear summaries of the cloud.

23.6 Algorithm: soft differentiable resampling

Algorithm 1 `soft_resampling_step`

Require: weighted particle cloud $\{x_i, w_i\}_{i=1}^N$, interpolation parameter $\alpha \in [0, 1]$, realized ancestor indices a_j if hard ancestors are retained, or a separately declared softened ancestor rule if they are not

Ensure: equal-weight surrogate cloud $\tilde{X} = (\tilde{x}^{(1)}, \dots, \tilde{x}^{(N)})$ and the declared ancestor policy

- 1: Compute the weighted mean $\bar{x} = \sum_{i=1}^N w_i x_i$
 - 2: **for** each output index $j = 1, \dots, N$ **do**
 - 3: Form $\tilde{x}^{(j)} = \alpha x_{a_j} + (1 - \alpha) \bar{x}$
 - 4: **end for**
 - 5: Return the equal-weight cloud $\tilde{X}$, the chosen α , and the declared ancestor policy used to define a_j
-

Approximation enters here. This algorithm is not another notation for categorical resampling. It is a new surrogate object that trades exactness for smoother dependence on the weights.

23.7 Filtering-loop insertion point

Inside the filtering loop, the soft surrogate sits exactly where hard resampling would have sat:

$$\{x_i, w_i\}_{i=1}^N \xrightarrow{\text{soft resampling}} \{\tilde{x}^{(j)}, 1/N\}_{j=1}^N. \quad (23.8)$$

It changes only the equal-weighting rule. It does not, by itself, change the preceding weighting rule or add an OT correction.

23.8 Verification contract

A minimal verification pass should check:

- (1) the weighted mean is preserved as in (23.7),
- (2) the surrogate becomes the hard categorical rule when $\alpha = 1$,
- (3) nonlinear test-function bias is visible when expected,
- (4) the declared ancestor policy is recorded explicitly,

- (5) any downstream gradient claim states whether ancestors remain discrete or are themselves relaxed.

A failed mean-preservation check is evidence that the surrogate has been implemented incorrectly. A successful mean-preservation check is not evidence that the classical categorical target has been preserved.

23.9 What this chapter does and does not claim

This chapter teaches the simplest smooth resampling surrogate and its bias. It does *not* claim:

- that the surrogate preserves the exact categorical resampling law,
- that preserving the weighted mean preserves the full distribution,
- that the resulting scalar is automatically the same target later used by HMC or training.

Its role is narrower: it shows, in the smallest possible setting, how resampling can be made smoother and where the resulting approximation enters.

Chapter 24

Deterministic OT Equal-Weighting Baseline

The previous chapter introduced the resampling bottleneck and explained why one might replace discrete ancestor selection by a transport object. The present chapter slows that idea down. Its purpose is to teach the deterministic optimal-transport equal-weighting baseline itself before entropy, Sinkhorn iteration, or learned acceleration are added.

The key conceptual move is simple. A weighted particle cloud and an equal-weight particle cloud are not the same object. Deterministic OT equal-weighting asks for a mass-redistribution rule that sends the weighted source cloud to an equal-weight target cloud as cheaply as possible under a declared cost. This already changes the resampling object relative to categorical ancestor sampling, but it does so in a clean, finite-dimensional, deterministic way.

24.1 What problem this chapter solves

Suppose a particle filter has produced support points $x_1, \dots, x_N \in \mathbb{R}^{d_x}$ with normalized weights $w_1, \dots, w_N$. A later stage of the filter may want an equally weighted cloud so that no single particle dominates future propagation or summary statistics. Classical resampling achieves this by random duplication and elimination. The deterministic OT baseline instead asks:

How should mass be reassigned from the weighted cloud to an equal-weight output cloud if we want to move it as little as possible under a declared transport cost?

This chapter answers that question in three layers:

- (i) define the weighted and equal-weight empirical objects;
- (ii) define the coupling matrix that records how source mass is reassigned;
- (iii) convert that coupling into the equal-weight output cloud by barycentric projection.

24.2 Running three-particle cell: deterministic OT version

Reuse the running one-dimensional cell from the previous chapter:

$$x_1 = -2, \quad x_2 = 0, \quad x_3 = 3, \quad w = \left(\frac{1}{2}, \frac{1}{3}, \frac{1}{6}\right), \quad u = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right).$$

Using the source support itself as a simple square target support, the quadratic cost matrix is

$$C = \begin{pmatrix} 0 & 4 & 25 \\ 4 & 0 & 9 \\ 25 & 9 & 0 \end{pmatrix}. \quad (24.1)$$

A feasible coupling for this problem is

$$\Pi^{\text{cell}} = \begin{pmatrix} \frac{1}{3} & \frac{1}{6} & 0 \\ 0 & \frac{1}{6} & \frac{1}{6} \\ 0 & 0 & \frac{1}{6} \end{pmatrix}. \quad (24.2)$$

The row sums are $(1/2, 1/3, 1/6)$, matching the source weights, and the column sums are $(1/3, 1/3, 1/3)$, matching the equal target marginal. The barycentric output cloud is therefore

$$\tilde{x}^{(1)} = 3 \left(\frac{1}{3}(-2) \right) = -2, \quad (24.3)$$

$$\tilde{x}^{(2)} = 3 \left(\frac{1}{6}(-2) + \frac{1}{6}(0) \right) = -1, \quad (24.4)$$

$$\tilde{x}^{(3)} = 3 \left(\frac{1}{6}(0) + \frac{1}{6}(3) \right) = 1.5. \quad (24.5)$$

So one deterministic equal-weight cloud associated with this feasible transport is

$$\{-2, -1, 1.5\} \quad \text{with equal weights } \frac{1}{3}.$$

This single worked cell is enough to make three key points visible:

- (i) the coupling is a mass-routing matrix, not yet the output cloud;
- (ii) the output particles need not be duplicates of the source support;
- (iii) deterministic equal-weighting already differs from categorical resampling even before entropy or differentiability are mentioned.

24.3 Objects and notation

The chapter uses the following core objects.

Object	Meaning in this chapter
$\hat{\pi}^N$	weighted empirical source measure
$\hat{\pi}_{\text{eq}}^N$	equal-weight empirical output measure
w	source weight vector
u	target equal-weight vector, usually $(1/N, \dots, 1/N)$
C	declared transport cost matrix
Π	coupling matrix sending source mass to target mass
$\mathcal{U}(w, u)$	feasible coupling set with row sums w and column sums u
Π_0^*	optimal coupling for the declared unregularized OT problem
$B(\Pi)$	barycentric equal-weight cloud induced by coupling Π

24.4 Weighted cloud, equal-weight cloud, and the coupling set

The weighted source cloud is

$$\hat{\pi}^N(dx) = \sum_{i=1}^N w_i \delta_{x_i}(dx), \quad w_i \geq 0, \quad \sum_{i=1}^N w_i = 1. \quad (24.6)$$

The equal-weight target cloud is

$$\hat{\pi}_{\text{eq}}^N(dx) = \frac{1}{N} \sum_{j=1}^N \delta_{\tilde{x}(j)}(dx). \quad (24.7)$$

A coupling is a nonnegative matrix telling us how much mass from source particle i contributes to output slot j . The feasible set is

$$\mathcal{U}(w, u) = \{ \Pi \in \mathbb{R}_+^{N \times N} : \Pi \mathbf{1} = w, \Pi' \mathbf{1} = u \}. \quad (24.8)$$

The row constraints say each source particle gives away exactly its original weight. The column constraints say each output slot receives exactly one equal share of total mass.

Reader checkpoint. At this stage the coupling Π is only a bookkeeping object. It is not yet a random ancestor rule and not yet the final equal-weight cloud. The next step is to choose which feasible coupling is preferred.

24.4.1 Why the filter needs a transport object, not just a scalar

The scalable-OT survey sharpens the main practical boundary here. For the BayesFilter filtering recursion, a fast scalar optimal-transport value is not by itself a usable resampling output. The filter needs an object that actually acts on the weighted particle cloud. In the deterministic equal-weighting baseline, that usable object is the coupling Π together with its barycentric action. Schematically, the required outputs are of the form

$$\text{coupling matrix, barycentric projection, factorized transport operator, or directly transported equal-w} \quad (24.9)$$

A reported scalar transport cost or value certificate may help compare candidate transports, but it does not yet produce the next particle representation used by the filter.

This is why the present chapter slows down on couplings and barycentric maps. Given the weighted source measure

$$\hat{\pi}^N(dx) = \sum_{i=1}^N w_i \delta_{x_i}(dx) \quad (24.10)$$

and equal-weight target marginal u , the mathematical output needed by later code is not merely the scalar

$$\min_{\Pi \in \mathcal{U}(w,u)} \langle C, \Pi \rangle, \quad (24.11)$$

but the minimizing coupling itself together with the map that turns that coupling into an equal-weight cloud. The later entropic and Sinkhorn chapter will replace this exact linear-program teacher by an entropically regularized teacher, but it inherits the same computational doctrine: a usable transport object must be produced, not only a scalar score.

24.5 The unregularized deterministic OT baseline

Given a declared cost matrix C , the deterministic transport baseline chooses a feasible coupling with minimal cost:

$$\Pi_0^* \in \arg \min_{\Pi \in \mathcal{U}(w,u)} \langle C, \Pi \rangle. \quad (24.12)$$

This is a linear program. It is exact for the chosen deterministic OT object, not for classical categorical resampling. The distinction matters. A reader who keeps only one sentence in mind should keep this one:

The deterministic OT baseline solves a transport projection problem exactly for that declared transport problem, but that problem is already different from random ancestor sampling.

24.6 Barycentric projection: from coupling to equal-weight cloud

Once a coupling has been chosen, the final equal-weight output cloud is obtained by barycentric projection. For each output slot j , define

$$B(\Pi)_j = \frac{1}{u_j} \sum_{i=1}^N \Pi_{ij} x_i. \quad (24.13)$$

In the equal-weight case $u_j = 1/N$, this becomes

$$B(\Pi)_j = N \sum_{i=1}^N \Pi_{ij} x_i. \quad (24.14)$$

Operationally, each column of the coupling is turned into an output particle by forming the weighted average of the source particles that contribute mass to that column.

This is the key step that makes the OT baseline implementation-facing. The coupling is a matrix of mass transfers; the barycentric map is the actual equal-weight cloud later code would use.

24.7 Algorithm: deterministic OT equal-weighting

Algorithm 2 `deterministic_ot_equalweight_step`

Require: weighted particle cloud $\{x_i, w_i\}_{i=1}^N$, equal target marginal u , declared target support rule z_j , and cost matrix $C_{ij} = c(x_i, z_j)$

Ensure: optimal coupling Π_0^* , barycentric equal-weight cloud $\tilde{X}$, and declared transport metadata

- 1: Form the feasible coupling set $\mathcal{U}(w, u)$
- 2: Solve the linear program $\Pi_0^* \in \arg \min_{\Pi \in \mathcal{U}(w, u)} \langle C, \Pi \rangle$
- 3: **for** each output slot $j = 1, \dots, N$ **do**
- 4: Compute the barycentric output particle

$$\tilde{x}^{(j)} = \frac{1}{u_j} \sum_{i=1}^N \Pi_{ij}^* x_i$$

5: **end for**

- 6: Return the equal-weight cloud $\tilde{X} = (\tilde{x}^{(1)}, \dots, \tilde{x}^{(N)})$, the coupling Π_0^* , the declared target support rule, and any transport diagnostics
-

Approximation enters here. This algorithm is exact for the unregularized deterministic OT object in (24.12). It is not exact for categorical resampling, and it is not yet the entropically smoothed object used later for differentiable computation.

24.8 Filtering-loop insertion point

Inside a filtering loop, this deterministic OT step sits after the weighted cloud has been constructed and before the equal-weight cloud is passed to the next propagation or summary stage. In symbols:

$$\{x_i, w_i\}_{i=1}^N \xrightarrow{\text{deterministic OT equal-weighting}} \{\tilde{x}^{(j)}, 1/N\}_{j=1}^N. \quad (24.15)$$

If auxiliary particle-local state is carried elsewhere in the filter, this chapter does not yet decide how that state is transported. That is a separate implementation contract. The present chapter fixes only the equal-weighting rule for the particle locations and masses themselves.

24.9 Verification and failure interpretation

A minimal verification contract should check:

- (1) row sums equal the source weights exactly,
- (2) column sums equal the equal target marginal exactly,
- (3) the barycentric output has the declared shape $N \times d_x$,

- (4) the target support rule is recorded explicitly,
- (5) any claim about downstream filtering is kept separate from this local OT equality check.

A failed marginal check is evidence that the coupling solver or bookkeeping is wrong. It is not evidence that OT is conceptually invalid. A good marginal check is evidence only that the deterministic OT object has been implemented correctly, not that the output cloud preserves the same law as categorical resampling.

24.10 What this chapter does and does not claim

This chapter teaches the clean deterministic transport baseline for equal-weighting. It does *not* claim:

- that deterministic OT equal-weighting is the same object as classical categorical resampling,
- that it is automatically differentiable in the sense required later,
- that barycentric projection preserves the original particle-filter target,
- that the output cloud is already the final HMC or training target.

Its role is narrower: it fixes the exact deterministic OT baseline so that the next chapter can explain what changes when entropy, Sinkhorn iteration, and solver differentiation are added.

Chapter 25

Entropic OT, Sinkhorn, and Barycentric Projection

The previous chapter fixed the deterministic OT equal-weighting baseline. This chapter explains the main differentiable computational route used in the OT lane: entropic regularization, finite Sinkhorn iteration, barycentric projection, and the distinction between the mathematical teacher object and the numerical object actually differentiated in code.

This chapter has one dominant goal: a reader should be able to understand what object finite Sinkhorn computes, how barycentric output is produced from it, where approximation enters, and what diagnostics must be checked before the result is used in a downstream filtering scalar.

25.1 What changes when entropy is added

The deterministic OT baseline chooses a coupling that minimizes transport cost exactly for the declared linear program. Entropic OT changes that object by adding a smoothing term to the optimization problem. In plain language, the transport plan is now rewarded for being more diffuse instead of concentrating all mass as sharply as possible.

The gain is numerical smoothness and a tractable matrix-scaling solver. The cost is that the transport object is no longer the unregularized deterministic OT solution. This is the first approximation layer of the chapter.

25.2 Running three-particle cell with entropic smoothing

Keep the same running cell as before:

$$x_1 = -2, \quad x_2 = 0, \quad x_3 = 3, \quad w = \left(\frac{1}{2}, \frac{1}{3}, \frac{1}{6}\right), \quad u = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right),$$

with quadratic cost matrix

$$C = \begin{pmatrix} 0 & 4 & 25 \\ 4 & 0 & 9 \\ 25 & 9 & 0 \end{pmatrix}. \tag{25.1}$$

For $\varepsilon = 1$, the Gibbs kernel is

$$K_1 = \begin{pmatrix} 1 & e^{-4} & e^{-25} \\ e^{-4} & 1 & e^{-9} \\ e^{-25} & e^{-9} & 1 \end{pmatrix}. \quad (25.2)$$

The entries are heavily concentrated near the diagonal, but not exactly zero away from it. That is a simple visual sign of what entropy has changed: the plan is no longer forced into the sharpest possible assignment.

25.3 Entropic OT teacher object

Let $\mathcal{U}(w, u)$ be the feasible coupling set from the previous chapter. The entropically regularized OT problem is

$$\Pi_\varepsilon^* = \arg \min_{\Pi \in \mathcal{U}(w, u)} \left\{ \langle \Pi, C \rangle + \varepsilon \sum_{i,j} \Pi_{ij} (\log \Pi_{ij} - 1) \right\}. \quad (25.3)$$

This chapter uses the same entropy convention as the earlier OT material, so the minimizer is the exact teacher for the chosen regularized object, not for the unregularized linear program and not for categorical resampling.

Reader checkpoint. At this stage, the chapter has changed the mathematical object once: from the unregularized deterministic OT coupling to the entropically smoothed OT coupling. Nothing numerical has happened yet. The next step explains how this exact regularized object is computed in practice.

25.3.1 Exact teacher versus exact engineering route

The scalable-OT survey suggests a useful extra distinction before moving on to finite Sinkhorn iterations. There are two different senses in which a route may still be called “exact” for the entropic OT object.

First, there is the mathematical teacher object Π_ε^* from (25.3). That is the exact minimizer of the entropically regularized problem. Second, there is an engineering route that preserves that same entropic object while changing how the matrix-scaling updates are executed in memory or on hardware. Exact online, streaming, and GPU Sinkhorn methods belong to this second category: they do not change the entropic teacher problem, but they avoid materializing the full cost, kernel, or coupling whenever possible.

This distinction matters for the BayesFilter transport lane. A method that recomputes the same entropic scaling updates through blocked reductions or streamed kernel application is still on the semantics-preserving reference lane. By contrast, a method that replaces the kernel, truncates the coupling family, or changes the transport objective has moved to a different approximation class even if it is motivated by scalability. The reader should therefore keep separate:

- (i) the exact entropic teacher object,
- (ii) the finite numerical object produced by a declared Sinkhorn execution path,

(iii) the engineering strategy used to realize that numerical path.

The next sections fix the second object explicitly and later staged chapters will discuss scalability routes that either preserve or alter it.

25.4 Why the optimizer factorizes

The first-order optimality conditions imply that the regularized coupling has the form

$$\Pi_\varepsilon^\star = \text{diag}(a)K_\varepsilon \text{diag}(b), \quad (K_\varepsilon)_{ij} = \exp(-C_{ij}/\varepsilon), \quad (25.4)$$

with positive scaling vectors a, b chosen so that the coupling has the correct row and column sums. Operationally, this means the hard OT problem has been reduced to the problem of finding two scaling vectors that balance a known kernel. That reduction is why Sinkhorn iteration is so useful.

25.5 Algorithm: finite Sinkhorn equal-weighting

Algorithm 3 sinkhorn_equalweight_step

Require: weighted particle cloud $\{x_i, w_i\}_{i=1}^N$, equal target marginal u , cost matrix C , regularization $\varepsilon > 0$, initialization $(a^{(0)}, b^{(0)})$, iteration budget K , and stabilization policy

Ensure: finite coupling $\Pi_{\varepsilon, K}$, barycentric equal-weight cloud $\tilde{X}$, row/column residuals, and declared numerical metadata

1: Form the Gibbs kernel $(K_\varepsilon)_{ij} = \exp(-C_{ij}/\varepsilon)$

2: **for** $k = 0, \dots, K - 1$ **do**

3: Update $a^{(k+1)} = w \oslash (K_\varepsilon b^{(k)})$

4: Update $b^{(k+1)} = u \oslash (K_\varepsilon' a^{(k+1)})$

5: **end for**

6: Form the finite coupling $\Pi_{\varepsilon, K} = \text{diag}(a^{(K)})K_\varepsilon \text{diag}(b^{(K)})$

7: Compute residuals

$$r_{\text{row}} = \|\Pi_{\varepsilon, K} \mathbf{1} - w\|_1, \quad r_{\text{col}} = \|\Pi_{\varepsilon, K}' \mathbf{1} - u\|_1$$

8: **for** each output slot $j = 1, \dots, N$ **do**

9: Compute $\tilde{x}^{(j)} = \frac{1}{u_j} \sum_{i=1}^N \Pi_{ij} x_i$

10: **end for**

11: Return the coupling, equal-weight cloud, residuals, ε , K , initialization policy, and stabilization metadata

25.5.1 Exact online and streaming Sinkhorn as the reference lane

The scalable-OT survey also clarifies how BayesFilter should talk about scaling without silently changing the transport object. A finite Sinkhorn route can be made substantially more practical by changing the execution strategy rather than the underlying entropic

teacher problem. Streaming, online, or GPU-specialized implementations still belong to the reference lane when they preserve the same entropic scaling equations and the same barycentric output rule while avoiding full materialization of the cost or kernel matrix.

In that reference-lane setting, the implementation may compute the needed row-wise or column-wise reductions in blocks, keep only partial transport state in memory, or fuse barycentric application with solver passes. What must remain fixed is the declared numerical object: the finite coupling $\Pi_{\epsilon,K}$, its residual checks, and the barycentric output derived from that same finite coupling. Thus a streamed or GPU-tiled Sinkhorn routine is not automatically a new approximation class. It becomes a new approximation class only when the scaling equations, kernel, feasible family, or output object itself are changed.

This chapter therefore treats exact-online and streaming implementations as engineering variants of the same entropic reference lane. Later staged chapters on retained-teacher approximations will take up the different case where the transport object itself is compressed, factorized, or otherwise altered for scalability.

This is the chapter's main callable procedure. A reader who implements only one OT route from the monograph should be able to implement this one.

25.6 Worked balancing pass on the running cell

Take the simplest initialization $b^{(0)} = (1, 1, 1)'$. Then the first row-balancing step computes

$$a^{(1)} = w \oslash (K_1 b^{(0)}). \quad (25.5)$$

For the running cell,

$$K_1 b^{(0)} \approx (1.0183, 1.0184, 1.0001),$$

so

$$a^{(1)} \approx (0.491, 0.327, 0.167).$$

The next column-balancing step then uses these row-repaired values to push the column sums toward $(1/3, 1/3, 1/3)$. The exact arithmetic is less important than the interpretation: finite Sinkhorn repeatedly repairs whichever marginal is currently wrong.

Plain-language takeaway. Sinkhorn is not a mysterious OT oracle. It is an alternating balancing procedure for a smoothed transport matrix. That is why later learned warm-start methods try to predict better initial balancing coordinates rather than replacing the solver outright.

25.7 Barycentric projection as the output map

The solver output is still only a coupling matrix. The equal-weight particle cloud that later code consumes is obtained by barycentric projection:

$$\tilde{x}^{(j)} = \frac{1}{u_j} \sum_{i=1}^N \Pi_{ij} x_i. \quad (25.6)$$

In the equal-weight case, this is

$$\tilde{x}^{(j)} = N \sum_{i=1}^N \Pi_{ij} x_i. \quad (25.7)$$

This step is where the numerical coupling becomes the actual equal-weight output cloud. It is therefore the place where an implementation must be especially clear about shapes, normalization, and any auxiliary state that is or is not carried alongside the particle locations.

25.8 Differentiation contracts

The final major distinction of the chapter is between three different derivative objects:

- (i) derivative of the exact regularized OT optimizer,
- (ii) derivative of the finite unrolled Sinkhorn computation,
- (iii) derivative of some scalar built from that finite computation.

In code, the second and third are the most common. A gradient is only the correct gradient of the exact executed scalar if the computational graph used in autodiff matches the scalar value actually reported by the code.

25.9 Filtering-loop insertion point

Inside a filter, the finite EOT/Sinkhorn equal-weighting step sits after the weighted particle cloud has been formed and before an equally weighted cloud is passed onward:

$$\{x_i, w_i\}_{i=1}^N \xrightarrow{\text{EOT/Sinkhorn}} \{\tilde{x}^{(j)}, 1/N\}_{j=1}^N. \quad (25.8)$$

If the broader filtering algorithm carries auxiliary particle-local state, this chapter does not yet assert how that state is transported. Its contract is narrower: it specifies the equal-weighting rule for the particle locations and the mass-routing matrix from which that equal-weight cloud is derived.

25.10 Verification contract

A reader implementing this route should verify at least:

- (1) row and column residuals are reported and decrease with additional iterations or stronger stabilization,
- (2) the initialization, ε , iteration budget, and stabilization policy are recorded,
- (3) the barycentric cloud has the declared shape and normalization,
- (4) any reported gradient is tied to the exact computational graph that was actually executed,
- (5) any stronger filtering or HMC claim is kept separate from these local numerical checks.

A failed residual check is evidence about the finite solver or stabilization path, not about the abstract EOT object. A successful residual check is evidence that the numerical route is behaving as declared, not that it preserves the original categorical target.

25.11 What this chapter does and does not claim

This chapter teaches the main differentiable OT computational route in a near-implementation style. It does *not* claim:

- that entropic OT is the same object as unregularized OT,
- that finite Sinkhorn is the exact regularized optimizer,
- that a barycentric equal-weight cloud is the same object as categorical ancestor sampling,
- that autodiff through the executed graph is automatically the gradient of the original filtering likelihood.

Its role is to fix the entropic teacher object, the finite numerical route, the output cloud construction, and the differentiation contract that the next chapter will accelerate with a learned warm start.

Chapter 26

Custom Gradients for LEDH-PFPPF-OT

The preceding chapters introduced three ingredients separately: local exact Daum–Huang particle flow, proposal-corrected PF-PF weighting, and entropy-regularized OT resampling. This chapter explains how those pieces can be differentiated without asking an automatic-differentiation system to retain the entire filtering graph in memory. The key object is a custom vector-Jacobian product, or VJP, for the executed LEDH-PFPPF-OT scalar.

The chapter is intentionally narrow. It does not claim that the relaxed OT-resampled particle filter is the same object as categorical resampling. It does not claim that finite Sinkhorn iteration is the exact unregularized OT optimizer. It does not claim HMC correctness for a nonlinear model. Its claim is more basic and more useful: if the value path is a declared finite LEDH-PFPPF-OT computation, then the gradient path must be the derivative of that same computation, and the expensive OT block has a structured adjoint.

26.1 The scalar whose derivative is wanted

Let θ denote the model parameter and let all random numbers used by the diagnostic or sampler target be fixed as part of the deterministic value path. For one filtering pass, write the executed scalar as

$$\mathcal{L}_K(\theta) = \text{Reduce}(\text{LEDH-PFPPF-OT}_K(\theta; y_{1:T}, \xi)), \quad (26.1)$$

where ξ denotes the fixed particle and process-noise streams, K denotes the finite Sinkhorn or annealed-transport iteration budget, and Reduce is the declared scalar reduction, such as a mean over fixed seeds. The derivative needed by HMC or optimization is

$$\nabla_{\theta} \mathcal{L}_K(\theta),$$

not the derivative of a different resampling rule, a different Sinkhorn stopping policy, or a different random seed construction.

Definition 26.1 (Same-scalar custom-gradient contract). *A custom-gradient implementation for LEDH-PFPPF-OT is same-scalar if its primal return is exactly $\mathcal{L}_K(\theta)$ and its VJP returns*

$$v^{\top} D_{\theta} \mathcal{L}_K(\theta)$$

for every upstream scalar multiplier v . For a scalar log target the usual case is $v = 1$.

This contract is the same HMC principle as Chapter 13, but the present chapter specializes it to the LEDH-PFPF-OT computation. The particle-flow part follows the Li–Coates invertible-particle-flow proposal logic [Li and Coates, 2017]; the OT relaxation follows the entropy-regularized resampling route of Corenflos et al. [2021], with the Sinkhorn background of Cuturi [2013] and Peyré and Cuturi [2019]. The derivative identities below are project derivations for the declared BayesFilter finite program; the citations explain the particle-flow, entropy-regularized OT, and Sinkhorn objects being differentiated.

26.2 Forward decomposition

At one observation time, the value path can be written as a composition

$$\theta \mapsto (X^-, \log w^-, A_{\text{flow}}) \mapsto (X, \log w) \mapsto \tilde{X} \mapsto \ell_t. \quad (26.2)$$

Here $X^- \in \mathbb{R}^{N \times d}$ is the pre-observation proposal cloud, $\log w^-$ are proposal-correction log weights, and A_{flow} denotes the LEDH auxiliary quantities needed for the proposal density and log-determinant correction. The local LEDH construction uses particle-local linearizations and particle-local covariance state as in Li–Coates PF-PF. The OT block then maps the weighted cloud to an equal-weight cloud

$$(X, \log w) \mapsto \tilde{X}.$$

This chapter focuses on the last map because it is the part that can dominate memory. Naive reverse-mode autodiff through all pairwise costs, Sinkhorn updates, and barycentric products may retain objects of size $O(KN^2)$. A custom VJP instead treats the OT block as a finite differentiable program and applies its adjoint by recomputation, streaming reductions, or checkpointing.

26.3 Transport orientation

The OT chapters used a coupling Π with source index first and output slot second. The TensorFlow LEDH-PFPF-OT route is more naturally described by the row-stochastic transport matrix

$$T = N\Pi^\top, \quad T\mathbf{1} = \mathbf{1}, \quad T^\top\mathbf{1} = Nw, \quad (26.3)$$

where $w = \text{softmax}(\log w)$. The transported equal-weight cloud is

$$\tilde{X} = TX, \quad \tilde{x}_j = \sum_{i=1}^N T_{ji}x_i. \quad (26.4)$$

Thus row j of T tells how output slot j averages the source particles. The row sums make each output particle a barycenter. The column sums say that source particle i contributes total mass proportional to its weight.

Remark 26.1 (Why orientation matters). *If a derivation uses Π but the code applies TX , transpose errors are easy to introduce. The VJP formulas below use T , not Π . A reader who wants the coupling form can substitute $T = N\Pi^\top$.*

26.4 VJP of the barycentric product

The simplest and most important adjoint identity is the VJP of the barycentric matrix product.

Proposition 26.1 (Barycentric VJP). *Let $\tilde{X} = TX$, with $T \in \mathbb{R}^{N \times N}$ and $X \in \mathbb{R}^{N \times d}$. Use the Frobenius inner product for matrix adjoints, and let $G_{\tilde{X}} \in \mathbb{R}^{N \times d}$ be the upstream adjoint for $\tilde{X}$. Then the direct adjoints are*

$$G_X^{\text{direct}} = T^\top G_{\tilde{X}}, \quad G_T = G_{\tilde{X}} X^\top. \quad (26.5)$$

Proof. Taking a differential gives

$$d\tilde{X} = dT X + T dX.$$

Pairing with $G_{\tilde{X}}$ under the Frobenius inner product,

$$\begin{aligned} \langle G_{\tilde{X}}, d\tilde{X} \rangle &= \langle G_{\tilde{X}}, dT X \rangle + \langle G_{\tilde{X}}, T dX \rangle \\ &= \langle G_{\tilde{X}} X^\top, dT \rangle + \langle T^\top G_{\tilde{X}}, dX \rangle. \end{aligned}$$

The coefficients of dT and dX are exactly (26.5). $\square$

Equation (26.5) is only the direct part of the particle adjoint. Because T itself depends on X and $\log w$, the complete VJP also pulls G_T backward through the Sinkhorn or annealed-transport construction.

26.5 A normalized transport matrix

For fixed final potentials $f, g \in \mathbb{R}^N$, fixed normalized log weights $\log w_i$, and cost $C_{ji} = c(z_j, z_i)$, the executed row-stochastic transport can be written in column-normalized form

$$T_{ji} = N w_i \frac{\exp\{(f_j + g_i - C_{ji})/\varepsilon\}}{\sum_{r=1}^N \exp\{(f_r + g_i - C_{ri})/\varepsilon\}}. \quad (26.6)$$

This expression enforces $T^\top \mathbf{1} = Nw$ exactly for the displayed normalization. The row condition $T\mathbf{1} \approx \mathbf{1}$ is enforced by the potentials produced by finite Sinkhorn or annealed Sinkhorn iteration. In the TensorFlow implementation this is the convention used by the final transport application: f indexes output rows, g indexes source columns, the normalization is a column log-sum-exp over output rows, and the transported particles are obtained by the matrix product TX .

Define

$$p_{ji} = \frac{T_{ji}}{N w_i}, \quad \sum_{j=1}^N p_{ji} = 1. \quad (26.7)$$

The vector $p_{\cdot i}$ is the normalized column distribution associated with source particle i .

Proposition 26.2 (VJP of the normalized transport output). *At fixed finite values of $f, g \in \mathbb{R}^N$, $C \in \mathbb{R}^{N \times N}$, $\varepsilon > 0$, and normalized weights $w_i > 0$, $\sum_i w_i = 1$, let T be defined by (26.6). Let $G_T \in \mathbb{R}^{N \times N}$ be the upstream adjoint for T . For each source column i , set*

$$\bar{G}_i = \sum_{r=1}^N p_{ri} (G_T)_{ri}, \quad S_{ji} = T_{ji} ((G_T)_{ji} - \bar{G}_i). \quad (26.8)$$

Then the VJP of (26.6) satisfies

$$(G_{\log w})_i += \sum_{j=1}^N (G_T)_{ji} T_{ji}, \quad (26.9)$$

$$(G_f)_j += \frac{1}{\varepsilon} \sum_{i=1}^N S_{ji}, \quad (26.10)$$

$$(G_g)_i += \frac{1}{\varepsilon} \sum_{j=1}^N S_{ji}, \quad (26.11)$$

$$(G_C)_{ji} += -\frac{1}{\varepsilon} S_{ji}. \quad (26.12)$$

If $\log w$ is itself produced by a logsoftmax, then (26.9) must be composed with the logsoftmax VJP.

Proof. Let

$$a_{ji} = \frac{f_j + g_i - C_{ji}}{\varepsilon}, \quad p_{ji} = \frac{\exp(a_{ji})}{\sum_r \exp(a_{ri})}, \quad T_{ji} = N w_i p_{ji}.$$

For a perturbation of one column i ,

$$dT_{ji} = T_{ji} \left[d \log w_i + da_{ji} - \sum_{r=1}^N p_{ri} da_{ri} \right].$$

Pairing this expression with $(G_T)_{ji}$ and collecting the coefficient of da_{ji} gives $S_{ji} = T_{ji}((G_T)_{ji} - \bar{G}_i)$. Since

$$da_{ji} = \frac{1}{\varepsilon} (df_j + dg_i - dC_{ji}),$$

the adjoints for f , g , and C are (26.10)–(26.12). The coefficient of $d \log w_i$ is $\sum_j (G_T)_{ji} T_{ji}$, giving (26.9). $\square$

This proposition is the core OT VJP product. It says that the upstream matrix G_T should first be column-centered under the transport-induced probability p_i . Only after that centering should the adjoint be pushed into the potentials and costs. In the displayed column-normalized map, $\sum_j S_{ji} = 0$, so the final-potential adjoint G_g is identically zero at this last normalization stage. This is not a bug: adding the same g_i to every logit in one normalized column does not change that column's probabilities. Earlier Sinkhorn-loop adjoints may still involve the variables that produced g .

26.6 VJP of the quadratic cost

For the common squared-distance cost

$$C_{ji} = \frac{1}{2} \|z_j - z_i\|^2,$$

where the same scaled particle cloud Z supplies the row and column points, the cost adjoint has another simple form.

Proposition 26.3 (Quadratic-cost VJP). *Let G_C be the upstream adjoint for $C_{ji} = \frac{1}{2}\|z_j - z_i\|^2$, where $Z = (z_1, \dots, z_N)^\top \in \mathbb{R}^{N \times d}$. The contribution to the adjoint of $z_k \in \mathbb{R}^d$ is*

$$(G_Z)_k += \sum_{i=1}^N (G_C)_{ki}(z_k - z_i) - \sum_{j=1}^N (G_C)_{jk}(z_j - z_k). \quad (26.13)$$

Proof. The differential of one cost entry is

$$dC_{ji} = (z_j - z_i)^\top (dz_j - dz_i).$$

Multiplying by $(G_C)_{ji}$, summing over j, i , and collecting the coefficient of dz_k gives the two sums in (26.13): the first from terms where k is the row index, and the second from terms where k is the column index. $\square$

If centering, scaling, or key construction is stopped by design, the VJP stops there too. If the selected mode differentiates through centering or scaling, then (26.13) must be further composed with the VJP of those preprocessing maps.

26.7 Softmin and Sinkhorn adjoints

The finite Sinkhorn or annealed-transport step is a loop of softmin operations. The exact schedule can differ between a fixed-target Sinkhorn route and the FilterFlow-style annealed route, but the local adjoint primitive is the same.

Definition 26.2 (Softmin block). *For a query index j , costs C_{jr} , values v_r , and temperature $\varepsilon > 0$, define*

$$m_j = -\varepsilon \log \sum_{r=1}^N \exp\{(v_r - C_{jr})/\varepsilon\}. \quad (26.14)$$

Proposition 26.4 (Softmin VJP for fixed temperature). *Assume $\varepsilon > 0$ is held fixed and all displayed exponentials are finite. Let $G_m \in \mathbb{R}^N$ be the upstream adjoint for m , and define*

$$q_{jr} = \frac{\exp\{(v_r - C_{jr})/\varepsilon\}}{\sum_s \exp\{(v_s - C_{js})/\varepsilon\}}.$$

For fixed ε ,

$$(G_v)_r += - \sum_{j=1}^N (G_m)_j q_{jr}, \quad (26.15)$$

$$(G_C)_{jr} += (G_m)_j q_{jr}. \quad (26.16)$$

Proof. Differentiating (26.14) gives

$$dm_j = - \sum_r q_{jr} (dv_r - dC_{jr}) = \sum_r q_{jr} dC_{jr} - \sum_r q_{jr} dv_r.$$

Pairing with G_m and collecting coefficients yields (26.15) and (26.16). $\square$

The sign and scale in Proposition 26.4 are for a potential-scale value v . Some implementations, including the TensorFlow helper used in the current BayesFilter route, call the same operation with a log-scale input h_r :

$$m_j = -\varepsilon \log \sum_r \exp\{h_r - C_{jr}/\varepsilon\}.$$

This is the same formula with $v_r = \varepsilon h_r$. Therefore the local VJP with respect to h_r is $(G_h)_r += -\varepsilon \sum_j (G_m)_j q_{jr}$, while the VJP with respect to a potential-scale component v_r remains (26.15). If $h_r = \log \alpha_r + b_r/\varepsilon$, then the adjoint to b_r again has the potential-scale coefficient $-\sum_j (G_m)_j q_{jr}$.

For an annealed schedule, the loop state contains several potentials, for example (a_y, b_x, a_x, b_y) . Abstractly write the finite loop as

$$s_{k+1} = F_k(s_k; Z, \log w, \varepsilon_k), \quad k = 0, \dots, K-1. \quad (26.17)$$

The reverse loop is then

$$(G_{s_k}, G_Z, G_{\log w}) += DF_k(s_k; Z, \log w, \varepsilon_k)^\top G_{s_{k+1}}, \quad k = K-1, \dots, 0. \quad (26.18)$$

Each multiplication by $DF_k^\top$ is implemented by applying the softmin VJP above, the averaging VJP for damped updates, and the cost VJP for the pairwise squared distances. This is ordinary reverse-mode differentiation of a finite program, but expressed as an explicit adjoint scan rather than as a raw tape over the whole program.

Proposition 26.5 (Finite-Sinkhorn VJP by reverse scan). *Assume the finite transport block is the deterministic program*

$$(T, s_K) = \mathcal{S}_K(Z, \log w)$$

obtained by a fixed number of differentiable softmin, averaging, cost, and normalization operations with fixed branch decisions. Then a custom VJP for $\mathcal{S}_K$ is obtained by:

- (i) *applying Proposition 26.2 to move G_T into final potentials, costs, and log weights;*
- (ii) *applying Proposition 26.3 to move final-cost adjoints into Z ;*
- (iii) *scanning (26.18) backward through the finite potential updates;*
- (iv) *composing any resulting G_Z with the chosen centering, scaling, and stop-gradient policy.*

The result equals the VJP of the executed finite program.

Proof. The finite transport block is a composition of differentiable maps under the fixed branch and fixed iteration assumptions. Propositions 26.2, 26.3, and 26.4 give the VJP for the nontrivial primitive maps. The VJP of a composition is obtained by applying the transposed local Jacobians in reverse order. That reverse composition is exactly the scan described above. $\square$

26.8 Why a custom VJP is memory efficient

The naive reverse graph for the OT block stores pairwise costs, logits, normalizers, potentials, and intermediate loop states for every time step. With N particles and K Sinkhorn updates, this can scale like $O(KN^2)$ in stored tensors before the rest of the filtering graph is considered.

The custom VJP changes the storage problem. The primal path must retain or reconstruct only enough information to replay the finite program:

- the input particles and log weights entering the OT block;
- the branch choices, masks, iteration count, temperature schedule, and stabilization policy;
- final potentials, and optionally sparse checkpoints of intermediate potentials;
- any stopped quantities needed to reproduce the executed scalar.

During the backward pass, pairwise blocks can be recomputed and reduced immediately. This trades additional arithmetic for lower peak memory. The trade is natural in HMC, because one scalar log target needs one reverse VJP to obtain all parameter-score components, while a forward-mode JVP would need one pass per requested direction.

26.9 Embedding the OT VJP in the filtering loop

The full LEDH-PFPPF-OT gradient is a reverse scan through time. At time t , the adjoint entering the OT block contains two sources: downstream dependence of future particles, and direct dependence of the current scalar contribution. The OT VJP returns adjoints for the pre-transport particles and log weights:

$$(G_{X_t}, G_{\log w_t}) = \text{VJP}_{\text{OT},t}(G_{\tilde{X}_t}).$$

The log-weight adjoint then flows through the PF-PF proposal-correction terms: transition density, observation density, proposal density, and flow log-determinant. The particle adjoint flows through the LEDH migration and the model transition and observation callbacks. Repeating this from $t = T$ down to $t = 1$ yields the parameter score.

Proposition 26.6 (Filtering-loop custom VJP). *Suppose each time-step map in the executed LEDH-PFPPF-OT value path is differentiable under fixed branch decisions, fixed random streams, fixed masks, and fixed iteration budgets. If each time-step custom VJP is the VJP of that same executed time-step map, then the reverse time scan returns the derivative of the executed scalar $\mathcal{L}_K(\theta)$.*

Proof. The full filter value is a finite composition of the time-step maps and the final scalar reduction. By the chain rule, the VJP of the full composition is the product of the time-step transposed Jacobians in reverse time order. The assumption says that each custom time-step VJP equals the corresponding transposed Jacobian multiplication for the executed map. Therefore their reverse-time composition equals the VJP of the executed scalar. $\square$

26.10 Implementation contract

A robust implementation should expose two separable functions:

$$\text{ot_forward}(X, \log w; \rho) = (\tilde{X}, \text{aux}), \quad \text{ot_vjp}(\text{aux}, G_{\tilde{X}}) = (G_X, G_{\log w}),$$

where ρ contains the declared OT settings: cost convention, epsilon, annealing schedule, iteration budget, row and column chunk sizes, stabilization, and stop-gradient policy. The auxiliary record should contain only mathematical or numerical data needed to reproduce the same finite map. It should not silently change the scalar by switching from streaming to dense semantics, from finite Sinkhorn to another optimizer, or from one random stream to another.

The current TensorFlow implementation follows this contract through an opt-in mode named `same_scalar_custom_vjp`. In dense mode, the custom-gradient primal calls the same finite FilterFlow-style annealed transport routine as the raw TensorFlow path, with the selected `transport_ad_mode` and any declared warm-start state. Its backward rule then recomputes that same transport matrix under a local `GradientTape` and applies the unclipped upstream transport adjoint. In streaming mode, the primal returns the transported particles without returning a dense transport matrix; the backward rule recomputes the same streaming transport under a local `GradientTape`. This is a same-scalar checkpointed or recomputed VJP route. It is not yet a fully hand-coded reverse scan of every softmax and normalization primitive in Proposition 26.5. The hand-derived reverse scan remains the mathematical destination for a future lower-level adjoint implementation.

The following checks belong with any implementation:

- (1) row and column residuals for the finite transport object;
- (2) equality of dense and streaming VJP on small problems where both are feasible;
- (3) finite-difference or raw-autodiff agreement on small fixed-seed cases;
- (4) explicit tests for stopped versus differentiated scale, keys, and potentials;
- (5) static-shape behavior under the intended compiled path;
- (6) finite value and finite score on the target model class before any HMC use.

These checks certify local derivative accounting for the declared finite computation. They do not certify the statistical accuracy of the relaxed filter, posterior convergence, or default production readiness.

26.11 Relation to forward-mode JVP

Forward-mode JVP is also mathematically legitimate. For a perturbation $\dot{\theta}$, it propagates tangents through the same filtering loop and returns

$$D_{\theta} \mathcal{L}_K(\theta) \dot{\theta}.$$

This can be memory efficient, and it is useful for diagnostics along a small number of directions. For HMC, however, the sampler usually needs the whole score vector. If the parameter dimension is d_{θ} , forward mode needs roughly d_{θ} directional passes unless it is batched over tangent directions. A VJP gives the full scalar-target score in one reverse pass. That is why the custom-gradient design in this chapter emphasizes the OT VJP.

26.12 What this chapter does and does not claim

This chapter gives a mathematical route for a custom gradient of the executed finite LEDH-PFPP-OT scalar. It claims:

- the barycentric OT product has the VJP in (26.5);
- the normalized finite-transport matrix has the VJP in (26.9)–(26.12);
- finite Sinkhorn or annealed Sinkhorn can be differentiated by a reverse scan through the executed softmin program;
- composing those local VJPs through the filtering loop gives the derivative of the same executed scalar under fixed branch and randomness assumptions.

It does *not* claim that the relaxed OT filter equals a categorical particle filter, that finite Sinkhorn equals unregularized OT, that a custom VJP improves Monte Carlo accuracy, or that the resulting score is automatically HMC-ready for a structural model. Those are separate validation questions.

Chapter 27

Retained-Teacher Neural OT Warm Starts

The previous chapter fixed the entropic OT and Sinkhorn layer as a fully declared teacher object. The present chapter introduces the narrowest neural extension of that teacher: do not replace the teacher; learn how to start it better.

This is the main implementation-bearing neural chapter of the OT block. Its question is narrower than “can a network learn transport?” It asks instead: can a network predict a good internal solver state so that a retained corrective Sinkhorn phase needs less work while still protecting the declared teacher object?

27.1 Objects and conventions

Let $X = (x_1, \dots, x_N)^\top \in \mathbb{R}^{N \times d_x}$ be the weighted source cloud and $w \in \Delta_N$ its normalized weights. Let $u = (1/N, \dots, 1/N)$ be the equal target marginal, and let $C(X, Z) \in \mathbb{R}^{N \times N}$ be the declared transport cost matrix. The teacher coupling is either the exact regularized OT optimizer Π_ε^* or the finite numerical teacher $\Pi_{\varepsilon, K}$. The barycentric teacher output is

$$Y_\tau(X, w, \varepsilon) = B(\Pi_\tau(X, w, \varepsilon)). \quad (27.1)$$

The student does not directly define a new target cloud at first. It predicts a solver-relevant latent quantity such as dual variables or scaling vectors. Write

$$h_\psi = S_\psi(X, w; \varepsilon, N, d_x), \quad (27.2)$$

where h_ψ is later translated into the initialization of the retained corrective Sinkhorn solver.

27.1.1 Retained-teacher approximation family, not one architecture

The scalable-OT survey suggests broadening the reader’s orientation here before committing to one neural warm-start architecture. The retained-teacher lane is best understood as a family of approximations that all try to preserve a declared entropic teacher object while reducing the cost of reaching or applying it. Warm-start predictors are the narrowest member of that family, but they are not the only one.

Representative retained-teacher approximation routes include:

- (i) learned warm starts for dual or scaling variables,
- (ii) Nystrom kernel factorizations that approximate the Gibbs kernel while keeping a teacher-style scaling structure,
- (iii) positive-feature Sinkhorn variants that replace the dense kernel by a positive low-rank feature factorization,
- (iv) direct low-rank transport factorizations when the resulting transported object is still audited against the teacher output,
- (v) stochastic or minibatch routes used only as teacher-side approximations rather than silently promoted to full replacement resampling rules.

The common doctrine is that these routes should still be judged first by how well they preserve the declared teacher coupling or teacher barycentric output. They are not automatically semantics-preserving merely because they are faster than a dense Sinkhorn pass. Their status depends on whether the approximation changes only the execution path, the kernel/coupling family, or the final transported object itself.

27.2 What the teacher computes

There is no single teacher unless the variant is named. In this chapter, the main teacher is the retained EOT/Sinkhorn route from the previous chapter. The most important teacher objects are:

- (i) the exact regularized coupling $\Pi_\varepsilon^\star$,
- (ii) the finite numerical teacher $\Pi_{\varepsilon,K}$,
- (iii) the barycentric teacher output $Y_\tau = B(\Pi_\tau)$.

The chapter's neural route is only intended to accelerate the finite teacher, not to redefine these objects.

27.3 Algorithm: retained-teacher warm-started Sinkhorn

What the student is allowed to change. Only the initialization of the retained teacher solver, or another explicitly named latent teacher coordinate.

What the teacher still protects. The declared transport constraints, the EOT objective, and the corrected barycentric output after refinement.

27.4 Training objective and deployment contract

Let h_τ denote the teacher latent target and let $\mathcal{D}(N, d_x, \varepsilon, m, \eta)$ be the training distribution over problem instances. A simple supervised teacher-student fitting problem is

$$\psi^\star \in \arg \min_{\psi} \mathbb{E}_{(X, w, \varepsilon, m) \sim \mathcal{D}} [\mathcal{L}\{S_\psi(X, w; \varepsilon, N, d_x), h_\tau(X, w, \varepsilon)\}]. \quad (27.3)$$

Algorithm 4 `train_and_deploy_warmstarted_sinkhorn`

Require: training distribution $\mathcal{D}$ over weighted particle clouds, declared cost construction, regularization schedule ε , teacher Sinkhorn solver, architecture class S_ψ , deployment-time corrective budget K_{corr}

Ensure: trained predictor S_ψ , corrected deployment coupling $\hat{\Pi}_{\psi,\varepsilon,K_{\text{corr}}}$, barycentric output cloud $\tilde{X}_{\psi,\text{corr}}$, and declared residual diagnostics

- 1: Generate teacher problems by solving the retained EOT/Sinkhorn route on sampled clouds from $\mathcal{D}$
- 2: Extract the latent teacher target to be predicted, e.g. dual/scaling coordinates
- 3: Fit the predictor S_ψ on this teacher data using the declared loss family
- 4: For a new weighted cloud (X, w) , compute the student prediction $h_\psi = S_\psi(X, w; \varepsilon, N, d_x)$
- 5: Translate h_ψ into the initial state of the retained corrective Sinkhorn solver
- 6: Run K_{corr} corrective Sinkhorn updates to obtain $\hat{\Pi}_{\psi,\varepsilon,K_{\text{corr}}}$
- 7: Form the barycentric equal-weight output cloud $\tilde{X}_{\psi,\text{corr}} = B(\hat{\Pi}_{\psi,\varepsilon,K_{\text{corr}}})$
- 8: Record row/column residuals, teacher-student discrepancy, and any downstream scalar checks

In practice, the deployment claim is not about this loss alone. It is about whether the corrected route preserves the teacher output well enough on held-out and stress-test clouds.

27.5 Permutation symmetry

Particle labels are arbitrary. For a permutation matrix $P \in \mathbb{R}^{N \times N}$, the predictor should at least satisfy a declared permutation symmetry. In the output-cloud case the natural condition is

$$S_\psi(PX, Pw; \varepsilon, N, d_x) = P S_\psi(X, w; \varepsilon, N, d_x) \quad \text{or the corresponding latent-state analogue.} \quad (27.4)$$

This is a necessary condition, not a target-correctness theorem.

27.6 Residual hierarchy

The first deployment residual is the teacher-student discrepancy after correction:

$$R_{\psi,\tau}(X, w, \varepsilon) = B(\hat{\Pi}_{\psi,\varepsilon,K_{\text{corr}}}) - B(\Pi_\tau(X, w, \varepsilon)). \quad (27.5)$$

This residual does not by itself control filtering or HMC target error. It only measures how well the corrected student reproduces the chosen teacher.

27.6.1 How the broader retained-teacher approximations fit here

The same residual logic gives the chapter a place to classify the broader scalable-OT approximations from the survey. If a Nystrom factorization, a positive-feature kernel, or a low-rank coupling factorization is proposed as a retained-teacher acceleration, then the

first audit question is still teacher preservation: how close is the transported cloud or coupling application to the declared teacher route from Chapter 25?

For example, a kernel-factor route may replace the dense Gibbs kernel by a factorized approximation while still producing a coupling application of the same schematic form. A low-rank coupling route may store and apply transport through structured factors rather than a dense matrix. A stochastic or minibatch route may reduce computation by solving smaller transport subproblems. In all such cases, this chapter's doctrine is the same: they are still retained-teacher candidates only if the approximation error is measured against the teacher object rather than hidden behind speedups alone.

That is why the residual hierarchy should be read broadly. The diagnostic is not restricted to one neural warm-start map. It is the first reusable test for any approximation family that claims to accelerate the teacher without changing the meaning of the transported output. Once the approximation stops being naturally judged by teacher-preservation residuals, it belongs in the later target-changing chapters instead of this retained-teacher one.

27.7 Filtering-loop insertion point

Inside the filtering loop, the retained-teacher neural route sits exactly where Chapter 25 placed the finite EOT/Sinkhorn layer:

$$\{x_i, w_i\}_{i=1}^N \xrightarrow{\text{student warm start+retained Sinkhorn}} \{\tilde{x}^{(j)}, 1/N\}_{j=1}^N. \quad (27.6)$$

It accelerates the equal-weighting layer after the weighted cloud has already been formed. It does not redefine the earlier particle-weighting rule itself.

27.8 Verification notes

A practical implementation should still check a few local quantities carefully:

- (1) teacher generation is declared and reproducible,
- (2) the predicted latent teacher object is named explicitly,
- (3) the corrected route satisfies the row/column residual contract,
- (4) teacher-student cloud discrepancy remains below the declared threshold on the tested envelope,
- (5) the downstream scalar is checked whenever the learned route is embedded in a filtering likelihood or HMC target.

These checks are mathematical and numerical, not bureaucratic. Their purpose is only to ensure that the reader can tell whether the implementation is still computing the declared teacher-student object or has drifted to a different one.

27.9 What this chapter does and does not claim

This chapter teaches the main implementation-bearing neural route in the OT block: retain the EOT/Sinkhorn teacher and learn how to start it better. It does *not* claim:

- that the student replaces the teacher object,
- that small teacher-student residuals automatically imply posterior or HMC correctness,
- that broader neural OT families are unimportant, only that they answer a different implementation question.

Its role is narrower: fix one teacher-student acceleration route in a form that a reader can nearly implement directly.

Chapter 28

ICNN, Brenier Maps, and Monge-Gap Direct Map Learning

The previous chapter studied the narrowest neural extension of the entropic OT teacher: retain the teacher and learn how to start its solver better. The present chapter changes the question. Instead of accelerating a declared solver, it asks how one can learn the transport map itself.

This is a more ambitious route. The potential gain is obvious: if a good map is learned directly, one may bypass repeated iterative solves at deployment time. The cost is equally important: the learned object is no longer an initialization for a retained teacher. It is the transport map itself, so every modeling and training choice now acts directly on the output cloud.

The chapter is deliberately slower than the earlier neural-OT material. Its job is not to survey every map-learning paper quickly. Its job is to teach the reader what a direct map-learning route is trying to learn, why Brenier/ICNN and Monge-gap methods are mathematically natural, and where the new approximation burden enters.

28.1 Reader orientation: what changes in this chapter?

The best way to read this chapter is to compare it with the retained-teacher warm-start chapter.

- In the warm-start chapter, the teacher still decides what the final transport plan and output cloud are; the learner only helps the teacher solve its own problem faster.
- In the present chapter, the learner itself proposes the transport map. There may be no corrective Sinkhorn phase at all.

So the central question is no longer “did we accelerate the same teacher?” It is “what map object have we learned, and under what assumptions should we trust it?”

Plain-language takeaway. If the previous chapter was about helping an expensive trusted calculation, this chapter is about replacing that calculation by a learned map. That is why the mathematical structure and the burden of proof both change.

28.2 Object inventory for direct map learning

A direct map-learning method tries to produce a deterministic transport map

$$T : \mathbb{R}^{d_x} \rightarrow \mathbb{R}^{d_x}. \quad (28.1)$$

The source measure μ and target measure ν are related by the pushforward relation

$$T_{\#}\mu = \nu, \quad (28.2)$$

meaning that if one samples $X \sim \mu$ and then computes $T(X)$, the law of that transformed sample is ν .

The main local objects are:

Object	Meaning in this chapter
μ, ν	source and target probability measures
T	direct transport map sending source points to target points
ψ	scalar potential whose gradient may define the transport map
ψ^*	convex conjugate of the potential when that representation is used
$T_{\#}\mu$	pushforward law obtained by applying T to source samples
$\mathcal{M}_{\rho}^c(T)$	Monge-gap regularizer measuring how OT-like the map is for cost c and reference measure ρ

Reader checkpoint. The important change relative to the warm-start chapter is immediate: the learned object is no longer an internal solver coordinate. It is the transport map itself, or a potential from which that map is derived.

28.3 Running direct-map cell

A tiny one-dimensional cell is enough to carry the main ideas. Let the source be the two-point cloud

$$X_{\text{src}} = \{-1, 1\} \quad \text{with equal weights } \frac{1}{2},$$

and let the target be

$$X_{\text{tgt}} = \{0, 2\} \quad \text{with equal weights } \frac{1}{2}.$$

The obvious translation map is

$$T^*(x) = x + 1. \quad (28.3)$$

Applying T^* to the source cloud sends $-1 \mapsto 0$ and $1 \mapsto 2$, so the pushforward equals the target cloud exactly.

For quadratic cost,

$$\frac{1}{2} \sum_{x \in X_{\text{src}}} |x - T^*(x)|^2 = \frac{1}{2}(1^2 + 1^2) = 1. \quad (28.4)$$

This simple cell already shows the difference between a direct map and a retained teacher solve. The output cloud is obtained directly by evaluating T on the source particles; there is no intermediate coupling matrix or balancing loop in this picture.

28.4 Brenier map viewpoint

In the quadratic-cost Euclidean setting, a classical route is to represent the map as the gradient of a convex potential:

$$T^*(x) = \nabla \psi^*(x), \quad \psi^* \text{ convex.} \quad (28.5)$$

This statement should be read with its theorem-level assumptions attached. The classical Brenier characterization is a continuous-measure result for quadratic cost in Euclidean space under regularity conditions such as finite second moments and absolute continuity of the source measure with respect to Lebesgue measure. In the finite empirical-cloud setting used throughout BayesFilter, a Monge map may fail to exist or fail to be unique in that theorem-strength sense. So the present chapter uses the Brenier viewpoint primarily as structured motivation for a learned map family, not as a blanket theorem that every finite cloud transport problem in the particle-filter setting is exactly represented by a gradient of a convex potential.

In one dimension, this means the derivative of the potential defines the map. For the translation cell above, one convenient choice is

$$\psi^*(x) = \frac{1}{2}x^2 + x. \quad (28.6)$$

Then

$$\nabla \psi^*(x) = x + 1 = T^*(x). \quad (28.7)$$

The second derivative of ψ^* is 1, so the potential is convex. This tiny calculation is the cleanest way to see what the Brenier/ICNN family is trying to exploit: instead of learning the map directly, it learns a structured potential whose gradient gives the map.

Reader checkpoint. The direct-map route is now more concrete. Instead of predicting the teacher’s internal solver coordinates, it tries to learn a map T , often through a potential ψ that imposes structure on what that map can be.

28.5 ICNN parameterization and training contract

Input-convex neural networks (ICNNs) provide a parametric family of scalar functions that remain convex in designated inputs. The implementation idea is:

- (i) parameterize the potential ψ by an ICNN ψ_θ ,
- (ii) obtain the transport map by differentiating the network, $T_\theta(x) = \nabla \psi_\theta(x)$,
- (iii) train ψ_θ so that the induced map sends source mass toward the target according to the chosen OT objective.

A useful objective to keep in view is the convex-potential dual form

$$J(\psi) = \mathbb{E}_{X \sim \mu}[\psi(X)] + \mathbb{E}_{Y \sim \nu}[\psi^*(Y)], \quad (28.8)$$

where ψ^* is the convex conjugate

$$\psi^*(y) = \sup_x \{x^\top y - \psi(x)\}. \quad (28.9)$$

This is best read as continuous-measure theorem language or as motivation for a structured parametric family, not as a claim that an arbitrary finite-cloud training loop automatically inherits the full optimal-transport theorem. In a finite empirical implementation, one still has to choose how μ and ν are represented, how gradients of the potential are evaluated on particle clouds, and how any conjugate or dual approximation is actually computed numerically. The attraction of this route is that the map is not a black-box regressor. It is constrained to come from a convex potential, which is exactly the sort of structure the quadratic-cost theory suggests under its stated assumptions.

The cost is also immediate from the equations. One must evaluate gradients of the potential and, depending on the chosen training formulation, may need to evaluate or approximate the convex conjugate. So the route trades a retained transport solver for a more structured but more delicate map-learning objective.

Algorithm 5 `train_icnn_brenier_map`

Require: source-cloud sampler for μ , target-cloud sampler for ν , ICNN potential family ψ_θ , declared quadratic-cost/Brenier setting, convexity-enforcement policy, optimizer, learning-rate schedule, batch size, stopping rule, and held-out validation clouds

Ensure: trained potential ψ_θ , induced map $T_\theta(x) = \nabla\psi_\theta(x)$, and validation diagnostics for pushforward mismatch and scalar drift

- 1: Initialize ICNN parameters θ and optimizer state
 - 2: **while** the stopping rule has not been met **do**
 - 3: Draw minibatches $x_b \sim \mu$ and $y_b \sim \nu$
 - 4: Evaluate the potential values $\psi_\theta(x_b)$
 - 5: Evaluate the gradient-defined transport map $T_\theta(x_b) = \nabla\psi_\theta(x_b)$
 - 6: Compute the declared dual or potential-based training objective, including any required convex-conjugate or surrogate terms
 - 7: Add any declared regularization enforcing smoothness, stability, or numerical tractability
 - 8: Backpropagate the total loss and update θ
 - 9: Re-enforce convexity according to the declared ICNN policy if it is not built directly into the parameterization
 - 10: **end while**
 - 11: On held-out validation clouds, compute mapped particles $T_\theta(x_i)$
 - 12: Evaluate endpoint pushforward mismatch, transport-cost proxy, and downstream scalar sensitivity on the held-out clouds
 - 13: Return ψ_θ , the induced map T_θ , and the declared diagnostics
-

Approximation enters here. The learned object is the transport map itself. Errors in the learned potential or its gradient directly change the transported output cloud.

28.6 Monge-gap regularization

A more flexible direct-map family regularizes a candidate map by how far it behaves from an OT map rather than forcing a particular architecture from the start. For cost c and reference measure ρ , the Monge gap is

$$\mathcal{M}_\rho^c(T) = \int c(x, T(x)) d\rho(x) - W_c(\rho, T_\#\rho). \quad (28.10)$$

As a mathematical definition, this is exact. But in a practical training loop, the second term is typically not available in closed form because the pushforward measure $T_{\#}\rho$ itself depends on the current learned map. So any finite- sample training objective that uses an estimated OT reference term should be read as a surrogate or approximate Monge-gap objective unless the estimator and its properties are stated explicitly. Operationally, the chapter’s point is: fit a useful map, but penalize it when it behaves unlike an OT map for the chosen cost.

A tiny example makes the gap visible. Keep the source cloud $\{-1, 1\}$ and target cloud $\{0, 2\}$. Consider two maps:

$$T_{\text{good}}(-1) = 0, \quad T_{\text{good}}(1) = 2, \quad (28.11)$$

$$T_{\text{bad}}(-1) = 2, \quad T_{\text{bad}}(1) = 0. \quad (28.12)$$

Both maps have the same pushforward target cloud $\{0, 2\}$, but they incur very different transport costs under the quadratic metric. The good map has cost

$$\frac{1}{2}(1^2 + 1^2) = 1,$$

whereas the bad map has cost

$$\frac{1}{2}(3^2 + 1^2) = 5.$$

So the bad map reaches the same endpoint law while moving mass much less efficiently. The Monge-gap idea is precisely to penalize such behavior.

Algorithm 6 `train_monge_gap_regularized_direct_map`

Require: source-cloud sampler for μ , target-cloud sampler for ν , reference-measure sampler for ρ , parametric map family T_θ , cost function c , endpoint-fit objective, Monge-gap weight λ_{MG} , optimizer, stopping rule, and held-out validation suite

Ensure: trained direct map T_θ , Monge-gap diagnostic history, endpoint-fit diagnostics, and downstream scalar sensitivity report

- 1: Initialize map parameters θ and optimizer state
 - 2: **while** the stopping rule has not been met **do**
 - 3: Draw minibatches from the source, target, and reference distributions
 - 4: Compute mapped source particles $z_i = T_\theta(x_i)$
 - 5: Evaluate the declared endpoint-fit loss between the mapped source batch and the target batch
 - 6: Compute the empirical transport-cost term on the reference samples from ρ
 - 7: Estimate the OT reference term needed for the empirical Monge-gap penalty; this estimate defines the practical training surrogate unless a stronger equivalence result is supplied
 - 8: Form the total loss as endpoint-fit term plus λ_{MG} times the empirical Monge-gap penalty, together with any declared regularization
 - 9: Backpropagate the total loss and update θ
 - 10: **end while**
 - 11: On held-out validation clouds, evaluate endpoint mismatch, Monge-gap magnitude, transport-cost proxy, and downstream scalar drift
 - 12: Return the trained map T_θ and the declared diagnostics
-

Reader checkpoint. The Monge-gap route still learns a direct map, but it relaxes the strong ICNN-only structural commitment. The gain is flexibility. The cost is that the learned map is now judged relative to a regularized OT-like behavior rather than by a retained teacher solver.

28.7 Structured direct-map variants

The same direct-map logic can be combined with additional structural priors, such as sparse displacement or factorized transport structure. These are worth naming because they may be useful when the filtering update itself has a scientifically justified sparse or low-active-subspace geometry. They should not be read as free improvements in all settings.

28.8 Why this chapter is not the default immediate route

The direct-map families are mathematically serious and potentially powerful, but there are two reasons they are not the monograph’s default first implementation route.

- (i) They learn the final map object itself rather than only accelerating a declared teacher solve.
- (ii) Their strongest mathematical support depends on more specific geometric assumptions than the retained-teacher acceleration route requires.

28.8.1 Direct-map families as a different scalable-OT answer

The scalable-OT survey makes the boundary with the retained-teacher chapter even clearer. Nystrom, positive-feature, and related entropic approximations begin by asking how to preserve or approximate a declared teacher transport object more cheaply. The direct-map route asks a different question: can one learn the final transport map itself well enough that repeated teacher solves are no longer the center of the method?

That difference is not merely architectural. It changes the meaning of the main residual. In the retained-teacher lane, the first residual is naturally a teacher-preservation discrepancy. In the direct-map lane, the primary residuals are pushforward mismatch, Monge-gap behavior, transport-cost proxy, and scalar drift under the learned map. So this chapter should be read as a target-boundary chapter: it explains why a direct-map learner is not just “a stronger warm start,” but a new answer to the scalable-OT problem with a different evidence contract.

In practical terms, this chapter occupies the middle position between the retained-teacher approximation lane and the broader replacement/dynamic lane. It is more radical than teacher-preserving factorized Sinkhorn, because the map itself is learned. But it is still narrower than the dynamic/operator families of the next chapter, because it aims to learn one endpoint map rather than an entire transport path or measure-level operator.

28.9 Verification and non-claims

A minimal implementation-facing reading of this chapter should distinguish:

- what map object is being fit,
- what assumptions justify that map family,
- what direct-map residuals are measured,
- what stronger filtering or HMC claims are still not justified.

This chapter does *not* claim:

- that direct learned maps are interchangeable with retained-teacher EOT acceleration,
- that a map-level fit automatically preserves the same filtering scalar,
- that broader direct-map families are already validated for BayesFilter.

Its role is to slow down the direct-map family itself so the reader can see that it is solving a different problem from the previous chapter.

Chapter 29

Dynamic Geodesic and Operator Learners, and the Filtering Target Contract

The previous chapter treated direct map-learning families such as ICNN/Brenier and Monge-gap approaches. The present chapter changes the learned object again. Instead of asking the network to learn one static transport map, it asks what happens when the learned object is a dynamic transport path, a velocity field, or an operator acting on families of measures.

This chapter is intentionally secondary. Its job is not to become another core implementation chapter. Its job is to explain why these broader learned families carry an even heavier target-status burden than retained-teacher acceleration or static direct-map learning.

29.1 Reader orientation: what changes here?

The easiest way to read this chapter is to keep the following contrast in mind.

- The retained-teacher neural chapter learned how to solve a declared transport teacher faster.
- The direct-map chapter learned one final transport map.
- This chapter asks what happens when the learned object is richer than one final map: a path, a velocity field, or an operator on entire families of measures.

So the main burden is not just implementation difficulty. It is target drift: what scalar is now being computed, and what target would a later optimizer or HMC routine actually be differentiating?

29.2 Object inventory for dynamic and operator learners

A dynamic transport learner no longer tries to produce one final map T . Instead it may try to learn:

- a geodesic path ρ_λ between source and target measures,
- a velocity field $v_\lambda(x)$ generating that path,
- or an operator $\mathcal{G}$ that maps input measures to output trajectories or output measures.

This is a larger algorithmic change because the learned object is not merely a replacement map. It is a representation of how transport unfolds.

Reader checkpoint. The present chapter has crossed another object boundary. A dynamic learner is not just another static map family. It tries to represent motion or evolution, not only the endpoint transformation.

29.3 Static map versus dynamic path: a toy contrast

A tiny one-dimensional toy example makes the distinction concrete. Suppose the source cloud is

$$X_{\text{src}} = \{-1, 1\}$$

and the target cloud is

$$X_{\text{tgt}} = \{0, 2\}.$$

A static-map learner tries to learn one rule such as

$$T(x) = x + 1, \tag{29.1}$$

which moves the source points directly to the target points.

A dynamic path learner instead tries to describe how the particles move as a function of pseudo-time $\lambda \in [0, 1]$. One simple path is

$$x(\lambda) = x + \lambda, \quad \lambda \in [0, 1]. \tag{29.2}$$

At $\lambda = 0$ the source points are unchanged, and at $\lambda = 1$ they have reached the target points. The endpoint transformation is the same as the static map, but the learned object is richer: it describes the whole path, not just the final location.

This toy distinction is enough for the present chapter. It shows why a dynamic learner carries more structure, more opportunity, and more target risk than a single direct map.

29.4 Dynamic geodesic learners

Some neural OT papers learn Wasserstein geodesics or associated velocity fields. These are mathematically appealing because particle-flow filtering already has a pseudo-time flavor. The attraction is therefore obvious: perhaps one could learn reusable transport dynamics instead of solving a fresh static problem each time.

The caution is equally important. A dynamic geodesic learner does not answer the same question as a retained finite Sinkhorn solver. It learns a richer object and must therefore be judged against a richer target contract.

Algorithm 7 `train_dynamic_transport_path`

Require: source-cloud sampler, target-cloud sampler, pseudo-time grid $0 = \lambda_0 < \dots < \lambda_K = 1$, learned velocity-field or path parameterization, rollout integrator, endpoint loss, path regularizers, optimizer, stopping rule, and validation suite

Ensure: trained path or velocity-field parameters, deployable terminal map or path object, and pathwise validation diagnostics

- 1: Initialize the path or velocity parameters and optimizer state
 - 2: **while** the stopping rule has not been met **do**
 - 3: Draw source and target cloud minibatches
 - 4: Initialize the source particles at pseudo-time λ_0
 - 5: **for** each pseudo-time step $k = 0, \dots, K - 1$ **do**
 - 6: Evaluate the learned velocity field or path increment at λ_k
 - 7: Advance the particles using the declared rollout or integration rule
 - 8: **end for**
 - 9: Compute the endpoint loss against the target cloud at $\lambda = 1$
 - 10: Compute any declared pathwise regularizers, such as smoothness or action penalties
 - 11: Backpropagate the total loss and update the parameters
 - 12: **end while**
 - 13: On held-out clouds, evaluate endpoint fit, path regularity, discretization sensitivity, and downstream scalar sensitivity
 - 14: Return the trained path/velocity object together with the declared diagnostics
-

Reader checkpoint. A geodesic learner is not just a faster static map and not just a warm-started solver. It is a learner for the entire transport path or for the dynamics generating that path. That greater ambition is why the chapter is more about burden and verification than about a single near-implementable core algorithm.

29.5 Operator-learning families

A related family learns operators on spaces of measures. In such approaches, the input is itself a measure or empirical cloud, and the output is either another measure, a path of measures, or a set of latent coordinates describing the transport. This is useful as a representation idea, but it moves even farther from the finite retained-teacher transport layer of the previous chapters.

The main conceptual advantage is reuse across families of problems. The main conceptual cost is that the learned object is even less tied to a single declared teacher solve. That makes these methods valuable as research-direction chapters, but not as the default first implementation route in this monograph.

29.6 Adjacent boundary-marker families

Several nearby families remain useful only as boundary markers in this monograph: - measure-to-measure regression operators, - local Wasserstein regression, - manifold or Riemannian neural OT.

These show that neural transport ideas extend beyond the current finite-cloud Euclidean BayesFilter setting. They do not, by themselves, answer the present implementation question.

29.6.1 Sliced, subspace, and localization families as target-changing lanes

The scalable-OT survey gives this chapter a more precise boundary around several important high-dimensional families. Sliced and subspace OT methods attack high dimension by projecting the measures onto one-dimensional directions or selected subspaces before solving lower-dimensional transport problems. Localization and block-structured OT instead exploit state-space structure by transporting local blocks or using localized costs rather than one global cloud-wide coupling.

These methods are mathematically important, but in the BayesFilter monograph they should not be described as mere compressed versions of the dense entropic teacher. A projected one-dimensional or subspace transport, or a block-localized transport, is generally a different transport object from the full-state dense coupling used in the previous chapters. That means the main burden here is target interpretation rather than just solver engineering.

In that sense, sliced/subspace and localization families fit the present chapter better than the retained-teacher chapter. Their primary promise is not simply to reach the same teacher object faster. Their promise is to define a more tractable transport rule in high dimension. The price is that the resulting output cloud must usually be read as a new resampling approximation unless a separate argument shows how the projected or localized transport reconstructs the intended full-state object.

This chapter therefore treats these families as target-changing or structure-changing candidates by default. They may still be useful and even necessary in high dimension, but their first audit question is no longer only teacher-preservation residual. It is: what transport object is now being applied to the filtering ensemble, and what scalar target would a later optimizer or HMC routine actually differentiate if this object were embedded downstream?

29.7 Same-scalar rule revisited

The most important target-contract warning survives every change of learned object. If a downstream routine uses a scalar $\tilde{L}$ built from a learned map, path, or operator, then the reported gradient must differentiate that same scalar. Even if this value-gradient parity holds, the result is coherent only for the learned target unless a correction returns to the teacher or original target.

The small toy example from the earlier chapter already showed how a learned output cloud can change a simple downstream scalar. The same logic becomes more severe here because the learned object is richer and therefore has more ways to drift from the teacher semantics. A path learner can change not only the final cloud, but also the whole trajectory through which intermediate summaries are defined.

Plain-language takeaway. A smooth learned path is not automatically a faithful path for the original target. It may simply be a coherent path for a different learned target.

29.8 Consistency notes for richer learned objects

For richer learned objects, a few mathematical consistency checks are especially important:

- (i) whether endpoint or pathwise improvement is accompanied by stability of the downstream scalar,
- (ii) whether the learned object behaves robustly under weight, dimension, or particle-count shift,
- (iii) whether a reported gradient is the gradient of the same scalar value that is later interpreted downstream,
- (iv) whether any claim of target preservation is supported by an explicit correction mechanism rather than by smoothness or speed alone.

These are not governance gates. They are simply the mathematical checks needed to understand what object is being computed and whether it remains identifiable from the declared scalar and learned path or operator.

Algorithm 8 `audit_dynamic_transport_target_contract`

Require: learned output object (map, path, or operator), downstream scalar-construction routine, reported gradient routine, optional teacher or corrected baseline scalar, shift-test suite, and declared tolerances for scalar and gradient mismatch

Ensure: a consistency report containing same-scalar parity status, shift-robustness status, any failed conditions, and the strongest mathematically supported interpretation of the learned object

- 1: **for** each evaluation problem in the declared suite **do**
 - 2: Construct the learned output object for that problem
 - 3: Compute the downstream scalar $\tilde{L}$ from that learned object
 - 4: Compute the reported gradient of that same scalar
 - 5: **if** a teacher or corrected baseline exists **then**
 - 6: Compute the baseline scalar and, where applicable, the baseline gradient
 - 7: Record scalar and gradient gaps relative to the baseline
 - 8: **end if**
 - 9: Check that the reported gradient is tied to the same scalar $\tilde{L}$
 - 10: Evaluate shift robustness under particle-count, weight, and context variation
 - 11: Record any failed scalar, gradient, or robustness condition
 - 12: **end for**
 - 13: Return the full consistency report and the strongest interpretation of the learned object consistent with those results
-

Reader checkpoint. This procedure does not approve or reject a method in an institutional sense. It simply organizes the mathematical question the chapter cares about: is the learned object still identifiable through the scalar and gradient that later code actually uses?

29.9 Relation to the full HMC target chapter

This chapter stops at the target-status burden created by broader dynamic and operator learners. It does not settle the HMC question. That full taxonomy still belongs to Chapter 30, which interprets the scalar and gradient contracts rung by rung across the whole DPF ladder.

29.10 What this chapter does and does not claim

This chapter explains why the broadest neural OT families should be read as secondary research directions in this monograph. It does *not* claim:

- that geodesic, operator, and adjacent measure/manifold families solve the same implementation problem as retained-teacher acceleration,
- that a richer learned object is automatically more useful for BayesFilter,
- that these families are already ready for the same implementation status as the earlier chapters.

Its role is to close the expanded OT block responsibly by keeping the broadest learned families in view while preserving the main algorithmic center of gravity in the earlier, more concrete chapters.

Chapter 30

DPF HMC Target Correctness and Structural-Model Suitability

The preceding DPF chapters built a ladder of increasingly differentiable filtering constructions: raw particle-flow proposals, proposal-corrected PF-PF, soft differentiable resampling, deterministic OT equal-weighting, entropic optimal-transport relaxation with Sinkhorn, retained-teacher learned transport acceleration, and broader learned-map or dynamic-transport surrogates. This chapter asks a narrower and more severe question. If one of those constructions is used inside Hamiltonian Monte Carlo for a nonlinear DSGE or MacroFinance model, what target is the Markov chain intended to leave invariant?

The answer cannot be inferred from the label “differentiable particle filter.” HMC is a target-specific algorithm. Its mathematical interpretation depends on the scalar log target used for accept/reject decisions, the gradient used by the numerical integrator, and the correction mechanism, if any [Neal, 2011, Betancourt, 2017]. Particle MCMC adds a separate target logic: an unbiased, nonnegative likelihood estimator can define a corrected extended-space chain under pseudo-marginal assumptions, but that is not the same object as a smooth surrogate likelihood differentiated by HMC [Andrieu and Roberts, 2009, Andrieu et al., 2010]. The DPF ladder must therefore separate three questions:

- (i) whether the scalar value supplied to HMC is the intended posterior value, an unbiased estimator inside a corrected construction, or a deliberately approximate value;
- (ii) whether the gradient supplied to the integrator is the derivative of that same scalar value;
- (iii) whether the evidence is strong enough for research, bank-facing, or production-governance language.

30.1 Objects in the HMC target contract

Let $\theta \in \Theta$ be the structural parameter and let $u \in \mathbb{R}^{d_\theta}$ be the unconstrained parameter used by the sampler, with transform $\theta = T(u)$. Let

$$\ell(u) = \log p(T(u)) + \log L(y_{1:T} \mid T(u)) + \log |\det DT(u)|$$

denote the scalar log target in unconstrained coordinates when the filtering likelihood is exact. This expression assumes that the transform is differentiable on the sampler-valid region and that $DT(u)$ is the square Jacobian relevant to the change of variables,

or the appropriate constrained- to-unconstrained volume term if the parameterization is more general. In a DPF construction the likelihood term may be replaced by a particle, relaxed, or learned object. The HMC contract is therefore stated for the actual scalar value $\ell_\star(u)$ used by the sampler, not for the method family name.

Table 30.1: Objects that must be named before a DPF construction can be called an HMC target.

Object	Mathematical role	Reviewer audit question
Parameter u and transform T	Coordinates integrated by HMC and the map back to structural parameters.	Are support constraints, Jacobian terms, and invalid parameter regions included in the scalar target?
Scalar log target $\ell_\star(u)$	The value whose density $\pi_\star(u) \propto \exp\{\ell_\star(u)\}$ is claimed as the invariant target.	Is $\ell_\star$ exact, pseudo-marginal extended-space, relaxed, or learned?
Potential $U_\star(u)$	$U_\star(u) = -\ell_\star(u)$, up to an irrelevant additive constant.	Is the sign convention consistent between value, gradient, and accept/reject?
Momentum p and mass M	Auxiliary Gaussian momentum $p \sim \mathcal{N}(0, M)$ used to define Hamiltonian dynamics.	Is M fixed or adapted under a declared warmup policy, and is it positive definite?
Hamiltonian $H_\star(u, p)$	$H_\star(u, p) = U_\star(u) + \frac{1}{2}p^\top M^{-1}p$ in ordinary separable HMC.	Does the accept/reject step evaluate this same $H_\star$?
Integrator $\Phi_{\epsilon, L}$	Numerical map used to propose (u', p') after L steps of size ϵ .	Is the proposal reversible and volume preserving for the stated correction, or is a different Metropolis ratio supplied?
Gradient $g(u)$	Vector supplied to the integrator in place of $\nabla_u \ell_\star(u)$.	Is $g(u) = \nabla_u \ell_\star(u)$ for the same scalar value, or is the mismatch explicitly corrected and diagnosed?
Accept/reject value	Scalar value used in the Metropolis correction.	Does the correction use the intended target, the relaxed target, or a learned surrogate?
Target-status category	Label attached to the resulting chain.	Does the label distinguish exact, pseudo-marginal, relaxed, learned, and diagnostic-only use?

Assume that $\ell_\star : \mathbb{R}^{d_\theta} \rightarrow \mathbb{R}$ is differentiable on the interior of the sampler's valid region, and that $g : \mathbb{R}^{d_\theta} \rightarrow \mathbb{R}^{d_\theta}$ is the vector field supplied to the integrator in the same coordinates. The mass matrix M in the ordinary separable Hamiltonian is a symmetric positive-definite $d_\theta \times d_\theta$ matrix, so $p^\top M^{-1}p$ is well defined. The minimum value-gradient consistency condition is

$$g(u) = \nabla_u \ell_\star(u). \quad (30.1)$$

Equation (30.1) is not a claim that $\ell_\star$ is the exact nonlinear filtering likelihood. It

is a claim that the gradient used by the integrator and the value used by the target interpretation are the same mathematical object.

If the value is ℓ_* but the integrator uses a vector field $g \neq \nabla \ell_*$, the usual leapfrog derivation no longer describes the Hamiltonian flow of H_* . A Metropolis correction may still be valid for an arbitrary proposal only after reversibility, volume preservation, and the correct acceptance ratio have been established. Without that separate proposal-level proof, a smooth chain with mismatched value and gradient is not evidence of correct HMC. If both value and gradient are changed to a relaxed or learned scalar, then the chain may be internally coherent, but the target has changed to π_* .

30.2 Target-status taxonomy

The DPF chapters use the following target-status categories. The categories are deliberately conservative because a reviewer should be able to tell exactly what posterior, extended posterior, or surrogate posterior is being discussed.

Exact-likelihood HMC

HMC evaluates a deterministic scalar log posterior and its exact or audited derivative. Numerical integration error is corrected by the standard Metropolis step, provided the implemented proposal satisfies the usual HMC reversibility and volume-preservation assumptions.

Pseudo-marginal MCMC

A random nonnegative likelihood estimator is included as part of an extended-space target. The claim is not that the estimator is a smooth likelihood surrogate. The claim is that marginal correctness follows from the pseudo-marginal construction under its assumptions [Andrieu and Roberts, 2009, Andrieu et al., 2010].

Noisy-gradient method

A stochastic or approximate gradient is used to move the chain. This is not exact HMC unless a valid correction theory is stated. It may be useful as an optimization, adaptation, or diagnostic device, but that usefulness does not by itself identify the invariant posterior.

Delayed-acceptance or surrogate-corrected MCMC

A surrogate or particle object proposes, screens, or accelerates moves, but a later correction evaluates the intended target. The target claim belongs to the corrected chain, not to the surrogate alone.

Relaxed-target HMC

The scalar log target deliberately replaces a discontinuous or discrete filtering operation by a smooth relaxed operation, such as soft resampling or entropic OT. HMC can be coherent for this scalar if Equation (30.1) holds, but the posterior is the relaxed posterior.

Learned-surrogate HMC

A neural or amortized map defines part of the scalar value. The chain targets the learned scalar only if value and gradient are consistent. It

targets the original or teacher posterior only after a separate correction or error-bound argument strong enough for that claim.

30.3 Rung-by-rung target analysis

30.3.1 EDH and LEDH flow proposals

At the raw EDH or LEDH rung, the value object is a flow-based approximate filtering quantity. The flow is derived from Gaussian closure or local linearization assumptions and is integrated numerically. If HMC differentiates the same scalar flow object that it evaluates, then the construction can satisfy value-gradient consistency for that scalar. Its target status is nevertheless limited:

- exact-target HMC only in recovery cases where the flow construction equals the exact filtering likelihood, such as the linear-Gaussian benchmark;
- approximate-target or value-side benchmark HMC outside those recovery cases;
- diagnostic-only use if the implementation differentiates a path that differs from the scalar value used for accept/reject.

Smoothness of the flow ODE, by itself, does not upgrade a closure-based filtering approximation into an exact nonlinear posterior target.

30.3.2 PF-PF proposal correction

At the PF-PF rung, the flow is used as a proposal and the scalar value includes the proposal-to-target correction through importance weights and change-of-variables terms. This is the first rung in the BayesFilter DPF ladder with an explicit proposal-correction interpretation. That statement is not a production or validation claim. It means only that the value-side target story is more defensible than raw flow transport because the proposal density, Jacobian or log determinant, and particle weight enter the same scalar object. The improvement is proposal-accounting clarity for the stated one-step conditional construction in Chapter 22; it is not evidence, by itself, of multi-step posterior fidelity, finite-particle stability, or HMC correctness.

For HMC, the required scalar is the corrected particle objective actually used by the sampler. The required gradient is the derivative of that same corrected objective, including every differentiable term in the proposal density, transition density, measurement density, flow map, and log determinant. The remaining evidence burden is substantial:

- finite-particle variability must be measured rather than hidden by a deterministic seed;
- log-determinant and Jacobian paths must be audited against the same value path;
- flow discretization error must be separated from Monte Carlo error;
- compiled repeated evaluation must remain finite and reproducible on the structural model class being claimed;

- if randomness enters the HMC target, the construction must state whether it is pseudo-marginal extended-space MCMC, a fixed-randomness deterministic approximation, or a noisy-gradient diagnostic.

Thus PF-PF is a plausible first research rung for DPF-HMC experiments, not an HMC-validity theorem and not a bank-governance result.

30.3.3 Soft differentiable resampling

Soft resampling replaces categorical ancestor selection by a differentiable mixture or shrinkage construction. The scalar value is therefore a soft-resampling filtering value. If HMC differentiates exactly that scalar, the resulting chain can be internally coherent for the soft target. It should not be described as sampling the categorical-resampling posterior unless a separate correction is supplied.

The core mathematical issue is nonlinear test-function bias. Even if a soft-resampling map preserves a weighted mean under special conditions, it does not generally preserve expectations of nonlinear functions, path genealogies, or the likelihood contribution of the original categorical particle filter. The HMC label must therefore be relaxed-target HMC, surrogate-target HMC, or diagnostic use, depending on the scalar value actually corrected by the sampler.

30.3.4 OT and entropic OT resampling

At the OT/EOT rung, resampling is written as a transport problem. Entropic regularization and a finite Sinkhorn solve make the construction differentiable and numerically structured, but they also define a different map from exact categorical resampling. The scalar value must specify the cost, entropy convention, regularization parameter, stopping tolerance, and whether the unrolled solver path or an implicit derivative is used.

The scalable-OT survey sharpens the target-status split inside this rung. Exact online or streamed Sinkhorn implementations can still belong to the same relaxed entropic target class if they preserve the same declared teacher object and the same executed scalar. Their main changes are engineering changes in storage, blocking, or GPU execution, not changes in the target definition itself. By contrast, factorized kernel routes, low-rank coupling routes, sliced/subspace transports, and localized/block transports should not all inherit that same status automatically. Some remain best read as retained-teacher approximations, while others are better read as explicit new transport rules.

For HMC, the target status is relaxed-target HMC when the accept/reject value and gradient both use the same EOT/Sinkhorn scalar. Finite solver residuals, underflow handling, stabilization, and ε sensitivity are target diagnostics, not mere implementation details. A finite Sinkhorn layer does not become an exact categorical resampling layer because its gradient exists. Nor does a faster implementation become a new target simply because its execution path is different, provided the same scalar and same transport object are still being evaluated.

30.3.5 Learned or amortized OT

Learned OT introduces a second approximation layer: a neural or amortized map is trained to imitate, accelerate, or residual-correct a transport teacher. The scalar value is then the learned scalar unless a correction evaluates the teacher or original target.

The gradient path is the derivative through the learned map and any explicitly retained teacher terms.

The scalable-OT survey helps separate two cases that should not be conflated. Retained-teacher accelerators, such as warm-start predictors or other teacher-preserving approximations, may still be judged first against the same transport teacher object. Their HMC burden is therefore to show that the value and gradient are consistent for the same learned-or-corrected scalar and that teacher-preservation residuals are understood on the tested envelope. Broader transport learners, sliced or localized replacement rules, and dynamic/operator families do not even begin with that teacher-preservation presumption. They are better read as learned or replacement targets unless a separate correction or error argument returns the chain to the teacher or original target.

The target-status label must therefore be learned-surrogate HMC unless the implementation supplies a valid correction or an error analysis strong enough to promote the claim. Runtime reduction, smoothness, and low teacher residual on training clouds are not target-correctness evidence. The missing validation includes out-of-distribution particle-cloud tests, seed sensitivity, posterior comparison against the EOT teacher, and stress tests on structural parameter regions that are invalid, nearly invalid, or geometrically stiff.

30.4 Structured target maps

Table 30.2: Rung-by-rung HMC target map.

Rung	Scalar value	Gradient path	Target status	Missing validation before promotion
EDH/LEDH	Gaussian-closure or local-flow filtering scalar.	Derivative of the same flow scalar, if implemented consistently.	Exact only in recovery cases; otherwise approximate benchmark.	Linear-Gaussian recovery, nonlinear likelihood-error studies, flow discretization checks, invalid-region behavior.
PF-PF	Proposal-corrected particle scalar with weights and flow Jacobian/log-det terms.	Derivative of the same corrected scalar, including correction terms.	First rung with explicit proposal-correction interpretation; research candidate only.	Finite-particle variance, Jacobian/log-det audit, value-gradient parity, compiled repeated evaluation, randomness policy.
Soft re-sampling	Soft or shrink-age resampling scalar.	Derivative through the same soft map.	Relaxed or surrogate target.	Posterior sensitivity against categorical or trusted references, nonlinear test-function bias, seed and temperature sensitivity.

Rung	Scalar value	Gradient path	Target status	Missing validation before promotion
OT/EOT	Transport-relaxed scalar with cost, entropy, ε , and solver budget.	Derivative through the same unrolled or implicit solver path.	Relaxed target.	ε and Sinkhorn-budget sensitivity, residual tolerances, stabilization audit, teacher posterior comparison.
Learned OT	Learned approximation to an OT/EOT or residual-corrected scalar.	Derivative through the learned map and retained teacher terms.	Learned surrogate unless corrected.	Teacher residual to posterior-error analysis, out-of-distribution cloud tests, structural stress tests, correction evidence.

Table 30.2 is a target map, not a performance table. It does not say that the lower rungs are useless or that the higher rungs are invalid. It says that every rung must name the scalar target before an HMC claim can be interpreted.

30.5 Pseudo-marginal and surrogate distinctions

The pseudo-marginal result and the DPF surrogate story are often confused because both involve particle approximations. They are different claims. Suppose $\widehat{L}(y \mid \theta, \omega) \geq 0$ is an unbiased estimator of $L(y \mid \theta)$ under auxiliary randomness ω :

$$\mathbb{E}_{\omega \mid \theta} [\widehat{L}(y \mid \theta, \omega)] = L(y \mid \theta).$$

Pseudo-marginal MCMC targets an extended density involving ω and can recover the exact marginal posterior under its assumptions. The validity argument is an extended-target argument. It is not an argument that $\log \widehat{L}$ is a smooth deterministic likelihood whose pathwise gradient is the gradient of $\log L$.

By contrast, a relaxed DPF target usually chooses a deterministic or fixed-randomness scalar $\ell_\star(\theta)$ and differentiates it. HMC may be valid for $\ell_\star$ if the value-gradient contract and proposal correction hold. That does not imply validity for the original posterior $\ell(\theta)$. A delayed-acceptance or surrogate-corrected scheme can bridge the gap only if the final correction evaluates the intended target with the right acceptance probability.

30.6 Why nonlinear DSGE models sharpen the issue

Nonlinear DSGE models make target drift especially costly because the filtering law is tied to structural economics rather than only to a generic latent-state transition. The DPF chapter does not need to solve the economics, but it must not hide target changes inside a numerical method. The main DSGE stress points are:

- (i) **Determinacy and invalid regions:** the sampler will propose parameters outside or near determinacy regions. The target must return a declared finite penalty or $-\infty$ policy, not an uncaught solver failure.

- (ii) **Pruning and approximation convention:** first-order, higher-order, pruned, and simulation-based approximations can define different latent laws. A DPF likelihood is only meaningful after the structural law is fixed.
- (iii) **Structural timing:** observables, controls, states, and shocks must enter the transition and measurement equations with the intended timing. A smooth filter for the wrong timing is still the wrong target.
- (iv) **Latent-state semantics:** deterministic-completion coordinates and stochastic state coordinates are not interchangeable. Particle clouds must represent the declared latent law.
- (v) **Local Jacobian fragility:** EDH, LEDH, and proposal-corrected flow methods inherit local-linearization and derivative fragility near boundaries or stiff regions.

For DSGE work, the correct conclusion is conservative. A DPF rung may be a useful research target after it states its scalar value and diagnostics. It is not bank-facing evidence until the structural law, invalid-region policy, value-gradient parity, and posterior sensitivity have been tested on the named model class.

30.7 Why MacroFinance models sharpen the issue

MacroFinance latent-factor systems create a different target-risk profile. Their long panels can amplify small likelihood changes, and their latent factors often interact with volatility, measurement noise, and missing-data patterns. A relaxed or learned resampling map that looks harmless on a short toy filter can move posterior mass materially in a long panel.

The minimum MacroFinance limitations to record are:

- latent dimension and particle degeneracy pressure;
- long observation spans that compound small per-period target changes;
- missing, ragged-edge, or mixed-frequency observation patterns;
- volatility and measurement-noise directions with sharp curvature;
- compiled repeatability across seeds, batches, and hardware settings;
- model-risk governance, including reproducible artifacts and claim boundaries.

These limitations do not rule out DPF research. They rule out treating a smooth differentiable filtering path as banking evidence before the target and validation ladder have been passed.

30.8 Relation to surrogate-HMC acceleration

Surrogate-HMC and Hamiltonian-neural-network ideas address a different point in the inference stack [Greydanus et al., 2019]. They attempt to improve movement through an already chosen target, or to use a surrogate proposal that is later corrected against that target. DPF methods may instead alter the filtering likelihood itself through flow approximation, relaxed resampling, or learned transport. The distinction is operational:

- a corrected surrogate-HMC proposal can preserve the intended target if the correction evaluates that target and the proposal assumptions hold;
- a relaxed DPF likelihood defines a new target unless a correction returns to the original likelihood;
- a learned DPF transport map defines a learned target unless the sampler corrects against the teacher or original target.

Table 30.3: Method-family distinctions relevant to DPF-HMC claims.

Method family	What is approximated?	Target implication	Required wording
Classical exact HMC	Hamiltonian trajectory by numerical integration.	Target preserved after Metropolis correction when value, gradient, and proposal conditions hold.	Exact for the named scalar target, not exact because the trajectory is exact.
Pseudo-marginal MCMC	Likelihood value by an unbiased nonnegative estimator on extended space.	Marginal posterior can be exact under pseudo-marginal assumptions.	Corrected extended-space chain, not smooth surrogate-gradient HMC.
Noisy-gradient method	Gradient or force field used for movement.	No exact posterior claim without correction theory.	Diagnostic or approximate unless corrected.
Delayed-acceptance surrogate	Early-stage value or proposal geometry.	Can preserve intended target if final correction evaluates intended target.	Surrogate-assisted, not target-defining, when corrected.
Relaxed DPF	Filtering likelihood construction itself.	Posterior changes to relaxed posterior.	Relaxed-target HMC.
Learned DPF	Transport, residual, or filtering map by a learned function.	Posterior changes to learned posterior unless corrected.	Learned-surrogate HMC pending correction or validation.

30.9 Evidence gates for banking language

The chapter’s recommendation is evidence-gated. A method can be mathematically interesting, credible as a research prototype, and still not ready for a bank-facing model-risk claim. The following gates constrain all wording in this chapter and in later experiments.

Table 30.4: Evidence ladder for DPF-HMC banking and governance language.

Claim level	Allowed claim	Required evidence	Disallowed short-cut
Exploratory use	The rung is worth studying on controlled examples.	Named scalar value, value-gradient check, finite smoke tests, clear target status.	Calling the method suitable because it is differentiable.
Credible re-search use	The rung has a defensible target interpretation for a named model class.	Source-grounded derivation, repeated fixed-seed and varied-seed tests, posterior comparisons, failure-region diagnostics.	Treating a toy result as evidence for DSGE or MacroFinance deployment.
Bank-facing re-search claim	The rung can support a restricted internal research conclusion under model-risk controls.	Reproducible artifacts, model-specific stress tests, challenger comparisons, audit trail, documented limitations.	Using runtime improvement as target-correctness evidence.
Production or governance claim	The rung is approved for controlled production use in a defined workflow.	Independent review, change control, monitoring, rollback, data and model governance, stability across releases.	Inferring production readiness from chapter-level mathematics.

No rung in this chapter is promoted to production or bank-governance status. The strongest current recommendation is a bounded experimental hypothesis: prioritize proposal-corrected PF-PF first because it is the first rung with an explicit proposal-correction interpretation, then compare relaxed and learned rungs as deliberately changed targets. The hypothesis remains conditional on the finite-particle, Jacobian, randomness, compilation, and posterior-comparison gates stated above.

30.10 BayesFilter recommendation

The disciplined recommendation is:

- (i) use EDH and LEDH as recovery and approximation benchmarks, not as generic exact HMC targets;
- (ii) treat proposal-corrected PF-PF as the first experimental hypothesis with an explicit proposal-correction interpretation and a defensible value-side target story, subject to finite-particle, Jacobian, randomness, compilation, and posterior-comparison gates;
- (iii) label soft resampling and OT/EOT as relaxed-target HMC unless a correction restores the original categorical or trusted target;

- (iv) label learned OT as learned-surrogate HMC unless a correction or posterior-error argument justifies stronger wording;
- (v) require DSGE and MacroFinance promotion evidence at the model-class level before any bank-facing claim.

For the current BayesFilter LEDH repair lane, the serious experimental route is therefore not a no-resampling shortcut. The route to test is Li-Coates Algorithm 1 UKF LEDH with PF-PF correction and a declared Corenflos-style OT, Sinkhorn, or annealed-transport resampling relaxation. Carrying the Algorithm 1 covariance state through that transport is an implementation contract for the selected route, because the covariance state belongs to Algorithm 1 while the transport map defines the relaxed resampling layer. A no-resampling graph or fixed-randomness run can still be useful as a kernel, shape, or gradient-plumbing diagnostic, but it is not by itself evidence that the DPF route is filter-ready or HMC-ready.

30.11 What this chapter does and does not claim

This chapter claims that DPF-HMC analysis must be organized around named scalar targets and value-gradient consistency. It does not claim:

- that PF-PF has already passed finite-particle, Jacobian, compiled-path, or posterior-comparison checks in BayesFilter;
- that relaxed or learned resampling targets are unusable;
- that pseudo-marginal validity transfers to deterministic relaxed gradients;
- that nonlinear DSGE or MacroFinance DPF-HMC has been validated in this repository.

Its role is narrower: it clarifies which scalar target is being differentiated, what correction mechanism is being invoked if any, and what mathematical interpretation the resulting chain can support.

Chapter 31

DPF Literature-to-Debugging Verification Contract

The preceding DPF chapters separate the mathematical layers: classical particle filtering, particle-flow transport, proposal correction, differentiable resampling, OT/Sinkhorn relaxation, learned transport, and HMC target interpretation. This chapter turns that separation into a verification contract. Its purpose is not to validate any implementation by prose. Its purpose is to tell a coding agent or reviewer which equation, controlled example, implementation quantity, tolerance, and failure interpretation belongs to each layer.

The contract is deliberately ordered. Target and value-gradient checks come before tuning. Teacher-map diagnostics come before learned-map diagnostics. Equation-local failures should be interpreted narrowly: a failed Sinkhorn marginal test is evidence about a finite solver, not evidence that particle flows are wrong; a learned-map residual is evidence about the student and its training distribution, not evidence that the OT teacher is invalid.

31.1 Diagnostic order

Use this order before changing code or tuning numerical parameters:

- (1) name the scalar object being estimated, relaxed, learned, or sampled;
- (2) verify that the gradient path corresponds to that same scalar object whenever HMC or autodiff claims are made;
- (3) test the classical particle-filter recursion and likelihood estimator on a controlled model;
- (4) test the particle-flow endpoint and homotopy derivative before testing PF-PF proposal correction;
- (5) test PF-PF weights and log-determinant accounting before interpreting resampling or OT behavior;
- (6) test categorical, soft, EOT, Sinkhorn, and barycentric objects before testing a learned student map;

(7) only after those checks pass, run posterior comparisons and sampler diagnostics.

This order prevents a common error: making a numerically smooth path smoother without first proving that it is the intended path.

31.2 Proposal log-probability autodiff topology

The most expensive debugging lesson in the FilterFlow/BayesFilter comparison was not a floating-point tolerance issue and not a seed issue. It was an autodiff-topology issue in the adapted proposal log probability. The forward numbers can be exactly right while the derivative is wrong if the proposal sample and the proposal log probability are wired through the same distribution object in the computation graph.

Write

$$f_\theta(x_t \mid x_{t-1}) \quad \text{and} \quad g_\theta(y_t \mid x_t)$$

for the transition and observation densities. For an adapted proposal

$$q_\theta(x_t \mid x_{t-1}, y_t),$$

the one-step log-weight update corresponding to (20.12) is

$$\log \tilde{w}_t^{(i)} = \log w_{t-1}^{(i)} + \log f_\theta(x_t^{(i)} \mid x_{t-1}^{(i)}) + \log g_\theta(y_t \mid x_t^{(i)}) - \log q_\theta(x_t^{(i)} \mid x_{t-1}^{(i)}, y_t). \quad (31.1)$$

The last term is not the log probability of a random-number-generation event. It is the density of the realized particle value under the declared proposal law.

In the linear-Gaussian adapted-proposal case used by the row-173 debug trace, the optimal proposal satisfies

$$q_\theta(x_t \mid x_{t-1}, y_t) = p_\theta(x_t \mid x_{t-1}, y_t) = \frac{f_\theta(x_t \mid x_{t-1})g_\theta(y_t \mid x_t)}{p_\theta(y_t \mid x_{t-1})}. \quad (31.2)$$

Substituting (31.2) into the signed increment in (31.1) gives

$$\begin{aligned} & \log f_\theta(x_t \mid x_{t-1}) + \log g_\theta(y_t \mid x_t) - \log q_\theta(x_t \mid x_{t-1}, y_t) \\ &= \log p_\theta(y_t \mid x_{t-1}). \end{aligned} \quad (31.3)$$

Thus, when the proposal is the exact adapted proposal, the signed increment is independent of the realized value x_t after cancellation. Equivalently,

$$\frac{\partial}{\partial x_t} [\log f_\theta(x_t \mid x_{t-1}) + \log g_\theta(y_t \mid x_t) - \log q_\theta(x_t \mid x_{t-1}, y_t)] = 0.$$

This zero derivative is a cancellation of three density derivatives. It is not obtained by making the proposal-density derivative vanish.

The correct implementation topology therefore has two conceptual steps:

- (i) sample the proposed particle from the proposal law;
- (ii) evaluate the proposal density at the realized particle as an ordinary density argument.

In pseudocode:


```
sample_dist = make_proposal_dist(previous_state, observation)
proposed_particles = sample_dist.sample(seed)
```

```
density_dist = make_proposal_dist(previous_state, observation)
proposal_ll = density_dist.log_prob(proposed_particles)
```

The second distribution has the same numerical parameters as the first, but it is a fresh density-evaluation node. Equivalently, one may call a helper such as `optimal_proposal_log_prob(...)` that rebuilds the proposal mean and covariance before applying `log_prob`. This is the topology used by FilterFlow’s executable contract: `propose` first returns a state whose `particles` field contains the sampled particles, and `loglikelihood(proposed_state, state, observation)` then recomputes the proposal distribution and evaluates its density at `proposed_state.particles`.

The tempting but wrong topology is

```
proposal_dist = make_proposal_dist(previous_state, observation)
proposed_particles = proposal_dist.sample(seed)
proposal_ll = proposal_dist.log_prob(proposed_particles)
```

This can be value-correct: the scalar value of `proposal_ll` may match the fresh-density value exactly. But the derivative can be wrong because the autodiff graph is no longer the graph of the mathematical density function $x_t \mapsto \log q_\theta(x_t \mid x_{t-1}, y_t)$. The proposed particle is now also the reparameterized sample produced by the same distribution node. In the row-173 trace, that wiring made the BayesFilter VJP

$$\frac{\partial}{\partial x_t} \log q_\theta(x_t \mid x_{t-1}, y_t)$$

with unit upstream equal to zero, while FilterFlow’s density-at-realized-particle route gave a maximum absolute VJP of

$$26.075772828701865.$$

The forward proposal log-probability values still agreed. The value agreement therefore hid the bug.

The downstream effect was exactly the cancellation failure predicted by (31.3). With the naive wiring, BayesFilter’s unit-upstream VJP of

$$\log f_\theta + \log g_\theta - \log q_\theta$$

with respect to the proposed particles had maximum absolute discrepancy

$$26.075772828710043,$$

and the normalized previous-log-weight carry VJP had discrepancy

$$7.930673258915529.$$

After evaluating the proposal log probability through a freshly recomputed proposal density, the proposal-log-probability VJP matched FilterFlow exactly in that probe, the signed-sum discrepancy fell to about

$$1.49 \times 10^{-11},$$

and the normalized carry discrepancy fell to about

$$1.45 \times 10^{-11}.$$

These numbers are row-specific evidence for the FilterFlow/BayesFilter comparison; the general rule is the topology rule above.

The audit implication is strict. A DPF implementation may pass value replay, seed replay, particle replay, and proposal-log-probability value replay while still failing the derivative contract. Every adapted-proposal implementation that is used for gradient comparison must therefore check the proposal log-probability VJP as a density evaluation at the realized particle, not only the scalar log-probability value.

31.3 BayesFilter/FilterFlow deterministic tie-out contract

The production BayesFilter/FilterFlow comparison is a same-object implementation tie-out, not an oracle test. The only statement it supports is conditional:

Under the frozen V2 deterministic comparison contract, BayesFilter and the executable local FilterFlow-side adapters match on the declared density, filter-path, fixed-ancestor, and fixed-branch AD-gradient surfaces.

The contract freezes the objects that would otherwise make a particle filter comparison ambiguous:

- (i) the model row and physical parameter vector;
- (ii) density probes for initial, transition, and observation log densities;
- (iii) initial particles, observations, and additive transition innovations;
- (iv) the scalar, namely the sum of predictive log normalizers for the path comparisons;
- (v) branch schedules: no resampling for one path test, and fixed ancestor indices for the branch-replay path test;
- (vi) the gradient scalar and physical knobs for the fixed-branch AD-gradient comparison.

The comparison deliberately avoids random-number-generator equality. When an ancestor schedule is needed, it is treated as data. Both sides receive the same schedule and replay the same branch.

The V2 production suite contains six rows:

```
lgssm_2d_h25_rich,  sv_1d_h18_rich,  range_bearing_4d_h20_rich,
structural_ar1_quadratic_h16,  spatial_sir_j3_rk4,  predator_preym_rk4.
```

The closed V2 run on 2026-06-07 reported:

- density tie-out: all six rows matched, with maximum absolute delta 0;
- deterministic no-resampling path tie-out: all six rows matched, with maximum absolute delta 0;

- deterministic fixed-ancestor path tie-out: all six rows matched, with maximum absolute delta 0, and the expected one resampling event was replayed on each executed row;
- fixed-branch AD-gradient tie-out: all five executable gradient rows matched on scalar and AD-gradient values, with maximum scalar delta 0 and maximum AD-gradient delta 0.

The spatial SIR row was not promoted to the gradient comparison. It remains a gradient-contract block under the frozen manifest, not a failure and not a success.

Finite differences are diagnostic-only in this contract. They are useful for finding gross scalar/gradient wiring mistakes, but they are not stable enough to be a gate for the DPF comparison. In the V2 gradient artifact the range-bearing same-implementation finite-difference diagnostic had maximum reported discrepancy about

$$1.16 \times 10^{-4},$$

while the BayesFilter and FilterFlow AD gradients still matched exactly under the fixed-branch scalar. This is recorded as a numerical diagnostic, not as a promotion criterion and not as a veto.

Two robustness diagnostics sit below the production V2 gate:

- a seeded fixed-ancestor diagnostic generated three frozen ancestor schedules and replayed them across all six V2 rows. All 18 executed rows matched with maximum absolute delta 0. This is branch-robustness evidence only; it is not a stochastic-resampling distribution claim;
- a controlled one-dimensional LGSSM horizon ladder compared $T = 2$ and $T = 4$. Both scenarios had implementation agreement. The $T = 4$ scenario retained a forward residual veto in the older Sinkhorn residual diagnostic, so the ladder is explanatory and does not change the V2 production conclusion.

The non-claims are part of the contract. The BayesFilter/FilterFlow tie-out does not prove filter correctness, does not prove that either implementation is mathematically correct, does not validate stochastic resampling distributions, does not validate gradients through random or discrete ancestor selection, and does not compare to student implementations, TT/SIRT, dense quadrature, paper tables, GPU behavior, HMC behavior, DSGE behavior, scalability, deployment, or production readiness.

31.4 Equation-to-test matrix

Table 31.1: Equation-indexed verification matrix for the DPF block.

Layer	Equation or claim	Controlled example	Implementation quantity	Pass signal	Narrow failure interpretation
Filtering recursion	(20.3), (20.4)	linear-Gaussian state-space model with Kalman reference	predict/update law, normalized weights, filtered mean/covariance	moments match reference within Monte Carlo error	recursion, normalization, or model-interface bug; not an OT or learned-map claim

Table 31.1: Equation-indexed verification matrix for the DPF block, continued.

Layer	Equation or claim	Controlled example	Implementation quantity	Pass signal	Narrow failure interpretation
Bootstrap likelihood estimator	(20.16), (20.17)	same linear-Gaussian model; fixed seeds and increasing N	per-period average weights and log likelihood estimator	Monte Carlo mean approaches Kalman likelihood; variance decreases with N	particle proposal or weight bug; not evidence about differentiability
Homotopy derivative	(21.1), (21.3)	scalar nonlinear observation with analytic log-likelihood derivative	finite difference in λ versus analytic derivative	central difference and analytic derivative agree under step refinement	homotopy algebra or likelihood derivative issue; not yet a PF-PF correction issue
EDH endpoint	(21.11), (21.15)	linear-Gaussian recovery with known posterior mean/covariance	flow endpoint particles, mean, covariance, and endpoint residual	endpoint moments match Kalman update within flow and Monte Carlo tolerances	flow ODE or Gaussian-closure implementation issue; not HMC target validity
LEDH local endpoint	(21.17), (21.24)	synthetic observation with known local Jacobian variation	particle-local Jacobians, local precision, and endpoint continuity	local quantities are finite and stable under small perturbations	local linearization fragility; not proof that global EDH is wrong
PF-PF weight	(22.2), (22.8), (22.3)	synthetic affine flow with known determinant	proposal density, target density, corrected log weight	manual density ratio and implemented weight agree	proposal-density or correction algebra bug; not a resampling diagnosis
Log-determinant ODE	(22.22), (22.24), (22.23)	affine flow $x_\lambda = A_\lambda x + b_\lambda$	integrated log determinant and finite-difference determinant	sign and magnitude match under step-size refinement	Jacobian sign, time direction, or integrator issue; not a likelihood theorem
Categorical discontinuity	(23.2)	two-particle resampling with weight crossing	ancestor index path and realized resampled cloud	jump occurs as expected; no false pathwise gradient is reported	discrete resampling is being differentiated incorrectly; not proof soft resampling is valid
Soft-resampling bias	(23.5), (23.7)	two-particle nonlinear test function	mean preservation and nonlinear test-function error	affine tests behave as derived; nonlinear bias is visible when expected	surrogate-law effect; not evidence of categorical-target preservation
EOT/Sinkhorn marginals	(25.3), (25.5), (25.4)	small cost matrix with prescribed marginals	row residual, column residual, cost, entropy, ϵ , iteration count	residuals fall below tolerance and are stable under log-domain implementation	finite-solver or stabilization issue; not proof of posterior invariance
Barycentric projection	(25.6)	small coupling with known column masses	projected equal-weight support cloud and dimensions	output has declared shape and column-mass normalization	projection or shape bug; not a coupling-optimality result
Learned-map residual	(27.2), (27.4), (27.5)	permutation-equivariance test and held-out teacher clouds	teacher/student residuals, permutation residual, training-distribution tags	residuals stay below threshold on the declared envelope	student or distribution-shift issue; not a teacher-map refutation
HMC value-gradient contract	(30.1)	finite-difference check on a fixed scalar target and fixed random seed policy	scalar value, autodiff gradient, finite-difference gradient, accept/reject value	all paths refer to the same scalar and gradients agree within tolerance	target-interface bug; do not tune HMC before resolving it

31.5 Minimal controlled examples

Table 31.2: Minimal controlled examples for DPF verification.

Example	Required inputs	Expected output	Does not prove
Linear-Gaussian recovery	Known transition, observation, covariance matrices, Kalman likelihood, and Kalman filtering moments.	Particle recursion, EDH endpoint, and likelihood estimates approach the Kalman reference under increasing particles or decreasing flow step size.	Nonlinear DSGE validity or learned-map generalization.
Synthetic affine flow	Invertible affine map, analytic determinant, and known proposal density.	PF-PF density ratio, weight, and log determinant match hand calculation.	Correctness for nonlinear flow fields.
Scalar nonlinear observation	One-dimensional observation function with analytic derivatives and controlled curvature.	Homotopy derivative and local-linearization diagnostics show expected error as curvature changes.	High-dimensional stability.
Two-particle soft-resampling bias	Two particles, weights crossing 1/2, and nonlinear test function.	Categorical path is discontinuous; soft path has the predicted surrogate bias.	Acceptability of soft resampling as an HMC target.
Small Sinkhorn problem	Positive cost matrix, weights, equal target marginal, fixed ε , and known residual tolerance.	Row/column residuals and stabilized objective behave consistently with the solver budget.	Posterior equivalence to categorical resampling.
Permutation-equivariance learned-map test	Teacher clouds, learned map, and permutation matrices.	Permuting input rows permutes output rows, and scalar summaries remain invariant when required.	Small teacher/student error or target preservation.
Finite-difference HMC gradient test	Fixed scalar target, fixed random seed policy, finite-difference step ladder, and autodiff gradient.	Autodiff and finite-difference gradients agree for the scalar used by accept/reject.	Convergence of the Markov chain or correctness for another scalar target.

31.6 Diagnostic specification

Every diagnostic result should be recorded with the following fields:

- **Equation or claim:** label from the chapter, not only a method name.

- **Inputs:** model, particles, weights, random seed policy, ε , iteration budget, architecture, and scalar target as applicable.
- **Expected output:** scalar, vector, matrix, empirical measure, residual, or posterior summary.
- **Tolerance:** absolute and relative thresholds, Monte Carlo standard-error logic, or convergence slope.
- **Failure interpretation:** the narrow mathematical layer that the failure implicates.
- **Non-implication:** what the failure does not prove.

Table 31.3: Diagnostic contract by layer.

Diagnostic	Inputs	Tolerance rule	Failure interpretation	Non-implication
Filtering recursion parity	Kalman reference, fixed particles, fixed model.	Monte Carlo error decreases with N ; deterministic quantities match reference when applicable.	State-space interface, normalization, or weight update bug.	Does not reject OT, learned OT, or HMC.
Flow endpoint parity	Linear-Gaussian recovery, flow step ladder.	Endpoint error decreases as flow integration is refined.	Flow ODE, closure, or integration issue.	Does not prove PF-PF correction is wrong.
PF-PF algebra parity	Affine flow, analytic determinant, explicit proposal density.	Log-weight difference below deterministic tolerance.	Density-ratio, Jacobian, or sign error.	Does not diagnose resampling.
Soft-resampling bias check	Two-particle non-linear test.	Bias sign and magnitude match derived expression within arithmetic tolerance.	Surrogate target is being confused with categorical target.	Does not prove soft resampling is unusable.
Sinkhorn residual check	Small cost matrix, fixed ε , solver budget.	Row/column residuals below declared threshold and improve with budget.	Finite-solver or stabilization issue.	Does not prove EOT is a valid posterior target.
Learned residual check	Teacher outputs, held-out clouds, distribution tags.	Residuals reported by regime; thresholds tied to training envelope.	Student approximation or extrapolation issue.	Does not invalidate the teacher.

Diagnostic	Inputs	Tolerance rule	Failure interpretation	Non-implication
HMC value-gradient check	Scalar value, autodiff path, finite-difference ladder.	Gradient errors shrink under step refinement until numerical limits.	Value-gradient mismatch or nondifferentiable scalar path.	Does not prove chain convergence.

31.7 Source and implementation map

Table 31.4: Source, chapter, equation, and implementation links.

Layer	Chapter location	Equation labels	Source family	Implementation quantity
Classical PF	Chapter 20, Sections 20.3–20.7	(20.4), (20.17)	SMC literature spine	weights, ESS, likelihood estimator, empirical moments
Particle flow	Chapter 21, Sections 21.3–21.7	(21.3), (21.14), (21.24)	Daum–Huang, PF-PF flow literature	flow field, endpoint residual, local Jacobians
PF-PF correction	Chapter 22, Sections 22.3–22.7	(22.8), (22.23)	proposal correction and change-of-variables spine	proposal density, target density, log determinant, corrected weight
Resampling and OT	Chapter 23; Chapter 24; Chapter 25	(23.2), (23.7), (24.12), (25.3)	differentiable resampling, OT, Sinkhorn sources	ancestor path, soft surrogate, coupling residuals, barycentric cloud
Learned OT	Chapter 27	(27.2), (27.5), (27.4)	EOT teacher and set-architecture sources	teacher output, student output, residuals, distribution tags
HMC target	Chapter 30, Sections 30.1–30.9	(30.1)	HMC, pseudo-marginal, surrogate-HMC spine	scalar value, gradient, accept/reject value, target-status label

31.8 Scalable OT reuse-risk and validation doctrine

The scalable-OT survey sharpens the verification contract for the OT block in three ways. First, code availability is useful but does not by itself justify promotion. Second, exact-online, retained-teacher approximate, and structure-changing transport families should not share the same pass/fail story. Third, the validation ladder must test transported particle objects, not only OT scalars or paper-reported benchmark values.

A practical BayesFilter review should therefore separate three lanes:

- (i) **Exact reference lane:** dense or streamed entropic OT/Sinkhorn implementations that preserve the same declared teacher object.
- (ii) **Retained-teacher approximation lane:** Nystrom, positive-feature, low-rank, or similar accelerators judged first by teacher-preservation residuals.
- (iii) **Structure-changing lane:** sliced, localized, block, or other replacement transport rules judged as explicit new resampling approximations.

The first question for any candidate is therefore not “is it faster?” but “which of these lanes does it belong to?” The rest of the diagnostic stack depends on that classification.

31.8.1 Code availability and reuse risk are prefilters, not verdicts

Source availability is a practical filter because BayesFilter needs an exposed transport object, not only a scalar loss. But a repository with a readable commit, a visible API, or even a working prototype does not become default-ready for BayesFilter on that basis alone. Reuse risk should therefore be recorded as a narrow implementation note:

- (1) whether the code exposes a usable transport object or only a scalar loss,
- (2) whether the backend is compatible with the BayesFilter TensorFlow/TFP default,
- (3) whether the source is complete and inspectable,
- (4) whether the code path matches the mathematical object cited in the literature,
- (5) whether any missing piece would force BayesFilter to invent new semantics locally rather than reproduce the cited object.

A failure here does not refute the mathematics. It only limits reuse value and raises implementation-surface risk.

31.8.2 Validation ladder before promotion

Before any scalable OT candidate is promoted beyond a literature note or local prototype, the review should follow a fixed ladder. The central rule is that transported-particle parity comes before downstream performance claims.

- (1) deterministic dense parity on small fixtures: compare transported particles or transported summaries, not only OT objective values;
- (2) marginal checks: source and target mass constraints under weighted inputs;
- (3) positivity, non-finite, and numerical-stability checks;
- (4) representation diagnostics appropriate to the lane: rank, feature dimension, sparsity, projection count, block locality, or minibatch design;
- (5) LEDH-PFPPF-OT value parity on existing BayesFilter fixtures;
- (6) gradient smoke tests only after value-side parity is understood;
- (7) runtime and memory ladder over particle count and state dimension;
- (8) downstream filtering diagnostics;
- (9) multi-seed or uncertainty-aware comparisons before ranking stochastic candidates;
- (10) default-readiness review only after downstream evidence, not from source availability or paper benchmarks alone.

This ladder is deliberately asymmetric. Passing an early rung does not justify a later claim, but failing an early rung can already block promotion.

31.8.3 Lane-specific narrow interpretations

The failure interpretation should remain lane-specific.

- In the exact reference lane, a failed marginal or barycentric parity check is evidence about the finite solver or implementation path.
- In the retained-teacher approximation lane, a large teacher-preservation residual is evidence about the approximation family or its tested envelope, not a refutation of the teacher object itself.
- In the structure-changing lane, disagreement with the dense teacher is not automatically failure; the question is whether the replacement criterion and downstream validation have been declared and met.

This prevents a common review error: treating every non-dense transport as if it were trying to prove exact parity with the same teacher object.

31.9 Promotion thresholds

Table 31.5: Promotion thresholds for DPF implementation claims.

Claim level	Required passing evidence	Disallowed interpretation	Gate
Exploratory implementation	Layer-local smoke tests pass on minimal controlled examples; failures are recorded with equation labels.	Treating smoothness or runtime as target correctness.	May run more experiments.
Credible re-search implementation	Classical PF, flow, PF-PF, resampling/OT, learned-map, and HMC value-gradient checks pass on named model classes with sensitivity reports.	Generalizing beyond tested N , d_x , ε , model class, or target status.	May make bounded research claims.
Bank-facing re-search claim	Research checks plus posterior comparisons, stress regimes, reproducible artifacts, and documented failure policies.	Claiming production suitability from a chapter or benchmark alone.	Requires model-risk review.
Production or governance claim	Independent review, change control, monitoring, rollback, data/model governance, release stability, and ongoing diagnostics.	Assuming academic derivation or local tests are sufficient approval.	Requires separate governance process.

31.10 Boundary of this contract

This chapter does not validate any DPF implementation. It defines the minimum diagnostic evidence needed to interpret failures and to avoid promoting a method beyond its evidence. The word “validate” is therefore reserved for a specific equation-local check or for a separately governed model-risk process; it is not used here as a blanket claim about DPF, learned OT, HMC, or banking deployment.

Chapter 32

Filter Choice

Filter choice is a target-definition decision, not only a speed decision. For HMC, the selected backend determines the log likelihood, the derivative path, the finite-failure policy, and the compilation surface. BayesFilter should choose the simplest backend that gives the required target fidelity and a validated gradient.

32.1 Decision Register

The backend decision should be recorded in a short register:

Exact Kalman, solve, or square-root LGSSM Best for linear Gaussian likelihoods and regression oracles. The main risks are SPD failures, mask handling, and large-scale factor failures. The HMC gate is analytic/custom score parity plus compiled value-gradient parity.

EKF Best as a cheap nonlinear baseline and affine recovery check. The main risks are linearization bias and Jacobian fragility. The HMC gate is affine Kalman recovery plus finite-gradient stress tests.

Sigma-point Best for moderate nonlinear Gaussian approximations. The main risks are an approximate target and covariance-update instability. The HMC gate is reference recovery, static sigma-point shapes, and derivative parity.

Square-root sigma-point Best for value-side robustness under near-singular covariance. The main risk is false confidence when the implementation reconstructs covariance matrices internally. The HMC gate is factor reconstruction, finite gradients, and compiled parity.

SVD sigma-point Best for robust approximate value paths and spectral diagnostics. The main risks are close or repeated spectral values and raw autodiff instability. The HMC gate is eigen-gap telemetry, stress gates, and a custom-gradient fallback.

Particle or differentiable particle filter Best for non-Gaussian diagnostics and frontier approximations. The main risks are Monte Carlo variance, resampling non-differentiability, and relaxed target bias. The HMC gate is an estimator-variance report and a source-backed correction policy.

32.2 Recommended Order

For a new model, BayesFilter should proceed in this order:

- (i) reduce to an LGSSM or affine test case and validate the exact Kalman likelihood and derivatives;
- (ii) add masks, missing-data patterns, and mixed-frequency timing while retaining exact references;
- (iii) test EKF or sigma-point approximations on low-dimensional nonlinear examples with dense or trusted references;
- (iv) only then attempt DSGE-scale SVD sigma-point HMC;
- (v) treat particle methods and transport methods as separate evidence streams unless a correction policy makes them target-preserving.

32.3 Industrial Default

For NAWM-sized or industrial applications, the default should be an exact or controlled approximate likelihood with an analytic or custom derivative path. Raw tape gradients through spectral decompositions are acceptable as diagnostic tools and small-case oracles, but they should not be the final production plan unless spectral-gap, finite-gradient, and compiled parity evidence survives the intended stress regime.

This policy is intentionally conservative. The SVD sigma-point debugging work showed that value robustness and HMC-gradient robustness are different problems. The next implementation phases should close that gap deliberately instead of discovering it during long chains.

Part V

**High-Dimensional Nonlinear
Filtering**

Chapter 33

High-Dimensional Nonlinear Filtering Foundations

33.1 The Filtering Problem In Its Least Forgiving Form

A high-dimensional nonlinear filter is not merely a large Kalman filter. It is an approximation to a sequence of conditional probability laws whose update can become singular, multimodal, sharply curved, or numerically inaccessible. This chapter fixes the mathematical objects that the later chapters approximate. It also separates exact identities from approximation policies. That separation matters because a filter used inside optimization or Hamiltonian Monte Carlo does not only produce state estimates; it defines a scalar likelihood target and possibly a gradient of that target.

Let $\theta \in \Theta$ be static parameters, $x_t \in \mathbb{R}^n$ a latent state, and $y_t \in \mathbb{R}^m$ an observation. In discrete time a state-space model is

$$x_0 \sim \mu_\theta, \quad x_t \mid x_{t-1}, \theta \sim f_\theta(\cdot \mid x_{t-1}), \quad y_t \mid x_t, \theta \sim g_\theta(\cdot \mid x_t). \quad (33.1)$$

The continuous-time diffusion analogue, used by the Zakai/DMZ and tensor-train PDE papers, is commonly written

$$dX_t = a_\theta(X_t, t) dt + B_\theta(X_t, t) dV_t, \quad (33.2)$$

$$dY_t = h_\theta(X_t, t) dt + dW_t, \quad (33.3)$$

with variants for correlated noise. The discrete recursion is the natural contract for production filtering APIs; the continuous formulation exposes the PDE and pathwise robust methods in [Davis \[1980\]](#), [Yau and Yau \[2000\]](#), [Yau and Yau \[2008\]](#), [Li et al. \[2019\]](#), and [Meng et al. \[2025\]](#).

For a reader coming from numerical analysis, physics, or computational chemistry, the central object is the conditional law $p_\theta(x_t \mid y_{1:t})$. The input object at time t is the previous conditional law and a new observation; the output object is a new conditional law plus the normalizer Z_t . The exact target is the Bayes recursion below. Every later method is an approximation to one of three objects: the law $p_\theta(x_t \mid y_{1:t})$, the likelihood scalar $\ell_T(\theta)$, or a derivative of that scalar. A diagnostic is useful only when it says which of these objects it is checking.

Running quadratic-observation cell. The same teaching example will be used through Chapters 33–40:

$$x_t = \rho x_{t-1} + \eta_t, \quad \eta_t \sim \mathcal{N}(0, Q), \quad y_t = x_t^2 + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, R). \quad (33.4)$$

It is deliberately small. Its purpose is to make the objects visible, not to validate a macro-finance model. With a one-step Gaussian prediction $X \sim \mathcal{N}(0, P^-)$, the filtering density after observing $y > 0$ is proportional to

$$\exp\left\{-\frac{x^2}{2P^-} - \frac{(y - x^2)^2}{2R}\right\}. \quad (33.5)$$

When R is small and y is away from zero, the likelihood rewards states near both $+\sqrt{y}$ and $-\sqrt{y}$. The exact filtering object is therefore a conditional law that may have two lobes. A mean alone, a covariance alone, or a finite scalar likelihood alone is only one view of that object.

Object	Role in the exact recursion	Why the running cell exposes it
Conditional law p_t	Probability law of x_t given observations	x^2 observations can create two separated lobes.
Normalizer Z_t	One-step predictive likelihood contribution	It integrates both lobes and is the scalar used in inference.
Likelihood ℓ_T	Sum of log normalizers over time	Changing Z_t changes the parameter posterior.
Sensitivity s_t	Derivative of the law or normalizer with respect to parameters	HMC needs the gradient of the same scalar it reports.

Assumption 33.1 (Dominated filtering model). *The kernels in (33.1) admit densities with respect to reference measures, the predictive normalizing constants below are finite and positive on the observed path, and every differentiation under an integral sign is justified by a local dominating function. Continuous-time PDE statements additionally use the regularity, ellipticity, growth, and boundary assumptions stated in the cited source.*

33.2 Exact Discrete-Time Recursion And Likelihood

Under Assumption 33.1, prediction and update are

$$p_\theta(x_t \mid y_{1:t-1}) = \int f_\theta(x_t \mid x_{t-1}) p_\theta(x_{t-1} \mid y_{1:t-1}) dx_{t-1}, \quad (33.6)$$

$$p_\theta(x_t \mid y_{1:t}) = \frac{g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})}{Z_t(\theta)}, \quad Z_t(\theta) = \int g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1}) dx_t. \quad (33.7)$$

The likelihood factors as

$$p_\theta(y_{1:T}) = \prod_{t=1}^T Z_t(\theta), \quad \ell_T(\theta) = \sum_{t=1}^T \log Z_t(\theta). \quad (33.8)$$

The derivation is short, but it is the contract that every later approximation must respect. Write the joint density through time t as

$$p_\theta(x_{0:t}, y_{1:t}) = \mu_\theta(x_0) \prod_{s=1}^t f_\theta(x_s | x_{s-1}) \prod_{s=1}^t g_\theta(y_s | x_s).$$

Integrating out $x_{0:t-1}$ after conditioning on $y_{1:t-1}$ gives (33.6). Multiplying the predictive density by the likelihood $g_\theta(y_t | x_t)$ gives an unnormalized density

$$\gamma_t(x_t) = g_\theta(y_t | x_t) p_\theta(x_t | y_{1:t-1}).$$

Its integral is exactly the one-step predictive density of the observation,

$$Z_t(\theta) = p_\theta(y_t | y_{1:t-1}).$$

Bayes' formula then gives (33.7). Finally, the chain rule for the observation density gives

$$p_\theta(y_{1:T}) = p_\theta(y_1) \prod_{t=2}^T p_\theta(y_t | y_{1:t-1}) = \prod_{t=1}^T Z_t(\theta).$$

Thus a filtering implementation that changes the normalizers changes the likelihood, not merely the state estimate.

Reader checkpoint. What should now be clear: filtering updates a probability law and a scalar normalizer at the same time. Object preserved: the conditional law after normalization. Approximation enters at: the representation of the predictive law, the integral for Z_t , or both. Exported to synthesis: any industrial method must say whether it preserves a law, a likelihood scalar, or only a moment summary.

Proposition 33.1 (Predictive score identity). *Let $Z_t(\theta) = \int a_t(x, \theta) dx$ with $a_t(x, \theta) = g_\theta(y_t | x) p_\theta(x | y_{1:t-1})$. If $a_t > 0$ on the posterior support and dominated differentiation holds, then*

$$\nabla_\theta \log Z_t(\theta) = \int \nabla_\theta \log a_t(x, \theta) p_\theta(x | y_{1:t}) dx. \quad (33.9)$$

Proof. Under dominated differentiation,

$$\nabla_\theta \log Z_t(\theta) = \frac{\nabla_\theta Z_t(\theta)}{Z_t(\theta)} = \frac{\int \nabla_\theta a_t(x, \theta) dx}{\int a_t(x, \theta) dx}.$$

On the support where $a_t > 0$, $\nabla_\theta a_t = a_t \nabla_\theta \log a_t$. Substitution gives

$$\nabla_\theta \log Z_t(\theta) = \int \nabla_\theta \log a_t(x, \theta) \frac{a_t(x, \theta)}{Z_t(\theta)} dx.$$

The last factor is the filtering density in (33.7). Hence (33.9) follows. The identity does not say that a numerical score is correct; it says that any reported score must correspond to the derivative of the same normalizer used in the reported likelihood. $\square$

This proposition is exact but not automatically implementable. A numerical filter supplies an approximate normalizer $\hat{Z}_t$ and an approximate history-dependent distribution. If value and gradient are not derivatives of the same scalar approximation, a downstream optimizer or HMC sampler is solving a different problem from the one reported by the filter.

33.3 Analytical Likelihood-Gradient And Filtering Sensitivities

This section is a project derivation for the dominated discrete-time model in (33.1). It is not a continuous-time adjoint theorem. The PDE and robust-DMZ settings require their own sensitivity or adjoint analysis under the regularity assumptions of the cited sources.

Write

$$p_t^-(x_t) = p_\theta(x_t \mid y_{1:t-1}), \quad p_t(x_t) = p_\theta(x_t \mid y_{1:t}),$$

and let the parameter-gradient sensitivities be

$$s_t^-(x_t) = \nabla_\theta p_t^-(x_t), \quad s_t(x_t) = \nabla_\theta p_t(x_t).$$

These sensitivities are vector-valued when θ is vector-valued; the derivation is component-wise. Differentiating the prediction equation (33.6) under Assumption 33.1 gives

$$s_t^-(x_t) = \int [\nabla_\theta f_\theta(x_t \mid x_{t-1}) p_{t-1}(x_{t-1}) + f_\theta(x_t \mid x_{t-1}) s_{t-1}(x_{t-1})] dx_{t-1}. \quad (33.10)$$

The first term is the direct parameter effect on the transition density. The second term is the inherited sensitivity of the previous filtering law. If the initial distribution depends on θ , the recursion starts from $s_0(x_0) = \nabla_\theta p_\theta(x_0)$, or from the corresponding post-initial-observation sensitivity if an initial update is used.

The one-step normalizer is

$$Z_t(\theta) = \int g_\theta(y_t \mid x_t) p_t^-(x_t) dx_t. \quad (33.11)$$

Differentiating it gives

$$\nabla_\theta Z_t(\theta) = \int [\nabla_\theta g_\theta(y_t \mid x_t) p_t^-(x_t) + g_\theta(y_t \mid x_t) s_t^-(x_t)] dx_t. \quad (33.12)$$

Therefore

$$\nabla_\theta \ell_T(\theta) = \sum_{t=1}^T \nabla_\theta \log Z_t(\theta) = \sum_{t=1}^T \frac{\nabla_\theta Z_t(\theta)}{Z_t(\theta)}. \quad (33.13)$$

It remains to differentiate the normalized update. Define

$$n_t(x_t, \theta) = g_\theta(y_t \mid x_t) p_t^-(x_t), \quad p_t(x_t) = \frac{n_t(x_t, \theta)}{Z_t(\theta)}.$$

The numerator sensitivity is

$$\nabla_\theta n_t(x_t, \theta) = \nabla_\theta g_\theta(y_t \mid x_t) p_t^-(x_t) + g_\theta(y_t \mid x_t) s_t^-(x_t).$$

Using the quotient rule and $\nabla_\theta Z_t/Z_t = \nabla_\theta \log Z_t$,

$$s_t(x_t) = \frac{\nabla_\theta g_\theta(y_t \mid x_t) p_t^-(x_t) + g_\theta(y_t \mid x_t) s_t^-(x_t)}{Z_t(\theta)} - p_t(x_t) \nabla_\theta \log Z_t(\theta). \quad (33.14)$$

Integrating (33.14) over x_t gives zero, because a normalized density has derivative of total mass equal to zero. This is the sensitivity analogue of probability conservation.

Proposition 33.2 (Discrete-time likelihood sensitivity recursion). *Under Assumption 33.1, with differentiable transition and observation densities and finite positive $Z_t(\theta)$, equations (33.10)–(33.14) give an analytical recursion for $\nabla_\theta \ell_T(\theta)$ in the exact discrete-time filter.*

Proof. Equation (33.10) is differentiation of the prediction integral. Equation (33.12) is differentiation of the predictive normalizer. Equation (33.13) follows from the likelihood factorization (33.8) and the identity $\nabla \log Z = \nabla Z / Z$ for $Z > 0$. Equation (33.14) is the quotient rule applied to $p_t = n_t / Z_t$. Each step is componentwise in θ and uses dominated differentiation to exchange gradient and integral. $\square$

This recursion is the analytical object needed by Chapter 39. An automatic-differentiation tape through an approximate filter is a different object unless it differentiates exactly the same scalar displayed to the sampler.

Reader checkpoint. What should now be clear: the likelihood gradient is not a separate numerical decoration. It is the derivative of the same normalizers that define ℓ_T . Object preserved: scalar value-gradient consistency. Approximation enters at: any approximate p_t^- , $\hat{Z}_t$, truncation, rounding, or adaptive gate. Exported to synthesis: HMC can start only after the scalar and gradient contract is named.

33.4 Zakai, Kushner–Stratonovich, And DMZ Equations

For diffusion models, the normalized conditional distribution satisfies a nonlinear filtering equation, while the unnormalized conditional measure satisfies a linear Zakai equation. We use van Handel [2007] as the standard checked derivation anchor for the normalized and unnormalized equations rather than relying on unchecked historical priority papers. In an informal density notation, the unnormalized density σ_t obeys a linear stochastic equation of the form

$$d\sigma_t = \mathcal{L}_\theta^* \sigma_t dt + h_\theta^\top \sigma_t dY_t, \quad (33.15)$$

where $\mathcal{L}_\theta^*$ is the Fokker–Planck adjoint associated with the signal generator. The normalized density is

$$\rho_t(x) = \frac{\sigma_t(x)}{\int \sigma_t(u) du}. \quad (33.16)$$

The Duncan–Mortensen–Zakai viewpoint emphasizes the density/PDE form of (33.15). It is attractive in high dimension because it turns filtering into density propagation; it is frightening for exactly the same reason, since a tensor-product grid has exponentially many degrees of freedom.

Remark 33.1 (Bibliographic scope). *For the equations used below, this chapter relies on checked technical sources: van Handel for the standard normalized and unnormalized filtering equations, and Davis/Yau papers for the pathwise robust DMZ transformation and memoryless algorithm. Classical priority names are retained for orientation, but a historical citation is not used as a substitute for an inspected theorem or derivation.*

33.5 Pathwise Robust DMZ And Memoryless Computation

[Davis \[1980\]](#) studies a multiplicative functional transformation for nonlinear filtering. In the simple scalar-observation diffusion setting, the idea is to absorb observation-path dependence into a path-dependent transformation of the unnormalized density. The transformed object evolves according to a generator/PDE that can be studied pathwise rather than only as a stochastic differential equation.

[Yau and Yau \[2000\]](#) and [Yau and Yau \[2008\]](#) build a robust DMZ algorithm around this principle. Their central computational move is to freeze observation increments on a time partition and repeatedly solve observation-independent Kolmogorov-type equations, correcting by observation-dependent multiplicative factors. In schematic form, over $[t_j, t_{j+1}]$ one computes

$$\partial_s u_j(s, x) = \mathcal{K}_{y_{t_j}} u_j(s, x), \quad s \in [t_j, t_{j+1}], \quad (33.17)$$

$$u_{j+1}(t_{j+1}, x) = \mathcal{M}(y_{t_{j+1}} - y_{t_j}, x) u_j(t_{j+1}, x), \quad (33.18)$$

where $\mathcal{K}_{y_{t_j}}$ and $\mathcal{M}$ denote the paper-specific frozen operator and multiplicative observation update. The details are not optional: the convergence and bounded-domain approximation theorems in [Yau and Yau \[2000, 2008\]](#) depend on their growth, regularity, weak-solution, and boundary assumptions.

Algorithm 9 Robust DMZ offline/online template

Require: Partition $0 = t_0 < \dots < t_K = T$, initial density, checked PR-DMZ assumptions, spatial domain or truncation rule.

- 1: Build or choose the observation-independent Kolmogorov solver required by the cited theorem.
 - 2: **for** $j = 0, \dots, K - 1$ **do**
 - 3: Freeze the observation path according to the cited rule.
 - 4: Solve the frozen Kolmogorov equation on $[t_j, t_{j+1}]$.
 - 5: Apply the multiplicative observation correction.
 - 6: Normalize only after preserving the unnormalized-density semantics.
 - 7: Record mass, positivity, boundary, and discretization residuals.
 - 8: **end for**
-

[Meng et al. \[2025\]](#) is a recent arXiv contribution rather than a settled classical reference. It is therefore used here provisionally: it links pathwise robust DMZ regularity estimates to sparse functional tensor-train approximations and a finite-difference QTT algorithm. The durable lesson for the rest of the monograph is methodological rather than evidentiary: a tensor filter must make regularity, rank/compressibility, discretization error, rounding error, and online-assimilation stability visible before small storage can be interpreted as a filtering result.

33.6 Approximate Targets And The HMC Boundary

For a prior $\pi(\theta)$, the exact parameter posterior is proportional to

$$\pi(\theta \mid y_{1:T}) \propto \exp\{\ell_T(\theta)\} \pi(\theta). \quad (33.19)$$

If a filter supplies $\widehat{\ell}_T(\theta)$, the posterior implied by the calculation is

$$\widehat{\pi}(\theta \mid y_{1:T}) \propto \exp\{\widehat{\ell}_T(\theta)\}\pi(\theta), \quad (33.20)$$

unless a pseudo-marginal or other exact correction is part of the algorithm. The corresponding gradient is

$$\nabla_{\theta} \widehat{\ell}_T(\theta) = \nabla_{\theta} \sum_{t=1}^T \log \widehat{Z}_t(\theta),$$

provided the reported gradient is the derivative of the reported scalar. Truncation, re-sampling, clipping, tensor rounding, adaptive stopping, and discrete accept/reject gates must therefore state which scalar is being differentiated. The HMC chapter returns to this point: exact Hamiltonian dynamics for (33.20) is not exact sampling from (33.19).

33.7 Why Dimension Breaks Filters

High dimension is a family of failure mechanisms.

Particle collapse.

The bootstrap and SIR filters of [Gordon et al. \[1993\]](#) and [Arulampalam et al. \[2002\]](#) represent the filtering law by particles and weights. [Bengtsson et al. \[2008\]](#) and [Snyder et al. \[2008\]](#) show that, under their assumptions, weight concentration can be governed by the variance of the log likelihood and can require enormous ensembles. These results do not say every structured particle method fails; they say proposal, localization, and correction must be central.

Grid explosion.

A density grid with M points per coordinate has M^n entries. Sparse grids, tensor trains, and transport maps are attempts to exploit smoothness, low-order interactions, low rank, or triangular structure. If the posterior lacks that structure, the format fails.

Gaussian projection bias.

EKF/UKF/CKF-style filters propagate only selected moments. Multimodality, skewness, sharp likelihoods, and nonlocal curvature can be invisible to the chosen moment rule.

Sampler geometry.

HMC and NUTS can fail through divergences, poor energy exploration, invalid regions, or a value/gradient mismatch even when the filter returns finite numbers.

33.8 Defects Passed To Synthesis

This chapter passes five defects to the synthesis chapter.

- (1) The exact object is a sequence of probability laws and normalizers. A later method that returns a mean but not a defensible Z_t has not supplied a likelihood target for parameter inference.

- (2) The approximate posterior in (33.20) is a different target unless an exact correction is part of the method. This is the first same-scalar obligation inherited by HMC.
- (3) Analytical score recursions such as (33.13) are available in the dominated discrete-time setting. Continuous-time/PDE score and adjoint formulas are not derived here and remain source-specific obligations.
- (4) Density/PDE formulations require mass, positivity, boundary, and regularity checks. A low residual in a transformed PDE does not by itself prove that the normalized density is a valid filtering law.
- (5) Source-local PDE assumptions matter. The Davis and Yau robust-DMZ transformations and the Meng tensor approximation results support the statements made under their hypotheses; they do not license an unrestricted high-dimensional filtering theorem for BayesFilter.

The synthesis chapter uses these defects as gates: exact target first, approximation target second, sampler or acceleration only downstream.

Exported to synthesis.

Object	Exact target	Approximation site	Failure mode	Diagnostic/veto	Cost variable	Promotion gate	Citation/derivation anchor
Conditional law	$p_\theta(x_t y_{1:t})$	density, particles, moments, tensor, or map	mean-only or invalid law	mass, positivity, reference moments	state dimension	law semantics named	(33.6)–(33.7)
Normalizer	$Z_t = p(y_t y_{1:t-1})$	quadrature, Monte Carlo, tensor/PDE solve	wrong hood	likelihood finite Z_t , parity	posi- integral scalar cost	scalar likelihood reported	(33.7)–(33.8)
Likelihood gradient	$\nabla_\theta \ell_T$	sensitivity approximation or autodiff path	gradient of different scalar	a finite-difference parity, same-scalar check	filter-gradient cost	value and gradient match	Proposition 33.2
DMZ/PDE density	unnormalized filtering density	spatial discretization, operator compression	mass, boundary, or regularity drift	residual, mass, positivity, boundary checks	grid/rank	source assumptions checked	Davis [1980], Yau and Yau [2000, 2008]

33.9 Methodological Boundary

The chapters that follow are literature-grounded design chapters, not production promotion notes. A numerical smoke test can show that a command ran, values were finite, shapes matched, or a stop rule triggered. It cannot establish posterior accuracy, NAWM readiness, tensor validation, HMC convergence, GPU/XLA readiness, or a default production policy.

33.10 Bibliographic Limits

Several historical or specialized sources were not available for direct technical inspection in this pass: Savostyanov-specific maxvol/quasioptimality, Stroud’s book, Smolyak’s original sparse-grid construction, Genz-specific cubature foundations, and Knothe’s original rearrangement source. When later chapters use successor or alternative sources, their claims are restricted to what those inspected sources actually state.

Chapter 34

Deterministic Gaussian Projection and Point-Rule Foundations

34.1 Introduction And Scope

High-dimensional nonlinear filtering forces a choice of computational object. The exact Bayesian filter propagates a full conditional law through time, but in nonlinear models that law usually does not remain available in a finite closed form. In low dimension one may still tolerate heavy deterministic quadrature or a richer non-Gaussian representation. In high dimension, however, both cost and structural complexity escalate quickly. This chapter studies one narrower lane: a deterministic Gaussian-surrogate filter that carries a single Gaussian object, uses explicit moment engines to approximate the nonlinear expectations it needs, and prepares the rule-family language that the next chapter will specialize to fixed sparse-grid quadrature.

This chapter imports the exact-target and same-scalar discipline from Chapter 33. Its task is not yet to build the full sparse-grid algorithm. It first fixes the deterministic Gaussian carried object, the Gaussian projection contract, and the point-rule family comparisons needed before sparse-grid specialization. The next chapter, Chapter 35, then turns this shared Gaussian-rule foundation into the explicit fixed-cloud sparse-grid lane.

The running quadratic-observation cell from Chapter 33 reappears here as a compact shared object, but the present chapter stays at the level of Gaussian carried objects and deterministic moment engines. Low-dimensional cloud construction, value recursion, and the fixed-cloud scalar belong to the sparse-grid chapter that follows.

The central limitation should be fixed at the outset. A deterministic Gaussian lane does not claim exact nonlinear filtering, exact nonlinear likelihood evaluation, or faithful preservation of arbitrary multimodality, skewness, or high-order global interaction structure. Its purpose is narrower: to ask whether one carried Gaussian, together with a declared deterministic moment engine, can provide a coherent and scientifically useful approximate likelihood lane in the regime under study.

Background assumed. The reader is assumed to know multivariate Gaussian notation, state-space-model notation, Jacobian-style derivative notation, and the use of a Cholesky factor for a positive-definite covariance matrix. Sparse-grid merging, same-scalar branch identities, and fixed-cloud derivative obligations are defined where they first become necessary.

34.1.1 Problem Setting And Deterministic Gaussian Lanes

The scientific problem is high-dimensional nonlinear filtering. At time t , the exact Bayesian filter propagates and updates the full conditional law $p_\theta(x_t \mid y_{1:t})$. In nonlinear models that law is generally not available in a finite-dimensional closed form, and in high dimension it becomes unrealistic to treat exact function-valued propagation as the practical computational object. The deterministic Gaussian family therefore does not attempt to rescue exactness by notation. It selects one approximate lane whose strengths and boundaries can both be stated explicitly.

Within this family, the carried object is a single Gaussian surrogate $\mathcal{N}(m_t, P_t)$. Different deterministic point-rule lanes then differ not in what object is carried, but in how the nonlinear Gaussian moments needed by the projection are approximated. This chapter fixes that shared carried-object and moment-contract language before the next chapter specializes it to the fixed sparse-grid lane.

34.1.2 The Deterministic Gaussian Lane In One Page

The deterministic Gaussian family studied here is defined by two deliberate approximations and one downstream derivative constraint.

1. **Gaussian carried state.** Instead of carrying the full filtering law, the method carries one Gaussian summary $\mathcal{N}(m_t, P_t)$.
2. **Deterministic moment engine.** The nonlinear Gaussian moments that define the prediction and observation projection are approximated by a fixed point-rule family in standardized coordinates.
3. **Same-scalar derivative discipline.** Any later reported gradient must be the derivative of one declared approximate scalar built from that same deterministic moment engine.

The exact object, the carried surrogate, and the eventual approximate scalar are therefore different in kind:

$$\begin{aligned}
 \text{exact object:} & \quad p_\theta(x_t \mid y_{1:t}), \\
 \text{carried surrogate:} & \quad \mathcal{N}(m_t, P_t), \\
 \text{declared approximate scalar:} & \quad \widehat{\ell}_T^{(M)}(\theta) \text{ for a chosen deterministic moment engine } M.
 \end{aligned}
 \tag{34.1}$$

This distinction is the governing contract of the deterministic Gaussian block. The next chapter will instantiate one specific engine, fixed sparse-grid quadrature, within this larger family.

34.1.3 Approximation Hierarchy And Reading Guide

The deterministic Gaussian side of the expanded block unfolds in two chapter movements. First, the exact filtering recursion and the Gaussian projection that replaces it are fixed formally. Second, nearby deterministic point-rule families are compared at the level of their moment engines. Only after that shared foundation is explicit does the next chapter specialize to sparse-grid construction, merged clouds, the filtering value path, and the fixed-cloud scalar.

The main coordinates are:

$$\xi \in \mathbb{R}^{n_x}, \quad \xi \sim \mathcal{N}(0, I_{n_x}) \quad \text{standard quadrature coordinate,} \quad (34.2)$$

$$x = m + C\xi, \quad CC^\top = P \quad \text{physical state coordinate under a carried Gaussian,} \quad (34.3)$$

$$a = f_\theta(x), \quad \text{transition image used for prediction,} \quad (34.4)$$

$$z = h_\theta(x), \quad \text{observation image used for the innovation.} \quad (34.5)$$

A deterministic point-rule lane builds a finite rule in ξ -space and then places that rule into physical state space through the current Gaussian factor. That sentence is the bridge from the present foundations chapter to the sparse-grid chapter that follows.

34.2 Exact Filtering And Gaussian Projection

Use the additive-noise state-space model

$$X_t = f_\theta(X_{t-1}) + \eta_t, \quad \eta_t \sim \mathcal{N}(0, Q_\theta), \quad (34.6)$$

$$Y_t = h_\theta(X_t) + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, R_\theta), \quad (34.7)$$

with $X_t \in \mathbb{R}^{n_x}$, $Y_t \in \mathbb{R}^{n_y}$, and parameter $\theta \in \mathbb{R}^p$. The exact Bayesian prediction and update are

$$p_\theta(x_t \mid y_{1:t-1}) = \int p_\theta(x_t \mid x_{t-1}) p_\theta(x_{t-1} \mid y_{1:t-1}) dx_{t-1}, \quad (34.8)$$

$$p_\theta(x_t \mid y_{1:t}) = \frac{g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})}{\int g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1}) dx_t}. \quad (34.9)$$

In nonlinear models, these integrals usually do not close in finite-dimensional form. Deterministic Gaussian filters replace those exact densities by a Gaussian moment projection.

34.2.1 Object Map And Deterministic Gaussian Conventions

The deterministic Gaussian family uses the following common objects.

symbol	meaning	shape
m_t, P_t	carried Gaussian mean and covariance after update	$n_x, n_x \times n_x$
m_t^-, P_t^-	predictive Gaussian mean and covariance before update	$n_x, n_x \times n_x$
C_t, C_t^-	covariance factors with $P_t = C_t C_t^\top$, $P_t^- = C_t^- (C_t^-)^\top$	$n_x \times n_x$
$\chi_t^{(r)}$	deterministic physical rule point	n_x
$z_t^{(r)}$	observation image of a rule point	n_y
$\bar{z}_t, S_t, C_{xz,t}$	predicted observation mean, innovation covariance, cross-covariance	$n_y, n_y \times n_y, n_x \times n_y$
v_t, K_t	innovation residual and Gaussian projection gain	$n_y, n_x \times n_y$

Three implementation conventions are fixed chapter-wide.

1. **Factor convention.** Every covariance factor is the lower triangular Cholesky factor with positive diagonal, written $P = CC^\top$, whenever the declared branch is valid.
2. **Solve convention.** Formulas may use S_t^{-1} for compact mathematics, but the implementation contract is to compute such actions by linear solves against the declared factor of S_t , not by forming an explicit inverse.
3. **Noise convention.** Additive process noise enters analytically through Q_θ in predictive moments and additive observation noise through R_θ in innovation moments. This chapter does not use state augmentation for these additive noises.

Gaussian Projection From Moments

Assume the predictive state is represented by

$$X_t^- \sim \mathcal{N}(m_t^-, P_t^-). \quad (34.10)$$

Let

$$Z_t = h_\theta(X_t^-) \quad (34.11)$$

be the noiseless predicted observation. The Gaussian projection needs

$$\bar{z}_t = \mathbb{E}[Z_t], \quad (34.12)$$

$$S_t = R_\theta + \mathbb{E}[(Z_t - \bar{z}_t)(Z_t - \bar{z}_t)^\top], \quad (34.13)$$

$$C_{xz,t} = \mathbb{E}[(X_t^- - m_t^-)(Z_t - \bar{z}_t)^\top]. \quad (34.14)$$

Given these moments, define

$$v_t = y_t - \bar{z}_t, \quad K_t = C_{xz,t} S_t^{-1}, \quad (34.15)$$

$$m_t = m_t^- + K_t v_t, \quad P_t = P_t^- - K_t S_t K_t^\top. \quad (34.16)$$

The approximation has two layers. First, these moments may be approximated by a deterministic point rule. Second, the resulting posterior information is compressed back to one Gaussian.

How This Matches The Jia–Xin–Cheng Gaussian Approximation Block

Jia, Xin, and Cheng first write the Gaussian approximation filter before they introduce sparse grids. In their additive-noise setting, the sparse-grid part of the method answers only how to approximate the Gaussian expectations needed for m_t^- , P_t^- , $\bar{z}_t$, S_t , and $C_{xz,t}$. It does not change the fact that the filter stores one Gaussian pair (m_t, P_t) .

In BayesFilter notation, that bridge has the generic form

$$\mathbb{E}[F(\xi)] \rightsquigarrow \mathcal{Q}_M[F] \rightsquigarrow \sum_r w_r^{(M)} F(\xi_r^{(M)}), \quad (34.17)$$

where M denotes the chosen deterministic point-rule family. The next chapter specializes this generic expectation-to-rule replacement to the fixed sparse-grid cloud and its stored scalar contract.

34.3 Deterministic Point-Rule Families

A deterministic Gaussian-surrogate lane is defined not only by the carried Gaussian object but also by the moment engine it uses. The key families for the expanded high-dimensional block are:

- local linearization (EKF-style moment closure),
- low-order sigma-point rules (UKF/CKF-style closures),
- dense Gauss–Hermite / tensor-product rules,
- sparse-grid rules that the next chapter will specialize.

The decisive question is therefore not whether all these methods are “Gaussian,” but what deterministic moment engine they use and what that engine implies for the approximate scalar that later chapters may differentiate.

The standard sigma-point interface already exists elsewhere in the monograph. Chapter 16 gives the generic sigma-point moment-matching interface, while Chapter 17 clarifies square-root backend contracts. The purpose here is not to reteach all sigma-point filters, but to position the deterministic point-rule family as the shared foundations of the sparse-grid specialization.

Rule-family comparison discipline. For a deterministic point rule with nodes $\xi_r^{(M)}$, mean weights $w_{r,m}^{(M)}$, and covariance/cross-covariance weights $w_{r,c}^{(M)}$, the family-level operator has the form

$$\bar{z}_t^{(M)} \approx \sum_r w_{r,m}^{(M)} h_\theta(m_t^- + C_t^- \xi_r^{(M)}), \quad (34.18)$$

$$S_t^{(M)} \approx R_\theta + \sum_r w_{r,c}^{(M)} \Delta_{t,r}^{(M)} \Delta_{t,r}^{(M)\top}, \quad (34.19)$$

with the matching cross-covariance operator defined analogously. The one-weight specialization covers symmetric rules such as CKF-like or fixed sparse-grid clouds. Standard scaled-UKF comparisons generally require distinct mean and covariance weights, so rule-family comparisons must preserve that distinction.

34.4 Exports To Sparse-Grid Specialization And Later Chapters

This chapter exports four things forward.

- (1) the exact-target versus Gaussian-carried-object distinction,
- (2) the Gaussian projection contract and its innovation objects,
- (3) the deterministic point-rule family language needed before sparse-grid specialization,
- (4) the rule-family comparison discipline needed later when sparse-grid, low-order sigma-point, and dense GHQ lanes are compared.

The next chapter, Chapter 35, begins where this one stops: it instantiates one specific deterministic point-rule lane, fixed sparse-grid quadrature, and develops the low-dimensional cloud geometry, Smolyak structure, filtering value path, and fixed-cloud scalar that this shared foundations chapter has intentionally deferred.

Chapter 35

Sparse-Grid Quadrature and the Fixed-Cloud Scalar

35.1 Purpose And Scope

This chapter is the fixed sparse-grid quadrature lane itself. The preceding chapter fixed the deterministic Gaussian carried object, the Gaussian projection contract, and the rule-family comparison language. The present chapter now commits to one specific deterministic moment engine: a fixed, merged sparse-grid cloud in standardized Gaussian coordinates.

The chapter therefore owns four burdens that should stay together:

1. low-dimensional cloud construction from one-dimensional rules,
2. active bands and Smolyak coefficients,
3. duplicate merging and the stored cloud object,
4. the fixed-cloud scalar object exported to the later SGQF continuation, fixed-branch, and validation chapters.

The later continuation chapter, Chapter 36, takes up the forward value path, the one-step numeric oracle, and the SGQF-specific same-branch validation burden. The present chapter stays on cloud construction mechanics so that the reader can understand the sparse-grid object before it is asked to carry a likelihood scalar.

35.2 Low-Dimensional Construction Of FixedSGQF

This chapter develops the sparse-grid construction in the order a reader usually needs it. Start from one nonlinear Gaussian moment in one coordinate. Lift that rule to a tensor-product cloud in two coordinates. Then ask how the same construction becomes too expensive in higher dimension and how the sparse-grid combination reduces that cost without changing the underlying standardized-coordinate logic.

35.2.1 One nonlinear Gaussian moment as the basic quadrature problem

Start with a scalar Gaussian input and a simple nonlinear observation map. Let

$$X \sim \mathcal{N}(m, P), \quad h(X) = X^2. \quad (35.1)$$

Suppose we want the Gaussian-projection observation mean

$$\bar{z} = \mathbb{E}[h(X)] = \mathbb{E}[X^2]. \quad (35.2)$$

Write $X = m + C\xi$ with $\xi \sim \mathcal{N}(0, 1)$ and $C^2 = P$. Then

$$\bar{z} = \int_{\mathbb{R}} (m + C\xi)^2 \phi(\xi) d\xi. \quad (35.3)$$

Once the state has been standardized, the moment problem becomes a standard-normal weighted integral. The quadrature rule is therefore a numerical approximation to a Gaussian expectation, not an extra modeling layer with a separate target.

35.2.2 Tensor-product growth in two dimensions

Now move from one coordinate to two. Let

$$X \sim \mathcal{N}(m, P), \quad X \in \mathbb{R}^2, \quad (35.4)$$

and write $X = m + C\xi$ with $\xi \sim \mathcal{N}(0, I_2)$. If each standardized coordinate uses the three-point rule $\{-\sqrt{3}, 0, \sqrt{3}\}$, then the direct tensor product uses all pairs

$$(\xi_1, \xi_2) \in \{-\sqrt{3}, 0, \sqrt{3}\} \times \{-\sqrt{3}, 0, \sqrt{3}\}, \quad (35.5)$$

so already in two dimensions the rule has $3^2 = 9$ points, while in three dimensions it has $3^3 = 27$ points. Sparse-grid construction tries to keep the same standardized-coordinate logic while avoiding the full tensor explosion.

35.2.3 One-Dimensional Rules And Their Tensor Products

A one-dimensional standard-normal quadrature rule at level ℓ is a weighted sum

$$I_\ell[\psi] = \sum_{a=1}^{s_\ell} \omega_{\ell,a} \psi(\zeta_{\ell,a}) \approx \int_{\mathbb{R}} \psi(\xi) \phi(\xi) d\xi. \quad (35.6)$$

The chapter fixes the odd-point standard-normal GHQ ladder as its concrete one-dimensional family:

$$I_1 = \text{1-point standard-normal GHQ}, \quad I_2 = \text{3-point standard-normal GHQ}, \quad I_3 = \text{5-point standard-normal GHQ}, \quad (35.7)$$

which is stronger than the source theorem's minimal exactness requirement and is particularly useful for transparent 1D/2D/3D teaching examples.

The tensor rule attached to a multi-index $\mathbf{i} = (i_1, \dots, i_b)$ is

$$I_{\mathbf{i}}[F] = (I_{i_1} \otimes \dots \otimes I_{i_b})[F]. \quad (35.8)$$

This is simply the Cartesian-product lift of the one-dimensional rules.

35.2.4 Three-dimensional preview: why sparse grids help

The 3D preview is the first place where sparse grids become visibly different from dense tensor rules. With the 3-point GHQ family in each coordinate, the dense rule has 27 points. The low sparse-grid level keeps only the tensor rules whose total level lies in a narrow active band, then combines them with signed Smolyak coefficients and merges repeated nodes. The practical message is that sparse grids are not “drop some points from a tensor product.” They are a signed combination of tensor rules followed by duplicate accumulation into one stored cloud.

A fully worked two-dimensional tensor example. Take $b = 2$ and use

$$I_1 : (0, 1), \quad I_2 : \left\{ \left(-\sqrt{3}, \frac{1}{6} \right), \left(0, \frac{2}{3} \right), \left(\sqrt{3}, \frac{1}{6} \right) \right\}. \quad (35.9)$$

Then the tensor rules $(1, 1)$, $(1, 2)$, and $(2, 1)$ are completely explicit:

$$(1, 1) : (0, 0) \text{ with weight } 1, \quad (35.10)$$

$$(1, 2) : (0, -\sqrt{3}), (0, 0), (0, \sqrt{3}) \text{ with weights } \frac{1}{6}, \frac{2}{3}, \frac{1}{6}, \quad (35.11)$$

$$(2, 1) : (-\sqrt{3}, 0), (0, 0), (\sqrt{3}, 0) \text{ with weights } \frac{1}{6}, \frac{2}{3}, \frac{1}{6}. \quad (35.12)$$

So the sparse-grid level-2 band in two dimensions already shows the key pattern: one origin contribution from $(1, 1)$ and two axis triples from $(1, 2)$ and $(2, 1)$.

A concrete 3D sparse-grid preview. For $b = 3$ and $L = 2$, the active band is not the full tensor rule $(2, 2, 2)$. Instead it contains

$$(1, 1, 1), \quad (1, 1, 2), \quad (1, 2, 1), \quad (2, 1, 1), \quad (35.13)$$

with coefficients

$$c_{(1,1,1)} = -2, \quad c_{(1,1,2)} = 1, \quad c_{(1,2,1)} = 1, \quad c_{(2,1,1)} = 1. \quad (35.14)$$

Before duplicate merging, those four tensor rules contribute

$$1 + 3 + 3 + 3 = 10 \quad (35.15)$$

raw node-weight contributions, not 27. After merging, the origin contribution from $(1, 1, 1)$ and the three level-mixed rules accumulates to

$$-2 + \frac{2}{3} + \frac{2}{3} + \frac{2}{3} = 0, \quad (35.16)$$

so the center cancels completely, leaving the six axis points

$$(\pm\sqrt{3}, 0, 0), (0, \pm\sqrt{3}, 0), (0, 0, \pm\sqrt{3}), \quad (35.17)$$

each with weight $1/6$. This is the concrete sense in which a low sparse-grid rule can collapse a 27-point dense picture into a 6-point stored cloud while still preserving selected low-order structure.

35.3 Sparse-Grid Rule Reconstructed In Source Order

For a declared sparse-grid level L , the active band may be written as

$$\mathcal{A}_{b,L} = \{\mathbf{i} \in \mathbb{N}^b : L \leq |\mathbf{i}| \leq L + b - 1\}, \quad |\mathbf{i}| = \sum_{j=1}^b i_j. \quad (35.18)$$

This notation is the key to the low-dimensional examples: for $b = 2, L = 2$, the active band gives $(1, 1), (1, 2), (2, 1)$, not $(2, 2)$. For $b = 3, L = 2$, it gives the four tensor rules that build the low-level 3D sparse-grid preview.

The Smolyak coefficient attached to $\mathbf{i}$ is

$$c_{\mathbf{i}} = (-1)^{L+b-1-|\mathbf{i}|} \binom{b-1}{L+b-1-|\mathbf{i}|}. \quad (35.19)$$

Hence the sparse-grid quadrature approximation is

$$A_{b,L}[F] = \sum_{\mathbf{i} \in \mathcal{A}_{b,L}} c_{\mathbf{i}} I_{\mathbf{i}}[F]. \quad (35.20)$$

This coefficient can be negative. That fact is central: it is why later covariance and innovation objects must be checked rather than assumed safe just because each component tensor rule individually looks Gaussian-friendly.

35.3.1 From Formula To Point Cloud

The runtime object used later by the filter is not a symbolic Smolyak expression and not a raw tensor rule. It is the merged, sorted list of standardized nodes and accumulated signed weights:

$$\mathcal{C}_{b,L} = \{(\xi^{(r)}, w_r)\}_{r=1}^{M_{b,L}}. \quad (35.21)$$

That means the cloud-construction contract must specify:

1. the one-dimensional rule family,
2. the active band and Smolyak coefficients,
3. duplicate-merge tolerance,
4. zero-weight pruning rule,
5. deterministic node ordering.

If two implementations use the same nominal level L but merge nearly equal nodes differently, they have not necessarily evaluated the same scalar.

Raw-to-merged 2D cloud walkthrough. For $(b, L) = (2, 2)$, the active indices are $(1, 1)$, $(1, 2)$, and $(2, 1)$ with coefficients $-1, +1, +1$. Before merging, the origin appears three times:

$$(0, 0) \text{ from } (1, 1), \quad (0, 0) \text{ from } (1, 2), \quad (0, 0) \text{ from } (2, 1). \quad (35.22)$$

Its accumulated signed weight is

$$-1 + \frac{2}{3} + \frac{2}{3} = \frac{1}{3}, \quad (35.23)$$

while the four axis nodes each receive weight $1/6$. So the final merged five-point cloud is

$$\{(0, 0), (\pm\sqrt{3}, 0), (0, \pm\sqrt{3})\} \quad \text{with weights} \quad \left\{ \frac{1}{3}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6} \right\}. \quad (35.24)$$

This is the minimal nontrivial example showing why the stored object is a merged cloud rather than a symbolic list of tensor rules.

Raw-to-merged 3D cloud walkthrough. For $(b, L) = (3, 2)$, the active indices from (35.13) contribute ten raw node-weight terms in total. Seven distinct node locations appear before zero-weight pruning: the origin and the six axis nodes $(\pm\sqrt{3}, 0, 0)$, $(0, \pm\sqrt{3}, 0)$, $(0, 0, \pm\sqrt{3})$. The origin weight cancels by (35.16), leaving six stored nodes after pruning. So the bookkeeping path is

$$10 \text{ raw contributions} \longrightarrow 7 \text{ merged node locations} \longrightarrow 6 \text{ stored nodes after zero-weight pruning.} \quad (35.25)$$

This is the concrete 3D sparse-grid reduction the panel asked to see: a 27-point dense tensor picture is replaced by a six-point merged sparse-grid cloud because the low-level signed combination preserves only the required axis-quadratic structure.

Deterministic merge / prune / order policy. A fixed-cloud builder routine should execute the following deterministic policy:

1. enumerate every active tensor rule and every raw tensor node in a fixed, lexicographic order,
2. for each raw node $\xi^{i,\alpha}$, look for an existing stored node ξ satisfying

$$\|\xi^{i,\alpha} - \xi\|_\infty \leq \tau_{\text{merge}} := 10^{-12} \max\{1, \|\xi\|_\infty\}, \quad (35.26)$$

3. if such a node exists, accumulate the signed weight $c_i w^{i,\alpha}$; otherwise add a new dictionary entry,
4. remove entries whose accumulated absolute weight is below the declared numerical-zero threshold,
5. sort the surviving nodes lexicographically and store the final cloud in that order.

For the fixed BayesFilter scalar, the cloud $\mathcal{C}_{b,L}$, the univariate rules, the merge tolerance, the zero-weight rule, the sorting convention, and the accumulated signed weights are all part of the target. If two implementations use the same nominal level but merge nearly equal floating-point nodes differently, they have not necessarily evaluated the same scalar.

Implementation-facing cloud-builder contract. The runtime object produced by this section is a stored cloud builder routine of the form `build_fixed_sparse_grid_cloud((b, L, {Iℓ}, τme` that returns the sorted merged cloud $\mathcal{C}_{b,L}$, its weight-total diagnostic, and any signed-weight diagnostics used later in validation. The mathematics above is the authoritative definition of that routine.

35.3.2 Exactness, Point Count, And The UKF Relation

The chapter uses the source exactness and UKF-relation results in one narrow way. They justify the low-level rule-family orientation and the fact that a merged sparse-grid cloud can look very UKF-like in special symmetric-axis cases. They do not justify identifying sparse-grid filtering with UKF filtering, and they do not remove Gaussian projection error. The point of the UKF bridge here is explanatory: it helps the reader see when two deterministic clouds can look similar geometrically even though their construction contracts differ.

35.4 What The Fixed Cloud Exports Forward

This construction chapter now hands the SGQF continuation chapter a fully specified sparse-grid object:

stored cloud: $\mathcal{C}_{b,L} = \{(\xi^{(r)}, w_r)\}_{r=1}^{M_{b,L}}$,
 construction metadata: {rule family, $\mathcal{A}_{b,L}$, c_1 , merge tolerance, zero-weight rule, node ordering},
 declared downstream role: $\mathcal{C}_{b,L}$ is the fixed quadrature object from which the SGQF scalar is built.
(35.27)

The next chapter, Chapter 36, uses this already-fixed cloud to define the forward value path, the one-step numeric oracle, and the lane-specific same-branch validation contract.

35.5 Methodological Boundary And Non-Claims

The sparse-grid lane should be read as a deterministic Gaussian-surrogate method with a declared fixed cloud, not as a full non-Gaussian posterior method. Its strength is transparency: the cloud, signed weights, and construction metadata are explicit. Its limitation is equally explicit: one Gaussian carried object can still lose multimodality, strong asymmetry, and broad interaction structure even when the sparse-grid moment engine itself is accurate for the Gaussian moments it is asked to approximate.

Exported forward: - Chapter 36 takes up the value-path, numeric-oracle, and SGQF validation burden on the fixed cloud defined here. - Chapter 37 takes up richer retained-object alternatives when one Gaussian is too narrow. - Chapter 39 may import the fixed-cloud scalar and branch-defining sparse-grid metadata only after the SGQF continuation chapter has stated them explicitly. - Chapter 40 may import only the declared Gaussian-projection defect, point-explosion / active-dimension defect, and sparse-grid scalar contract stated across this chapter and its continuation.

Chapter 36

Fixed-Cloud Filtering, Same-Branch SGQF, and Lane-Specific Validation

36.1 Purpose And Scope

The previous chapter fixed the sparse-grid object itself: the one-dimensional rule family, the active band, the signed Smolyak combination, and the merged cloud stored in standardized Gaussian coordinates. That was the construction burden. The present chapter takes up the burden that follows once the cloud has already been fixed: what scalar the fixed cloud computes, how the forward value path is assembled, what counts as the same branch for this lane, and what validation burden must be discharged before the sparse-grid surrogate can be promoted beyond a pedagogical or exploratory object.

This split is deliberate. A reader first needs to understand how the cloud is built. Only after that does it become useful to ask which one-step scalar the cloud defines, which branch metadata must stay fixed when differentiating that scalar, and which diagnostics certify that a BayesFilter implementation is still evaluating the same declared object rather than a nearby adaptive variant.

36.2 Filtering Value Path On A Fixed Cloud

Fix a standardized sparse-grid cloud

$$\mathcal{C} = \{(\xi^{(r)}, w_r)\}_{r=1}^M, \quad \xi^{(r)} \in \mathbb{R}^{n_x}, \quad w_r \in \mathbb{R}. \quad (36.1)$$

and freeze that cloud before likelihood evaluation. The sparse-grid value path then has two quadrature stages at each time: prediction and observation.

36.2.1 Prediction Stage

Let $P_{t-1} = C_{t-1}C_{t-1}^\top$ on the declared square-root branch. Place the previous Gaussian cloud by

$$\chi_{t-1}^{(r)} = m_{t-1} + C_{t-1}\xi^{(r)}, \quad (36.2)$$

push points through the transition,

$$a_t^{(r)} = f_\theta(\chi_{t-1}^{(r)}), \quad (36.3)$$

and form

$$m_t^- = \sum_{r=1}^M w_r a_t^{(r)}, \quad (36.4)$$

$$P_t^- = Q_\theta + \sum_{r=1}^M w_r (a_t^{(r)} - m_t^-)(a_t^{(r)} - m_t^-)^\top. \quad (36.5)$$

After (36.5), BayesFilter symmetrizes P_t^- and checks the declared positive-definite branch. For the same-branch lane studied here there is no adaptive jitter, rank switch, node rewrite, or cloud rebuild inside the scalar. If the predictive branch fails, the value path returns a veto instead of silently changing the object being evaluated.

36.2.2 Observation Stage

Factor $P_t^- = C_t^-(C_t^-)^\top$ on the declared branch and place the predictive cloud:

$$\chi_t^{(r)} = m_t^- + C_t^- \xi^{(r)}, \quad (36.6)$$

Propagate through the observation map,

$$z_t^{(r)} = h_\theta(\chi_t^{(r)}), \quad (36.7)$$

and compute

$$\bar{z}_t = \sum_{r=1}^M w_r z_t^{(r)}, \quad (36.8)$$

$$\Delta_t^{(r)} = z_t^{(r)} - \bar{z}_t, \quad (36.9)$$

$$S_t = R_\theta + \sum_{r=1}^M w_r \Delta_t^{(r)} \Delta_t^{(r)\top}, \quad (36.10)$$

$$C_{xz,t} = \sum_{r=1}^M w_r (\chi_t^{(r)} - m_t^-) \Delta_t^{(r)\top}. \quad (36.11)$$

The one-step innovation scalar is then

$$v_t = y_t - \bar{z}_t, \quad (36.12)$$

$$\hat{\ell}_t^{\text{FSGQ}} = -\frac{1}{2} \{ \log \det S_t + v_t^\top S_t^{-1} v_t + n_y \log(2\pi) \}. \quad (36.13)$$

If S_t fails the declared branch, the value path returns a veto. If it passes, the Gaussian projection update is

$$K_t = C_{xz,t} S_t^{-1}, \quad (36.14)$$

$$m_t = m_t^- + K_t v_t, \quad (36.15)$$

$$P_t = P_t^- - K_t S_t K_t^\top. \quad (36.16)$$

with the final carried covariance again checked on the declared branch.

Value-path execution order. For this lane the runtime order is part of the scalar identity:

1. symmetrize and factor P_{t-1} ,
2. place previous-state points and propagate through f_θ ,
3. form (m_t^-, P_t^-) and veto if the predictive branch fails,
4. factor P_t^- , place predictive points, and propagate through h_θ ,
5. form $\bar{z}_t$, S_t , and $C_{xz,t}$, and veto if the innovation branch fails,
6. form v_t , $\hat{\ell}_t^{\text{FSGQ}}$, K_t , m_t , and P_t , then veto if the carried covariance branch fails.

Any implementation that changes the cloud, node ordering, factor branch, or veto policy in the middle of this sequence is no longer evaluating the same fixed-cloud scalar.

36.3 Worked One-Step Example With A Numeric Oracle

Use the one-dimensional transition-observation model

$$X_0 \sim \mathcal{N}(m_0, P_0), \quad m_0 = 0.5, \quad P_0 = 1, \quad (36.17)$$

$$X_1 = f_\beta(X_0) + \eta_1, \quad f_\beta(x) = \beta x, \quad \eta_1 \sim \mathcal{N}(0, Q), \quad (36.18)$$

$$Y_1 = h_\theta(X_1) + \varepsilon_1, \quad h_\theta(x) = x^2, \quad \varepsilon_1 \sim \mathcal{N}(0, R), \quad (36.19)$$

with

$$Q = 0.25, \quad R = 1, \quad y_1 = 2. \quad (36.20)$$

Evaluate at $\beta = 1$ on the positive scalar Cholesky branch and use the three-point standard-normal cloud

$$\mathcal{C}_{1,2} = \left\{ \left(-\sqrt{3}, \frac{1}{6} \right), \left(0, \frac{2}{3} \right), \left(\sqrt{3}, \frac{1}{6} \right) \right\}. \quad (36.21)$$

Placed points and moments. Since $C_0 = \sqrt{P_0} = 1$, the previous-state placed points are

$$\chi_0^{(r)} = m_0 + C_0 \xi^{(r)} \in \{0.5 - \sqrt{3}, 0.5, 0.5 + \sqrt{3}\}. \quad (36.22)$$

At $\beta = 1$, the transition images satisfy

$$a_1^{(r)} = f_\beta(\chi_0^{(r)}) = \chi_0^{(r)}. \quad (36.23)$$

The predicted Gaussian moments are therefore

$$m_1^- = \sum_r w_r a_1^{(r)} = 0.5, \quad P_1^- = Q + \sum_r w_r (a_1^{(r)} - m_1^-)^2 = 1.25 = \frac{5}{4}. \quad (36.24)$$

The declared Cholesky factor is

$$C_1^- = \sqrt{P_1^-} = \sqrt{1.25} = \frac{\sqrt{5}}{2}. \quad (36.25)$$

Hence the predictive placed points are

$$\chi_1^{(r)} = m_1^- + C_1^- \xi^{(r)} \in \left\{ 0.5 - \frac{\sqrt{15}}{2}, 0.5, 0.5 + \frac{\sqrt{15}}{2} \right\}. \quad (36.26)$$

Passing those points through $h_\theta(x) = x^2$ gives

$$z_1^{(r)} \in \{2.0635083269, 0.25, 5.9364916731\}. \quad (36.27)$$

The centered observation values are

$$\Delta_1^{(r)} = z_1^{(r)} - \bar{z}_1 \in \{0.5635083269, -1.25, 4.4364916731\}. \quad (36.28)$$

and the Gaussian update objects are

$$\bar{z}_1 = \sum_r w_r z_1^{(r)} = 1.5 = \frac{3}{2}, \quad (36.29)$$

$$S_1 = R + \sum_r w_r (z_1^{(r)} - \bar{z}_1)^2 = 5.375 = \frac{43}{8}, \quad (36.30)$$

$$C_{xz,1} = \sum_r w_r (\chi_1^{(r)} - m_1^-)(z_1^{(r)} - \bar{z}_1) = 1.25 = \frac{5}{4}, \quad (36.31)$$

$$v_1 = y_1 - \bar{z}_1 = 0.5 = \frac{1}{2}, \quad (36.32)$$

$$K_1 = C_{xz,1} S_1^{-1} = \frac{10}{43} \approx 0.2325581395, \quad (36.33)$$

$$m_1 = m_1^- + K_1 v_1 = \frac{53}{86} \approx 0.6162790698, \quad (36.34)$$

$$P_1 = P_1^- - K_1 S_1 K_1^\top = \frac{165}{172} \approx 0.9593023256. \quad (36.35)$$

The one-step deterministic approximate log likelihood is

$$\widehat{\ell}_1^{\text{FSGQ}} = -\frac{1}{2} \{ \log S_1 + v_1^2 S_1^{-1} + \log(2\pi) \} \approx -1.7830736342. \quad (36.36)$$

Same-branch derivative with respect to β . The same cloud and the same positive scalar Cholesky branch are reused in the derivative lane. Because the parameter enters only through the transition in this example, $\partial_\beta h_\theta(x) = 0$. Since m_0 and P_0 are fixed in β , the previous-state placed points satisfy

$$\dot{\chi}_0^{(r)} = 0. \quad (36.37)$$

From the transition derivative rule,

$$\dot{a}_1^{(r)} = D_x f_\beta(\chi_0^{(r)}) \dot{\chi}_0^{(r)} + \partial_\beta f_\beta(\chi_0^{(r)}) = \chi_0^{(r)} \in \{0.5 - \sqrt{3}, 0.5, 0.5 + \sqrt{3}\}. \quad (36.38)$$

Hence

$$\dot{m}_1^- = \sum_r w_r \dot{a}_1^{(r)} = 0.5, \quad \dot{d}_1^{(r)} = \dot{a}_1^{(r)} - \dot{m}_1^- \in \{-\sqrt{3}, 0, \sqrt{3}\}, \quad (36.39)$$

and

$$\dot{P}_1^- = 2. \quad (36.40)$$

Because $C_1^- = (P_1^-)^{1/2}$, the declared branch derivative is

$$\dot{C}_1^- = \frac{\dot{P}_1^-}{2C_1^-} = \frac{2}{\sqrt{5}} \approx 0.8944271910. \quad (36.41)$$

Therefore the predictive point sensitivities are

$$\dot{\chi}_1^{(r)} = \dot{m}_1^- + \dot{C}_1^- \xi^{(r)} \in \{-1.0491933385, 0.5, 2.0491933385\}. \quad (36.42)$$

Since $h_\theta(x) = x^2$,

$$\dot{z}_1^{(r)} = 2\chi_1^{(r)} \dot{\chi}_1^{(r)} \in \{3.0143149884, 0.5, 9.9856850116\}. \quad (36.43)$$

The centered observation sensitivities are

$$\dot{\Delta}_1^{(r)} = \dot{z}_1^{(r)} - \dot{z}_1 \in \{0.5143149884, -2, 7.4856850116\}. \quad (36.44)$$

The derived moment sensitivities are

$$\dot{z}_1 = 2.5 = \frac{5}{2}, \quad \dot{v}_1 = -2.5 = -\frac{5}{2}, \quad (36.45)$$

$$\dot{S}_1 = 14.5 = \frac{29}{2}, \quad \dot{C}_{xz,1} = 3.25 = \frac{13}{4}. \quad (36.46)$$

Let

$$u_1 = S_1^{-1} v_1 = \frac{4}{43} \approx 0.0930232558. \quad (36.47)$$

Then the one-step score ingredients are

$$\text{tr}(S_1^{-1} \dot{S}_1) = \frac{116}{43} \approx 2.6976744186, \quad (36.48)$$

$$2\dot{v}_1 u_1 = -\frac{20}{43} \approx -0.4651162791, \quad (36.49)$$

$$-u_1^2 \dot{S}_1 = -\frac{232}{1849} \approx -0.1254732288. \quad (36.50)$$

The same-branch one-step score is therefore

$$\dot{\ell}_1^{\text{FSGQ}} = -\frac{1}{2} \left[\frac{\dot{S}_1}{S_1} + 2\dot{v}_1 u_1 - u_1^2 \dot{S}_1 \right] = -\frac{1948}{1849} \approx -1.0535424554. \quad (36.51)$$

For later propagation,

$$\dot{K}_1 = \dot{C}_{xz,1} S_1^{-1} - K_1 \dot{S}_1 S_1^{-1} = -\frac{42}{1849} \approx -0.0227149811, \quad (36.52)$$

so

$$\dot{m}_1 = \dot{m}_1^- + \dot{K}_1 v_1 + K_1 \dot{v}_1 = -\frac{343}{3698} \approx -0.0927528394, \quad (36.53)$$

$$\dot{P}_1 = \dot{P}_1^- - \dot{K}_1 S_1 K_1 - K_1 \dot{S}_1 K_1 - K_1 S_1 \dot{K}_1 = \frac{2353}{1849} \approx 1.2725797729. \quad (36.54)$$

This makes the chapter's limitation concrete. Every quantity above is deterministic and internally checkable on the declared branch, yet the nonlinear observation law can still generate non-Gaussian posterior structure. FixedSGQF therefore gives a transparent Gaussian-surrogate value-and-gradient oracle, not a full non-Gaussian posterior representation.

Implementation checklist for this oracle. A first implementation should be able to reproduce, up to declared floating-point tolerance, at least m_1^- , P_1^- , $\bar{z}_1$, S_1 , $C_{xz,1}$, K_1 , m_1 , P_1 , $\hat{\ell}_1^{\text{FSGQ}}$, and $\dot{\ell}_1^{\text{FSGQ}}$. This is the compact end-to-end numerical oracle for both the value path and the derivative path.

This example is not a global correctness theorem. Its role is narrower and practical: a first BayesFilter implementation should be able to reproduce these numbers before any same-branch derivative claim is trusted.

36.4 General Analytical Derivative Framework For FixedSGQF

The worked oracle fixes one concrete same-branch derivative instance. The next task is to state the general derivative framework so that the SGQF lane does not depend on the toy example alone. Differentiate with respect to one parameter component θ_i , and let a dot denote ∂_i .

Why the derivative is operationally delicate. The one-step scalar depends on the previous carried Gaussian, the covariance factor used to place the cloud, the nonlinear transition and observation images, the innovation covariance, and the posterior update. The derivative must therefore propagate through the entire recursion, and it remains meaningful only when the value and derivative paths stay on the same declared branch.

Current-score objects versus propagation objects. For the current likelihood contribution, the derivative closes once $\dot{v}_t$ and $\dot{S}_t$ are known. The derivative $\dot{C}_{xz,t}$ is not needed to close the current score, but it is needed to form $\dot{K}_t$ and hence to propagate $\dot{m}_t$ and $\dot{P}_t$ to the next time step. A correct implementation must keep these two roles separate.

36.4.1 Derivative Ledger

The chain is

$$\begin{aligned} \theta \mapsto (m_{t-1}, P_{t-1}) \mapsto C_{t-1} \mapsto \chi_{t-1}^{(r)} \mapsto a_t^{(r)} \mapsto (m_t^-, P_t^-) \mapsto C_t^- \\ \mapsto \chi_t^{(r)} \mapsto z_t^{(r)} \mapsto (\bar{z}_t, S_t, C_{xz,t}) \mapsto (v_t, K_t, \hat{\ell}_t^{\text{FSGQ}}, m_t, P_t). \end{aligned} \quad (36.55)$$

The sparse-grid nodes and weights do not move with θ . They are saved branch objects. The Gaussian factors C_{t-1} and C_t^- , however, do move with θ , because they are functions of P_{t-1} and P_t^- .

36.4.2 Square-Root Branch

For the declared lower-triangular Cholesky branch $P = LL^\top$, set

$$A = L^{-1} \dot{P} L^{-\top}. \quad (36.56)$$

Define the lower-triangular operator

$$\Phi(A)_{jk} = \begin{cases} A_{jk}, & j > k, \\ A_{jj}/2, & j = k, \\ 0, & j < k. \end{cases} \quad (36.57)$$

Then

$$\dot{L} = L\Phi(A). \quad (36.58)$$

Every factor derivative in this SGQF lane uses this declared branch recipe.

36.4.3 Prediction Sensitivities

From the placed-point equation,

$$\dot{\chi}_{t-1}^{(r)} = \dot{m}_{t-1} + \dot{C}_{t-1}\xi^{(r)}. \quad (36.59)$$

From the transition map,

$$\dot{a}_t^{(r)} = D_x f_\theta(\chi_{t-1}^{(r)})\dot{\chi}_{t-1}^{(r)} + \partial_i f_\theta(\chi_{t-1}^{(r)}). \quad (36.60)$$

The predicted mean derivative is

$$\dot{m}_t^- = \sum_{r=1}^M w_r \dot{a}_t^{(r)}. \quad (36.61)$$

Let

$$d_t^{(r)} = a_t^{(r)} - m_t^-, \quad \dot{d}_t^{(r)} = \dot{a}_t^{(r)} - \dot{m}_t^-. \quad (36.62)$$

Then

$$\dot{P}_t^- = \dot{Q}_\theta + \sum_{r=1}^M w_r \left\{ \dot{d}_t^{(r)} d_t^{(r)\top} + d_t^{(r)} \dot{d}_t^{(r)\top} \right\}. \quad (36.63)$$

36.4.4 Observation Sensitivities And Innovation Score

On the predictive factor branch,

$$\dot{\chi}_t^{(r)} = \dot{m}_t^- + \dot{C}_t \xi^{(r)}. \quad (36.64)$$

Observation sensitivities satisfy

$$\dot{z}_t^{(r)} = D_x h_\theta(\chi_t^{(r)})\dot{\chi}_t^{(r)} + \partial_i h_\theta(\chi_t^{(r)}). \quad (36.65)$$

Then

$$\dot{\tilde{z}}_t = \sum_{r=1}^M w_r \dot{z}_t^{(r)}, \quad (36.66)$$

$$\dot{v}_t = -\dot{\tilde{z}}_t, \quad (36.67)$$

$$\dot{\Delta}_t^{(r)} = \dot{z}_t^{(r)} - \dot{\tilde{z}}_t, \quad (36.68)$$

$$\dot{S}_t = \dot{R}_\theta + \sum_{r=1}^M w_r \left\{ \dot{\Delta}_t^{(r)} \Delta_t^{(r)\top} + \Delta_t^{(r)} \dot{\Delta}_t^{(r)\top} \right\}, \quad (36.69)$$

$$\dot{C}_{xz,t} = \sum_{r=1}^M w_r \left\{ \dot{\chi}_t^{(r)} \Delta_t^{(r)\top} + (\chi_t^{(r)} - m_t^-) \dot{\Delta}_t^{(r)\top} \right\}. \quad (36.70)$$

The one-step fixed-Gaussian innovation score is

$$\partial_i \widehat{\ell}_t^{\text{FSGQ}} = -\frac{1}{2} \left[\text{tr}(S_t^{-1} \dot{S}_t) + 2\dot{v}_t^\top S_t^{-1} v_t - v_t^\top S_t^{-1} \dot{S}_t S_t^{-1} v_t \right]. \quad (36.71)$$

This closes the current score role. The propagation role then uses

$$\dot{K}_t = \dot{C}_{xz,t} S_t^{-1} - K_t \dot{S}_t S_t^{-1} \quad (36.72)$$

and

$$\dot{m}_t = \dot{m}_t^- + \dot{K}_t v_t + K_t \dot{v}_t, \quad (36.73)$$

$$\dot{P}_t = \dot{P}_t^- - \dot{K}_t S_t K_t^\top - K_t \dot{S}_t K_t^\top - K_t S_t \dot{K}_t^\top. \quad (36.74)$$

These become the differentiated inputs to the next time step.

36.5 Same-Branch Fixed-Cloud Scalar Contract

For the sparse-grid lane, the saved branch identity is not exhausted by the symbol $\mathcal{C}_{b,L}$. A branch-valid finite-difference comparison must reuse the full fixed ledger:

$$\mathfrak{B}_{\text{FSGQ}} = \{\mathcal{C}_{b,L}, \text{one-dimensional rule family, merge tolerance, zero-weight pruning rule, node ordering}\} \quad (36.75)$$

For parameter component i , the same-branch central difference is

$$D_i(h) = \frac{\widehat{\ell}_T^{\text{FSGQ}}(\theta + h e_i; \mathfrak{B}_{\text{FSGQ}}) - \widehat{\ell}_T^{\text{FSGQ}}(\theta - h e_i; \mathfrak{B}_{\text{FSGQ}})}{2h}. \quad (36.76)$$

A row is branch-valid only if both perturbations reuse the same saved structural branch record and realize the same accepted/failure stage-time pattern through every likelihood contribution included in the comparison. Equal cloud family but different first failure time counts as a branch mismatch; equal first failure time but different accepted-prefix pattern also counts as a branch mismatch.

That stricter logic matters because sparse-grid differentiation is not just “differentiate some nearby Gaussian filter.” It is “differentiate the same fixed-cloud scalar.” The later shared same-scalar chapter may import that contract, but the sparse-grid lane must state it locally because the cloud, merge policy, and branch-veto discipline are part of the lane-specific scalar identity.

Frozen versus recomputed objects on a saved branch. The branch identity freezes structure, not numerical values. Within a fixed cloud branch,

$$\begin{aligned} \text{frozen structural objects: } & \mathcal{C}_{b,L}, \tau_{\text{merge}}, \tau_{\text{zero}}, \text{ordering}, C(\cdot), \text{preprocessing policy, init} \\ \text{parameter-dependent numerical objects: } & m_t, P_t, C_t, m_t^-, P_t^-, C_t^-, \bar{z}_t, S_t, C_{xz,t}, K_t, \widehat{\ell}_t^{\text{FSGQ}}. \end{aligned} \quad (36.77)$$

Thus same-scalar finite differences recompute the parameter-dependent numerical objects by the same declared branch rule; they do not reuse old numerical values. In particular, $m_0(\theta \pm h e_i)$ and $P_0(\theta \pm h e_i)$ are recomputed by the same initial-condition policy. What must remain unchanged is the structural route by which those values are produced.

Finite-difference ladder and pass criterion. For each parameter component, use a ladder such as

$$h \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\} \max\{1, |\theta_i|\}. \quad (36.78)$$

For valid rows, compare the central difference against the analytical score G_i by

$$e_i(h) = \frac{|D_i(h) - G_i|}{\max\{1, |D_i(h)|, |G_i|\}}. \quad (36.79)$$

The diagnostic passes when at least two consecutive valid rows satisfy $e_i(h) \leq 10^{-5}$ and the smaller-step row is no worse than ten times the larger-step row. Failure means one of three things is likely: the derivative formula is wrong, the value and gradient paths are not using the same scalar, or the declared branch is too nonsmooth or ill-conditioned at that point.

Implementation-facing FD checklist. A routine of the form `same_scalar_fd_check(i, h_ladder,  $\theta$ ,  $\mathfrak{B}_F$ )` should:

1. evaluate the value path at $\theta \pm he_i$ using the same saved structural branch metadata,
2. mark a row branch-invalid if either side changes the structural branch or the realized accepted/failure stage-time pattern,
3. compute the central difference only for branch-valid rows,
4. compare that central difference against the analytical score with (36.79), and
5. report whether two consecutive valid rows satisfy the declared tolerance rule.

The equations above remain the authoritative mathematical definition; this list is the operational review contract.

36.6 Lane-Specific Validation Burden

Fixed SGQF should be validated as an approximate deterministic likelihood lane, not as an exact posterior filter. The minimum validation suite is:

1. **Construction checks.** Verify duplicate-node accumulation, weight totals, signed-weight counts, and deterministic cloud reproduction for small (b, L) cases.
2. **Linear-Gaussian recovery.** For affine f_θ, h_θ , the Gaussian projection recursion should match the Kalman filter up to floating-point tolerance.
3. **Worked scalar oracle.** Reproduce the one-step values in Section 36.3 exactly enough that the cloud, factor, innovation, and likelihood assembly are demonstrably aligned.
4. **Same-branch finite differences.** Use (36.76) on representative parameters and step sizes while logging branch-valid versus branch-invalid rows.
5. **Signed-weight stability.** Record the minimum eigenvalues of symmetrized P_t^- , S_t , and P_t , plus any branch veto.

6. **Memory and runtime scaling.** Record $M_{b,L}$, model evaluations, wall time, and peak memory as b , L , and T change.
7. **Accuracy ladder.** Compare fixed SGQF against tensor-product GHQ in small dimension and against other controlled references where available.

A useful test-model ladder mirrors the p47 source burden:

- **Model A:** a linear-Gaussian recovery case,
- **Model B:** the scalar quadratic-observation cell from Chapter 33,
- **Model C:** a cloud-sensitive two-dimensional nonlinear observation where duplicate merging and fourth moments matter,
- **Model D:** a small high-dimensional block-local model where signed weights, active-dimension growth, and covariance checks become visible,
- **Model E:** a deliberately hard all-coordinate interaction model to expose when sparse-grid savings stop compensating for Gaussian-surrogate bias,
- **Model F:** a Jia–Xin–Cheng-style orbit / sparse-grid benchmark that distinguishes sparse-grid exactness claims from posterior-accuracy claims.

The first three models should be understood concretely, not only as names:

$$\text{Model A: } X_t = A_\theta X_{t-1} + \eta_t, \quad \eta_t \sim \mathcal{N}(0, Q_\theta), \quad Y_t = H_\theta X_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, R_\theta), \quad (36.80)$$

$$\text{Model B: } X_t = \rho X_{t-1} + \eta_t, \quad \eta_t \sim \mathcal{N}(0, q), \quad Y_t = X_t^2 + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, r), \quad (36.81)$$

$$\text{Model C: } X_t = \rho X_{t-1} + \eta_t, \quad \eta_t \sim \mathcal{N}(0, qI_2), \quad Y_t = \theta^2(X_{t,1}^2 + X_{t,2}^2) + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, r). \quad (36.82)$$

Model A checks exact-reference recovery, Model B exposes posterior-shape failure of one-Gaussian surrogates, and Model C catches duplicate-node or signed-weight assembly errors because the relevant fourth moments change when the cloud is built incorrectly.

The implementation report for this lane should log at least:

$$\mathcal{R}_{\text{SGQF}} = \{M_{b,L}, \text{ signed-weight count}, \min \lambda(P_t^-), \min \lambda(S_t), \min \lambda(P_t), \text{ FD table}, \text{ branch-veto count}, \quad (36.83)$$

Accuracy claims should be made only against a named comparator:

$$\text{error} = \begin{cases} |\widehat{\ell}_T - \ell_T^{\text{Kalman}}|, & \text{linear-Gaussian test,} \\ |\widehat{\ell}_T - \ell_T^{\text{dense}}|, & \text{small dense-quadrature test,} \\ \text{RMSE or posterior diagnostic against PF/TT reference,} & \text{nonlinear large test.} \end{cases} \quad (36.84)$$

These are not mere software counters. They record the places where the declared sparse-grid scalar can fail mathematically or operationally: duplicate merging, signed-weight stability, covariance branch validity, same-branch derivative parity, and cost growth.

Passing this suite supports the claim that the implementation evaluates and differentiates the declared fixed-cloud surrogate scalar. It does not prove exact posterior accuracy, HMC convergence, or default readiness.

36.7 Implementation Contract For FixedSGQF

A compact implementation contract makes the SGQF lane auditable without forcing the reader back to the source note. The key runtime objects are:

object	definition	shape	reuse
$\xi^{(r)}$	standardized sparse-grid node	n_x	value and gradient
w_r	accumulated signed weight after duplicate merge	scalar	value and gradient
C_{t-1}, C_t^-	covariance factors on declared branch	$n_x \times n_x$	value, gradient, diagnostics
$\chi_{t-1}^{(r)}$	previous placed point	n_x	transition value and derivative
$a_t^{(r)}$	transition image	n_x	prediction moments
$\chi_t^{(r)}$	predictive placed point	n_x	observation value and derivative
$z_t^{(r)}$	observation image	n_y	observation moments
$\bar{z}_t, S_t, C_{xz,t}$	Gaussian projection moments	$n_y, n_y \times n_y, n_x \times n_y$	likelihood and update
v_t, K_t, u_t	innovation, gain, solve variable with $S_t u_t = v_t$	$n_y, n_x \times n_y, n_y$	likelihood, update, gradient
$\dot{\chi}, \dot{a}, \dot{z}$	point sensitivities	matching value objects	gradient only
$\dot{m}, \dot{P}, \dot{S}, \dot{C}_{xz}$	moment and filter sensitivities	matching value objects	gradient and next step
data record	$y_{1:T}$, preprocessing, missing-data policy if any	metadata	value, gradient, finite difference
initial law	$m_0(\theta), P_0(\theta), \dot{m}_0, \dot{P}_0$	$n_x, n_x \times n_x$	value and gradient start
branch record	cloud, rules, merge tolerance, zero-weight rule, node ordering, preprocessing policy, initial-condition policy, factor branch, symmetrize-then-veto rule, thresholds, accepted/failure stage-time pattern	metadata	same-scalar finite difference

36.7.1 Input, Output, Invariant, And Failure Contract

The full lane-level routine has inputs

$$\begin{aligned}
\mathcal{I}_{\text{in}} = & (y_{1:T}, \theta, L, \{I_\ell\}_{\ell=1}^{L+b-1}, m_0(\theta), P_0(\theta), \\
& \dot{m}_0^{(1:p)}(\theta), \dot{P}_0^{(1:p)}(\theta), f_\theta, h_\theta, Q_\theta, R_\theta, \\
& D_x f_\theta, \partial_{1:p} f_\theta, D_x h_\theta, \partial_{1:p} h_\theta, \dot{Q}_\theta^{(1:p)}, \dot{R}_\theta^{(1:p)}, \\
& \text{merge tolerance, factor map, branch thresholds}).
\end{aligned} \tag{36.85}$$

It returns either

$$\left(\widehat{\ell}_T^{\text{FSGQ}}(\theta), \nabla_\theta \widehat{\ell}_T^{\text{FSGQ}}(\theta), \{m_t, P_t\}_{t=0}^T, \mathcal{D}_{\text{diag}} \right) \tag{36.86}$$

or a failure record

$$\mathcal{F} = (\text{time, stage, reason, diagnostic values}). \quad (36.87)$$

The invariants are:

$$\sum_{r=1}^M w_r = 1 \quad \text{up to construction tolerance,} \quad (36.88)$$

$$P_t^- = (P_t^-)^\top, \quad S_t = S_t^\top, \quad P_t = P_t^\top \quad \text{after explicit symmetrization,} \quad (36.89)$$

$$S_t \succ 0 \quad \text{on every accepted likelihood step,} \quad (36.90)$$

$$\mathcal{B}_{\text{value}} = \mathcal{B}_{\text{gradient}} \quad \text{same saved branch.} \quad (36.91)$$

The routine fails rather than repairs if

$$\lambda_{\min}(S_t) < \epsilon_S, \quad \lambda_{\min}(P_t^-) < \epsilon_P, \quad \lambda_{\min}(P_t) < \epsilon_P, \quad (36.92)$$

or if the declared factor map cannot be evaluated on the symmetrized covariance. A smoothed repair would define a different scalar and therefore would need a new derivative contract.

36.7.2 One-Step Value-Path Contract

At one accepted time step, the value path is the ordered map

$$\begin{aligned} (m_{t-1}, P_{t-1}, y_t, \theta, \mathcal{C}) &\mapsto C_{t-1} \mapsto \chi_{t-1}^{(r)} \mapsto a_t^{(r)} \mapsto (m_t^-, P_t^-) \\ &\mapsto C_t^- \mapsto \chi_t^{(r)} \mapsto z_t^{(r)} \mapsto (\bar{z}_t, S_t, C_{xz,t}) \\ &\mapsto (v_t, \hat{\ell}_t^{\text{FSGQ}}, K_t) \mapsto (m_t, P_t). \end{aligned} \quad (36.93)$$

The accepted map is conditioned on two successful branch checks: the predictive covariance P_t^- must pass the declared factor and positive-definite test before observation-point placement, and the innovation covariance S_t must pass the declared branch test before the Gaussian projection update is accepted. The carried outputs propagated to the next step are only (m_t, P_t) together with the global fixed branch record $\mathcal{B}$. Point clouds, moment arrays, gains, and diagnostics may be cached for derivative reuse, but they are not part of the carried filtering state itself.

36.7.3 Default Numerical Constants

The constants below are defaults for diagnostics, not claims of universal validity:

quantity	default	role
duplicate merge tolerance	$10^{-12} \max(1, \ \xi\ _\infty)$	groups standardized nodes that should be identical under the declared rule
weight zero tolerance	10^{-14}	removes accumulated numerical-zero weights after merge
ϵ_S	10^{-10}	veto threshold for innovation covariance positive definiteness
ϵ_P	10^{-10}	veto threshold for carried and predictive covariance factors
ϵ_ℓ	$10^{-3} \max(1, \widehat{\ell}_T)$	pilot level-ladder change threshold
finite-difference relative tolerance	10^{-5}	same-scalar gradient diagnostic
maximum branch-invalid FD rows	0 in final audit	branch changes mean the derivative is not being tested

These values are deliberately conservative. They are meant to reveal when the fixed branch is fragile. They are not tuning knobs for making a failing branch look smooth.

36.8 End-To-End Mathematical Algorithm

FixedSGQF value and gradient, with all branch exits.

1. Build the sparse-grid cloud from $b = n_x$, level L , and $\{I_\ell\}$. Save $\mathcal{C} = \{(\xi^{(r)}, w_r)\}_{r=1}^M$.
2. Check the weight-sum invariant. If it fails, return $\mathcal{F} = (0, \text{cloud}, \text{weight-sum failure}, \dots)$.
3. Initialize $\widehat{\ell}_T^{\text{FSGQ}} \leftarrow 0$, $G_i \leftarrow 0$ for $i = 1, \dots, p$, and compute $m_0, P_0, \dot{m}_0^{(i)}, \dot{P}_0^{(i)}$.
4. For $t = 1, \dots, T$, symmetrize P_{t-1} . If the factor map fails, return a factor-branch failure. Otherwise set $C_{t-1} = C(P_{t-1})$ and compute the matching factor derivatives.
5. Place previous points, compute transition images, and form (m_t^-, P_t^-) together with their derivatives.
6. Symmetrize P_t^- . If the predictive covariance fails the branch, return predictive covariance failure.
7. Factor P_t^- , place observation points, evaluate observation values, and form $(\bar{z}_t, S_t, C_{xz,t}, v_t)$.
8. Symmetrize S_t . If the innovation covariance fails the branch, return innovation covariance failure.
9. Only on that accepted innovation branch, form $\widehat{\ell}_t^{\text{FSGQ}}$, update $\widehat{\ell}_T^{\text{FSGQ}}$, and accumulate the componentwise score.
10. Form K_t , then propagate m_t, P_t and their sensitivities. If the carried covariance branch fails, return carried covariance failure.
11. Return $(\widehat{\ell}_T^{\text{FSGQ}}, G_{1:p}, \{m_t, P_t\}, \mathcal{D}_{\text{diag}})$.

The algorithm is deliberately mathematical rather than Pythonic. Its purpose is to say what an implementation must compute, save, and refuse to change if the reported gradient is to be the derivative of the reported value.

36.9 Exports To Later Chapters

This SGQF continuation chapter exports five specific objects forward:

1. the fixed-cloud value-path recursion,
2. the one-step numeric oracle,
3. the lane-specific branch-identity record,
4. the branch-valid finite-difference contract for SGQF,
5. the SGQF validation and non-claim boundaries.

The later fixed-branch chapter may now treat the sparse-grid lane as an already specified scalar object rather than re-explaining the cloud mechanics. The closing validation/promotion chapter may consume the SGQF diagnostics and veto logic without needing to rederive the lane's runtime map.

Chapter 37

Low-Rank Density Filters, KR Maps, and Retained Objects

37.1 Orientation And Scope

This chapter studies a high-dimensional nonlinear filtering problem through the lens of one specific computational object: a low-rank non-Gaussian retained representation built from tensor-train density approximations, conditional Knothe–Rosenblatt maps, and squared-density repair. The exact target remains the filtering law and normalizer from Chapter 33; the approximation here is the retained low-rank density or retained-object representation that numerically replaces that law.

The scientific tension is the same one that appears across many high-dimensional filtering problems. The exact Bayesian recursion propagates a full conditional law, but once the model is nonlinear that law is usually not available in a finite-dimensional closed form. Zhao and Cui answer that tension with a non-Gaussian density-approximation lane based on tensor trains, squared-density repair, conditional KR maps, and preconditioning. The purpose of this chapter is to make that retained-object construction readable in source order and to state clearly what object is being carried, normalized, updated, and later handed to the shared fixed-branch derivative chapter.

What this chapter does not claim. This chapter does not claim that adaptive TT/KR algorithms are globally smooth, that moderate TT rank is always sufficient in high dimension, or that a retained low-rank object automatically yields an exact posterior target for HMC. Its purpose is narrower: define the retained object, explain when that object is scientifically meaningful, and state the diagnostics that later chapters must use before promotion.

37.2 Running Example And Notation

The running nonlinear example should stay conceptually continuous with the rest of the block, but the retained-object lane needs a slightly richer notation than the deterministic Gaussian lane. The key distinction is between:

- the exact Bayesian target law and its normalizer,
- the adaptive low-rank approximation family that numerically replaces it,

- and the fixed-branch scalar/derivative objects that later chapters may define from a frozen member of that family.

The method uses state variables x_t , observation variables y_t , and a static parameter α . Coordinate group notation such as $x_{t,<j}$, $x_{t,>j}$, $x_{t,\leq j}$, $x_{t,\geq j}$ matters because conditional transport and conditional density construction are coordinate-local operations. For TT/KR methods, the essential bookkeeping objects are coordinate groups, conditional densities, retained marginal/filter objects, and any Jacobian terms needed when the retained object is moved into preconditioned coordinates.

37.3 Exact Filtering Target And Recursive Bottleneck

The retained-object lane starts from the same target discipline as the rest of the block. At time t , the filtering problem is to update

$$p(x_t, \alpha \mid y_{1:t}),$$

and the path problem is to update

$$p(x_{0:t}, \alpha \mid y_{1:t}).$$

After observing y_t , the exact adjacent-state target has the three-block form

$$q_t(x_t, \alpha, x_{t-1}) = p(x_{t-1}, \alpha \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t),$$

and the next filter is obtained by integrating out x_{t-1} and normalizing. The bottleneck is therefore always the same: how to represent this object and how to integrate the represented object without returning to an infeasible full grid.

Implementation object. A minimal implementation must expose evaluators for the previous retained filter, the transition density, and the observation density, then build an evaluator for the three-block target. The first failure check is that this target must be finite and nonnegative at all fitting points; otherwise neither a square root nor an importance weight is meaningful.

37.4 Tensor-Train Approximation Toolkit

A tensor train is a compressed representation of a multidimensional array or function. For a grid tensor $A(i_1, \dots, i_D)$, the TT format writes

$$A(i_1, \dots, i_D) = G_1(i_1) G_2(i_2) \cdots G_D(i_D), \quad (37.1)$$

where the $G_k(i_k)$ are small matrices with connecting ranks. If the mode size is M and ranks are bounded by r , storage is $O(DMr^2)$ instead of $O(M^D)$. The key condition is rank stability: if ranks grow with the effective dimension, the compression disappears.

The chapter uses this toolkit for filtering objects, not for abstract storage arithmetic alone. The object being approximated may be a density, an unnormalized adjacent-state target, a retained marginal, or a conditional map. The approximation only helps if the represented object remains interpretable as a probability or retained filtering quantity after truncation, marginalization, and normalization.

37.4.1 Rank-Count Plausibility

The method is plausible when the posterior geometry has limited interaction width after ordering and preconditioning. If every coordinate uses p basis functions and the function has D coordinates, a full tensor stores p^D coefficients, while a TT with moderate ranks stores approximately DpR^2 coefficients. This is the theorem-to-implementation bridge: low-rank structure is not decoration, but the empirical condition under which the method escapes the full-grid curse of dimensionality.

Rank diagnostics therefore belong to the method contract. If fitted ranks grow persistently toward the cap or if conditioning deteriorates at the same time, then the chosen ordering, basis, or preconditioner is not expressing the target compactly.

37.5 Coordinate Systems, Conditional Maps, And Retained Objects

The retained-object lane uses several coordinate systems because different operations are easiest in different spaces. Let

$$r = (x_t, \alpha, x_{t-1}) \quad (37.2)$$

denote physical filtering coordinates. A domain map sends a reference cube coordinate $z \in [-1, 1]^D$ to physical coordinates:

$$r = \Psi_t(z), \quad J_{\Psi,t}(z) = |\det \nabla \Psi_t(z)|. \quad (37.3)$$

The target seen by the TT in reference coordinates is

$$\tilde{q}_t(z) = q_t(\Psi_t(z)) J_{\Psi,t}(z). \quad (37.4)$$

Thus z is the coordinate in which basis functions, fitting points, and mass matrices are declared.

Preconditioning introduces another coordinate u . Let T_t be an invertible map from physical coordinates to a preconditioned coordinate:

$$u = T_t(r), \quad r = T_t^{-1}(u). \quad (37.5)$$

If $\eta(u)$ is the reference density used in u -space, then

$$q_t(r) dr = q_t(T_t^{-1}(u)) |\det \nabla T_t^{-1}(u)| du. \quad (37.6)$$

The retained coordinate z_t is the part of the reference coordinate retained after marginalizing the previous state:

$$z = (z_t, z_{t-1}), \quad a_t(z_t) = \int \hat{q}_t(z_t, z_{t-1}) dz_{t-1}, \quad (37.7)$$

and

$$\hat{p}_t(z_t) = \frac{a_t(z_t)}{\hat{Z}_t}. \quad (37.8)$$

Most implementation mistakes in this lane are coordinate mistakes: using a physical density in reference coordinates without the Jacobian, applying a KR map in u -space to a point still in r -space, or carrying a retained filter in z_t while evaluating it as if it were already a physical density.

37.5.1 One Observation Step Through All Coordinate Systems

The symbols r, z, u, z_t become easier to track when one observation update is written as a single chain. Begin in physical coordinates

$$r = (x_t, \alpha, x_{t-1}), \quad q_t(r) = \hat{p}_{t-1}(x_{t-1}, \alpha) f_\alpha(x_t | x_{t-1}) g_\alpha(y_t | x_t). \quad (37.9)$$

A domain map

$$r = \Psi_t(z), \quad z \in [-1, 1]^D, \quad J_{\Psi,t}(z) = |\det \nabla \Psi_t(z)| \quad (37.10)$$

turns the physical density into a reference-coordinate density

$$\tilde{q}_t(z) = q_t(\Psi_t(z)) J_{\Psi,t}(z). \quad (37.11)$$

The squared-TT route fits a source-scale square root in reference coordinates and then returns to the unshifted evidence scale through

$$q_{t,\text{src}}(z) = \phi_t(z)^2 + \tau_t \lambda_t(z), \quad \hat{q}_t(z) = e^{-c_t} q_{t,\text{src}}(z). \quad (37.12)$$

The carried filter is not the full joint object. If $z = (z_t, z_{t-1})$, then

$$a_t(z_t) = \int \hat{q}_t(z_t, z_{t-1}) dz_{t-1}, \quad \hat{p}_t^{\text{ref}}(z_t) = \frac{a_t(z_t)}{\hat{Z}_t}. \quad (37.13)$$

If a later formula needs the filter as a physical density, write

$$(x_t, \alpha) = \Psi_{t,\text{ret}}(z_t), \quad \hat{p}_t^{\text{phys}}(x_t, \alpha) = \hat{p}_t^{\text{ref}}(\Psi_{t,\text{ret}}^{-1}(x_t, \alpha)) |\det \nabla \Psi_{t,\text{ret}}^{-1}(x_t, \alpha)|. \quad (37.14)$$

The bookkeeping rule is: $\tilde{q}_t(z)$, $a_t(z_t)$, $\hat{Z}_t$, and $\hat{p}_t^{\text{ref}}(z_t)$ are reference-coordinate objects integrated with dz ; $\hat{p}_t^{\text{phys}}$ is the corresponding physical object after the declared Jacobian correction.

Preconditioning inserts one additional coordinate u . A bridge density $\rho_t(r)$ and map $T_t : r \mapsto u$ are chosen so that the dominant shape of q_t is easier in u -space. The residual target is

$$q_t^\sharp(u) = \frac{q_t(T_t^{-1}(u))}{\hat{\rho}_t(T_t^{-1}(u))} \eta(u), \quad (37.15)$$

and the pullback relation after fitting is

$$\hat{q}_t(r) = \frac{\hat{\nu}_t^\sharp(T_t(r))}{\eta(T_t(r))} \hat{\rho}_t(r). \quad (37.16)$$

Thus the full coordinate story of one observation step is

$$q_t(r) \rightarrow \tilde{q}_t(z) \rightarrow \hat{q}_t(z) \rightarrow \hat{p}_t^{\text{ref}}(z_t) \rightarrow \tilde{q}_{t+1}(z_{t+1}, z_t), \quad (37.17)$$

with the preconditioned route adding

$$q_t(r) \rightarrow q_t^\sharp(u) \rightarrow \hat{\nu}_t^\sharp(u) \rightarrow \hat{q}_t(r). \quad (37.18)$$

The filtering task does not change; only the coordinate system in which the hard part of the density is approximated changes.

37.6 TT Sequential Learning And Conditional KR Transports

Zhao and Cui [2024] is the central discrete-time TT source for this lane. Its checked role here is direct: the chapter uses the paper’s sequential state-and-parameter learning setup, its posterior-density approximation view, and its conditional Knothe–Rosenblatt map construction from a TT approximation.

A nonnegative TT approximation supplies conditional density factors or conditional cumulative distribution functions. A triangular conditional map can then be constructed from one-dimensional conditional CDFs, which is the KR transport idea in operational form. For BayesFilter this is attractive because it connects low-rank density approximation to transport preconditioning. It remains conditional because normalization, monotonicity, rank error, and Jacobian stability are part of the method, not afterthoughts.

37.7 What This Chapter Exports Forward

At the end of this front-half chapter, the retained-object lane has fixed the objects that later operational chapters are allowed to manipulate:

1. the adjacent-state target and its coordinate systems,
2. the TT approximation toolkit and rank-count plausibility logic,
3. the retained numerator a_t and normalized carried filter $\hat{p}_t$,
4. the KR / preconditioning map perspective,
5. the core diagnostics that decide whether the retained object remains scientifically meaningful.

The next chapter, Chapter 38, then takes up the operational middle: squared-TT recursion, fixed-branch likelihood construction, stored fields, and the exact scalar object exported to the same-scalar derivative chapter.

37.8 Tensor-Network Kalman And Square-Root Caution

Tensor-network covariance compression is included here only as a cautionary adjacent lane. If a covariance-like object is compressed, the method must expose either a PSD reconstruction check or a factor/square-root representation whose product is positive semidefinite by construction. This material remains secondary to the retained TT/KR density lane; it is kept because it clarifies one important boundary between density compression and covariance compression.

37.9 Complexity, Diagnostics, And Exports

The retained-object lane exports five primary diagnostics:

- (1) rank growth,
- (2) normalization/mass error,
- (3) positivity / nonnegativity construction,
- (4) support / Jacobian / map validity,
- (5) retained-object consistency for the next time step.

These diagnostics are not side comments; they are the mechanism by which the chapter decides whether the retained object remains scientifically meaningful.

Exported forward. The next operational chapter may import only the retained-object scalar ingredients and branch-defining structural choices that this chapter makes explicit. The later validation/promotions chapter may import only the defects, diagnostics, promotion gates, and source-risk boundaries stated here. Neither later chapter should silently strengthen TT/KR transport existence, branch-local differentiability, or retained-object feasibility beyond what the checked sources support.

37.10 Methodological Boundary And Sources

The substantive claims in this chapter are tied to TT, KR, transport, and adjacent tensor-network sources where their checked constructions are used. Savostyanov maxvol theory and Knothe’s original rearrangement source were not inspected directly here; the chapter therefore uses Oseledets–Tyrtysnikov for TT-cross and Rosenblatt plus modern transport-filtering papers for triangular maps. No source here is used to claim production readiness, NAWM readiness, posterior accuracy, tensor-method validation, transport-method validation, HMC convergence, GPU/XLA readiness, or default readiness.

Chapter 38

Squared-TT Recursion and Fixed-Branch Likelihood Construction

38.1 Purpose And Scope

The previous chapter fixed the retained-object front half of the non-Gaussian lane: the exact adjacent-state target, the TT approximation toolkit, the coordinate systems, the KR/preconditioning map logic, and the meaning of the retained marginal a_t and normalized carried filter $\hat{p}_t$. Those objects answer what the method is trying to represent. The present chapter answers the operational question that follows: once the branch has committed to a finite-dimensional TT/KR construction, what exact approximate likelihood object is produced, what must be saved for the next step, and which numerical fields are part of the scalar identity rather than merely transient implementation detail?

This chapter is therefore the operational middle of the p50 lane. It is not yet the full derivative chapter. It builds the squared-TT recursion, the retained marginal and normalizer, the fixed least-squares branch specialization, and the stored-field contract that later allows the shared same-scalar gradient chapter to differentiate one declared scalar rather than an adaptively changing routine.

38.2 Squared-TT Filtering Object And Recursive Closure

The retained-object lane must answer two questions at the same time:

$$\text{How do we represent } q_t? \quad \text{How do we integrate the represented object?} \quad (38.1)$$

The exact adjacent-state target is a many-variable function rather than a finite-dimensional Gaussian summary. In the TT/KR lane, the target is first moved into reference coordinates and then approximated by a nonnegative low-rank object, typically through a square or squared-density representation.

Write the represented adjacent-state object in retained coordinates as

$$\hat{q}_t(z_t, z_{t-1}), \quad (38.2)$$

and define the retained numerator and normalizer by

$$a_t(z_t) = \int \widehat{q}_t(z_t, z_{t-1}) dz_{t-1}, \quad \widehat{Z}_t = \int a_t(z_t) dz_t. \quad (38.3)$$

The carried filter is then

$$\widehat{p}_t(z_t) = \frac{a_t(z_t)}{\widehat{Z}_t}. \quad (38.4)$$

This is the recursive closure the lane must preserve: a represented adjacent object, a retained numerator, a normalizer, and a normalized carried filter that can be queried at the next time step. Saving only samples, only a low-rank core without normalization metadata, or only $\widehat{Z}_t$ is not enough to close the same branch recursively.

38.3 What Must Be Saved For The Next Time Step

The next subtlety is what must survive from one time step to the next. If too little is stored, the next target cannot be reconstructed on the same branch. If the wrong object is stored, the implementation may look internally coherent while no longer evaluating the same scalar as the fixed branch.

For a fixed-branch likelihood implementation, the forward object that must be available at time $t + 1$ is not merely a cloud or a normalizer. It is the retained filter evaluator and the ingredients required to differentiate that retained filter on the same branch. A useful stored-field contract is

$$\mathcal{F}_t = \{C_{t,k}, M_{t,>k}, M_{t,<k}, \widehat{Z}_t, a_t, \widehat{p}_t, \Psi_t, \lambda_t, \tau_t\}_{k=1}^D, \quad (38.5)$$

with the derivative chapter later augmenting it by the corresponding dotted objects.

For the derivative-aware fixed branch, the stronger stored-field contract is

$$\dot{\mathcal{F}}_t = \{\dot{C}_{t,k}^{(i)}, \dot{M}_{t,>k}^{(i)}, \dot{M}_{t,<k}^{(i)}, \dot{\widehat{Z}}_t^{(i)}, \dot{a}_t^{(i)}, \dot{\widehat{p}}_t^{(i)}\}_{k=1}^D, \quad (38.6)$$

so that the next transformed target may be evaluated together with its product-rule sensitivity on the same branch. The carried-filter derivative is the quotient object

$$\partial_{\beta_i} \widehat{p}_t(x_t; \beta) = \frac{\partial_{\beta_i} a_t(x_t; \beta)}{\widehat{Z}_t(\beta)} - \frac{a_t(x_t; \beta) \partial_{\beta_i} \widehat{Z}_t(\beta)}{\widehat{Z}_t(\beta)^2}, \quad (38.7)$$

which is precisely the object needed by the next-step fixed-branch derivative pass.

The point is conceptual before it is algorithmic: the next-step target depends on evaluating the previous carried filter at new fitting points. Therefore the retained-object lane must store the represented numerator/filter object in a form that the next-step evaluator can query without rebuilding an adaptive branch. This is what makes the fixed-branch recursion meaningfully recursive rather than a sequence of disconnected local fits.

38.4 Fixed-Branch Likelihood Construction

The adaptive Zhao–Cui algorithm contains discrete choices—ranks, fitting points, sweep decisions, domain maps, and stabilization choices—that should not be treated as globally

differentiable with respect to the scientific parameter. The fixed-branch specialization freezes those discrete choices first, then asks what scalar is defined when the corresponding numerical equations are recomputed on that same frozen structure.

The fixed branch is therefore more than a density formula. At time t , the finite-dimensional branch data may be summarized as

$$\mathcal{B}_t = \{\Omega_{t,k}, \Psi_{t,k}, \psi_{t,k,\ell}, C_{t,k}, R_{t,k}, B_{t,k}, \lambda_{t,k}, \tau_t, c_t, \text{mass contractions, map data}\}. \quad (38.8)$$

Changing domains, bases, rank tuples, TT cores, reference densities, defensive mass conventions, or log-scale shifts changes the approximate likelihood. Those objects are part of the scalar identity, not optional implementation metadata.

38.4.1 What A Core Object Looks Like

For coordinate k , store

$$C_k \in \mathbb{R}^{R_{k-1} \times p_k \times R_k}. \quad (38.9)$$

Given a scalar coordinate value r_k , first compute the basis vector

$$b_k(r_k) = (\psi_{k,0}(r_k), \dots, \psi_{k,p_k-1}(r_k))^\top. \quad (38.10)$$

Then form the matrix slice

$$H_k(r_k)[a, b] = \sum_{\ell=0}^{p_k-1} C_k[a, \ell, b] b_k(r_k)[\ell]. \quad (38.11)$$

Evaluation of the TT is the loop

$$v_0 = 1, \quad v_k = v_{k-1} H_k(r_k), \quad k = 1, \dots, D, \quad \widehat{h}(r) = v_D. \quad (38.12)$$

The square-density object additionally stores mass contractions

$$M_{>k} = \int H_{k+1}(r_{k+1}) \cdots H_D(r_D) H_D(r_D)^\top \cdots H_{k+1}(r_{k+1})^\top dr_{k+1:D}, \quad (38.13)$$

To make one retained-coordinate marginal explicit, suppose the retained state is stored as the last coordinate $z_D = z_t$ and the previous-state coordinates are $z_1, \dots, z_{D-1}$. Then the reverse-order marginal is represented by the left-side contractions

$$M_{<0} = 1, \quad M_{<j}[b, b'] = \sum_{a, a', \ell, \ell'} M_{<j-1}[a, a'] C_j[a, \ell, b] C_j[a', \ell', b'] B_j[\ell, \ell'], \quad j = 1, \dots, D-1, \quad (38.14)$$

where B_j is the frozen basis mass matrix. The retained numerator is then the finite-dimensional contraction

$$\begin{aligned} a_t(z_D; \beta) &= e^{-c_t} \sum_{b, b', \ell, \ell'} M_{<D-1}[b, b'] C_D[b, \ell, 1] C_D[b', \ell', 1] b_D(z_D)[\ell] b_D(z_D)[\ell'] \\ &\quad + \tau_t \int \lambda_t(z_D, z_{1:D-1}) dz_{1:D-1}, \end{aligned} \quad (38.15)$$

which is the saved carried object queried by the next step. Its derivative on the same frozen branch is obtained by the dotted left-side recursion

$$\begin{aligned}\dot{M}_{<j}[b, b'] &= \sum_{a, a', \ell, \ell'} \dot{M}_{<j-1}[a, a'] C_j[a, \ell, b] C_j[a', \ell', b'] B_j[\ell, \ell'] \\ &+ \sum_{a, a', \ell, \ell'} M_{<j-1}[a, a'] \dot{C}_j[a, \ell, b] C_j[a', \ell', b'] B_j[\ell, \ell'] \\ &+ \sum_{a, a', \ell, \ell'} M_{<j-1}[a, a'] C_j[a, \ell, b] \dot{C}_j[a', \ell', b'] B_j[\ell, \ell'],\end{aligned}\quad (38.16)$$

starting from $\dot{M}_{<0} = 0$, and the resulting retained-numerator derivative

$$\begin{aligned}\dot{a}_t(z_D; \beta) &= e^{-c_t} \sum_{b, b', \ell, \ell'} \dot{M}_{<D-1}[b, b'] C_D[b, \ell, 1] C_D[b', \ell', 1] b_D(z_D)[\ell] b_D(z_D)[\ell'] \\ &+ e^{-c_t} \sum_{b, b', \ell, \ell'} M_{<D-1}[b, b'] \dot{C}_D[b, \ell, 1] C_D[b', \ell', 1] b_D(z_D)[\ell] b_D(z_D)[\ell'] \\ &+ e^{-c_t} \sum_{b, b', \ell, \ell'} M_{<D-1}[b, b'] C_D[b, \ell, 1] \dot{C}_D[b', \ell', 1] b_D(z_D)[\ell] b_D(z_D)[\ell'].\end{aligned}\quad (38.17)$$

These are not decorative arrays; together with the right-side contractions they are the finite-dimensional objects through which marginals, normalizers, retained evaluators, and later same-branch derivatives are actually computed.

The resulting fixed-branch scalar has the form

$$\widehat{\ell}_T^{\text{TT/KR}}(\beta; B) = \sum_{t=1}^T \log \widehat{Z}_t(\beta; B_t), \quad (38.18)$$

where the adaptive selection procedure is no longer being differentiated. What is being differentiated later is the scalar induced by recomputing the fitted numerical objects on this saved branch.

38.5 Consolidated Fixed Least-Squares Formulation

The implementable fixed-branch route in BayesFilter should be stated narrowly. It is not adaptive TT-cross. It is a fixed least-squares specialization with fixed points, fixed ranks, fixed sweep order, and deterministic stabilization. In symbols,

fixed-branch TT likelihood route = fixed points+fixed ranks+fixed sweep order+deterministic least squares (38.19)

This narrower statement matters because it isolates the branch whose value and later derivative can be audited. If the implementation silently switches pivot sets, changes rank caps, or changes stabilization rules under perturbation, then it has left the fixed-branch scalar described here.

A representative one-step input/output contract is:

$$\mathcal{I}_t = \{y_t, \beta, \widehat{p}_{t-1}, f_\beta, g_\beta, \Psi_t, Z_{\text{fit}}^{(t)}, W^{(t)}, \text{basis, ranks, sweeps, } \rho_t, c_t, \tau_t\}, \quad (38.20)$$

$$\mathcal{O}_t = \{C_{t,k}, M_{t,>k}, M_{t,<k}, \widehat{Z}_t, a_t, \widehat{p}_t\}_{k=1}^D. \quad (38.21)$$

The important point is not the exact symbol list but the discipline: the branch must state what is input, what is output, and which outputs are required to close the next-step recursion on the same branch.

A concrete shape contract helps keep this branch auditable. One useful table is

object	shape
C_k	$R_{k-1} \times p_k \times R_k$
$L_{j,k}$	$1 \times R_{k-1}$
$R_{j,k}$	$R_k \times 1$
A_k	$n_{\text{fit}} \times (R_{k-1} p_k R_k)$
$g_k = \text{vec}(C_k), d_k$	$R_{k-1} p_k R_k$
N_k	$(R_{k-1} p_k R_k) \times (R_{k-1} p_k R_k)$

(38.22)

with vectorization convention

$$\iota_k(a, \ell, b) = (a - 1)p_k R_k + \ell R_k + b, \quad g_k[\iota_k(a, \ell, b)] = C_k[a, \ell, b]. \quad (38.23)$$

This is the level at which the later derivative chapter can say, concretely, what it means to recompute the same least-squares solve rather than copying a stale core from the base parameter.

38.5.1 One Concrete Branch, Fully Instantiated

A branch record is easier to audit when one concrete instantiation is stated. For a declared run, that record should pin down:

1. coordinate order and domain policy,
2. basis family and basis degree in each coordinate,
3. deterministic fitting points and weights,
4. rank vector and deterministic rank ladder before branch freezing,
5. sweep order and number of sweeps,
6. defensive reference density, defensive mass, and any frozen shift convention,
7. the boxed convention that the fit is performed in source scale while the reported scalar returns to the unshifted evidence scale.

This list is not busywork. If one of these items changes under perturbation, then the implementation has left the fixed-branch scalar even if all later notation still looks similar.

A useful first mathematical instantiation is intentionally narrow. For a scalar state, use the two-block coordinate order

$$r_t = (x_t, x_{t-1}) \in \mathbb{R}^2. \quad (38.24)$$

For vector states, concatenate coordinates as

$$r_t = (x_{t,1}, \dots, x_{t,m_x}, x_{t-1,1}, \dots, x_{t-1,m_x}). \quad (38.25)$$

Choose a fixed interval for each coordinate,

$$r_{t,k} \in [\ell_{t,k}, u_{t,k}], \quad r_{t,k} = a_{t,k} + h_{t,k} z_k, \quad a_{t,k} = \frac{u_{t,k} + \ell_{t,k}}{2}, \quad h_{t,k} = \frac{u_{t,k} - \ell_{t,k}}{2}, \quad (38.26)$$

so the Jacobian is constant,

$$J_t = \prod_{k=1}^D h_{t,k}. \quad (38.27)$$

The branch stores $\ell_{t,k}, u_{t,k}, a_{t,k}, h_{t,k}$, and these remain fixed through same-branch finite differences.

A deterministic domain-selection policy should be declared before branch freezing. Let a pilot location and scale be

$$\mu_{t,k}^{\text{pilot}}, \quad s_{t,k}^{\text{pilot}} > 0, \quad (38.28)$$

and set

$$[\ell_{t,k}, u_{t,k}] = [\mu_{t,k}^{\text{pilot}} - \gamma s_{t,k}^{\text{pilot}}, \mu_{t,k}^{\text{pilot}} + \gamma s_{t,k}^{\text{pilot}}], \quad (38.29)$$

with fixed expansion factor γ . In the current fixed-variant fitting story, these pilot moments may come from a Gaussian scout such as a UKF warm start. The scout provides a local frame and an initialization region; it is not a certificate that the represented density is already correct. The out-of-domain diagnostic at fitting or proposal points is

$$\delta_{\text{out}} = \frac{1}{N} \sum_{j=1}^N \mathbf{1}\{r_{t,k}^{(j)} \notin [\ell_{t,k}, u_{t,k}] \text{ for some } k\}, \quad (38.30)$$

and the branch is accepted only if

$$\delta_{\text{out}} \leq \delta_{\text{max}}. \quad (38.31)$$

If this fails, the branch must be rebuilt before differentiation rather than expanded during a same-branch derivative check.

Use one declared basis family and fixed point design. A useful first choice is normalized Legendre basis of common degree p in every coordinate together with deterministic Halton-type fitting points and equal weights. The rank vector is then selected by a small deterministic ladder before branch freezing, for example

$$\mathcal{R}_{\text{ladder}} = \{(1, \dots, 1), (2, \dots, 2), (4, \dots, 4), (6, \dots, 6), (8, \dots, 8)\}, \quad (38.32)$$

with the first candidate passing the declared residual, conditioning, and defensive-mass tests becoming the frozen branch rank. If no candidate passes, the correct outcome is branch failure, not an unrecorded adaptive rank change.

The defensive reference and shift convention should also be fixed explicitly. A minimal deterministic choice is uniform reference density on $[-1, 1]^D$, a source-scale floor $\epsilon_{\text{floor}} > 0$, defensive mass $\tau_t = 2^D \epsilon_{\text{floor}}$, and shift c_t computed once at the base parameter and then reused for same-branch finite differences. This is the meaning of the chapter's boxed convention that the fit is carried out in source scale while the reported scalar returns to the unshifted evidence scale.

38.5.2 Square-Root Fitting Object Versus Density Audit Object

The fixed-variant fitting lane needs one further distinction. The TT sweep may be written in square-root form, but the scientific object being evaluated is a density. Write the represented nonnegative object as

$$q_\theta(z) = h_\theta(z)^2 + \tau q_0(z), \quad Z_\theta = \int q_\theta(z) dz, \quad p_\theta(z) = \frac{q_\theta(z)}{Z_\theta}, \quad (38.33)$$

where h_θ is the trainable square-root TT field, q_0 is the fixed defensive reference density, and $\tau > 0$ is the defensive mixture weight. If the target square-root field is $s(z)$, then the target density is

$$p_\star(z) \propto s(z)^2. \quad (38.34)$$

Therefore a raw square-root residual,

$$h_\theta(z_i) \approx s(z_i), \quad (38.35)$$

is not yet the primary heldout criterion. The reported density depends on the square, the defensive component, and the normalizer Z_θ . Sign, scale, and local amplitude errors in h_θ need not map one-for-one to density quality after normalization. Square-root residuals are therefore fitting diagnostics, not promotion criteria by themselves.

For a weighted batch

$$B = \{(z_i, w_i, s_i)\}_{i=1}^n, \quad (38.36)$$

the target-only empirical measure is

$$\alpha_i^{\text{hold}} = \frac{w_i s_i^2}{\sum_j w_j s_j^2}, \quad (38.37)$$

and the corresponding density cross-entropy is

$$\hat{\mathcal{L}}_{\text{hold}}(\theta) = - \sum_i \alpha_i^{\text{hold}} \log q_\theta(z_i) + \log Z_\theta. \quad (38.38)$$

This target-only measure is the corrected heldout audit object for the fixed-variant lane. The historical helper

$$\alpha_i^{\text{old}} \propto w_i (s_i^2 + \tau q_0(z_i)) \quad (38.39)$$

mixes the defensive reference density into the empirical target. That helper may still be useful as a boundary comparator or internal training-support convention, but it is not the heldout target measure and should not be reported as if it were the scientific density criterion.

This correction also changes how fitting evidence should be narrated. A UKF warm start can supply local geometry and a mini-batch training run can improve the represented density over that geometry, but success is judged by the target-only heldout cross-entropy in (38.38), not by the old τq_0 -smoothed helper and not by square-root residuals alone.

38.6 Initialization And Sweep Order

The branch must also save the deterministic initialization and sweep rule. A constant-channel initialization is a useful mechanical baseline, but it should not be confused with the preferred fixed-variant fitting narrative once a deterministic geometric warm start is available. When a Gaussian scout such as a UKF provides the pilot frame, the current mathematical story is that the branch record should freeze a warm start derived from that scout rather than revert to random or purely constant initialization. Let one bidirectional sweep mean updates in the order

$$1, 2, \dots, D, D-1, \dots, 1. \quad (38.40)$$

Fix the number of bidirectional sweeps S . The exact deterministic initialization rule matters less than saving it in the branch record, because changing initialization changes the fixed fitting map.

38.6.1 Forward Core Update

At fitting point $z^{(j)}$, define environments for core k by

$$L_{j,k} = H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)}), \quad R_{j,k} = H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)}). \quad (38.41)$$

For endpoint ranks this means $L_{j,k} \in \mathbb{R}^{1 \times R_{k-1}}$ and $R_{j,k} \in \mathbb{R}^{R_k \times 1}$. The design row for coefficient (a, ℓ, b) is

$$A_k[j, (a, \ell, b)] = L_{j,k}[1, a] b_k(z_k^{(j)})[\ell] R_{j,k}[b, 1]. \quad (38.42)$$

The local ridge system is

$$N_k g_k = d_k, \quad N_k = A_k^\top W A_k + \rho I, \quad d_k = A_k^\top W y. \quad (38.43)$$

After solving (38.43), reshape g_k back into the core by

$$C_k[a, \ell, b] = g_k[(a, \ell, b)]. \quad (38.44)$$

This is the forward fitting rule whose identity later same-branch finite differences must preserve.

38.6.2 Derivative Core Update

The derivative environments are obtained by differentiating the matrix products:

$$\dot{L}_{j,k} = \sum_{r=1}^{k-1} H_1 \cdots H_{r-1} \dot{H}_r H_{r+1} \cdots H_{k-1}, \quad (38.45)$$

$$\dot{R}_{j,k} = \sum_{r=k+1}^D H_{k+1} \cdots H_{r-1} \dot{H}_r H_{r+1} \cdots H_D. \quad (38.46)$$

Because basis functions and fitting points are frozen, the derivative design row contains only environment terms:

$$\begin{aligned} \dot{A}_k[j, (a, \ell, b)] &= \dot{L}_{j,k}[1, a] b_k(z_k^{(j)})[\ell] R_{j,k}[b, 1] \\ &\quad + L_{j,k}[1, a] b_k(z_k^{(j)})[\ell] \dot{R}_{j,k}[b, 1]. \end{aligned} \quad (38.47)$$

The derivative normal equations are

$$\dot{N}_k = \dot{A}_k^\top W A_k + A_k^\top W \dot{A}_k, \quad \dot{d}_k = \dot{A}_k^\top W y + A_k^\top W \dot{y}, \quad (38.48)$$

and differentiating $N_k g_k = d_k$ gives

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k. \quad (38.49)$$

After solving (38.49), reshape

$$\dot{C}_k[a, \ell, b] = \dot{g}_k[(a, \ell, b)]. \quad (38.50)$$

This is the derivative fitting rule. A later implementation must use this same stored computational path rather than inventing a separate approximate derivative solve.

38.6.3 Alternating-Sweep Recomputations

The symbols $L_{j,k}$, $R_{j,k}$, A_k , N_k , and d_k are not static arrays during an alternating sweep. They are recomputed from the current cores at the moment a core is updated. Let

$$H_{j,k} = H_k(z_k^{(j)}; C_k), \quad \dot{H}_{j,k} = H_k(z_k^{(j)}; \dot{C}_k), \quad (38.51)$$

where basis values and fitting points are fixed and only the coefficients change. For a current core collection C^{cur} , define prefix and suffix products by

$$L_{j,1}^{\text{cur}} = 1, \quad L_{j,k}^{\text{cur}} = L_{j,k-1}^{\text{cur}} H_{j,k-1}^{\text{cur}}, \quad k = 2, \dots, D, \quad (38.52)$$

and

$$R_{j,D}^{\text{cur}} = 1, \quad R_{j,k}^{\text{cur}} = H_{j,k+1}^{\text{cur}} R_{j,k+1}^{\text{cur}}, \quad k = D-1, \dots, 1. \quad (38.53)$$

The dotted prefix and suffix products follow the same recursions with one product-rule term added. At a core update k , build the current design matrix and normal equations from the current environments, solve for the new core, then build the dotted objects and solve the derivative system. After updating one core, every cached prefix with index $\ell > k$ and every cached suffix with index $\ell < k$ is stale and must be recomputed before use.

This stale-cache rule is part of the branch definition. It is what makes the phrase “the same fitting rule” operational rather than rhetorical.

38.6.4 Stopping Rule And Failure Exits

After each full sweep compute

$$\varepsilon_s = \frac{\|A(C^{(s)}) - y\|_W}{\max\{\|y\|_W, \epsilon_{\text{abs}}\}}, \quad \Delta_s = \frac{\|C^{(s)} - C^{(s-1)}\|}{\max\{\|C^{(s-1)}\|, \epsilon_{\text{abs}}\}}, \quad (38.54)$$

The sweep stops successfully only if

$$\varepsilon_s \leq \epsilon_{\text{fit}} \quad \text{and} \quad \Delta_s \leq \epsilon_{\text{sweep}}. \quad (38.55)$$

If this does not happen by S sweeps, the branch reports sweep failure rather than silently accepting a different scalar. The full accepted/failure path must therefore distinguish at least the following exits:

Failure condition	Mathematical test	Report field
Domain miss	$\delta_{\text{out}} > \delta_{\text{max}}$	domain failure
Rank ladder fail	no R satisfies residual/conditioning/floor tests	rank failure
Ill-conditioned solve	$\text{cond}(N_k) > \kappa_{\text{max}}$	solve failure
Sweep stagnation	$\varepsilon_s > \epsilon_{\text{fit}}$ or $\Delta_s > \epsilon_{\text{sweep}}$ after S sweeps	sweep failure
Nonpositive target for smooth derivative	$\min_j \tilde{q}_{t,j} \leq 0$ without declared floor	target failure
Mass failure	$\hat{Z}_t \leq 0$ or not finite	normalizer failure
Finite-difference branch mismatch	frozen fields or accepted/failure path differ across $\beta - h, \beta, \beta + h$	branch mismatch

A failed branch is not a successful derivative with a warning attached; it is a declared failure of the fixed scalar. This accepted/failure vocabulary is what staged *ch37* later consumes when it says a branch-valid FD row requires the same realized acceptance/failure path on both sides of the perturbation.

38.7 One Observation Step Through The Stored Objects

A reader should be able to narrate one time step in the following order:

1. evaluate the previous carried filter on the current branch's fitting points,
2. combine it with the transition and observation densities to form the adjacent-state target in the declared coordinates,
3. fit the squared-TT / low-rank object on the fixed branch,
4. contract out the previous-state block to form a_t ,
5. compute $\hat{Z}_t$ and normalize to $\hat{p}_t$,
6. save the branch fields needed for the next-step forward evaluation and later same-branch derivative evaluation.

The dependency flow should be explicit enough to implement:

$$\begin{aligned}
\text{reference points} &\longrightarrow z^{(j)} \text{ from the fixed point design,} \\
\text{physical points} &\longrightarrow r^{(j)} = \Psi_t(z^{(j)}), \\
\text{target values} &\longrightarrow \tilde{q}_{t,j} = \hat{p}_{t-1}(x_{t-1}^{(j)}; \beta) f_\beta(x_t^{(j)} | x_{t-1}^{(j)}) g_\beta(y_t | x_t^{(j)}) J_t, \\
\text{square-root fit values} &\longrightarrow y_j = e^{c_t/2} \sqrt{\tilde{q}_{t,j}}, \\
\text{initial cores} &\longrightarrow \text{deterministic constant-channel initialization,} \\
\text{fixed sweep objects} &\longrightarrow L_{j,k}, R_{j,k}, A_k, N_k, d_k, C_k, \\
\text{mass objects} &\longrightarrow M_{>k}, M_{<k}, R_t, \hat{Z}_t, \\
\text{carried objects} &\longrightarrow a_t, \hat{p}_t, \\
\text{saved objects for } t+1 &\longrightarrow \mathcal{F}_t \text{ and, when needed, } \dot{\mathcal{F}}_t.
\end{aligned} \tag{38.56}$$

The forward environments are recomputed from the current cores during each sweep; they are not static arrays. For fitting point $z^{(j)}$, the environments are

$$L_{j,k} = H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)}), \quad R_{j,k} = H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)}), \quad (38.57)$$

and the local ridge system is built from those current environments. After a core update, cached prefixes and suffixes on the wrong side of that core are stale and must be recomputed before use. That bookkeeping is part of the fixed fitting map.

The same principle governs next-step closure. The next transformed target is not formed from a vague “previous TT object,” but from the saved carried evaluator:

$$\tilde{q}_{t+1}(z; \beta) = \hat{p}_t(x_t(z); \beta) f_\beta(x_{t+1}(z) \mid x_t(z)) g_\beta(y_{t+1} \mid x_{t+1}(z)) J_{t+1}(z). \quad (38.58)$$

If the derivative lane is active, the saved dotted object from (38.6) is what makes the next-step product-rule derivative possible on the same branch. Without that saved object, a later derivative pass would be differentiating a different computational route.

This is the operational middle that the monograph must carry so that the later derivative chapter can start from a declared scalar object rather than a vague reference to “the TT filter.”

38.8 Exports To The Same-Scalar Derivative Chapter

This chapter exports the following objects forward:

1. the represented adjacent-state object and retained marginal recursion,
2. the stored-field contract for next-step closure,
3. the fixed-branch scalar $\hat{\ell}_T^{\text{TT/KR}}(\beta; B)$,
4. the fixed-branch least-squares specialization,
5. the lane-specific statement that domains, points, ranks, maps, stabilization rules, and stored-field conventions belong to the scalar identity.

The next chapter may now focus on derivative theory, branch-valid finite differences, and HMC-facing same-scalar discipline, because the present chapter has already fixed what scalar object is being differentiated.

Chapter 39

Fixed-Branch Approximate Likelihoods and Same-Scalar Gradients

39.1 Why Fixed-Branch Discipline Is Shared Across The Two Lanes

Both method chapters in the restarted high-dimensional block face the same structural problem. The exact Bayesian target is a filtering law and normalizer sequence, but the implemented approximation uses discrete numerical structure that may change with parameters: cloud level, merge policy, factor branch, coordinate domains, ranks, pivots, grids, fitting points, or map choices. If those structural choices move while the parameter moves, then the reported derivative is not automatically the derivative of one declared scalar.

This chapter therefore treats fixed-branch discipline as a cross-method target contract, not as an appendix to one particular algorithm.

39.2 A Canonical Branch Record

A branch record should be read as a three-ledger object,

$$\mathcal{H}(B) = \{\mathcal{H}_{\text{fixed}}, \mathcal{H}_{\text{diff}}, \mathcal{H}_{\text{diag}}\}, \quad (39.1)$$

where:

- $\mathcal{H}_{\text{fixed}}$ contains the structural choices that define the scalar on the saved branch,
- $\mathcal{H}_{\text{diff}}(\theta)$ contains the parameter-dependent numerical values recomputed on that branch,
- $\mathcal{H}_{\text{diag}}$ contains residuals, condition numbers, rank vectors, branch-failure markers, and other diagnostic quantities.

For the sparse-grid lane, the fixed ledger includes the cloud, one-dimensional rule family, merge/order policy, factor family, thresholds, preprocessing policy, and initial-condition policy. For the retained-object lane, the fixed ledger includes coordinate domains, basis

families, fitting points, weights, ranks, sweep order, shifts, stabilization choices, retained-object construction policy, and the defensive mass / reference-density conventions that define the same branch.

Two evaluations use the same branch if and only if their fixed ledgers match:

$$B(\theta_a) = B(\theta_b) \iff \mathcal{H}_{\text{fixed}}(\theta_a) = \mathcal{H}_{\text{fixed}}(\theta_b). \quad (39.2)$$

The differentiable ledger is allowed to change with the parameter; otherwise the same-scalar derivative would not be testing the actual fixed branch.

39.3 Fixed-Branch Approximate Likelihoods Across The Three Operational Layers

The restarted high-dimensional part now separates the operational burdens that the old compressed block used to mix together. The deterministic sparse-grid lane exports a declared fixed-cloud scalar from Chapter 36. The retained-object TT/KR lane exports a declared fixed-branch scalar from Chapter 38. The present chapter sits above those lane-specific constructions and states the common same-scalar derivative logic they must both obey.

The three operational layers are:

$$\begin{aligned} \text{exact target: } \ell_T(\theta) &= \sum_{t=1}^T \log Z_t(\theta), \\ \text{declared approximate scalar: } \widehat{\ell}_T(\theta; B), \\ \text{same-scalar derivative requirement: } \nabla_{\theta} \widehat{\ell}_T(\theta; B) &\text{ differentiates the same declared } \widehat{\ell}_T. \end{aligned} \quad (39.3)$$

For the sparse-grid lane, B contains the fixed cloud, merge/order policy, factor branch, and lane-specific veto rules. For the retained-object lane, B contains the fixed domains, fitting points, ranks, sweeps, shifts, stabilization choices, and stored-field conventions that define the TT/KR scalar. The chapter therefore treats fixed-branch discipline as a cross-method target contract, not as an appendix to one particular algorithm.

39.4 Finite-Difference Contract And Branch-Validity Logic

The finite-difference contract is

$$D_i(h) = \frac{\widehat{\ell}_T(\theta_0 + he_i; B) - \widehat{\ell}_T(\theta_0 - he_i; B)}{2h}, \quad (39.4)$$

with both perturbed evaluations reusing the same fixed ledger while recomputing all parameter-dependent numerical values through the same declared equations.

For the retained-object lane, this means the domains, points, ranks, shifts, and sweep order are unchanged, but the fitted core values

$$C_{t,k}(\theta_0 + he_i; B) \quad \text{and} \quad C_{t,k}(\theta_0 - he_i; B) \quad (39.5)$$

are recomputed from the same saved fitting rule. They are not copied from the base parameter. A row is branch-valid only if the two perturbations share:

- (i) the same saved fixed ledger, and
- (ii) the same realized acceptance/failure path through the declared branch logic.

If either side changes the accepted/failure pattern, the row is branch-invalid and is not evidence about the derivative of the same scalar.

Copied-core and branch-identity audit protocol. The operational diagnostic should promote these prose rules into explicit indicators. For a step ladder h_i , define

$$I_{\pm}(h_i) = \mathbf{1}\{\mathcal{H}_{\text{fixed}}(\theta_0 \pm h_i e_j) = \mathcal{H}_{\text{fixed}}(\theta_0)\}, \quad (39.6)$$

which checks branch-identity equality of the frozen fields, and

$$K_{\pm}(h_i) = \mathbf{1}\{C_{t,k}(\theta_0 \pm h_i e_j; B) \text{ was recomputed by the fixed fitting rule}\}, \quad (39.7)$$

which checks that perturbed cores were not copied from the base parameter. Record a decreasing-window indicator

$$W_i = \mathbf{1}\{e(h_i) > e(h_{i+1}) > e(h_{i+2})\}, \quad (39.8)$$

and classify the diagnostic status as one of:

$$\text{status} = \begin{cases} \text{branch identity failure,} & \min_{i,\pm} I_{\pm}(h_i) = 0, \\ \text{copied-core failure,} & \min_{i,\pm} K_{\pm}(h_i) = 0, \\ \text{decreasing-window agreement,} & \max_i W_i = 1, \\ \text{inconclusive: no decreasing window,} & \text{otherwise.} \end{cases} \quad (39.9)$$

This makes the same-branch contract auditable. If a frozen field differs, the centered difference is comparing different functions. If a differentiable field is copied rather than recomputed, the centered difference is not evaluating the fixed fitting rule.

39.5 Value Path, Derivative Path, And What Is Actually Differentiated

The implementation burden of a same-scalar chapter is not just to state that a gradient exists. It must say which objects belong to the value path and which objects belong to the derivative path. A generic fixed-branch workflow has the shape

$$\begin{aligned} \text{value path:} & \quad \text{frozen structure} \rightarrow \hat{Z}_t \rightarrow \hat{\ell}_T, \\ \text{derivative path:} & \quad \text{same frozen structure} \rightarrow \dot{\hat{Z}}_t \rightarrow \partial_i \hat{\ell}_T. \end{aligned} \quad (39.10)$$

For the sparse-grid lane this means differentiating the same fixed cloud and the same branch-conditioned Gaussian value path. For the retained-object lane it means differentiating the same retained-object fitting equations, mass contractions, quotient updates, and carried-filter update equations without changing domains, points, ranks, shifts, or sweep order.

The most important implementation distinction is between:

- objects needed to close the current scalar derivative, and
- objects needed only to propagate the branch-consistent state to the next step.

This distinction is what keeps a same-scalar derivative from being confused with a numerically similar but structurally different adaptive algorithm.

Retained-object derivative mechanics on the frozen branch. For the TT/KR lane, the forward and derivative paths should be read as the same finite sequence of fixed equations with different outputs. A useful teaching layer is:

Forward scalar path	Derivative path
$\tilde{q}_{t,j} = \hat{p}_{t-1,j} f_j g_j J_j$	$\dot{\tilde{q}}_{t,j} = J_j(\dot{\hat{p}}_{t-1,j} f_j g_j + \hat{p}_{t-1,j} \dot{f}_j g_j + \hat{p}_{t-1,j} f_j \dot{g}_j)$
$y_j = e^{c_t/2} \sqrt{\tilde{q}_{t,j}}$	$\dot{y}_j = e^{c_t/2} \dot{\tilde{q}}_{t,j} / (2\sqrt{\tilde{q}_{t,j}})$
$A_k \rightarrow N_k = A_k^\top W A_k + \rho I, \quad d_k = A_k^\top W y$	$\dot{A}_k \rightarrow \dot{N}_k = \dot{A}_k^\top W A_k + A_k^\top W \dot{A}_k, \quad \dot{d}_k = \dot{A}_k^\top W y + A_k^\top W \dot{y}$
$N_k g_k = d_k$	$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k$
$g_k \rightarrow C_k \rightarrow M_{>k} \rightarrow \hat{Z}_t$	$\dot{g}_k \rightarrow \dot{C}_k \rightarrow \dot{M}_{>k} \rightarrow \dot{\hat{Z}}_t$
$\hat{p}_t = a_t / \hat{Z}_t$	$\dot{\hat{p}}_t = \dot{a}_t / \hat{Z}_t - a_t \dot{\hat{Z}}_t / \hat{Z}_t^2$

(39.11)

The fixed linear solve is the key finite-dimensional derivative step:

$$N_k g_k = d_k \implies N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k. \quad (39.12)$$

Computation should solve (39.12) with the same numerical linear algebra used in the forward pass; it should not form an explicit inverse. After the core derivative is obtained, the remaining objects follow by repeated product-rule and quotient-rule propagation through the same frozen branch.

Mass-contraction and normalizer block. The squared-TT normalizer is not a separate conceptual object from the fitted cores; it is a contraction of those same cores against the frozen basis mass matrices. Let the right mass contraction satisfy a recursion of the form

$$M_{>j-1}[a, a'] = \sum_{b, b', \ell, \ell'} C_j[a, \ell, b] M_{>j}[b, b'] C_j[a', \ell', b'] B_j[\ell, \ell'], \quad (39.13)$$

with basis mass matrix B_j frozen by the branch. Differentiate it by the product rule:

$$\begin{aligned} \dot{M}_{>j-1}[a, a'] &= \sum_{b, b', \ell, \ell'} \dot{C}_j[a, \ell, b] M_{>j}[b, b'] C_j[a', \ell', b'] B_j[\ell, \ell'] \\ &\quad + \sum_{b, b', \ell, \ell'} C_j[a, \ell, b] \dot{M}_{>j}[b, b'] C_j[a', \ell', b'] B_j[\ell, \ell'] \\ &\quad + \sum_{b, b', \ell, \ell'} C_j[a, \ell, b] M_{>j}[b, b'] \dot{C}_j[a', \ell', b'] B_j[\ell, \ell']. \end{aligned} \quad (39.14)$$

Starting from the terminal contraction, this recursion produces the square mass R_t and its derivative $\dot{R}_t$. If

$$\zeta_t = R_t + \tau_t, \quad \hat{Z}_t = e^{-c_t} \zeta_t, \quad (39.15)$$

then, with c_t and τ_t frozen on the branch,

$$\dot{\hat{Z}}_t = e^{-c_t} \dot{R}_t. \quad (39.16)$$

This is the mass-contraction block the derivative must preserve. Omitting one of the product-rule terms in (39.14) means the reported gradient no longer differentiates the same scalar normalizer.

Carried-filter and next-step closure block. The retained numerator is another contraction of the same represented adjacent-state object. Write

$$a_t(z_t; \beta) = \int \hat{q}_t(z_t, z_{t-1}; \beta) dz_{t-1}, \quad \dot{a}_t(z_t; \beta) = \int \dot{\hat{q}}_t(z_t, z_{t-1}; \beta) dz_{t-1}. \quad (39.17)$$

The normalized carried filter then satisfies the quotient rule

$$\hat{p}_t(z_t; \beta) = \frac{\dot{a}_t(z_t; \beta)}{\hat{Z}_t(\beta)} - \frac{a_t(z_t; \beta) \dot{\hat{Z}}_t(\beta)}{\hat{Z}_t(\beta)^2}. \quad (39.18)$$

This is the crucial bridge between time steps. At time $t + 1$, the fixed-branch target contains $\hat{p}_t$, so its derivative contains $\hat{p}_t$. Without saving the carried-filter derivative, the next-step gradient is incomplete even if the current-step score is numerically correct.

Saved retained-filter object and query rule. The phrase “save the carried object” must denote a finite-dimensional evaluator, not an informal function name. For retained state dimension $m > 1$, let the retained block use the product basis

$$b_t(z_t) = b_{t,1}(z_{t,1}) \otimes \cdots \otimes b_{t,m}(z_{t,m}) \in \mathbb{R}^{p_{\text{ret}}}, \quad p_{\text{ret}} = \prod_{j=1}^m p_j. \quad (39.19)$$

The retained numerator is represented by a coefficient matrix

$$a_t(z_t; \beta) = b_t(z_t)^\top Q_t(\beta) b_t(z_t), \quad Q_t \in \mathbb{R}^{p_{\text{ret}} \times p_{\text{ret}}}, \quad (39.20)$$

and its derivative uses the matching dotted matrix

$$\dot{a}_t(z_t; \beta) = b_t(z_t)^\top \dot{Q}_t(\beta) b_t(z_t), \quad \dot{Q}_t \in \mathbb{R}^{p_{\text{ret}} \times p_{\text{ret}}}. \quad (39.21)$$

Normalize the stored evaluator by

$$P_t = \frac{Q_t}{\hat{Z}_t}, \quad \dot{P}_t = \frac{\dot{Q}_t}{\hat{Z}_t} - \frac{Q_t \dot{\hat{Z}}_t}{\hat{Z}_t^2}. \quad (39.22)$$

For next-step query points $z_t^{\text{query}}(j)$, build the retained-basis matrix

$$B^{\text{query}}[j, :] = b_t(z_t^{\text{query}}(j))^\top, \quad B^{\text{query}} \in \mathbb{R}^{M \times p_{\text{ret}}}, \quad (39.23)$$

and evaluate the saved carried object by

$$\hat{p}_t^{\text{ref}}[j] = B^{\text{query}}[j, :] P_t B^{\text{query}}[j, :]^\top, \quad \dot{\hat{p}}_t^{\text{ref}}[j] = B^{\text{query}}[j, :] \dot{P}_t B^{\text{query}}[j, :]^\top. \quad (39.24)$$

The next-step query rule is that $z_t^{\text{query}}(j)$ is the previous-state block of the fixed branch’s fitting point at time $t + 1$. A later implementation may store Q_t in TT, low-rank, or sparse form, but only if it preserves the same evaluator and derivative evaluator in (39.24). This is the stored-field convention that closes the derivative recursion on the same branch.

Branch-manifest reporting hook. Derivative and finite-difference reports for this lane should therefore carry a TT/KR branch-manifest identifier together with the saved-field convention, so that branch-valid rows certify equality of the actual retained evaluator rather than only superficial notation.

Compact derivative-reading checklists. A later implementation review should be able to verify the derivative lane block by block:

$$\begin{aligned}
\text{squared-density block: } & \bar{q}_t = e^{-c_t}(\phi_t^2 + \tau_t \lambda_t), \quad \partial(\phi_t^2) = 2\phi_t \dot{\phi}_t; \\
\text{mass-contraction block: } & R_t = \int \phi_t^2, \quad \dot{R}_t \text{ from product-rule contractions;} \\
\text{fixed-solve block: } & N_k g_k = d_k \mapsto N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k; \\
\text{carried-filter block: } & \hat{p}_t = a_t / \hat{Z}_t, \quad \dot{\hat{p}}_t = \dot{a}_t / \hat{Z}_t - a_t \dot{\hat{Z}}_t / \hat{Z}_t^2.
\end{aligned} \tag{39.25}$$

The corresponding failure-mode summaries are:

- **squared-density failure:** differentiating a different fitted square root than the one actually squared,
- **mass-contraction failure:** omitting a product-rule term in the contraction recursion,
- **fixed-solve failure:** changing the normal equations or copying a stale core at perturbed parameters,
- **carried-filter failure:** feeding time $t + 1$ without the saved dotted retained object or with an undeclared query rule.

These lists are not replacement derivations; they are compact audit objects that make it harder for a future implementation to silently drift away from the mathematical branch defined here.

39.6 Approximate Targets, Exact-Target Corrections, And HMC Admissibility

For an unconstrained coordinate $q \in \mathbb{R}^d$ and transform $\theta = \tau(q)$, the exact-target HMC potential is

$$U(q) = -\ell_T(\tau(q)) - \log \pi(\tau(q)) - \log |\det D\tau(q)|. \tag{39.26}$$

If the filter supplies only $\hat{\ell}_T$, then ordinary HMC samples the declared approximate posterior defined by $\hat{\ell}_T$. Recovering the exact posterior from an approximate or randomized likelihood would require a separately defined exact-target extended-state or Metropolis correction scheme; such a scheme is outside the scope of this chapter.

The HMC admissibility burden is therefore downstream of same-scalar discipline:

- (1) the filter must define the scalar likelihood or approximate likelihood before any sampler can be interpreted,
- (2) the gradient used by HMC must be the gradient of the same scalar used in the Metropolis correction and posterior interpretation,
- (3) transformed targets, including TT/KR-style maps, require the Jacobian-corrected target and valid support.

39.7 Exports To Validation And Promotion Chapters

The later validation chapter may import only the following consequences from the present chapter:

- the branch-valid finite-difference contract,
- the distinction between fixed structural ledgers and recomputed differentiable ledgers,
- the exact-target versus approximate-target HMC admissibility boundary,
- the rule that divergences, nonfinite gradients, and same-scalar mismatch are veto diagnostics rather than efficiency datapoints.

39.8 Implementation Boundary

BayesFilter has no nonlinear HMC convergence result in this lane, no NeuTra backend, no RMHMC backend, no tensor/KR transport-preconditioned HMC backend, and no NAWM posterior recovery benchmark. This chapter defines a shared branch, scalar, and derivative contract. It does not promote any sampler or backend as production-ready for nonlinear state-space models.

Chapter 40

Validation, Defect Calculus, and Method Promotion

This chapter closes the restarted high-dimensional block. It synthesizes the validation and promotion logic shared by the deterministic Gaussian/sparse-grid lane and the non-Gaussian low-rank retained-object lane. Its purpose is not to re-derive the methods, but to consume their exported defects, diagnostics, approximate-target contracts, and HMC-admissibility consequences.

The previous chapters have already separated:

- exact target from approximate target,
- carried object from moment or retained-object engine,
- frozen branch-defining structure from recomputed numerical values.

The role here is therefore downstream and comparative: which diagnostics veto a method, which diagnostics only explain it, and under what conditions a chapter- local approximation becomes an industrially credible component rather than a merely interesting computation.

40.1 Validation Architecture And Benchmark Roles

The restarted high-dimensional block uses three ledgers of evidence that must not be conflated:

- (1) **Engineering correctness.** Did the declared scalar build, factor, and update without violating its own branch rules?
- (2) **Numerical or sampler validity.** Did finite-difference parity, ESS, PSD, rank, support, Jacobian, or divergence diagnostics pass?
- (3) **Scientific interpretation.** Given the surviving diagnostics, what may be concluded about approximation quality, target fidelity, or method promotion?

A result in one ledger cannot be silently upgraded into the next. A green build is engineering evidence. A valid same-scalar finite-difference row is numerical evidence. Neither by itself is a scientific proof that the approximation is the right industrial method.

The benchmark roles are therefore tiered:

- exact or controlled references test whether the declared scalar behaves as intended on small models,
- local diagnostics test whether the approximation object stays meaningful,
- promotion ladders decide whether a method deserves more expensive downstream comparison.

40.2 Finite-Difference Verification As The First Shared Gate

The first shared validation gate is finite-difference parity on a fixed branch. For a base parameter θ_0 and coordinate direction e_i , define

$$D_i(h) = \frac{\widehat{\ell}_T(\theta_0 + he_i; B) - \widehat{\ell}_T(\theta_0 - he_i; B)}{2h}, \quad G_i = \partial_i \widehat{\ell}_T(\theta_0; B), \quad (40.1)$$

where both perturbed evaluations reuse the same fixed branch ledger and recompute the parameter-dependent numerical objects through the same declared equations. The row is branch-valid only if the branch identity is unchanged on both sides.

A useful reporting contract is:

$$e_i(h) = \frac{|D_i(h) - G_i|}{\max\{1, |D_i(h)|, |G_i|\}}, \quad (40.2)$$

with a small ladder such as

$$h \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\} \max\{1, |\theta_i|\}. \quad (40.3)$$

The diagnostic supports the same-scalar claim only when branch-valid rows show a stable decreasing error window. Failure may indicate a wrong derivative, a branch mismatch, or a nonsmooth/ill-conditioned value path. It is a validation gate, not a posterior-accuracy theorem.

40.3 Why The Synthesis Must Start With Defects

The preceding chapters explain high-dimensional nonlinear filtering methods. An industrial macro-finance program needs a harsher object: a calculus of failure. A method is useful for a global-bank or central-bank model only when the failure mode is explicit, the mitigation has a mathematical contract, and the cost variable that can kill the method is visible before implementation. Academic novelty is not the target here. The target is a defensible composition of known methods under stated assumptions.

This chapter therefore treats every candidate as a conditional component. A Gaussian scaffold is useful if it exposes local curvature and covariance failure. Sparse-grid or high-degree cubature is useful if it diagnoses local nonlinearity without pretending to be a global filter. Tensor methods are useful if economic structure keeps ranks stable and preserves probability or covariance semantics. Transport maps are useful if they reduce geometry while leaving a checkable target. HMC is useful only downstream of a same-scalar likelihood contract.

The reader-facing synthesis map is this: start with the exact conditional law and likelihood normalizers from Chapter 33; replace them only by named approximations; attach one diagnostic to each object being approximated; and promote a method only after the previous object exports the input required by the next. The chapter’s exact target is not a new algorithm. It is an auditable decision procedure for composing existing methods without losing the scalar likelihood, probability-law, support, Jacobian, rank, or covariance semantics needed by industrial inference.

Running quadratic-observation cell. The scalar cell

$$x_t = \rho x_{t-1} + \eta_t, \quad y_t = x_t^2 + \epsilon_t$$

has now served five roles. Chapter 33 used it to show that the exact filtering law can have two lobes. Chapter 35 used it to show why Gaussian projection can preserve moments while losing shape. Chapter 36 used it to turn one fixed sparse-grid cloud into a concrete value-path and numeric-oracle contract. Chapter 37 and Chapter 38 used it to separate retained-object semantics from fixed-branch likelihood construction. Chapter 39 used it to insist that HMC needs a scalar likelihood and a same-scalar gradient. Here the same cell is not evidence for a macro-finance model. It is the small mechanism that tells a mixed numerical reader what the architecture is trying not to lose.

Imports from the preceding chapters.

Source chapter	Exported object	Exported defect	Exported diagnos- tic/veto	Consumed here as
Ch. 33	conditional law, normalizer, likelihood gradient	wrong target, missing normalizer, value-gradient mismatch	finite Z_t , mass, positivity, same-scalar check	target and likelihood gates
Ch. 35	Gaussian moment pair and point-rule family	projection bias, exactness without accuracy, point explosion	innovation, PSD, active dimension, comparator residual	scaffold and local curvature diagnostic
Ch. 36	fixed sparse-grid scalar, numeric oracle, branch record	merge mismatch, signed-weight instability, branch-invalid finite differences	oracle reproduction, branch-valid minimum-eigenvalue veto, cloud reproduction	deterministic Gaussian lane validation
Ch. 37	retained object, coordinate map, KR / pre-conditioning semantics	support failure, Jacobian mismatch, rank/semantic failure	support/logdet, rank, mass, positivity	retained-object meaning gate
Ch. 38	fixed-branch scalar, stored-field contract	wrong saved object, branch-identity loss, incomplete next-step closure	stored-field audit, retained-marginal closure, branch-fixed structural check	operational middle gate
Ch. 39	scalar potential, gradient, transformed target	same-scalar mismatch, divergences, invalid form	scalar parity, finite support/Jacobian, energy diagnostics	downstream sampler gate

40.4 Lane-Specific Validation Ladders Before Promotion

The high-dimensional part now contains two distinct approximation lanes with different local failure modes, so the validation ladder must stay lane-aware before the closing promotion comparison begins.

40.4.1 FixedSGQF ladder

The sparse-grid lane should not be promoted from elegant construction alone. Before any cross-method comparison, it should pass a ladder that includes:

1. deterministic cloud reproduction and duplicate-merge checks,
2. the one-step numeric oracle from Chapter 36,
3. branch-valid finite differences on a saved fixed-cloud branch,
4. signed-weight stability checks through P_t^- , S_t , and carried covariance objects,
5. controlled GHQ / linear-Gaussian comparators where available.

These diagnostics answer whether the implementation is evaluating the declared fixed-cloud scalar, not whether Gaussian projection has become an exact high-dimensional posterior method.

40.4.2 Retained-object TT/KR ladder

The TT/KR lane carries a richer retained object and therefore faces richer local failure modes. Before cross-method promotion, it should pass a ladder that includes:

1. coordinate-system and Jacobian-consistency checks,
2. positivity / nonnegativity and normalization checks for the represented object,
3. rank-growth and conditioning diagnostics,
4. stored-field and retained-marginal closure checks for the next step,
5. branch-valid finite differences on the fixed-branch likelihood route.

These diagnostics do not prove that moderate TT rank is always sufficient or that the adaptive source algorithm is globally smooth. They only establish whether the frozen retained-object branch remains a scientifically interpretable scalar object on the tested route.

Copied-core and branch-status reporting. When the derivative claim is part of the result, the TT/KR lane should report not only a finite-difference error table but also the branch-status fields imported from Chapter 39: branch-identity indicators, copied-core indicators, decreasing-window indicators, and the resulting status classification. A small error alone is insufficient if the perturbed values were computed on different frozen fields or with copied fitted cores.

A useful derivative reporting object is:

$$\mathcal{R}_{\text{FD}} = \{\widehat{\ell}_T(\theta_0; B), G_i, (h_i, D_i(h_i), e_i(h_i))_i, I_{\pm}(h_i), K_{\pm}(h_i), W_i, \text{status}\}. \quad (40.4)$$

Here the report should also retain the scalar values $\widehat{\ell}_T(\theta_0 + h_i e_j; B)$ and $\widehat{\ell}_T(\theta_0 - h_i e_j; B)$ for each h_i , not merely the final error summary. The derivative table is meaningful only when the branch record is identical in the three columns base / plus / minus.

Benchmark-specific interpretation. The benchmark ledger should stay explicit about what each model can and cannot establish. A linear-Gaussian benchmark can establish exact-reference agreement for the declared run contract, but not nonlinear posterior accuracy. A stochastic-volatility or predator-prey benchmark can establish long-horizon nonlinear stability or preconditioning benefit under the declared run contract, but not universal scalability. A spatial SIR benchmark can establish whether the method remains useful at the tested state dimension, horizon, rank ladder, and basis ladder, but not that all spatial epidemic models admit moderate TT rank. One-seed demonstrations remain diagnostic unless the result artifact explicitly justifies a stronger interpretation.

Runtime, memory, and veto interpretation. Runtime and memory should be interpreted only after veto diagnostics have passed. When reported, they should be split into mathematically meaningful components: forward target evaluation, fitting solves, mass contractions, KR inversion or sampling work, retained-object storage work, and derivative overhead. Likewise, peak memory should separate value-only storage from value-plus-gradient storage. These are not mere profiling counters; they are the places where the fixed-branch approximation can become unaffordable or mathematically fragile.

40.5 Mathematical Veto Ledger Before Interpretation

The validation report should mark a run as failed before interpreting posterior plots, trajectory summaries, or method rankings if any mathematical veto fires. Let

$$\mathcal{F}_t = \{Z_t, \hat{Z}_t, \hat{p}_t, C_{t,k}, \dot{C}_{t,k}, w_i, D_h, G_{\text{FB}}\} \quad (40.5)$$

collect the scalar, tensor, weight, and derivative objects used at time t . The finite-value veto is

$$V_{\text{finite}} = \mathbf{1}\{\exists t, \exists a \in \mathcal{F}_t : a \text{ contains NaN or Inf}\}. \quad (40.6)$$

The normalization veto is

$$V_{\text{norm}} = \mathbf{1}\{\exists t : \hat{Z}_t \leq 0 \text{ or } |\int \hat{p}_t(z) dz - 1| > \varepsilon_{\text{norm}}\}. \quad (40.7)$$

The least-squares conditioning veto is

$$V_{\text{cond}} = \mathbf{1}\{\max_{t,k} \kappa(N_{t,k}) > \kappa_{\text{max}}^{\text{allowed}}\}. \quad (40.8)$$

The rank-saturation veto is

$$V_{\text{rank}} = \mathbf{1}\{\exists t : R_t = R_{\text{max}} \text{ and residual}_t > \varepsilon_{\text{fit}}\}. \quad (40.9)$$

The KR monotonicity veto is

$$V_{\text{KR}} = \mathbf{1}\{\exists j, z_{<j}, a < b : F_j(b | z_{<j}) < F_j(a | z_{<j})\}. \quad (40.10)$$

Finally, the fixed-branch derivative veto is

$$V_{\text{grad}} = \mathbf{1}\{B(\beta+h) \neq B(\beta) \text{ or } B(\beta-h) \neq B(\beta) \text{ or no branch-stable decreasing window holds}\}. \quad (40.11)$$

A run should be interpreted only if

$$V_{\text{finite}} + V_{\text{norm}} + V_{\text{cond}} + V_{\text{rank}} + V_{\text{KR}} + V_{\text{grad}}^{\text{required}} = 0, \quad (40.12)$$

where $V_{\text{grad}}^{\text{required}} = V_{\text{grad}}$ only when the claim includes validating the derivative. The tolerances in these vetoes must be declared before a run. A minimal default table is:

quantity	default	meaning
$\varepsilon_{\text{norm}}$	$10^2 \varepsilon_{\text{mach}} + 10 \varepsilon_{\text{quad}}$	normalization tolerance
$\kappa_{\text{max}}^{\text{allowed}}$	10^{10} in double precision	least-squares conditioning ceiling
ε_{fit}	$\max(10^{-8}, 10 \text{tol}_{\text{ALS}})$	rank-saturation residual ceiling
ε_{KR}	$10^2 \varepsilon_{\text{mach}} + 10 \varepsilon_{\text{quad}}$	monotonicity/inversion tolerance

(40.13)

These are implementation tolerances, not scientific outcomes. Changing them after seeing posterior plots invalidates the benchmark interpretation.

40.6 Defect 1: Log-Weight Dispersion Can Become Exponential Without Structure

The bootstrap and SIR filters in Chapter 37 are honest baselines because their collapse mechanism is visible. The aim of this section is not to prove the strongest available particle-collapse theorem. It is to record a heuristic defect calculus showing why, without locality, guided proposals, or tempering, the variance scale entering the weights can grow rapidly with effective dimension. The sharper asymptotic statements belong to the cited particle-filter literature; the present derivation is a warning model, not a complete theorem of exponential collapse.

$$\text{ESS} = \frac{1}{\sum_{i=1}^N \bar{W}_i^2}. \quad (40.14)$$

Suppose the log-weight decomposes across conditionally independent observation blocks,

$$L_i = \sum_{k=1}^{d_{\text{eff}}} \ell_{ik}, \quad \mathbb{E}[\ell_{ik}] = \mu, \quad \text{Var}(\ell_{ik}) = \sigma_\ell^2, \quad (40.15)$$

with weak dependence sufficient for $\text{Var}(L_i) \asymp d_{\text{eff}} \sigma_\ell^2$. This is the informal calculus behind the high-dimensional particle-filter warnings of Bengtsson et al. [2008] and Snyder et al. [2008].

Proposition 40.1 (Log-weight variance warning). *If the log-weight variance grows like $\text{Var}(L_i) = v_d \asymp d_{\text{eff}} \sigma_\ell^2$, then the associated importance-weight dispersion scale grows exponentially in v_d under the lognormal heuristic below. In that regime, a particle filter with proposal independent of the current observation can enter a practically severe collapse regime unless ensemble size grows very aggressively with that dispersion scale.*

Derivation. For a lognormal heuristic, $\mathbb{E}[W] = \exp(\mu_d + v_d/2)$ and $\mathbb{E}[W^2] = \exp(2\mu_d + 2v_d)$, so

$$\frac{\mathbb{E}[W^2]}{\mathbb{E}[W]^2} = \exp(v_d).$$

The usual importance-sampling variance factor is therefore exponential in v_d . Since the denominator of (40.14) is driven by the largest normalized weights, stable ESS typically requires sample growth that offsets this exponential dispersion. The cited papers give sharper asymptotic statements under stronger assumptions; the monograph uses this calculation only to justify a heuristic defect warning about log-weight dispersion, not to claim a full theorem of exponential particle collapse for every proposal/model pair. $\square$

40.7 Validation Benchmarks And Promotion Logic

The validation ladder should combine exact or controlled references, local object-validity diagnostics, and promotion rules. Exact or reference-density metrics such as Hellinger distance, relative L^1 error, moment error, trajectory RMSE, coverage, and ESS quantiles are meaningful only when the target and branch contracts have already been named. Their role is not to replace the same-scalar contract, but to test whether a declared approximation remains useful after the engineering and branch-validity gates have passed.

For the restarted high-dimensional block, the benchmark architecture should at least cover:

- (1) an exact or controlled linear-Gaussian reference where available,
- (2) a nonlinear toy case that reveals posterior-shape or retained-object limitations,
- (3) a branch-valid finite-difference check on the declared approximate scalar,
- (4) structural diagnostics specific to the lane: rank, mass, positivity, PSD, support, Jacobian, or ESS as appropriate.

These ladders are not optional appendices. They are what stops a chapter-local approximation from being promoted on speed or elegance alone.

The benchmark backbone should be named explicitly so the reader does not have to reconstruct it from isolated examples:

- (V1) exact-reference linear Gaussian benchmark,
- (V2) long-horizon stochastic-volatility benchmark,
- (V3) spatial SIR benchmark for high-dimensional partial-observation filtering,
- (V4) predator-prey benchmark for nonlinear preconditioning,
- (V5) derivative validation table whenever a fixed-branch gradient is part of the claim.

A useful compact benchmark table should pair the resource ledger with the model-specific accuracy ledger and, when derivatives are claimed, with a separate finite-difference table containing D_h , G_{FB} , $|D_h - G_{FB}|$, and the branch-status fields for each h . The derivative table is meaningful only when the branch record is identical in the three columns $\theta - h$, θ , and $\theta + h$.

What each benchmark can and cannot establish. The linear-Gaussian benchmark can establish exact-reference agreement for the declared run contract, but it cannot establish nonlinear posterior accuracy. The stochastic-volatility benchmark can establish long-horizon nonlinear stability and credible agreement with references or synthetic truth under the declared run contract, but it cannot by itself establish high-dimensional scalability. The spatial SIR benchmark can establish whether the method remains useful at the tested latent dimension, horizon, rank ladder, and basis ladder, but it cannot prove that all spatial graphs or epidemic models admit moderate TT rank. The predator-prey benchmark can establish that nonlinear preconditioning yields a measurable benefit on the tested nonlinear dynamical system, but it cannot prove that such preconditioning is always worth its cost. One-seed demonstrations are diagnostic unless the result artifact explicitly justifies stronger interpretation.

40.8 Defect Calculus And Promotion Rules

The chapter closes with a simple rule: runtime, memory, ESS per gradient, rank, and point count can compare methods only after the target, semantic, and numerical veto diagnostics have passed. The industrial promotion logic is therefore conditional:

$$V \text{ passes} \implies \text{compare performance}; \quad V^c \implies \text{veto, not ranking.} \quad (40.16)$$

This chapter does not conclude that high-dimensional nonlinear filtering is solved for BayesFilter, that any method is NAWM-ready, that HMC converges, or that any lane should become a production default. It concludes only that a candidate method must pass the declared validation and defect calculus before speed, elegance, or novelty are allowed to matter.

Part VI

HMC, Geometry, and Diagnostics

Chapter 41

HMC for State-Space Likelihoods

This chapter will develop Hamiltonian Monte Carlo and NUTS for filtering-based posterior targets. The mathematical foundations are supported by the Phase 2B literature gate, but implementation claims remain gated by BayesFilter’s target, gradient, and compilation contracts.

41.1 Target Contract

For BayesFilter, an HMC transition is valid only relative to the target in Definition 3.1. The sampler needs the scalar log target and its gradient in the coordinates used by the integrator. If the filter supplies a custom score, that score must correspond to the same log likelihood value used in the Metropolis correction.

The HMC chapter therefore depends on the earlier derivative chapters. A filtering backend that can evaluate $\ell(\theta)$ but has no declared gradient policy is value-only evidence. It can be used for optimization, diagnostics, or likelihood comparison, but it should not be promoted to HMC production use.

41.2 Implementation Position

TensorFlow Probability provides useful reference implementations of HMC and NUTS, but BayesFilter should not treat TFP NUTS as the strategic production backend. This is a project implementation decision, not a mathematical criticism of NUTS. The decision is recorded in the source map and reset memo and is supported by the minimal Gaussian benchmark in `docs/benchmarks/benchmark_tfp_nuts_gaussian.py`.

The reasons to keep TFP NUTS as a reference rather than the default production path are:

- full-trajectory NUTS control flow is hard to keep reliably XLA-compiled across model-specific likelihoods, adaptation logic, and diagnostics;
- nested TensorFlow Probability kernels can obscure whether the complete value-and-gradient pipeline is compiled or only partially traced;
- debugging numerical failures inside TFP kernel internals is slower than debugging a small BayesFilter-owned HMC/NUTS transition;

- large DSGE targets require explicit policies for finite log density, finite gradients, static shapes, filter failure handling, and custom derivative backends.

Consequently, this chapter will consolidate NUTS foundations from the approved literature sources while treating TFP NUTS as a reference and diagnostic baseline. BayesFilter’s production direction is a small, inspectable HMC layer whose contracts are designed around filtering likelihoods, analytic/custom gradients, and XLA/JIT discipline. Any future NUTS implementation should satisfy the same filter-aware contracts before it is promoted beyond diagnostic use.

41.3 Diagnostic Use of NUTS

NUTS remains valuable as an algorithmic reference and as a diagnostic sampler on small controlled targets. The bounded BayesFilter decision is narrower: the project should not treat TFP NUTS as the default answer to every convergence, geometry, or compilation problem. The Gaussian benchmark shows that TFP NUTS can XLA-compile on a toy target, but it is still materially heavier than fixed-step HMC there. Larger filtering targets add the hard parts omitted by that benchmark: spectral derivative risk, missing-data branches, model-specific invalid regions, and backend fallbacks.

Chapter 42

Mass Matrices and Local Geometry

Mass matrices translate derivative information into sampler geometry. In a linear Gaussian validation problem, a local posterior Hessian or observed information matrix can be a useful preconditioner. In a nonlinear or weakly identified model, the same matrix can be misleading unless it is regularized and checked against diagnostics.

42.1 Sources of Curvature

BayesFilter recognizes several curvature sources:

- analytic likelihood Hessian plus prior curvature;
- observed-information or expected-information approximations;
- finite-difference Hessian diagnostics at a MAP point;
- empirical covariance from warmup draws;
- diagonal fallbacks when dense curvature is unstable.

The source must be recorded because each source has a different failure mode. MacroFinance mass-matrix tests check eigenvalue regularization, dense versus diagonal choices, whitening finiteness, and covariance construction from Hessian helpers. These are geometry utilities; they are not identification proofs.

42.2 Regularization Policy

A mass matrix used by HMC must be symmetric positive definite. If a curvature estimate is indefinite or ill-conditioned, BayesFilter should regularize it, record the regularization, and expose the eigenvalue diagnostics. Silently using an indefinite curvature matrix as if it were a covariance is a target geometry error.

Chapter 43

Boundary Handling and Safe Gradients

Boundary handling is part of the posterior target. Parameters can leave the stationary, positive-definite, or economically admissible region during HMC warmup. A robust implementation must handle those proposals without crashing the chain or returning non-finite gradients.

43.1 Finite Invalid Returns

The preferred policy is:

- (i) detect invalid regions before fragile factorizations where possible;
- (ii) return a deterministic low log density;
- (iii) return a finite fallback gradient compatible with rejection;
- (iv) record the invalid reason in diagnostics;
- (v) keep the behavior stable under JIT compilation.

This policy is not a mathematical trick to make an invalid model valid. It is an engineering rule that keeps sampler control flow alive long enough for HMC to reject bad proposals and continue exploring the admissible region.

43.2 Boundary Checklist

Every HMC target should state its policy for prior support, parameter transforms, stationarity or determinacy restrictions, covariance pivots, solve residuals, non-finite likelihoods, and non-finite gradients.

Chapter 44

XLA and JIT Compilation

BayesFilter uses JIT compilation as a production tool, not as a correctness argument. A sampler is not fully compiled merely because an outer function is wrapped in `tf.function`. The complete value-and-gradient path must be inside the compiled region, with static shapes and compatible device placement.

44.1 Full-Pipeline Requirement

For HMC, the compiled path includes:

- parameter transform;
- model-provider maps;
- filtering likelihood;
- custom or autodiff gradient path;
- invalid-region branch;
- leapfrog transition or equivalent integrator.

If any component falls back to Python, retraces dynamically, or executes an opaque noncompiled kernel, the implementation should be described as partially compiled.

44.2 Device and Shape Constraints

The DSGE XLA source notes emphasize two lessons that carry over to BayesFilter. First, XLA compiles a function for a target platform, so mixing CPU-only custom calls with GPU-targeted operations inside one JIT block can fail. Second, XLA requires static shapes. Dynamic metadata should be encoded as values or opaque compile-time metadata, not as changing tensor ranks.

The practical policy is phase separation: solve or model-construction phases may be compiled separately from the filtering/HMC phase if they have different device requirements. A monolithic JIT block is valuable only when it actually contains compatible operations.

Chapter 45

Diagnostics and Failure Taxonomy

Diagnostics are evidence about failures and geometry. They are not proofs of convergence. BayesFilter should report diagnostics at three levels: filter evaluation, derivative evaluation, and sampler behavior.

45.1 Filter and Derivative Diagnostics

Filter diagnostics include factor pivots, reconstruction errors, solve residuals, condition numbers, invalid-region counts, and finite-value checks. Derivative diagnostics include score/Hessian finite checks, symmetry errors, backend parity deltas, finite-difference residuals, and custom-gradient parity. For SVD or eigendecomposition paths, singular-value or eigenvalue gap telemetry is mandatory before HMC claims are made.

45.2 Sampler Diagnostics

Sampler diagnostics include acceptance rate, divergences, energy behavior, effective sample size, split R-hat, mass-matrix conditioning, and tree depth if NUTS is used diagnostically. Short clean runs are smoke tests. They can justify the next experiment, but they do not establish convergence for an industrial target.

45.3 Failure Taxonomy

Common failure classes are:

- target-domain failures;
- factorization or solve failures;
- non-finite value or gradient failures;
- spectral derivative instability;
- JIT retracing or partial compilation;
- sampler geometry failures;
- unsupported missing-data or mask patterns.

Each failure should point to a next action: fix the model contract, change the backend, add jitter or regularization, strengthen derivative validation, or downgrade the backend to diagnostic use.

Chapter 46

Transport Maps and Surrogate Geometry

Transport maps and surrogates are downstream geometry tools. They can make an HMC target easier to explore, but they cannot repair an invalid likelihood, an inconsistent custom gradient, or a non-finite filter boundary. This chapter therefore depends on the target, derivative, and diagnostics contracts already defined in the earlier chapters.

46.1 Transported Targets

Let $q \in \mathbb{R}^d$ denote the original unconstrained parameter and let

$$q = T_\psi(z)$$

be an invertible differentiable map from transported coordinates z to the original coordinates. If the original target density is

$$\pi_Q(q) \propto \exp[-U_Q(q)],$$

then the transported target is

$$U_Z(z) = U_Q(T_\psi(z)) - \log |\det J_{T_\psi}(z)|. \quad (46.1)$$

Running HMC in z -space is target-preserving only when the sampler evaluates (46.1) and its gradient. The Jacobian term is not optional. A learned map that is imperfect can still be useful as a preconditioner, but omitting the change-of-variables correction changes the target.

46.2 NeuTra Position

The Phase 2B literature gate accepted NeuTra only as transport and geometry support. That is exactly the right status for BayesFilter. A NeuTra-like map can improve posterior geometry by making the transformed target closer to a simple reference distribution. It is not a correctness guarantee, and it is not a substitute for a valid filtering likelihood or a derivative provider that matches the Metropolis value.

The strategic use is:

- (i) certify the filtering target and gradient first;
- (ii) train or choose a transport map to improve geometry;
- (iii) run HMC on the Jacobian-corrected transported target;
- (iv) compare diagnostics and effective sample size per unit time to the untransported or affine baseline.

If the map has poor tail behavior, bad inverse stability, or large Jacobian curvature, it can make sampling worse even when it improves a training loss.

46.3 Surrogate Targets

A surrogate likelihood or surrogate Hamiltonian is a different object from a transport map. If the surrogate changes the scalar log target used in the Metropolis correction, the Markov chain targets the surrogate posterior unless a correction mechanism is added. BayesFilter should record one of three statuses for every surrogate:

diagnostic surrogate used only for exploration, initialization, or comparison; no posterior correctness claim is made;

corrected surrogate used inside a delayed-acceptance or Metropolis-corrected scheme that evaluates the intended target before a sample is accepted;

target-defining surrogate deliberately defines the approximate posterior being sampled, with approximation error documented separately.

This distinction prevents a common failure mode: using a fast approximate filter or neural surrogate for gradients and then interpreting the resulting chain as if it sampled the exact filtering posterior.

46.4 Diagnostics

Transport diagnostics should be tied to sampler performance and target integrity:

- transformed-target finite value and finite gradient checks;
- log-Jacobian finite checks and inverse/forward reconstruction residuals;
- tail probes for large $\|z\|$;
- acceptance rate, divergences, E-BFMI, and effective sample size per second relative to an affine baseline;
- shell-level Hessian or force diagnostics where available.

These diagnostics explain why a map helps or fails. They do not override a failed target contract.

46.5 BayesFilter Policy

Transport and surrogate methods should enter BayesFilter after the exact or controlled approximate likelihood path has been validated. They are valuable for large models because geometry can dominate HMC cost, but they sit on top of the filter contract rather than underneath it. In particular, they should not be proposed as a fix for SVD spectral-gradient instability, missing derivative guards, or partial XLA compilation. Those problems must be solved at the filter and sampler-contract layer first.

Part VII

**Industrial Applications and Case
Studies**

Chapter 47

LGSSM Validation Ladder

The linear Gaussian state-space model is the first validation target for BayesFilter because it supplies an exact likelihood, controlled derivatives, and tractable regression tests. Every nonlinear or DSGE backend should be reduced to an LGSSM special case before it is trusted in HMC.

47.1 Evidence Already Available

The MacroFinance project provides the strongest local evidence stream:

- exact prediction-error likelihood formulas and analytic derivative notes;
- covariance, solve, Cholesky square-root, QR square-root, TensorFlow, and SVD-flavored backends;
- finite-difference and autodiff parity tests on small models;
- large-scale backend-ladder tests that report reconstruction errors, solve residuals, QR pivots, and finite derivative deltas;
- missing-data policy tests that fail unsupported sparse panels before the likelihood silently changes meaning.

This evidence justifies using the LGSSM as BayesFilter’s regression oracle. It does not yet replace the BayesFilter-specific packaging and public API tests that will be needed when the code is moved into this project.

47.2 Validation Rungs

The LGSSM ladder is:

- (i) fixed-parameter likelihood agreement against an independent Kalman implementation;
- (ii) score agreement against finite differences and small-model autodiff;
- (iii) Hessian agreement where the backend claims observed-information support;
- (iv) backend parity among solve, Cholesky, QR, and TensorFlow paths;

- (v) mask and mixed-frequency parity against dense special cases;
- (vi) compiled/eager parity for the full value-and-gradient target;
- (vii) short HMC plumbing checks with finite diagnostics;
- (viii) medium and strict HMC runs only after the preceding rungs pass.

47.3 Interpretation

Passing the LGSSM ladder means the sampler interface, derivative convention, and linear algebra backends are coherent on an exact target. It does not prove that a nonlinear approximation is accurate, and it does not prove that a DSGE posterior has good HMC geometry. It is the floor, not the ceiling.

Chapter 48

Nonlinear SSM Validation Ladder

The nonlinear SSM ladder tests whether approximate nonlinear filtering can be used inside HMC before DSGE perturbation solvers and economic boundaries enter the picture. It should use small synthetic models with known parameters and independent references, not large application models.

48.1 Controlled Models

The first nonlinear validation suite is deliberately small. It is designed to test the filtering law, structural deterministic completion, and sigma-point moment approximation before DSGE perturbation solvers, client adapters, or HMC geometry enter the picture. The suite contains three first-rung models.

48.1.1 Model A: Affine Gaussian Structural Oracle

The affine oracle is a BayesFilter-local synthetic model:

$$\begin{aligned} x_t &= (m_t, \ell_t)^\top, & m_t &= 0.35m_{t-1} - 0.10\ell_{t-1} + 0.25\varepsilon_t, & \ell_t &= m_{t-1}, \\ y_t &= m_t + \eta_t, & \varepsilon_t &\sim N(0, 1), & \eta_t &\sim N(0, 0.15^2). \end{aligned}$$

The initial law is Gaussian and the deterministic coordinate ℓ_t stores the previous m . The oracle is the exact linear Gaussian Kalman likelihood after converting the structural law to its full-state LGSSM. The test must verify that SVD cubature, SVD-UKF, and SVD-CUT4 collapse to the same prediction-error likelihood and that deterministic-completion residuals are zero up to numerical tolerance.

48.1.2 Model B: Nonlinear Accumulation

The nonlinear accumulation model is a smooth BayesFilter-local structural fixture:

$$\begin{aligned} m_t &= \rho m_{t-1} + \sigma \varepsilon_t, & k_t &= \alpha k_{t-1} + \beta \tanh(m_t), \\ y_t &= m_t + k_t + \eta_t, & \varepsilon_t &\sim N(0, 1), & \eta_t &\sim N(0, \sigma_\eta^2). \end{aligned}$$

The first V1 defaults are

$$\rho = 0.70, \quad \sigma = 0.25, \quad \alpha = 0.55, \quad \beta = 0.80, \quad \sigma_\eta = 0.30.$$

This model tests nonlinear deterministic completion without a time-indexed transition hook. The reference oracle is a dense one-step Gaussian moment-projection quadrature: it integrates the augmented predictive Gaussian, computes the transformed prediction and observation moments, and then applies the same Gaussian update used by the sigma-point filters. It is not an exact nonlinear likelihood.

48.1.3 Model C: Nonlinear Growth Benchmark

The scalar nonlinear growth benchmark is the standard model associated with Kitagawa’s Monte Carlo filtering examples [Kitagawa, 1996] and widely reused in sequential Monte Carlo discussions. Its time-inhomogeneous law is

$$x_t = \frac{x_{t-1}}{2} + \frac{25x_{t-1}}{1 + x_{t-1}^2} + 8 \cos(1.2(t-1)) + \sigma_x \varepsilon_t,$$

$$y_t = \frac{x_t^2}{20} + \sigma_y \eta_t, \quad \varepsilon_t, \eta_t \sim N(0, 1).$$

Because the current BayesFilter structural TensorFlow transition does not pass an explicit time index, the V1 testing fixture uses an autonomous deterministic phase coordinate:

$$\tau_t = \tau_{t-1} + 1, \quad \tau_1 = 1,$$

$$x_t = \frac{x_{t-1}}{2} + \frac{25x_{t-1}}{1 + x_{t-1}^2} + 8 \cos(1.2\tau_{t-1}) + \sigma_x \varepsilon_t.$$

The observation equation remains $y_t = x_t^2/20 + \sigma_y \eta_t$. The first defaults are $x_1 \sim N(0, 0.2)$, $\sigma_x = 1$, and $\sigma_y = 1$. As in Model B, the first oracle is dense one-step Gaussian moment projection, not the exact nonlinear likelihood.

48.1.4 Deferred Models

Bearings-only tracking. Gordon et al. [1993] and Arulampalam et al. [2002] are useful sources, but the test should wait until angle residual and wrapping policy are fixed.

Radar range-bearing tracking. This should follow the bearings-only residual policy and should add mixed-scale observation diagnostics.

Stochastic volatility. This should wait until the sigma-point interface supports non-additive observation noise or an explicit approximation contract, since current filters assume additive observation covariance after $h(x_t)$.

Deferred means that the model is useful later, not that it is rejected. The first V1 suite stops at Models A–C so that value, score, branch, and benchmark tests share a stable set of fixtures.

48.2 Current V1 Filter Evidence

The first CPU nonlinear benchmark compares three SVD/eigen sigma-point value filters:

$$\mathcal{B} = \{\text{SVD cubature}, \text{SVD-UKF}, \text{SVD-CUT4}\}.$$

For Model A the reference is the exact Kalman prediction-error likelihood after structural-to-LGSSM conversion. The benchmark rows must satisfy

$$|\ell_b(y_{1:T}) - \ell_{\text{KF}}(y_{1:T})| \approx 0, \quad b \in \mathcal{B},$$

up to floating-point tolerance, and the filtered means and covariances must match the exact Kalman recursion. This verifies that the nonlinear sigma-point code collapses to the correct affine Gaussian law.

For Models B and C the benchmark uses only a dense one-step Gaussian projection reference. If Q_{dense} denotes a tensor-product Gaussian rule and Π_b denotes one of the sigma-point rules, the artifact reports

$$\left| \ell_{\Pi_b}^{(1)} - \ell_{Q_{\text{dense}}}^{(1)} \right|, \quad \left\| \hat{x}_{\Pi_b}^{(1)} - \hat{x}_{Q_{\text{dense}}}^{(1)} \right\|_2, \quad \left\| \hat{P}_{\Pi_b}^{(1)} - \hat{P}_{Q_{\text{dense}}}^{(1)} \right\|_F.$$

These are approximation-quality diagnostics for the first filtering step, not exact full-likelihood errors. The current artifact therefore records the full Models B–C log likelihoods as filter outputs and blocks any claim that those values have been certified against an exact nonlinear likelihood.

The same artifact records the shared branch diagnostics

$$r_{\text{det}}, \quad r_{\text{supp}}, \quad g_{\text{place}}, \quad g_{\text{innov}}, \quad c_{\text{floor}}, \quad N_{\text{points}},$$

where r_{det} is the deterministic-completion residual, r_{supp} is the residual in the inactive SVD support, g_{place} and g_{innov} are placement and innovation spectral-gap diagnostics, c_{floor} counts active covariance floors, and N_{points} is the rule size. These diagnostics keep model-law failures separate from numerical regularization. They are also the gate before analytic score or HMC claims.

The analytic first-order score now has a second rung beyond the affine fixture. For Model B we use

$$\theta_B = (\rho, \sigma, \beta)$$

with α , the initial law, the innovation variance, and the observation variance held fixed. The derivative provider supplies

$$\frac{\partial F}{\partial x_{t-1}} = \begin{bmatrix} \rho & 0 \\ \beta(1 - \tanh^2 m_t)\rho & \alpha \end{bmatrix}, \quad \frac{\partial F}{\partial \varepsilon_t} = \begin{bmatrix} \sigma \\ \beta(1 - \tanh^2 m_t)\sigma \end{bmatrix},$$

and parameter partials

$$F_\rho = \begin{bmatrix} m_{t-1} \\ \beta(1 - \tanh^2 m_t)m_{t-1} \end{bmatrix}, \quad F_\sigma = \begin{bmatrix} \varepsilon_t \\ \beta(1 - \tanh^2 m_t)\varepsilon_t \end{bmatrix}, \quad F_\beta = \begin{bmatrix} 0 \\ \tanh(m_t) \end{bmatrix}.$$

The observation derivative is $H_x = [1, 1]$. Centered finite differences of the implemented SVD cubature, SVD-UKF, and SVD-CUT4 likelihoods match the analytic scores on the selected smooth branch.

For Model C we use

$$\theta_C = (\sigma_u, \sigma_y, P_{0,x}).$$

The derivative provider supplies

$$F_x = \begin{bmatrix} \frac{1}{2} + 25 \frac{1-x^2}{(1+x^2)^2} & -9.6 \sin(1.2\tau) \\ 0 & 1 \end{bmatrix}, \quad F_\varepsilon = \begin{bmatrix} \sigma_u \\ 0 \end{bmatrix},$$

$$F_{\sigma_u} = (\varepsilon_t, 0)', \quad \partial_{\sigma_y} R = 2\sigma_y, \quad \partial_{P_{0,x}} P_0 = \text{diag}(1, 0),$$

and $H_x = [x/10, 0]$. Score parity is certified on a nondegenerate phase-state testing variant with positive phase variance. The default Model C value benchmark still has zero phase variance. Under the smooth simple-spectrum/no-active-floor SVD score branch, that default law remains blocked by an active placement floor. This is an intentional diagnostic. The structural fixed-support branch in Chapter 18 instead keeps the phase direction as a declared structural null direction: the factor column and its derivative are zero, the pre-transition sigma-point variable is (x_{t-1}, ε_t) , and the propagated phase coordinate is completed pointwise by the transition map. On that branch, default Model C is now a score-ready testing target for SVD cubature, SVD-UKF, and SVD-CUT4, subject to the fixed-null covariance, fixed-null derivative, active-gap, and finite-difference diagnostics.

Consequently, GPU/XLA and HMC evidence for nonlinear Models B–C remains deferred until the score-ready parameter boxes and branch diagnostics have been selected and tested.

48.3 Required Outputs

A nonlinear SSM HMC validation run should write:

- JSON diagnostics with chain counts, warmup/draw counts, step sizes, acceptance rate, divergence counts, R-hat, ESS, MCSE/SD, and runtime;
- NPZ chain artifacts for independent audit;
- true-parameter coverage when synthetic truth exists;
- filter failure counts, conditioning-guard counts, and finite-gradient failures.

Medium runs are sampler-usability evidence. Strict runs require at least four chains, finite log targets, zero or justified divergences, R-hat below 1.01, bulk and tail ESS above the planned thresholds, and saved artifacts.

48.4 Stop Rules

If the LGSSM rung fails, the nonlinear rung should not start. If the nonlinear SSM rung fails while LGSSM passes, the failure should be attributed first to the approximation, nonlinear target geometry, transition hook, or derivative path. DSGE/NK debugging should wait until this simpler explanation is ruled out.

Chapter 49

NK SVD Case Study

The NK SVD case study is a warning label with useful engineering evidence. It shows how far a numerically hardened SVD sigma-point path can get, and it also shows why BayesFilter should not equate bounded HMC output with convergence or industrial readiness.

49.1 What Passed

The source reset memos record several meaningful passes:

- focused SVD, target-boundary, adjoint, and generic nonlinear SSM contracts passed during the post-refactor work;
- generic SSM SVD likelihoods and gradients compile under production XLA;
- NK SVD fixed-solution and full-solve likelihood/gradient paths compile under XLA with explicit opt-in;
- tiny NK SVD production-XLA HMC smoke tests run to completion with finite output;
- generic nonlinear SSM medium HMC writes diagnostics with finite chains and zero divergences under tuned short-run defaults.

These results are valuable sampler-usability and compilation evidence.

49.2 What Remains Blocked

The same reset memos record why NK convergence remains blocked. Earlier bounded stress runs produced nonfinite adjoint and lam1 inputs, SVD conditioning guard activations, nonfinite intermediate arrays, and extreme but finite transformed-theta states. The later production-XLA smoke gates do not erase that artifact history, because they are tiny runs and not clean stress experiments.

Therefore the next NK claim must be a clean stress gate, not a convergence claim. A valid clean gate requires no checked-nonfinite adjoint captures, no nonfinite lam1 inputs, no nonfinite SVD-filter captures, no unexplained transformed-theta scale explosion, and zero or separately justified divergences.

49.3 Lesson for BayesFilter

The case study separates four statuses:

filter-correct likelihood and gradients agree with controlled references;

sampler-usable bounded HMC runs produce finite chains and useful diagnostics;

converged strict multi-chain diagnostics pass on the intended target;

production-XLA-gated the full value-and-gradient path compiles and runs under the declared production policy.

NK SVD has meaningful production-XLA and smoke evidence, but it is not yet a converged case study. That distinction should stay visible in every future BayesFilter report.

Chapter 50

CIP/AFNS Case Study

The CIP/AFNS material is useful to BayesFilter because it is a large structured linear Gaussian application with economically meaningful observation equations, missing-data pressure, and analytic derivative requirements. The economics belongs in the source CIP monograph. BayesFilter should import only the filtering lessons.

50.1 Filtering Lessons

The reusable lessons are:

- affine or linearized macro-finance measurement systems can become large LGSSMs with structured observation matrices;
- yield-curve blocks create dense measurement panels with heterogeneous maturities and noise scales;
- missing observations must be represented by explicit masks, not by silent imputation or dynamic shape changes;
- analytic derivatives and backend parity matter because HMC mass matrices and MAP curvature depend on the same likelihood being sampled.

50.2 BayesFilter Boundary

BayesFilter should not re-derive AFNS economics, currency-basis theory, or asset-pricing restrictions in this monograph. It should define the state-space objects that an application supplies:

$$F_{\theta}, \quad Q_{\theta}, \quad H_{\theta}, \quad R_{\theta}, \quad m_{0|0,\theta}, \quad P_{0|0,\theta},$$

and then validate that the filter, derivative provider, masks, and HMC diagnostics behave as promised. This boundary keeps BayesFilter reusable and prevents application monographs from becoming maintenance forks of the filtering code.

50.3 Readiness Use

CIP/AFNS should be used as a large LGSSM readiness case once the BayesFilter package exists. The required evidence is backend parity, missing-data policy, finite analytic gradi-

ents and Hessians, MAP/mass-matrix diagnostics, and short HMC plumbing checks. It is not the right first target for SVD sigma-point HMC claims, because its main reusable strength is exact or controlled linear Gaussian filtering.

Chapter 51

NAWM-Scale Design Target

NAWM-scale models define the industrial target for BayesFilter. They should not be presented as completed evidence in this monograph pass. Instead, they set the design requirements for robust filtering and HMC on high-dimensional state-space systems.

51.1 Why NAWM Changes the Standard

As state dimension, parameter dimension, and observation dimension grow, several risks become routine rather than exceptional:

- covariance spectra become clustered or nearly singular;
- HMC proposals visit invalid or barely valid structural regions;
- raw autodiff through eigendecompositions or SVDs becomes fragile;
- compile boundaries and dynamic branches become performance bottlenecks;
- short smoke runs become less informative about long-chain behavior.

This is why BayesFilter should prefer analytic or custom derivative paths for production filtering wherever feasible.

51.2 Design Requirements

A NAWM-grade BayesFilter backend needs:

- (i) a fixed, documented target contract and parameter transform;
- (ii) explicit state-space shape contracts;
- (iii) finite invalid-region returns and finite gradients where HMC moves;
- (iv) analytic/custom gradients for exact or controlled approximate likelihoods;
- (v) spectral-gap and factor diagnostics for any spectral backend;
- (vi) full-pipeline JIT evidence, not only outer-wrapper tracing;
- (vii) reproducible diagnostics and reset memos for every long experiment.

51.3 Hypothesis

The working hypothesis is that NAWM-scale HMC will be more robust with source-backed analytic or custom filtering gradients than with raw tape gradients through spectral decompositions. This is not yet a theorem or a completed benchmark. It is the prevention-first design target motivated by the SVD debugging history and by the derivative risks documented in the Phase 2B literature gate.

Chapter 52

Production Checklist

This checklist is the operating definition of BayesFilter production readiness. It is intentionally stricter than a successful smoke test.

52.1 Target and Source

Before HMC:

- (i) the scalar log target is defined in writing;
- (ii) every formula has provenance in source notes, local code/tests, or reviewed literature;
- (iii) parameter transforms and Jacobian terms are included;
- (iv) invalid structural regions return documented finite values;
- (v) approximate filters are labeled as target-defining approximations or corrected surrogates.

52.2 Derivative and Filter

The filter backend must provide:

- (i) shape, dtype, mask, and static-compilation contracts;
- (ii) finite value and finite gradient checks;
- (iii) derivative parity against finite differences or autodiff on controlled cases;
- (iv) backend parity where multiple linear algebra forms exist;
- (v) factor, solve, condition-number, and spectral-gap diagnostics;
- (vi) an explicit custom-gradient policy when raw autodiff is not the final production path.

52.3 Sampler and JIT

The sampler must report:

- (i) whether the complete value-and-gradient path is compiled;
- (ii) eager/compiled parity on small targets;
- (iii) mass-matrix source and conditioning;
- (iv) acceptance rate, divergences, E-BFMI or equivalent energy diagnostics, R-hat, ESS, MCSE/SD, and runtime;
- (v) saved JSON and chain artifacts for medium and strict runs;
- (vi) sidecar capture artifacts only for bounded debugging, not for every production transition.

52.4 Claim Discipline

Use the narrowest honest label:

value-valid the likelihood value is finite and reference-checked;

gradient-valid the derivative matches the target value and passes parity checks;

sampler-usable finite HMC diagnostics justify the next experiment;

converged strict multi-chain diagnostics pass and artifacts are saved;

production-ready all preceding gates pass under the intended model, data, hardware, and compilation policy.

Anything less should stay in the reset memo as a gap, not disappear into prose.

Appendices

Appendix A

Notation and Shape Conventions

This appendix fixes the notation used throughout BayesFilter. Source projects use overlapping but not identical symbols; this monograph uses the following conventions unless a chapter states otherwise.

Symbol	Meaning
$t = 1, \dots, T$	observation time index
$x_t \in \mathbb{R}^{n_x}$	latent state
$y_t \in \mathbb{R}^{n_y}$	observation vector before masking
$\theta \in \mathbb{R}^{d_\theta}$	model parameter vector
$u \in \mathbb{R}^{d_u}$	unconstrained sampler coordinate
c, F, Q	linear Gaussian transition offset, matrix, covariance
a, H, R	linear Gaussian observation offset, matrix, covariance
$m_{t s}, P_{t s}$	conditional state mean and covariance
v_t, S_t	innovation and innovation covariance
K_t	Kalman gain
$\ell(\theta)$	filtering log likelihood
$g(\theta)$	likelihood score $\nabla_\theta \ell(\theta)$
$G(\theta)$	likelihood Hessian $\nabla_{\theta\theta}^2 \ell(\theta)$

Matrix transposes are written with $^\top$. Positive definiteness and positive semidefiniteness are written $A \succ 0$ and $A \succeq 0$. The operator $\text{tr}(\cdot)$ denotes trace, and $\text{diag}(\cdot)$ forms or extracts a diagonal depending on context.

A.1 Derivative Shapes

For a parameter-dependent vector $b(\theta) \in \mathbb{R}^n$, first derivatives have shape $d_\theta \times n$ and second derivatives have shape $d_\theta \times d_\theta \times n$. For a matrix $A(\theta) \in \mathbb{R}^{m \times n}$, first derivatives have shape $d_\theta \times m \times n$ and second derivatives have shape $d_\theta \times d_\theta \times m \times n$.

This convention matches the MacroFinance derivative-provider layout used as the current implementation reference. Chapters that discuss TensorFlow or JAX execution may store tensors differently for performance, but must state the mathematical shape convention before changing memory layout.

A.2 Label Namespace

New labels use a BayesFilter namespace:

```
ch:bf-*,  sec:bf-*,  def:bf-*,  asm:bf-*,  eq:bf-*,  tab:bf-*
```

Source-project labels should not be copied verbatim. When a source label is important for provenance, it should be recorded in the reset memo or source map.

Appendix B

Matrix Calculus Identities

The derivative chapters repeatedly use a small set of matrix identities. This appendix records them as proof obligations to be checked against the source derivations.

For an invertible matrix $S(\theta)$,

$$\partial_i S^{-1} = -S^{-1}(\partial_i S)S^{-1}.$$

For a symmetric positive definite matrix $S(\theta)$,

$$\partial_i \log \det S = \text{tr}(S^{-1} \partial_i S).$$

For $q = v^\top S^{-1} v$ and $Sw = v$,

$$\partial_i q = 2(\partial_i v)^\top w - w^\top (\partial_i S) w.$$

These identities are the local algebra behind the solve-form score in Chapter 9. Second-order formulas should be expanded only with explicit source labels or proof notes because the noncommutative matrix products are easy to reorder incorrectly.

Appendix C

Factor Derivative Proofs

This appendix is a placeholder for full Cholesky and QR factor derivative proofs. The current writing pass records the required evidence policy rather than attempting to re-prove every factor identity.

Before this appendix is promoted beyond outline form, it should include:

- the source MacroFinance labels for Cholesky derivative formulas;
- the QR perturbation equations used by the square-root Kalman backend;
- reconstruction identities connecting factor derivatives to covariance derivatives;
- code references to QR and Cholesky differentiated Kalman tests;
- MathDevMCP or manual proof-obligation results for each nontrivial step.

Appendix D

MathDevMCP Workflows

MathDevMCP is used as an audit assistant, not as an oracle. Its results should be recorded with their status and limitations.

Useful commands for this monograph include:

- `search-latex` to find source labels and neighborhoods;
- `extract-latex-context` to record exact source provenance;
- `audit-derivation-label` for bounded symbolic checks;
- `audit-kalman-recursion` for AST-level implementation structure.

The current solve-form Kalman audit found the expected solve-oriented operation graph but reported missing strict log-determinant and trace classification, as well as missing explicit shape and covariance guards. The correct response is to document those gaps and use the MacroFinance parity tests as additional evidence, not to relabel the audit as a proof.

Appendix E

ResearchAssistant Workflows

ResearchAssistant is the literature-ingestion sidecar for this monograph. It should be used when a chapter needs primary-source support rather than local implementation provenance. It is not a replacement for reading the source and checking whether the cited paper supports the exact BayesFilter claim.

E.1 Workspace Policy

Use a persistent, ignored workspace under the BayesFilter tree:

```
/home/chakwong/BayesFilter/.research/ra-bayesfilter-monograph.
```

Downloaded papers, source packages, extracted text, and inbox files stay out of Git. Durable decisions belong in ‘docs/plans’ result notes, the reset memo, and the source map.

E.2 Ingestion Workflow

The standard workflow is:

- (i) search or fetch the primary source;
- (ii) verify metadata against title, authors, venue, DOI or arXiv id;
- (iii) reject mismatches explicitly;
- (iv) extract the citation neighborhood or source labels relevant to the BayesFilter claim;
- (v) write a short result note stating supported claims and limits;
- (vi) only then add or use a bibliography entry.

The Phase 2B result note is the model: it accepted exact arXiv source packages for HMC, NUTS, NeuTra, Matrix Backprop, Kitagawa score/Hessian material, and the differentiable particle-filter frontier, while rejecting several metadata mismatches.

E.3 Claim Gate

A literature-heavy paragraph is draftable only when one of the following is true:

- an accepted ResearchAssistant note supports the claim and records its limits;
- a local source document plus code/test audit supports the claim;
- the paragraph is explicitly labeled as a project decision or an open hypothesis.

Raw ‘.bib’ entries, downloaded PDFs, and search results are not claim support.

E.4 Current Blockers

The current blockers are:

- primary Julier–Uhlmann or equivalent UKF support for detailed sigma-point literature claims;
- primary PMCMC and pseudo-marginal support before particle-filter correctness claims;
- square-root filtering primary sources if final square-root bibliographic prose is expanded;
- a dedicated derivation/code audit before Kalman score and Hessian formulas are marked peer-review ready;
- industrial-scale SVD-HMC safety evidence before NAWM-scale claims.

Until these blockers are closed, the monograph should keep the conservative wording used in the current draft.

Appendix F

Source Map

The machine-readable source map is

`docs/source_map.yml`.

It records which source project, chapter, reset memo, code file, or test file supports each BayesFilter chapter. The source map is part of the monograph’s quality-control system, not merely an index.

Each imported item should move through explicit states:

- **candidate**: relevant but not yet drafted;
- **drafted**: consolidated into BayesFilter prose;
- **audited**: checked against source math, code, tests, or reviewed literature;
- **superseded**: replaced by another source;
- **excluded**: intentionally outside BayesFilter scope.

The source map also records duplicate source groups. For example, Kalman likelihood material appears in the DSGE monograph, CIP monograph, and the MacroFinance analytic derivative note. BayesFilter treats the MacroFinance note and associated tests as the derivative spine while using the monographs as expository sources.

Generated artifacts, paper caches, and local PDFs are not provenance by themselves. A claim becomes monograph-ready only when the relevant source or test has been summarized in a committed note, mapped in the source map, or explicitly cited in the chapter.

Appendix G

Experiment Templates

This appendix gives reusable experiment templates for future BayesFilter implementation work. The templates are intentionally short. Each experiment should write a reset-memo entry with commands, artifacts, interpretation, and the next justified phase.

G.1 Filter Reference Template

Use this template before HMC:

- (i) define the model, parameter transform, data seed, and fixed structural parameters;
- (ii) compute a reference likelihood from an independent exact, dense, or trusted implementation;
- (iii) compare the BayesFilter backend value to the reference;
- (iv) compare gradients by finite differences or small-model autodiff;
- (v) record finite-value, factor, solve, mask, and failure-code diagnostics;
- (vi) save the command and tolerance rationale.

Passing this template earns value-valid or gradient-valid status, depending on which checks are included. It does not earn sampler-usability status.

G.2 HMC Smoke Template

Use this template to test sampler plumbing:

- (i) use a small model and static shapes;
- (ii) run one or two short chains with deliberately small draw counts;
- (iii) require finite chains, finite log targets, and no unhandled exceptions;
- (iv) record acceptance rate, divergence count, step size, runtime, and whether the complete value-gradient path was compiled;
- (v) state explicitly that the run is not convergence evidence.

Smoke runs justify the next experiment only when the failure taxonomy is clean.

G.3 Medium Recovery Template

Use this template for controlled posterior recovery:

- (i) run at least four chains when feasible;
- (ii) save JSON diagnostics and NPZ chain artifacts;
- (iii) report R-hat, bulk ESS, tail ESS, MCSE/SD, divergences, acceptance, and runtime;
- (iv) compare posterior intervals to synthetic truth or a trusted reference;
- (v) classify the result as sampler-usable or not sampler-usable.

Medium recovery is allowed to have looser thresholds than strict convergence, but the looseness must be stated in the result note.

G.4 Strict Convergence Template

Use this template only after the preceding gates pass:

- (i) run at least four chains with planned warmup and draw counts;
- (ii) require finite log targets and finite gradients throughout accepted trajectories;
- (iii) require zero divergences unless a bounded exception is justified;
- (iv) require the planned R-hat, ESS, and MCSE/SD thresholds;
- (v) audit saved artifacts before writing a convergence claim.

If any required diagnostic fails, the reset memo should name the blocker and the next diagnostic hypothesis instead of weakening the claim label.

Bibliography

- Nagavenkat Adurthi, Puneet Singla, and Tarunraj Singh. The conjugate unscented transform: An approach to evaluate multi-dimensional expectation integrals. In *2012 American Control Conference*, pages 5556–5561, 2012a. Article 6314970.
- Nagavenkat Adurthi, Puneet Singla, and Tarunraj Singh. Conjugate unscented transform and its application to filtering and stochastic integral calculation. In *AIAA Guidance, Navigation, and Control Conference 2012*, 2012b. AIAA Guidance, Navigation, and Control Conference, Minneapolis, Minnesota.
- Sungbae An and Frank Schorfheide. Bayesian analysis of dsge models. *Econometric Reviews*, 26(2–4):113–172, 2007.
- Martin M. Andreasen, Jesús Fernández-Villaverde, and Juan F. Rubio-Ramírez. The pruned state-space system for non-linear dsge models: Theory and empirical applications. *Review of Economic Studies*, 85(1):1–49, 2018.
- Christophe Andrieu and Gareth O. Roberts. The pseudo-marginal approach for efficient monte carlo computations. *The Annals of Statistics*, 37(2):697–725, 2009.
- Christophe Andrieu, Arnaud Doucet, and Roman Holenstein. Particle markov chain monte carlo methods. *Journal of the Royal Statistical Society: Series B*, 72(3):269–342, 2010.
- M. Sanjeev Arulampalam, Simon Maskell, Neil Gordon, and Tim Clapp. A tutorial on particle filters for online nonlinear/non-gaussian bayesian tracking. *IEEE Transactions on Signal Processing*, 50(2):174–188, 2002. doi: 10.1109/78.978374.
- Thomas Bengtsson, Peter Bickel, and Bo Li. Curse-of-dimensionality revisited: Collapse of the particle filter in very large scale systems. *Probability and Statistics: Essays in Honor of David A. Freedman*, pages 316–334, 2008.
- Michael Betancourt. A conceptual introduction to hamiltonian monte carlo. *arXiv preprint arXiv:1701.02434*, 2017.
- Nicolas Chopin and Omiros Papaspiliopoulos. *An Introduction to Sequential Monte Carlo*. Springer Series in Statistics. Springer, 2020. doi: 10.1007/978-3-030-47845-2.
- Adrien Corenflos, James Thornton, George Deligiannidis, and Arnaud Doucet. Differentiable particle filtering via entropy-regularized optimal transport. In *Proceedings of the 38th International Conference on Machine Learning*, pages 2100–2111, 2021.
- Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems*, volume 26, pages 2292–2300, 2013.

- Fred Daum and Jim Huang. Nonlinear filters with particle flow. In *Signal Processing, Sensor Fusion, and Target Recognition XVII*, 2008.
- M. H. A. Davis. On a multiplicative functional transformation arising in nonlinear filtering theory. *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete*, 54:125–139, 1980.
- Arnaud Doucet, Nando de Freitas, and Neil Gordon, editors. *Sequential Monte Carlo Methods in Practice*. Springer, 2001.
- J. Durbin and S. J. Koopman. *Time Series Analysis by State Space Methods*. Oxford University Press, 2012.
- Neil J. Gordon, David J. Salmond, and Adrian F. M. Smith. Novel approach to nonlinear/non-gaussian bayesian state estimation. *IEE Proceedings F: Radar and Signal Processing*, 140(2):107–113, 1993.
- Samuel Greydanus, Misko Dzamba, and Jason Yosinski. Hamiltonian neural networks. In *Advances in Neural Information Processing Systems*, volume 32, pages 15379–15389, 2019.
- Edward P. Herbst and Frank Schorfheide. *Bayesian Estimation of DSGE Models*. Princeton University Press, 2015.
- Chao Hu and Peter Jan van Leeuwen. A particle flow filter for high-dimensional system applications. *Quarterly Journal of the Royal Meteorological Society*, 147(739):2358–2377, 2021.
- Catalin Ionescu, Orestis Vantzos, and Cristian Sminchisescu. Training deep networks with structured layers by matrix backpropagation. *arXiv preprint arXiv:1509.07838*, 2015. doi: 10.48550/arXiv.1509.07838.
- Simon J. Julier and Jeffrey K. Uhlmann. A general method for approximating nonlinear transformations of probability distributions. Technical report, Robotics Research Group, Department of Engineering Science, University of Oxford, 1996. 1 November 1996.
- Simon J. Julier and Jeffrey K. Uhlmann. A new extension of the kalman filter to nonlinear systems. *Proceedings of AeroSense*, 1997.
- Jinill Kim, Sunghyun Kim, Ernst Schaumburg, and Christopher A. Sims. Calculating and using second-order accurate solutions of discrete time dynamic equilibrium models. *Journal of Economic Dynamics and Control*, 32(11):3397–3414, 2008.
- Genshiro Kitagawa. Monte carlo filter and smoother for non-gaussian nonlinear state space models. *Journal of Computational and Graphical Statistics*, 5(1):1–25, 1996. doi: 10.1080/10618600.1996.10474692.
- Sijing Li, Zhongjian Wang, Stephen S.-T. Yau, and Zhiwen Zhang. Solving high-dimensional nonlinear filtering problems using a tensor train decomposition method. *arXiv preprint arXiv:1908.04010*, 2019.

- Yanghai Li and Mark Coates. Particle filtering with invertible particle flow. *IEEE Transactions on Signal Processing*, 65(15):4102–4116, 2017.
- Yuhua Meng, Zhongjian Wang, Stephen S.-T. Yau, and Zhiwen Zhang. Regularity estimate and sparse approximation of pathwise robust duncan-mortensen-zakai equation. *arXiv preprint arXiv:2509.19093*, 2025.
- Radford M. Neal. Mcmc using hamiltonian dynamics. In Steve Brooks, Andrew Gelman, Galin Jones, and Xiao-Li Meng, editors, *Handbook of Markov Chain Monte Carlo*, pages 113–162. Chapman and Hall/CRC, 2011.
- Gabriel Peyré and Marco Cuturi. Computational optimal transport. *Foundations and Trends in Machine Learning*, 11(5–6):355–607, 2019.
- Sebastian Reich. A nonparametric ensemble transform method for bayesian inference. *SIAM Journal on Scientific Computing*, 35(4):A2013–A2024, 2013.
- Chris Snyder, Thomas Bengtsson, Peter Bickel, and Jeffrey Anderson. Obstacles to high-dimensional particle filtering. *Monthly Weather Review*, 136(12):4629–4640, 2008.
- Rudolph van der Merwe. *Sigma-Point Kalman Filters for Probabilistic Inference in Dynamic State-Space Models*. PhD thesis, OGI School of Science and Engineering at Oregon Health and Science University, 2004.
- Ramon van Handel. Stochastic calculus, filtering, and stochastic control, 2007. ACM 217 lecture notes.
- Shing-Tung Yau and Stephen S.-T. Yau. Real time solution of nonlinear filtering problem without memory i. *Mathematical Research Letters*, 7:671–693, 2000.
- Shing-Tung Yau and Stephen S.-T. Yau. Real time solution of the nonlinear filtering problem without memory ii. *SIAM Journal on Control and Optimization*, 47(1):163–195, 2008. doi: 10.1137/050648353.
- Yiran Zhao and Tiangang Cui. Tensor-train methods for sequential state and parameter learning in state-space models. *Journal of Machine Learning Research*, 25:1–51, 2024.